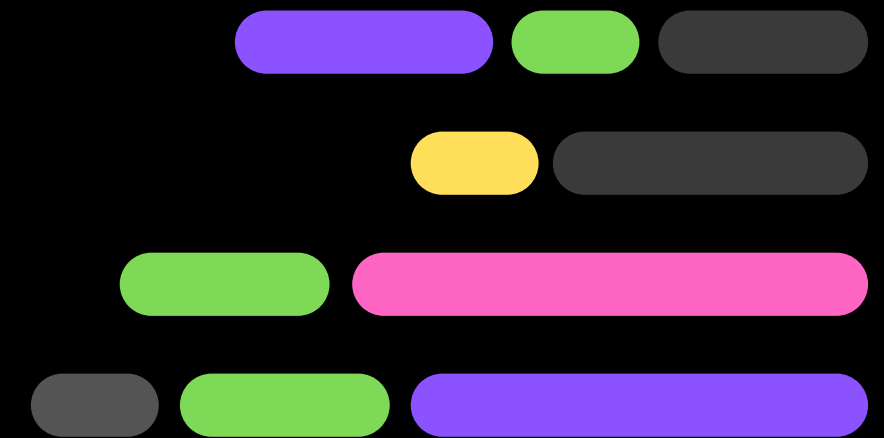
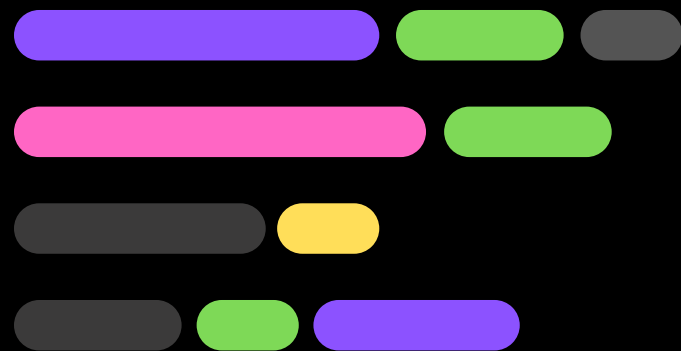




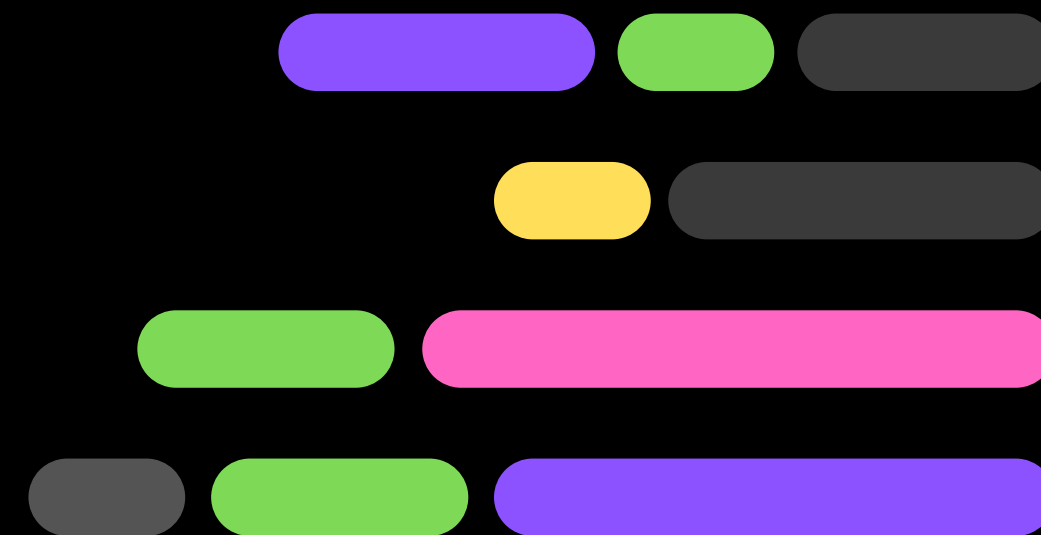
ARTIFICIAL INTELLIGENCE & { MACHINE LEARNING }

Welcome Aboard!





INTRODUCTION



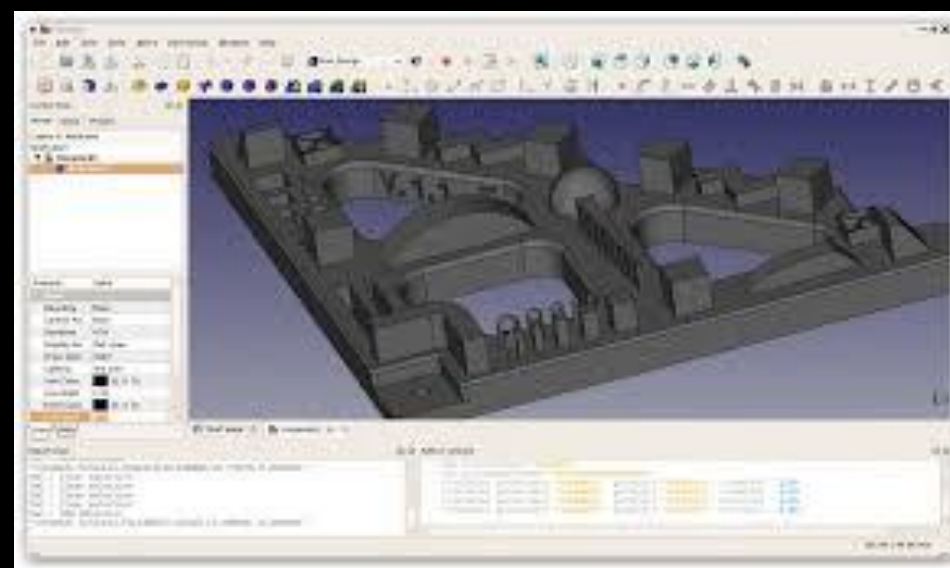
WHY PYTHON

Factors that make Python great for learning:

- 1.It is easy to learn
- 2.It is easy to use for writing new software
- 3.It is easy to obtain, install and deploy
- 4.Hundreds of Python libraries and frameworks (Pypi)
- 5.Big Data, Machine Language, Cloud Computing, and Automation
- 6.Mature and supportive Python community

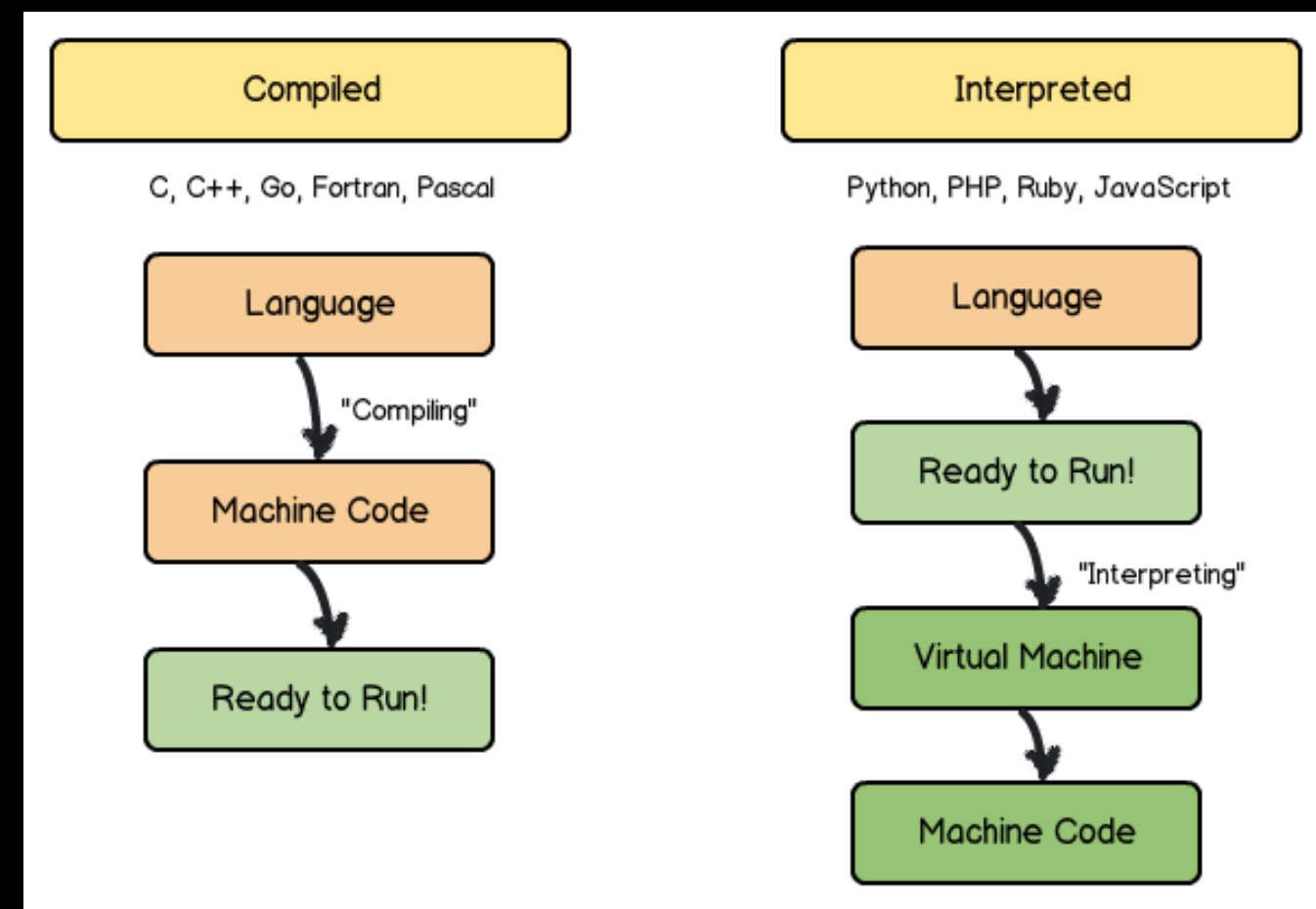


PYTHON BEING USED?



Compilation vs Interpretation

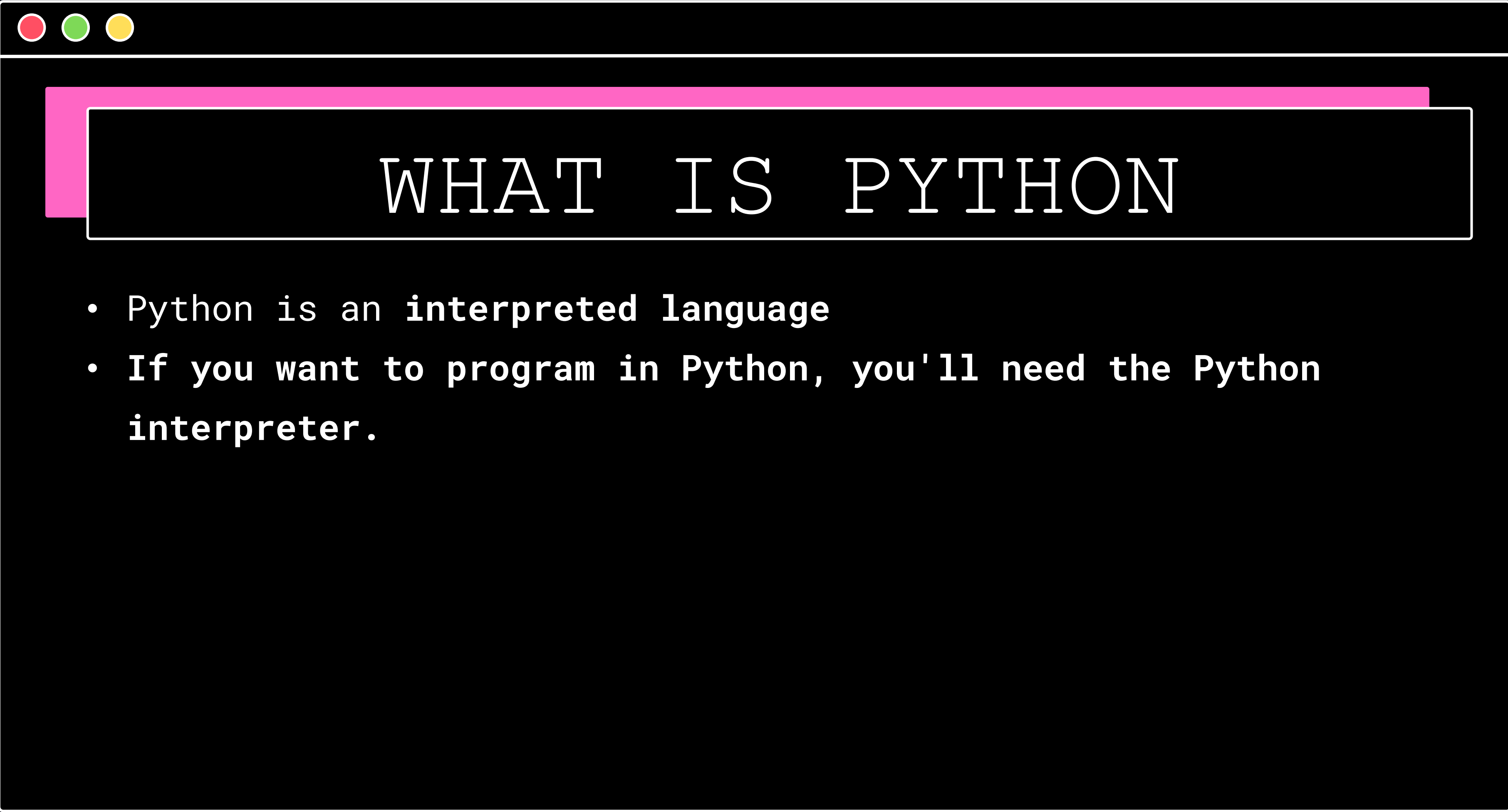
Compilation vs Interpretation



Compilation vs Interpretation

	Compilation	Interpretation
Advantages	- Faster execution	- Immediate code execution
	- End-user doesn't need compiler	- Portable across different architectures
	- Code stored in machine language	- No separate compilation needed
	- Protection of programming tricks	- Language flexibility
Disadvantages	- Time-consuming compilation process	- Slower execution due to interpreter
	- Need for multiple compilers for platforms	- Dependency on interpreter for execution

What does It
mean?



WHAT IS PYTHON

- Python is an **interpreted language**
- If you want to program in Python, you'll need the Python **interpreter.**

CREATOR OF PYTHON

Guido
van
Rossum

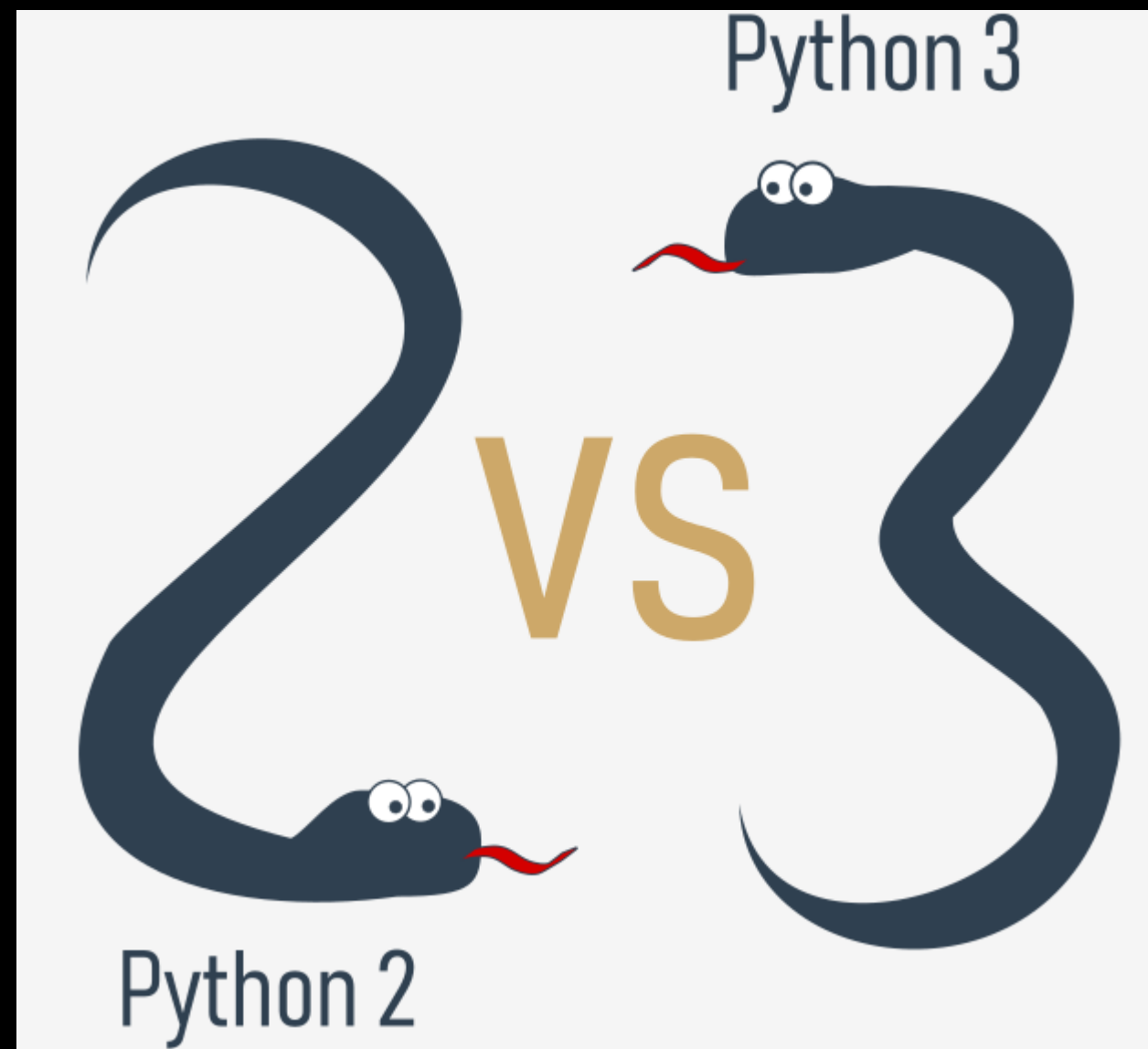




WHY NOT PYTHON

- `low-level programming` (sometimes called "`close to metal programming`"): if you want to implement an extremely effective driver or graphical engine, you wouldn't use Python.
- `applications for mobile devices`: although this territory is still waiting to be conquered by Python, it will most likely happen someday.

PYTHON VERSION



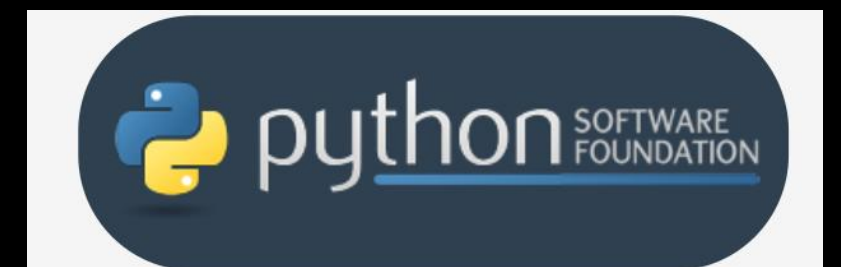
- The course focuses on Python 3, as it is the current version and recommended for new projects.
- Code samples in the course are tested against multiple versions of Python 3 to ensure compatibility.
- Important differences between Python 3 versions may be highlighted throughout the course when necessary.

PYTHON VERSION



CPython

- CPython is the primary implementation of Python maintained by the Python Software Foundation (PSF).
- Guido van Rossum, the creator of Python, implemented the first version of Python using the C programming language, and this approach continues to be used in CPython.
- CPython is considered the reference implementation of Python, as it follows all standards established by the PSF.
- CPython is highly influential and widely used, as it can be easily ported and migrated to various platforms with the ability to compile and run C language programs.



Cython

- **Purpose:** Addresses Python's lack of efficiency by translating Python code into C for faster execution.
- **Problem:** Python can be slow for complex mathematical calculations.
- **Solution:** Write code in Python, then use Cython to automatically translate it into C.
- **Process:** Write Python code, compile with Cython into C, then compile C into a shared library or executable.
- **Benefits:** Combines Python's ease of writing with C's efficiency, useful for performance-critical tasks and numerical computing.



Jython

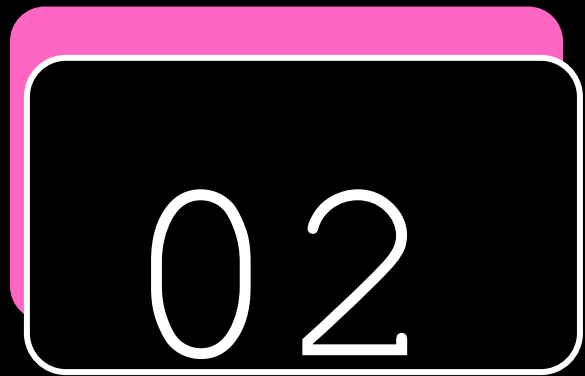
- **Purpose:** A version of Python implemented in Java, allowing seamless integration with Java environments.
- **Advantages:** Facilitates communication with existing Java infrastructure, making it useful for projects developed in Java.
- **Use Cases:** Particularly beneficial for large and complex systems written entirely in Java that require Python flexibility.
- **Compatibility:** Current Jython implementations follow Python 2 standards, with no Jython version conforming to Python 3 standards available yet.



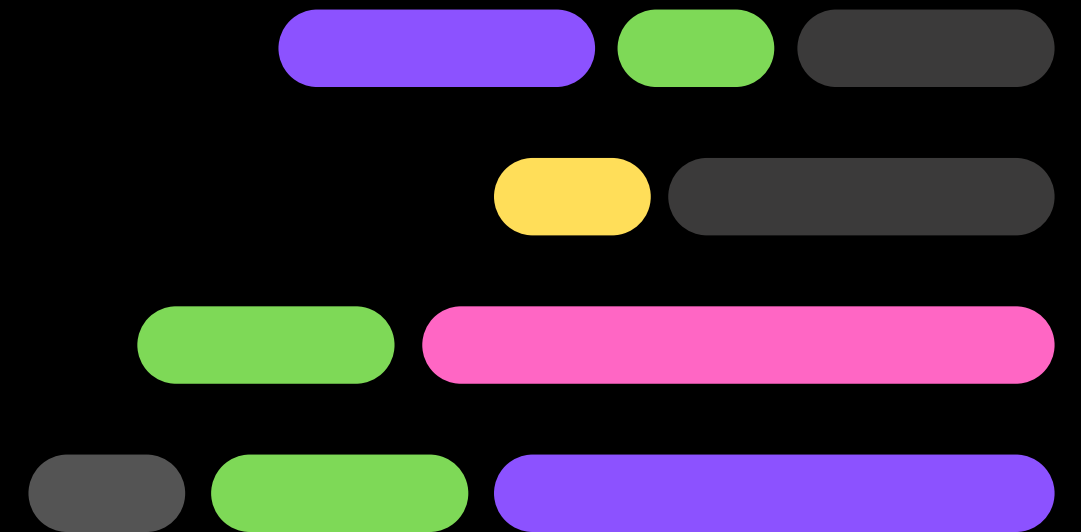
PyPy and Rpython

- **PyPy**: A Python implementation written in RPython, a subset of Python.
- **RPython**: A restricted subset of Python used to implement PyPy.
- **Execution**: PyPy source code is translated into C and then executed separately, rather than being interpreted like CPython.
- **Usefulness**: PyPy is useful for testing new features in Python implementations more easily than with CPython.
- **Compatibility**: PyPy is compatible with Python 3 language features.
- **Focus of the Course**: The course primarily focuses on CPython, but there are various other Python implementations in the world, including PyPy.





SETTING UP PYTHON ENVIRONMENT





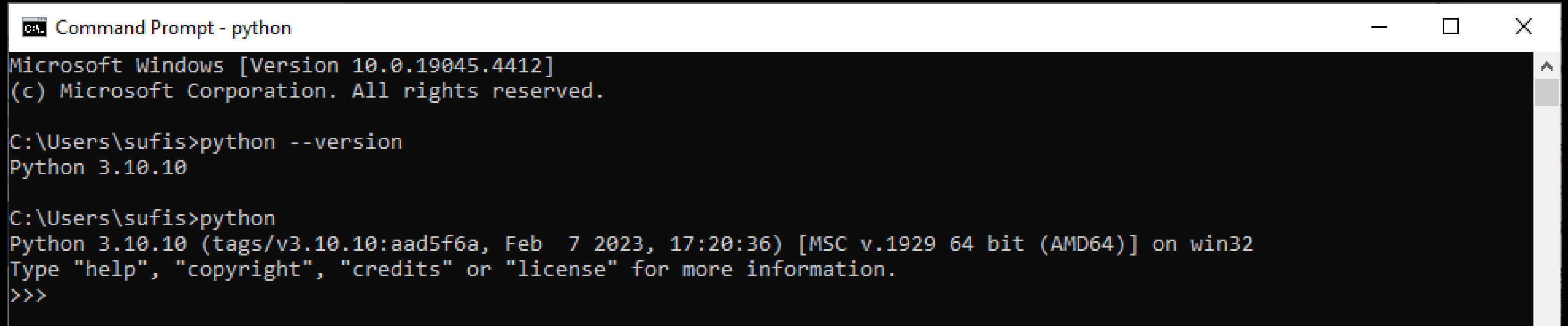
GETTING STARTED

- Installing Python
- Using IDLE and the basic interactive mode
- Writing a simple program
- Using Python shell
- Installing and Using PyCharm

GETTING STARTED

- Verify Python Installation:

On CMD: `python --version` / `python`



```
Command Prompt - python
Microsoft Windows [Version 10.0.19045.4412]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sufis>python --version
Python 3.10.10

C:\Users\sufis>python
Python 3.10.10 (tags/v3.10.10:aad5f6a, Feb 7 2023, 17:20:36) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```



GETTING STARTED

Starting your Work:

- **editor** which will support you in writing the code
- **console** in which you can launch your newly written code and stop it forcibly when it gets out of control.
- **debugger** able to launch your code step-by-step, which will allow you to inspect it at each moment of execution.

IDLE: Integrated Development and Learning Environment



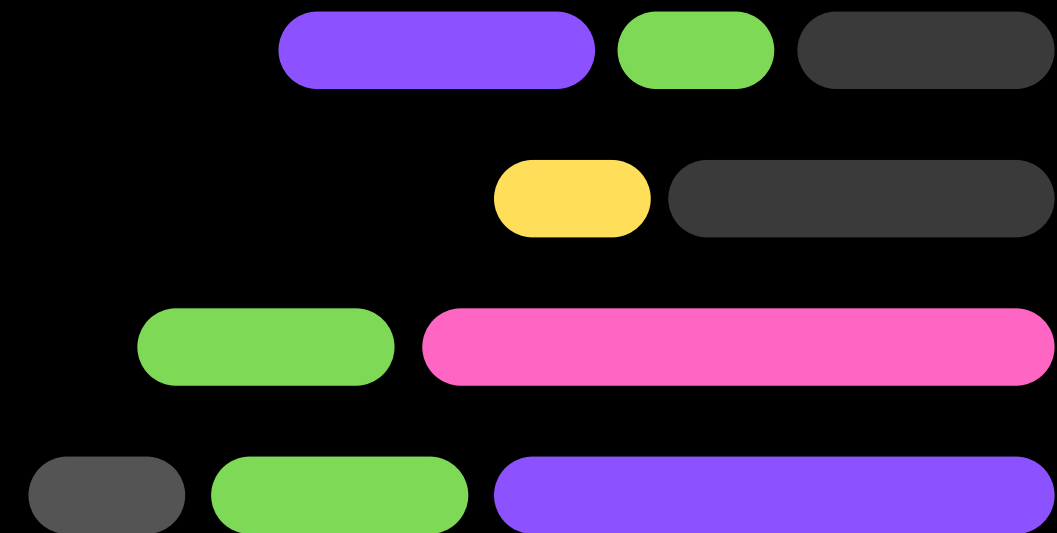
VIRTUAL ENVIRONMENT

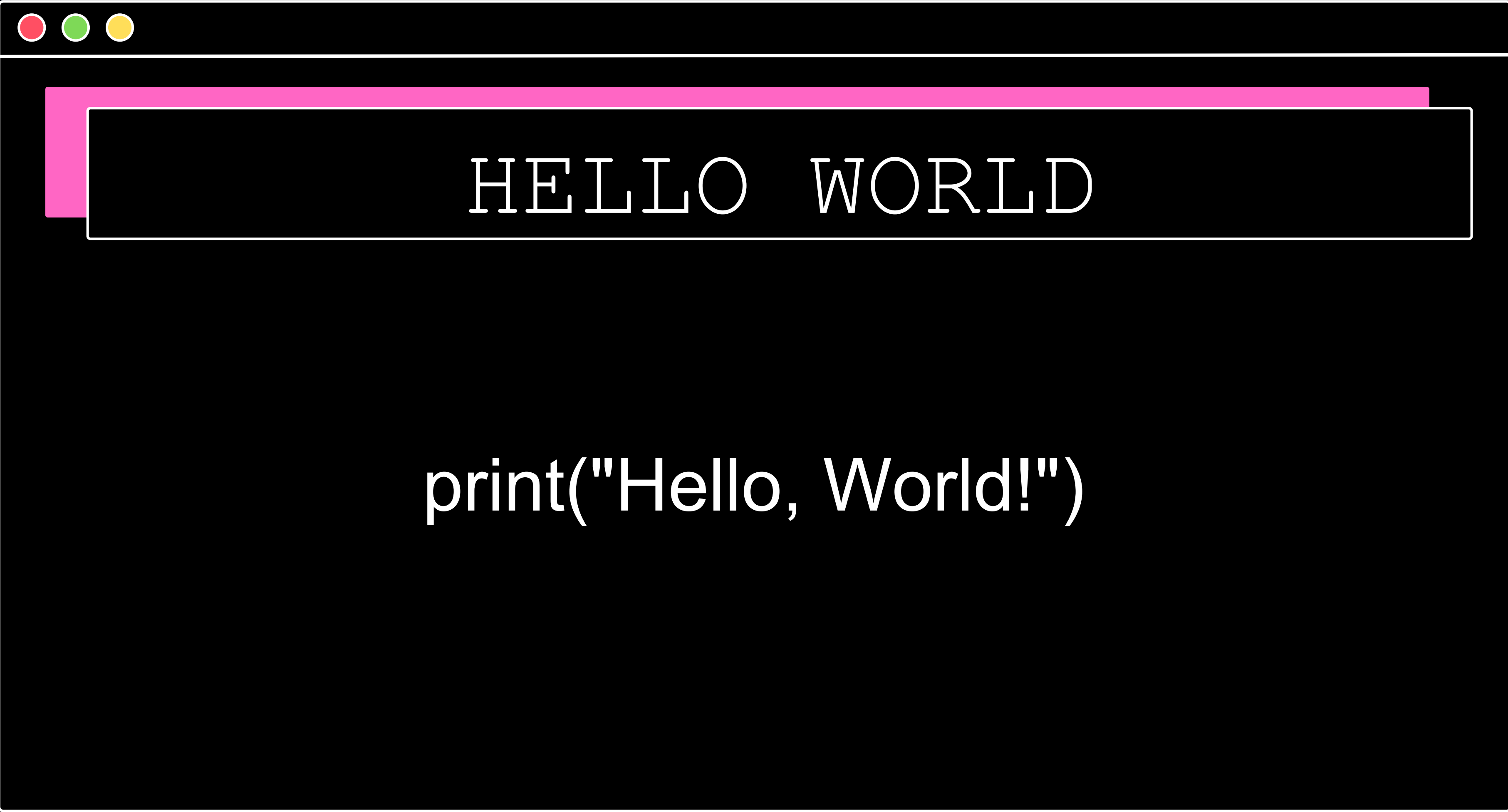
A virtual environment in Python is a self-contained directory that contains a Python interpreter and a set of libraries.

It allows you to isolate dependencies for different projects so they don't interfere with each other.



HELLO WORLD





HELLO WORLD

```
print("Hello, World!")
```



Print () Function

- `print()` Function: A built-in function that outputs specified messages to the screen.
- To call a function (this process is known as **function invocation** or **function call**)
- Escape Character: The backslash `\` is used to introduce special characters like `\n` for a newline.
- Formatting with `print()`: The '`sep`' parameter specifies the separator between arguments, and the '`end`' parameter defines what is printed at the end of the output

LAB 1

Print()



Literal

- Integer Literals: Numbers without a decimal point.

Example: 123, 0, -456

- Floating-Point Literals: Numbers with a decimal point.

Example: 3.14, 0.001, -7.5

- String Literals: Text enclosed in quotes.

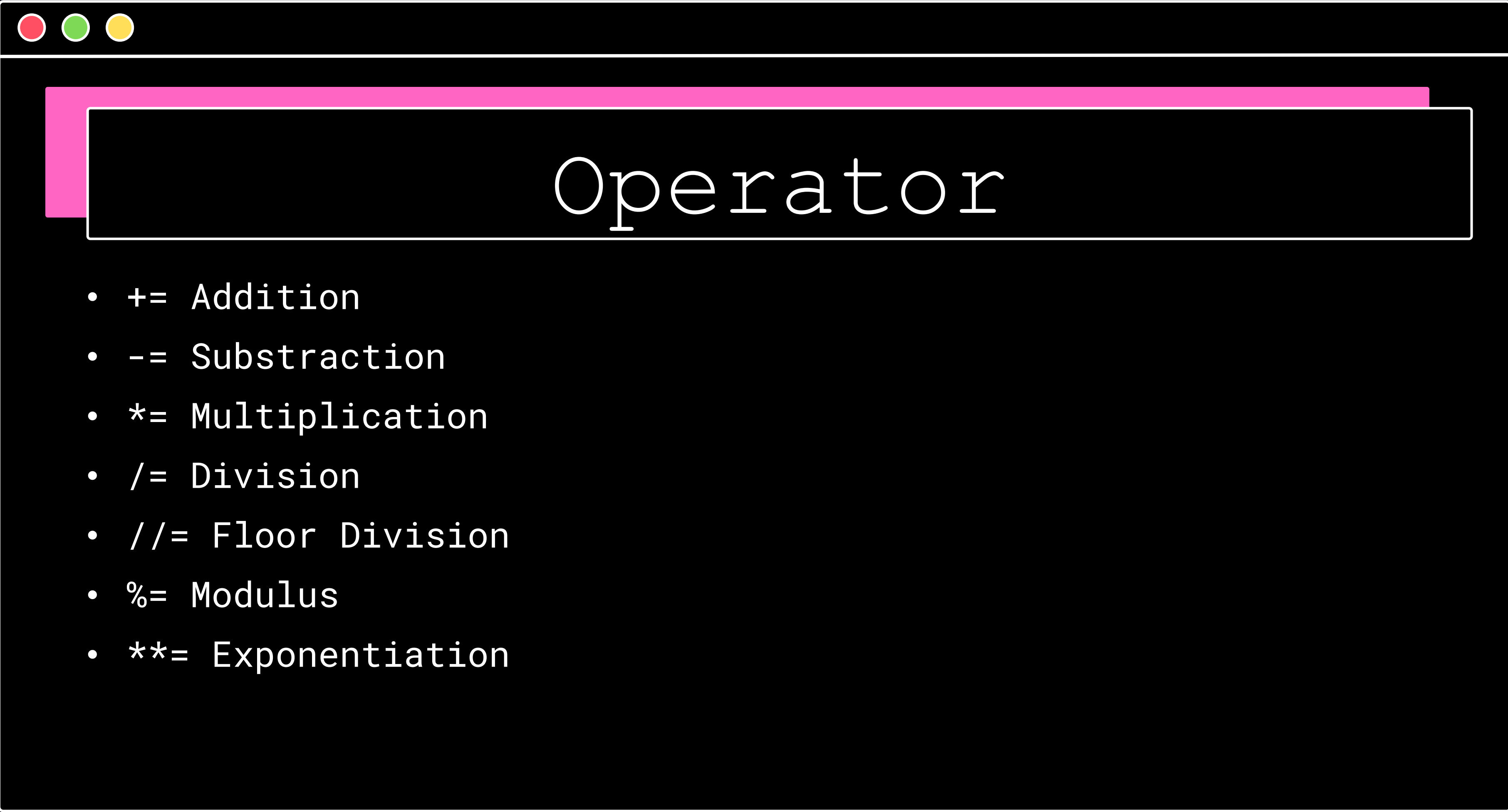
Example: "Hello, world!", 'Python', "I'm learning"

- Boolean Literals: The values True and False.

Example: True, False

LAB 2

Literal



Operator

- += Addition
- -= Subtraction
- *= Multiplication
- /= Division
- //= Floor Division
- %= Modulus
- **= Exponentiation

Operator

Priority	Operator	
1	<code>**</code>	
2	<code>+</code> , <code>-</code> (note: unary operators located next to the right of the power operator bind more strongly)	unary
3	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	
4	<code>+</code> , <code>-</code>	binary

Variables

Key Features:

NAME & VALUE

`num = 34`



Variables

Python is Dynamically-Typed Language:

python

 Copy code

```
# No need to declare variables or specify types
x = 10 # x is dynamically typed and assigned the value 10
y = "Hello" # y is dynamically typed and assigned the string "Hello"
```

c++

 Copy code

```
// Variables must be declared with a specific type
int x; // Declaration of an integer variable
x = 10; // Assignment of value 10 to variable x

std::string y; // Declaration of a string variable
y = "Hello"; // Assignment of string "Hello" to variable y
```

LAB 3

Variable

LAB 4

Variable & Operator



Comment

A remark inserted into the program, which is **omitted at runtime**, is called **a comment**. In Python, a comment is a piece of text that begins with a **#** (hash) sign

```
# This program evaluates the hypotenuse c.  
# a and b are the lengths of the legs.  
a = 3.0  
b = 4.0  
c = (a ** 2 + b ** 2) ** 0.5 # We use ** instead of square root.  
print("c =", c)
```



Comment

You can comment multiple lines in Python using triple quotes (''' or ''').

```
'''
```

```
This is a multi-line comment in Python.  
You can write comments spanning multiple lines  
by enclosing them in triple quotes.
```

```
'''
```

```
print("Hello, World!")
```




INPUT () FUNCTION

- **Purpose:** While `print()` outputs data to the console, `input()` retrieves data entered by the user.
- **Usage:** The `input()` function is invoked without any arguments.
- **Variable Assignment:** It's essential to assign the result of `input()` to a variable to capture the user's input. Without this step, the entered data will be lost.
- **Interactive Programs:** `input()` enables the creation of interactive programs that can receive and process user input, making the code more dynamic and user-friendly.

INPUT () FUNCTION

python

 Copy code

```
print("Tell me anything...")  
anything = input()  
print("Hmm...", anything, "... Really?")
```

- The program prompts the user to input data.
- The input() function receives the inputted data (string) and assigns it to the variable anything.
- The program then outputs the received data along with additional remarks.



TYPE CASTING

```
anything = float(input("Enter a number: "))  
something = anything ** 2.0  
print(anything, "to the power of 2 is", something)
```

- The float() function takes one argument a string: float(string) and tries to convert it into a float(the rest is the same)



TYPE CASTING

```
leg_a = float(input("Input first leg length: "))  
leg_b = float(input("Input second leg length: "))  
print("Hypotenuse length is " + str((leg_a**2 + leg_b**2) ** .5))
```

- This type of conversion is not a one-way street. You can also convert a number into a string, which is way easier and safer – this kind of operation is always possible. A function capable of doing that is called `str()`:

LAB 5

INPUT & OPERATOR



CONCATENATION

- The + (plus) sign, when applied to two strings, becomes a concatenation operator: (string + string)
- For Replication is * (asterisk) Sign
- It simply **concatenates** (glues) two strings into one
- operator is not commutative, i.e., "ab" + "ba" is not the same as "ba" + "ab".
- you must ensure that both its arguments are strings.



CONCATENATION

Replication:

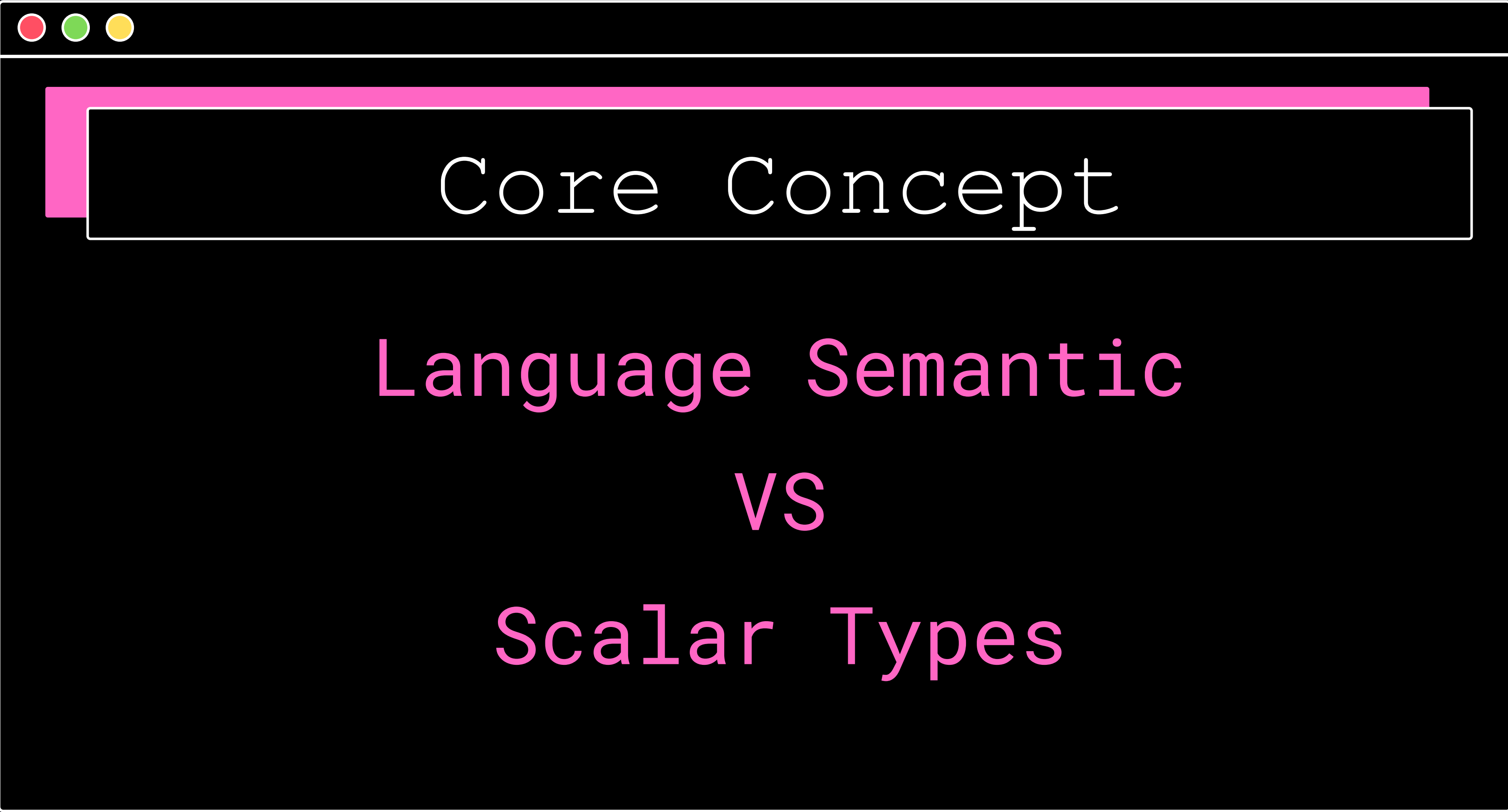
- The * (asterisk) sign, when applied to a string and number (or a number and string, as it remains commutative in this position) becomes a replication operator:

For example:

- "James" * 3 = "JamesJamesJames"
- 3 * "an" = "ananan"
- 5 * "2" (or "2" * 5) gives "22222" (not 10!)

LAB 6

INPUT



Core Concept

Language Semantic
VS
Scalar Types

Language Semantic

Variables are references, not boxes

Python does not copy the value, x and y both refer to the same object (10)

```
x = 10
y = x
x = 20
print(y)  # still 10
```


Language Semantic

Python is Dynamically-Typed Language:

python

 Copy code

```
# No need to declare variables or specify types
x = 10 # x is dynamically typed and assigned the value 10
y = "Hello" # y is dynamically typed and assigned the string "Hello"
```

c++

 Copy code

```
// Variables must be declared with a specific type
int x; // Declaration of an integer variable
x = 10; // Assignment of value 10 to variable x

std::string y; // Declaration of a string variable
y = "Hello"; // Assignment of string "Hello" to variable y
```

Language Semantic

Indentation defines code blocks:

- No {} like other languages – spaces matter.

```
if x > 0:
    print("Positive")    # inside block
print("Done")           # outside block
```

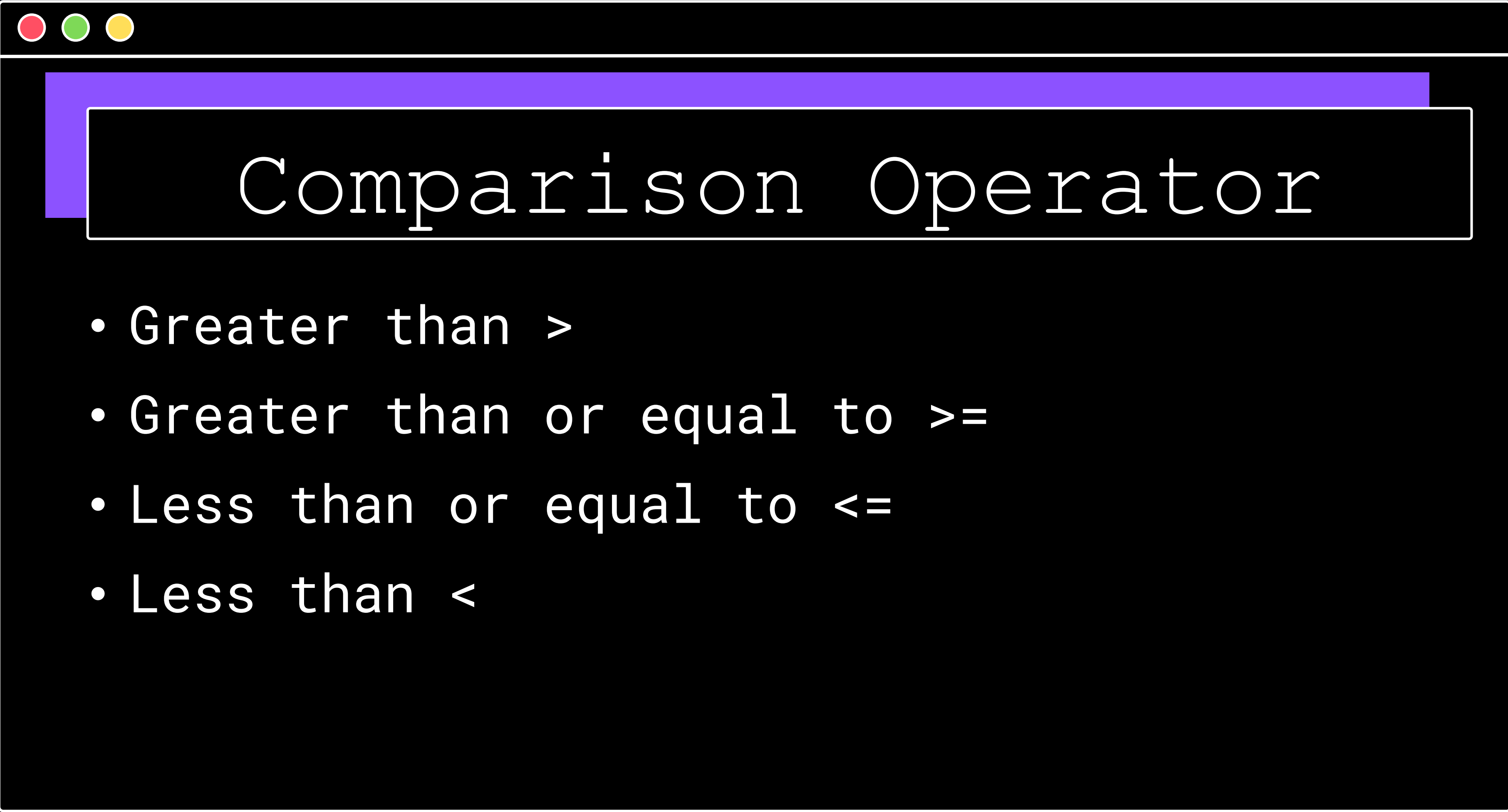



Language Semantic

Everything is an object:

- Even numbers and functions:

```
x = 10  
print(type(x))    # <class 'int'>
```



Comparison Operator

- Greater than >
- Greater than or equal to >=
- Less than or equal to <=
- Less than <

LAB 7

Comparison

CONDITIONS EXECUTION

```
if the_weather_is_good:  
    go_for_a_walk()  
have_lunch()
```

IDENTED

LAB 8

Comparison Operator

LAB 9

Number Classification

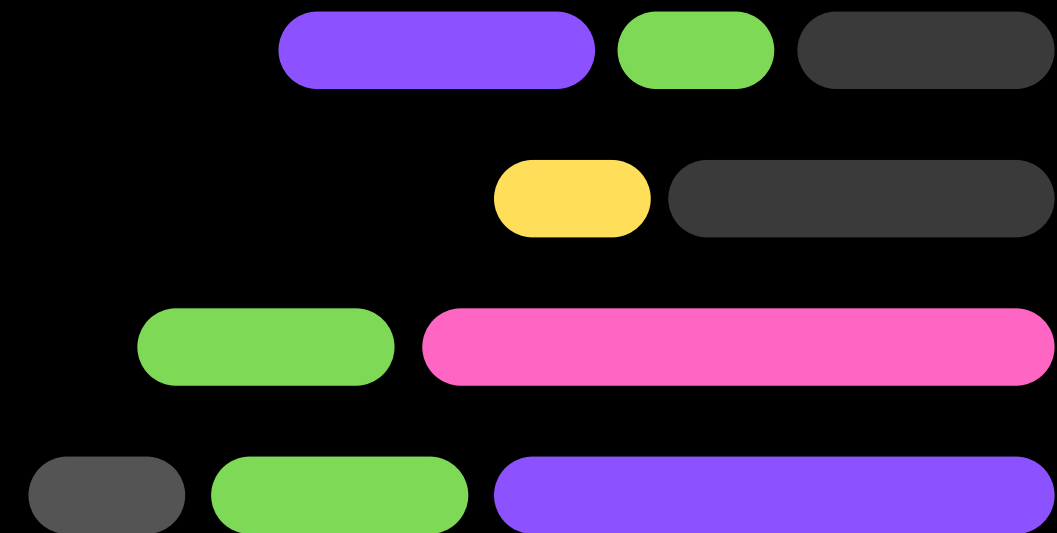
LAB 10

Year Leap



ITERATION

LOOPING



while Statement

- `while conditional_expression:`

`instruction_one`

`instruction_two`

`instruction_three`

`:`

`:`

`instruction_n`

- Indent all statements inside a while loop similarly, just like with if.
- Instructions within the while loop are called. the **loop's body**
- If the condition is False initially, the loop's body is not executed.
- The body should alter the condition's value to avoid an infinite loop.

LAB 11

Secret Number



for Statement

- Actually, the for loop is designed to do more complicated tasks - it can "browse" large collections of data item by item.

```
for i in range(100):  
    # do_something()  
    pass
```



for Statement

- The for keyword starts a for loop; no condition is needed.
- The variable after for is the **control variable**, automatically counting the loop's iterations.
- The in keyword defines the range of values for the control variable.
- The range() function generates values for the control variable, starting from 0 up to but not including the specified argument.
- The pass keyword is used as a placeholder when no action is required in the loop body.



for Statement

- the first argument determines the initial (first) value of the control variable. The last argument shows the first value the control variable will not be assigned.
- the range() function accepts only integers as its arguments, and generates sequences of integers.
- The first value shown is 2 (taken from the range()'s first argument.)
- The last is 7 (although the range()'s second argument is 8).

```
for i in range(2, 8):  
    print("The value of i is currently", i)
```


for Statement

- Range() function can accept three argument and it is for increment. The default value is 1

```
for i in range(2, 8, 3):  
    print("The value of i is currently", i)
```

The Value of i is currently 2
The Value of i is currently 5

LAB 12

Counting

break Statement

- `break` - exits the loop immediately, and unconditionally ends the loop's operation; the program begins to execute the nearest instruction after the loop's body;

```
# break - example

print("The break instruction:")
for i in range(1, 6):
    if i == 3:
        break
    print("Inside the loop.", i)
print("Outside the loop.")
```

continue Statement

- continue - behaves as if the program has suddenly reached the end of the body; the next turn is started and the condition expression is tested immediately.

```
# continue - example

print("\nThe continue instruction:")
for i in range(1, 6):
    if i == 3:
        continue
    print("Inside the loop.", i)
print("Outside the loop.")
```


LAB 13

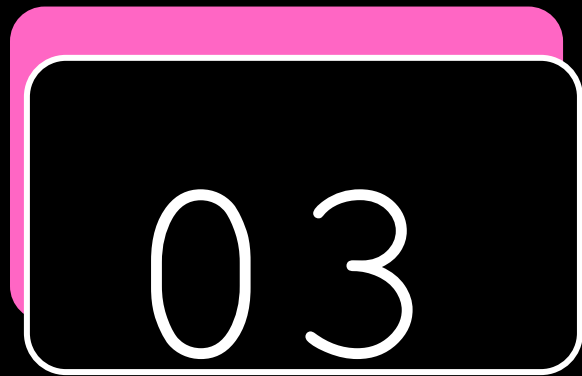
break

LAB 14

continue

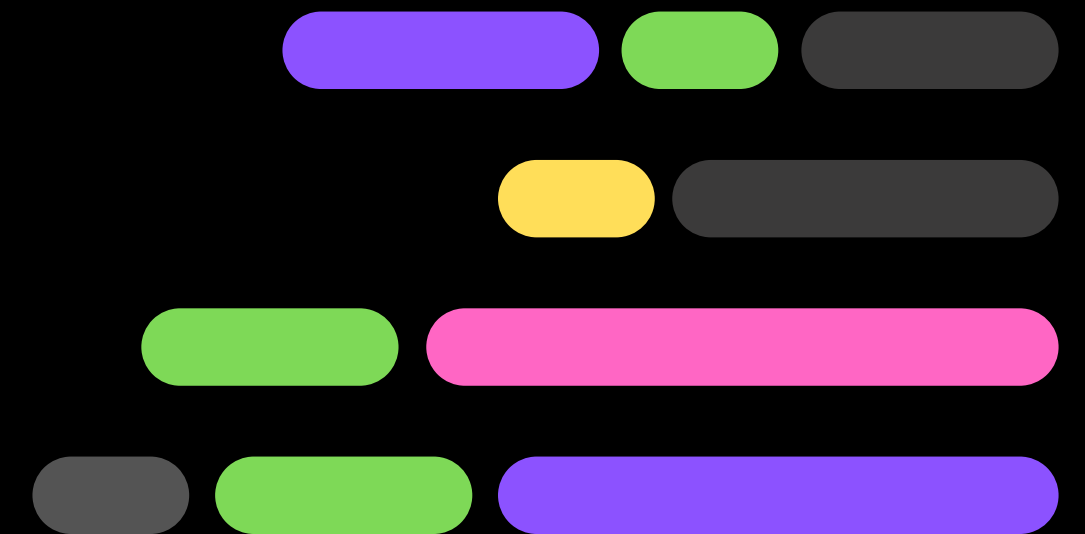
LAB 15

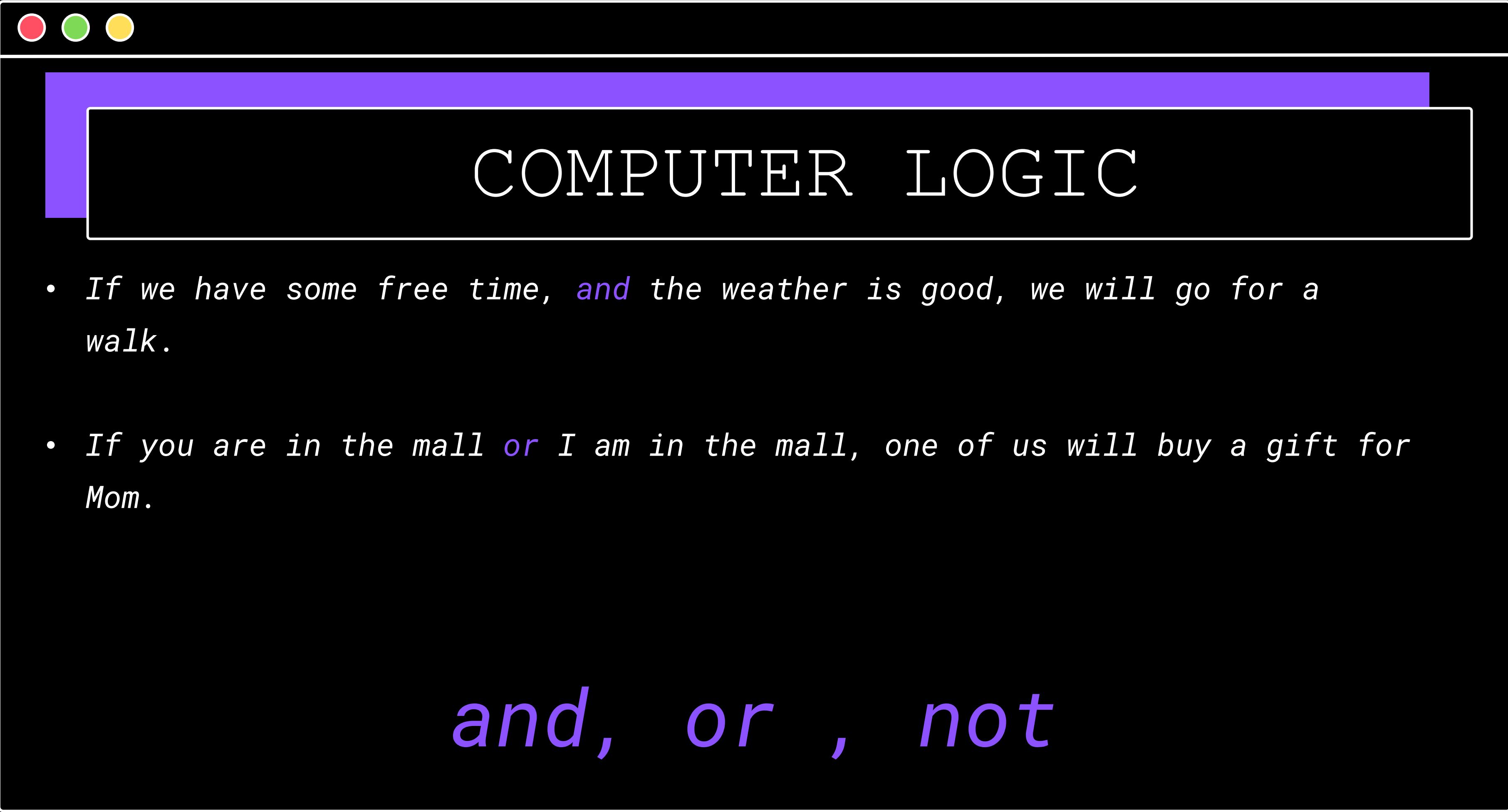
The Pyramid



LOGIC AND

BIT OPERATION





COMPUTER LOGIC

- *If we have some free time, **and** the weather is good, we will go for a walk.*
- *If you are in the mall **or** I am in the mall, one of us will buy a gift for Mom.*

and, or , not



COMPUTER LOGIC

- *One logical conjunction operator in Python is the word `and`. It's a **binary operator** with a **priority** that is **lower** than the one expressed by the **comparison operators**. It allows us to code complex conditions without the use of parentheses like this one:*

```
counter > 0 and value == 100
```

Logical operator: and

Argument A	Argument B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Logical operator: or

Argument A	Argument B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

Logical operator: not

Argument

not Argument

False

True

True

False

LAB 16

School

Admission

Why do we need List

```
numbers = [10, 5, 7, 2, 1]
```

- *list starts with an open square bracket and ends with a closed square bracket*
- *numbers is a list consisting of five values, all of them numbers.*
- *The elements inside a list may have different types. Some of them may be integers, others floats, and yet others may be lists.*
- *Indexable starting from [0]*

LAB 17

Basic List

Function vs Method

- *A function is a block of code that performs a specific task. It is defined independently and can be called by its name from anywhere in the code.*
- *Functions are called by their name followed by parentheses, and can be invoked with or without parameters.*

python

 Copy code

```
def my_function(param1, param2):  
    return param1 + param2  
  
result = my_function(3, 4)
```


Function vs Method

- *A method is a function that is associated with an object and is defined within a class. Methods are functions that belong to the instances of a class (objects).*
- *Methods are called on objects using the dot notation.*

```
python Copy code  
  
class MyClass:  
    def __init__(self, value):  
        self.value = value  
  
    def my_method(self, param):  
        return self.value + param  
  
obj = MyClass(10)  
result = obj.my_method(5)
```

Function vs Method

- **Function:**

```
python Copy code  
  
def add(a, b):  
    return a + b  
  
print(add(2, 3)) # 5
```

- **Method:**

```
python Copy code  
  
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
calc = Calculator()  
print(calc.add(2, 3)) # 5
```


LAB 18

The Beatles



Slicing List

A slice is an element of Python syntax that allows you to make a brand new copy of a list, or parts of a list. It actually copies the list's contents, not the list's name.

my_list[start:end]

A slice of this form makes a new (target) list, taking elements from the source list - the elements of the indices from start to end - 1.

Slicing List

```
# Copying the entire list.
list_1 = [1]
list_2 = list_1[:]
list_1[0] = 2
print(list_2)

# Copying some part of the list.
my_list = [10, 8, 6, 4, 2]
new_list = my_list[1:3]
print(new_list)
```

[1]
[8,6]

In and not in operators

- *Python offers two very powerful operators, able to look through the list in order to check whether a specific value is stored inside the list or not*

```
my_list = [0, 3, 12, 8, 2]

print(5 in my_list)
print(5 not in my_list)
print(12 in my_list)
```

List Comprehension

- *The special syntax used by Python in order to fill massive lists.*
- *A list comprehension is actually a list, but created on-the-fly during program execution, and is not described statically.*

```
squares = [x ** 2 for x in range(10)]
```

```
twos = [2 ** i for i in range(8)]
```

```
odds = [x for x in squares if x % 2 != 0]
```



List Comprehension

```
two_d_squares = [[j ** 2 for j in range(4)] for _ in range(4)]  
print(two_d_squares)
```

[[0, 1, 4, 9], [0, 1, 4, 9], [0, 1, 4, 9], [0, 1, 4, 9]]

LAB 19

Operator on List



Built-in Sequence Functions

They are *Python built-in functions* that work on *sequence types*, such as:

- *list*
- *tuple*
- *string*
- *range*
- *(also works on other iterables like set, dict keys)*

These functions *don't belong to the sequence* (unlike methods such as *.append()*), but can be used *on any sequence*.



Built-in Sequence Functions

len() – Length of a sequence

- *Returns number of elements.*

```
len([1, 2, 3])           # 3
```

```
len("Python")           # 6
```

```
len((10, 20))           # 2
```




Built-in Sequence Functions

min() – Smallest element

- Elements must be comparable.

```
min([3, 1, 5])           # 1
min("hello")             # 'e' (alphabetical)
```




Built-in Sequence Functions

max() – Largest element

Elements must be comparable.

```
min([3, 1, 5])           # 1
min("hello")             # 'e' (alphabetical)
```



Built-in Sequence Functions

sum() – Sum of numeric Elements

Not for strings

```
sum([1, 2, 3, 4])    # 10
```



Built-in Sequence Functions

sorted() – Returns a sorted list

Works on any sequence, returns a new list.

```
sorted([3, 1, 2])           # [1, 2, 3]
sorted("python")           # ['h', 'n', 'o', 'p', 't', 'y']
```



Built-in Sequence Functions

reversed() – Reverse iterator

Does NOT modify the original sequence.

```
list(reversed([1, 2, 3]))    # [3, 2, 1]
```


Built-in Sequence Functions

enumerate() – Index + value

Very useful in loops.

```
names = ["Ali", "Siti", "John"]  
  
for i, name in enumerate(names):  
    print(i, name)
```

```
0 Ali  
1 Siti  
2 John
```

Built-in Sequence Functions

zip() – Combine sequences

Stops at the shortest sequence.

```
names = ["Ali", "Siti"]  
scores = [80, 90]  
  
list(zip(names, scores))
```

```
[('Ali', 80), ('Siti', 90)]
```



Built-in Sequence Functions

any() – Is any element True?

```
any([False, False, True])    # True
```



Built-in Sequence Functions

all() – Are all elements True?

```
all([True, True, False])    # False
```


Built-in Sequence Functions

slice() - Create a slice object

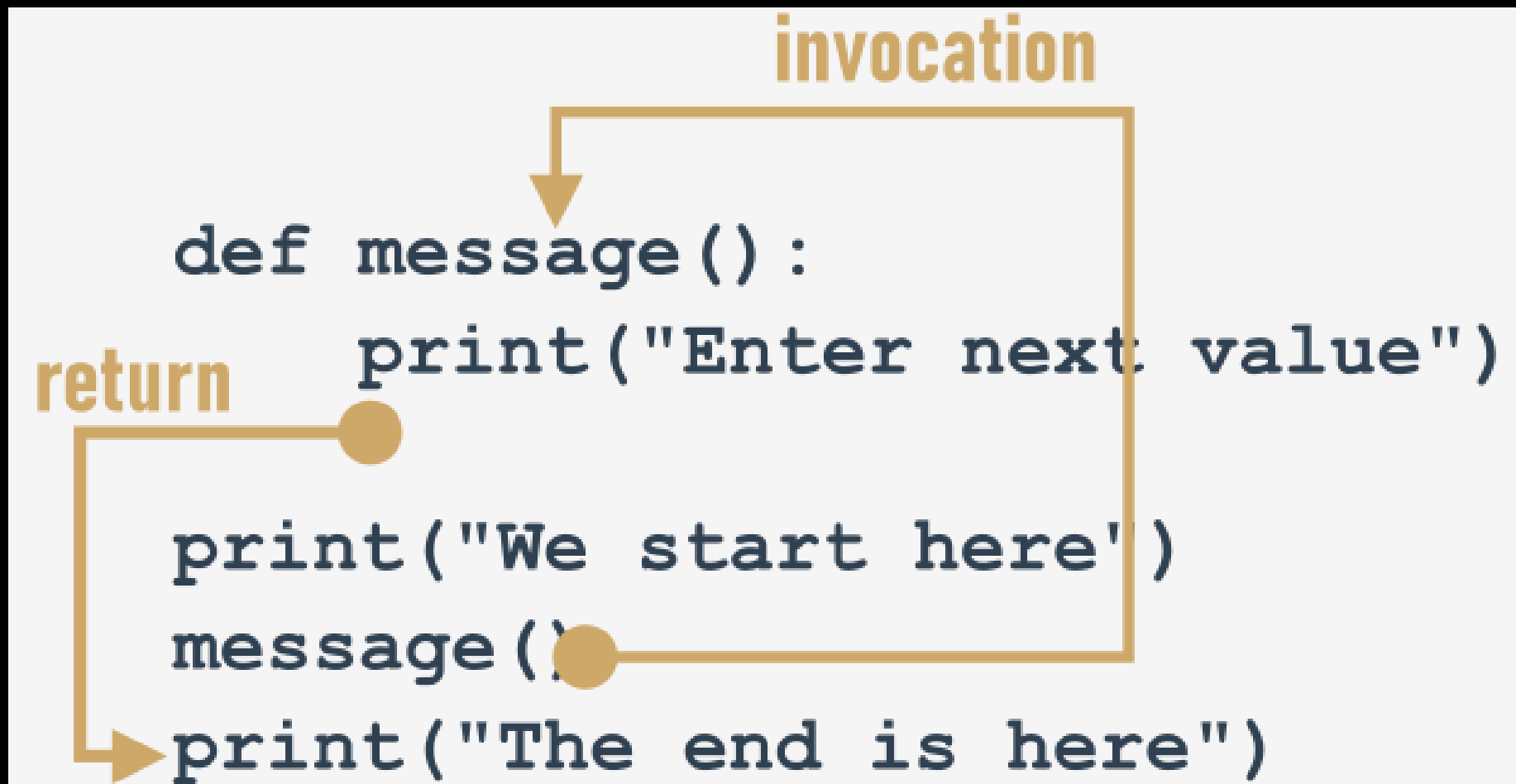
```
s = slice(1, 4)
[10, 20, 30, 40, 50][s]    # [20, 30, 40]
```

Equivalent to:

```
python
```

```
[10, 20, 30, 40, 50][1:4]
```

How Function Work



Parameterized Functions

- parameters exist only inside functions in which they have been defined, and the only place where the parameter can be defined is a space between a pair of parentheses in the def statement
- Assigning a value to the parameter is done at the time of the function's invocation, by specifying the corresponding argument.

```
def message(number):  
    print("Enter a number:", number)
```



Parameterized Functions

- parameters exist only inside functions in which they have been defined, and the only place where the parameter can be defined is a space between a pair of parentheses in the def statement
- Assigning a value to the parameter is done at the time of the function's invocation, by specifying the corresponding argument.

```
def message(number):  
    print("Enter a number:", number)
```


Positional Parameter

- A technique which assigns the *ith* (first, second, and so on) argument to the *ith* (first, second, and so on) function parameter is called **positional parameter passing**, while arguments passed in this way are named **positional arguments**.

```
def my_function(a, b, c):  
    print(a, b, c)  
  
my_function(1, 2, 3)
```

Keyword Argument Passing

- Python offers another convention for passing arguments, where **the meaning of the argument is dictated by its name**, not by its position
 - it's called **keyword argument passing**.

```
def introduction(first_name, last_name):  
    print("Hello, my name is", first_name, last_name)  
  
introduction(first_name = "James", last_name = "Bond")  
introduction(last_name = "Skywalker", first_name = "Luke")
```

Arbitrary Argument Passing

- Parameter that will pack all arguments into a dictionary
- Useful so that a function can accept a varying amount of keyword argument
- `**kwargs` or `**`

```
def hello(**kwargs):  
    #print("Hello " + kwargs['first'] + " " + kwargs['last'])  
    print("Hello",end=" ")  
    for key,value in kwargs.items():  
        print(value,end=" ")  
  
hello(title="Mr.",first="Bro",middle="Dude",last="Code")
```



Return Instruction

- To get `functions to return a value` (but not only for this purpose) you use the `return` instruction.
- There is 2 variant `return` without expression and `return` with expression

Return Instruction

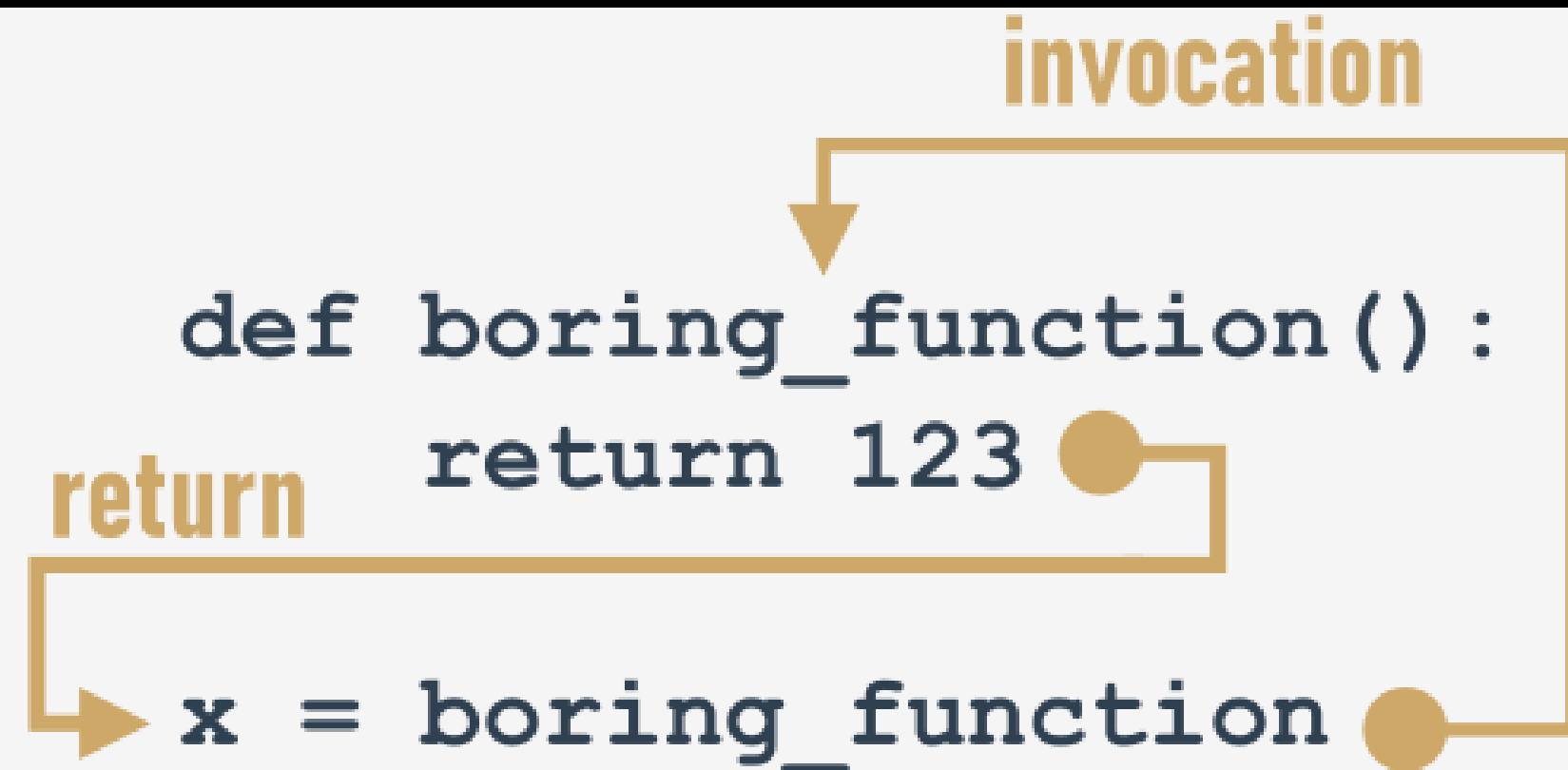
Return without expression

When used inside a function, it causes the immediate termination of the function's execution, and an instant return (hence the name) to the point of invocation.

```
def happy_new_year(wishes = True):  
    print("Three...")  
    print("Two...")  
    print("One...")  
    if not wishes:  
        return  
  
    print("Happy New Year!")
```

Return Instruction

The result may be freely used here, e.g., to be assigned to a variable.





None Value

Its data doesn't represent any reasonable value - actually, it's not a value at all; hence, it `mustn't take part in any expressions`.

`None` is a keyword.

There are only two kinds of circumstances when `None` can be safely used:

- when you `assign it to a variable` (or return it as a function's result)
- when you `compare it with a variable` to diagnose its internal state.



None Value

Don't forget this: if a function doesn't return a certain value using a return expression clause, it is assumed that it **implicitly returns None**.

```
value = None
if value is None:
    print("Sorry, you don't carry any value")
```


Simple Function: BMI

$$\text{BMI} = \frac{\text{(weight in kilograms)} \text{$$

Simple Function: Factorial

Factorial

$$n! = n * (n - 1) * (n - 2) * (n - 3) * \dots * 3 * 2 * 1$$

$$0! = 1$$

$$1! = 1$$

$$2! = 2 \times 1 = 2$$

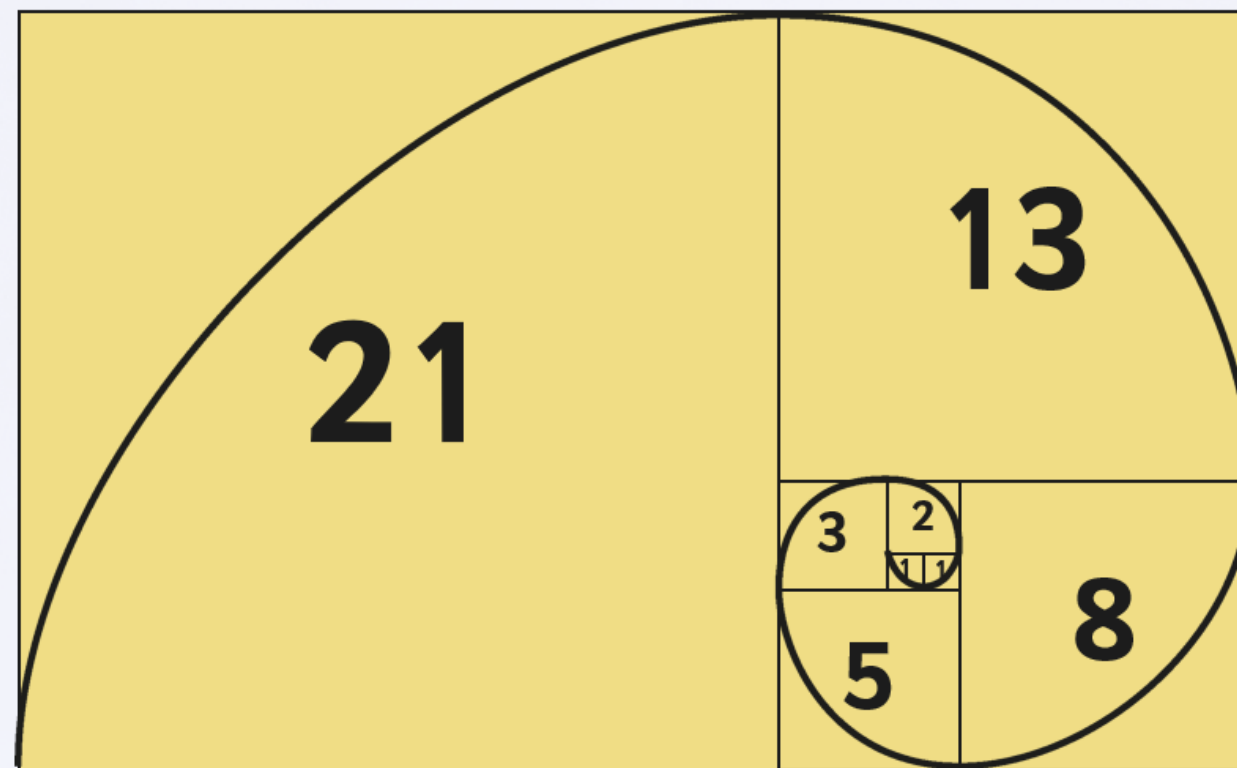
$$3! = 3 \times 2 \times 1 = 6$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$



Simple Function: Fibonacci

Fibonacci Spirals





Recursion

- recursion is a technique where a function invokes itself.
- The Fibonacci numbers definition is a clear example of recursion.

```
def fib(n):  
    if n < 1:  
        return None  
    if n < 3:  
        return 1  
    return fib(n - 1) + fib(n - 2)
```


LAB 20

Leap Year Function

LAB 21

Prime Number

TUPLE

Sequence Types

- **Definition:** Sequence types in Python are data types that can store more than one value and can be iterated through element by element using a for loop.
- **Example:** The list is a classic example of a Python sequence.

Mutability

- **Mutable Data:** Data that can be freely changed during program execution. Lists are an example of mutable data, as you can modify them in situ (e.g., `list.append(1)`).
- **Immutable Data:** Data that cannot be modified in situ. Any changes require the creation of a new data structure. Tuples are an example of immutable data.

TUPLE

tuples prefer to use parenthesis, whereas lists like to see brackets, although it's also possible to create a tuple just from a set of values separated by commas.

```
tuple_1 = (1, 2, 4, 8)
tuple_2 = 1., .5, .25, .125
```


How to use TUPLE

- If you want to get the elements of a tuple in order to read them over, you can use the same conventions to which you're accustomed while using lists.

```
my_tuple = (1, 10, 100, 1000)

print(my_tuple[0])
print(my_tuple[-1])
print(my_tuple[1:])
print(my_tuple[:-2])

for elem in my_tuple:
    print(elem)
```

How to use TUPLE

- The similarities may be misleading - don't try to modify a tuple's contents! *It's not a list!*
- All of these instructions (except the topmost one) will cause a runtime error:

```
my_tuple = (1, 10, 100, 1000)

my_tuple.append(10000)
del my_tuple[0]
my_tuple[1] = -10
```

How to use TUPL

- `len()` – return number of element
- `+` = can join tuples together
- `*` = multiply tuples
- `in/not in` = for expression

```
my_tuple = (1, 10, 100)

t1 = my_tuple + (1000, 10000)
t2 = my_tuple * 3

print(len(t2))
print(t1)
print(t2)
print(10 in my_tuple)
print(-10 not in my_tuple)
```

How to use TUPLE

- One of the most useful tuple properties is their ability to appear on the left side of the assignment operator.

```
var = 123

t1 = (1, )
t2 = (2, )
t3 = (3, var)

t1, t2, t3 = t2, t3, t1

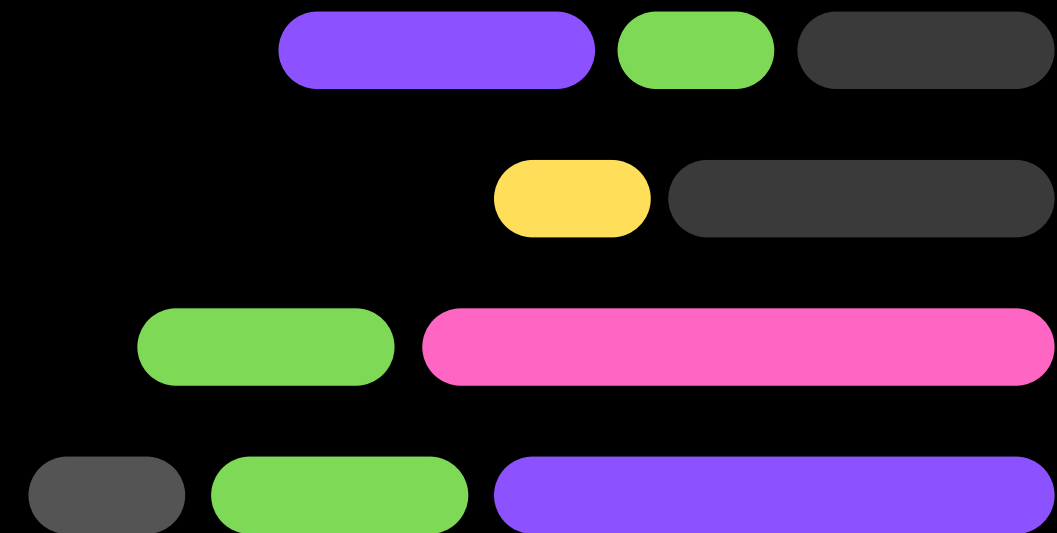
print(t1, t2, t3)
```


LAB 22

Tuple



DICTIONARY



DICTIONARY

- The `dictionary` is another Python data structure. It's `not a sequence` type (but can be easily adapted to sequence processing) and it is `mutable`.

```
var = 123

t1 = (1, )
t2 = (2, )
t3 = (3, var)

t1, t2, t3 = t2, t3, t1

print(t1, t2, t3)
```


DICTIONARY

- In Python's world, the word you look for is named a **key**. The word you get from the dictionary is called a **value**

Definition: A dictionary is a set of key-value pairs.

- **Unique Keys**: Each key must be unique.
- **Key Types**: Keys can be any immutable type (e.g., numbers, strings), but not lists.
- **Comparison to Lists**: Unlike lists, which have ordered values, dictionaries hold pairs of values.
- **Length**: The `len()` function returns the number of key-value pairs in a dictionary.
- **One-Way Tool**: Dictionaries allow lookup of values based on keys, but not vice versa (e.g., English-French dictionary only allows English to French lookup).

How to make a DICTIONARY

- The list of pairs is surrounded by curly braces, while the pairs themselves are separated by commas, and the keys and values by colons.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
phone_numbers = {'boss': 5551234567, 'Suzy': 22657854310}  
empty_dictionary = {}  
  
print(dictionary)  
print(phone_numbers)  
print(empty_dictionary)
```

How to use a DICTIONARY

If you want to get any of the values, you have to deliver a valid key value:

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
phone_numbers = {'boss' : 5551234567, 'Suzy' : 22657854310}  
empty_dictionary = {}
```

```
# Print the values here.
```

```
    print(dictionary['cat'])  
    print(phone_numbers['Suzy'])
```



How to use a DICTIONARY

And now the most important news: you **mustn't use a non-existent key**.

```
print(phone_numbers['president'])
```

Fortunately, there's a simple way to avoid such a situation. The `in` operator, together with its companion, `not in`, can salvage this situation.

How to use a DICTIONARY

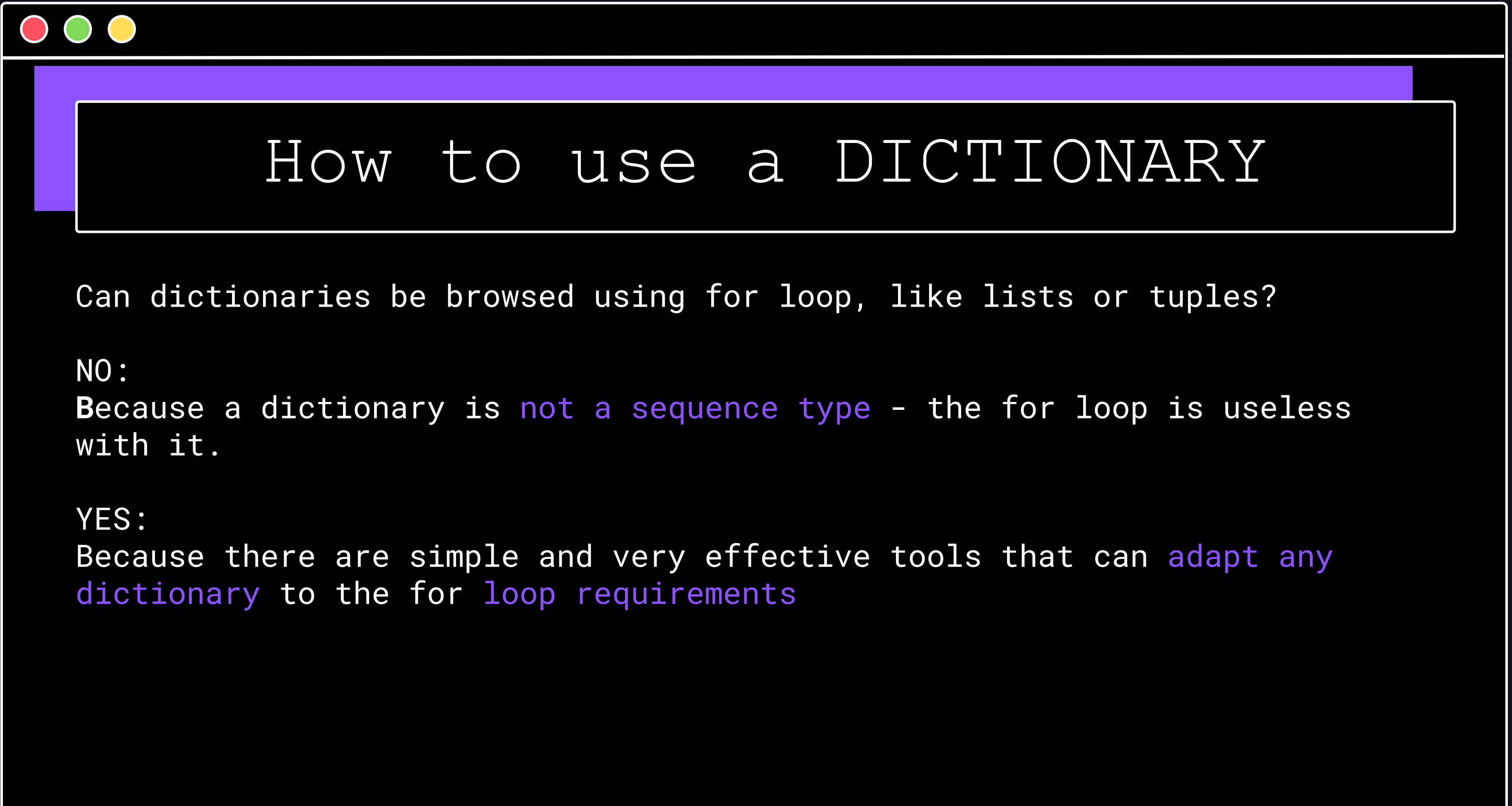
```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}
words = ['cat', 'lion', 'horse']

for word in words:
    if word in dictionary:
        print(word, "->", dictionary[word])
    else:
        print(word, "is not in dictionary")
```

Cat -> chat

Lion is not in dictionary

Horse -> cheval



How to use a DICTIONARY

Can dictionaries be browsed using for loop, like lists or tuples?

NO:

Because a dictionary is **not a sequence type** - the for loop is useless with it.

YES:

Because there are simple and very effective tools that can **adapt any dictionary** to the for **loop requirements**



How to use a DICTIONARY

The first of them is a method named `keys()`, possessed by each dictionary. The method returns an iterable object consisting of all the keys gathered within the dictionary. Having a group of keys enables you to access the whole dictionary in an easy and handy way.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
for key in dictionary.keys():  
    print(key, "->", dictionary[key])
```

How to use a DICTIONARY

There is also a method named `values()`, which works similarly to `keys()`, but `returns values`.

As the dictionary is not able to automatically find a key for a given value, the role of this method is rather limited.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
for french in dictionary.values():  
    print(french)
```

How to use a DICTIONARY

`items()` method

- The method `returns tuples` (this is the first example where tuples are something more than just an example of themselves) `where each tuple is a key-value pair`.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
for english, french in dictionary.items():  
    print(english, "->", french)
```


How to use a DICTIONARY

Modifying:

- Assigning a new value to an existing key is simple - as dictionaries are fully **mutable**, there are no obstacles to modifying them.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}
```

```
dictionary['cat'] = 'minou'  
print(dictionary)
```

How to use a DICTIONARY

Adding and update():

- Adding a new key-value pair to a dictionary is as simple as changing a value - you only have to assign a value to a new, previously non-existent key.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
dictionary['swan'] = 'cygne'  
print(dictionary)|
```

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
dictionary.update({"duck": "canard"})  
print(dictionary)|
```

How to use a DICTIONARY

Remove a key():

- Removing a key will always cause the removal of the associated value. Values cannot exist without their keys.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
del dictionary['dog']  
print(dictionary)|
```

How to use a DICTIONARY

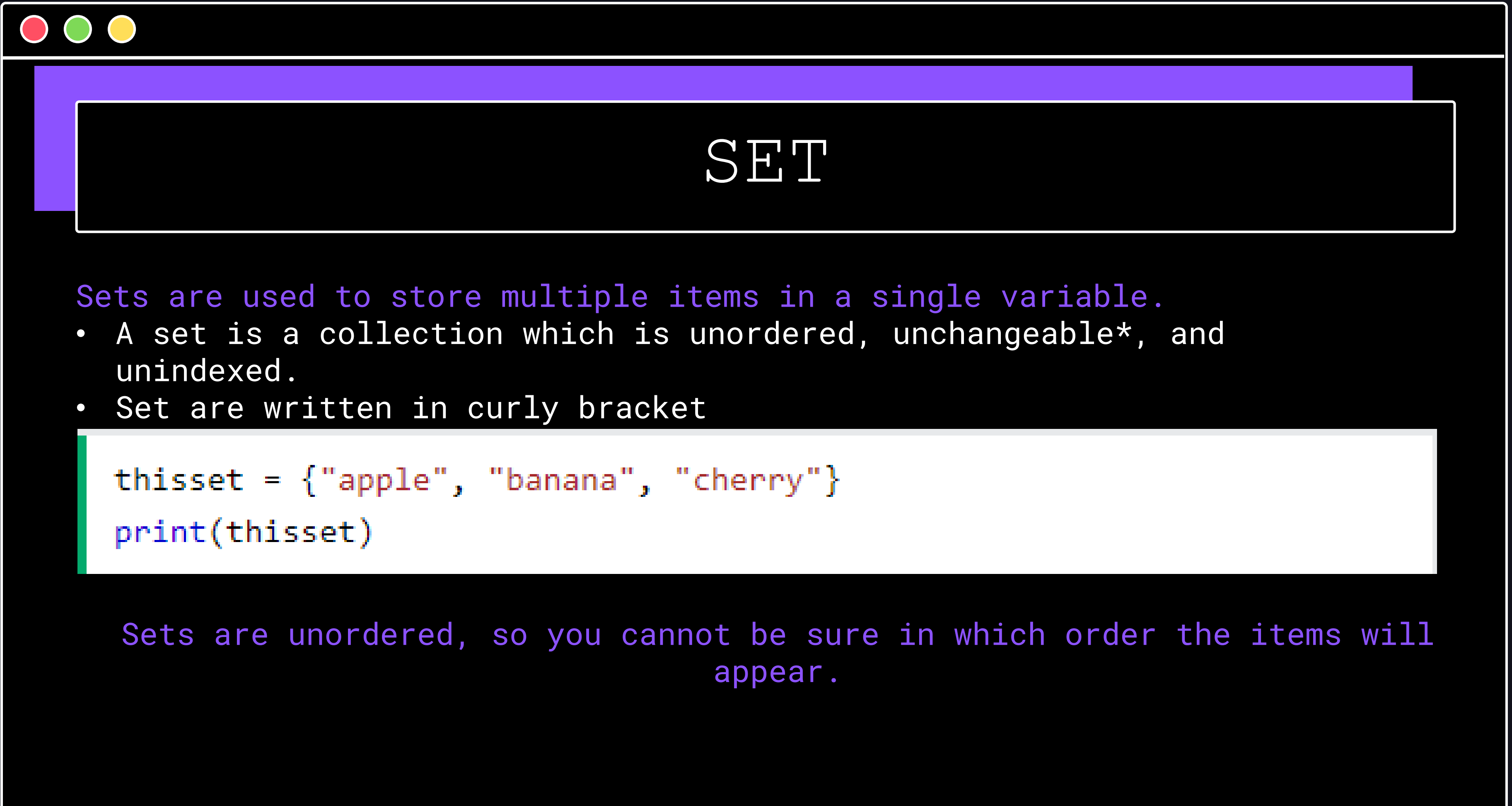
Remove last item in dictionary:

- To remove the last item in a dictionary, you can use the `popitem()`.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}  
  
dictionary.popitem()  
print(dictionary)      # outputs: {'cat': 'chat', 'dog': 'chien'}
```


LAB 23

Dict



SET

Sets are used to store multiple items in a single variable.

- A set is a collection which is unordered, unchangeable*, and unindexed.
- Set are written in curly bracket

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

Sets are unordered, so you cannot be sure in which order the items will appear.

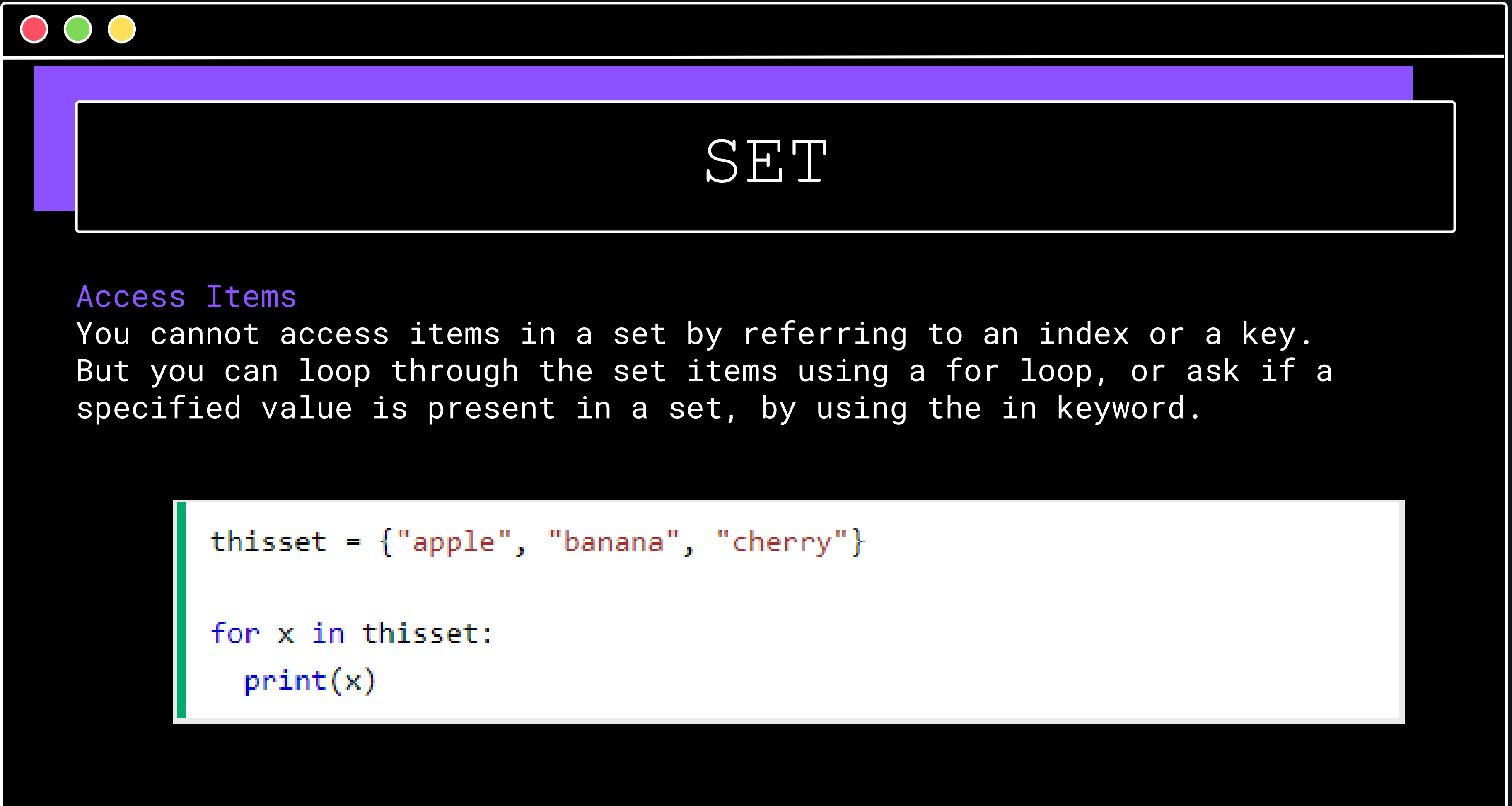


SET

Duplicates Not Allowed

```
thisset = {"apple", "banana", "cherry", "apple"}  
  
print(thisset)
```

- The values True and 1 are considered the same value in sets, and are treated as duplicates:
- Same goes to False and 0



SET

Access Items

You cannot access items in a set by referring to an index or a key. But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.

```
thisset = {"apple", "banana", "cherry"}  
  
for x in thisset:  
    print(x)
```




SET

Add Set Items

Once a set is created, you cannot change its items, but you can add new items.

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
  
print(thisset)
```

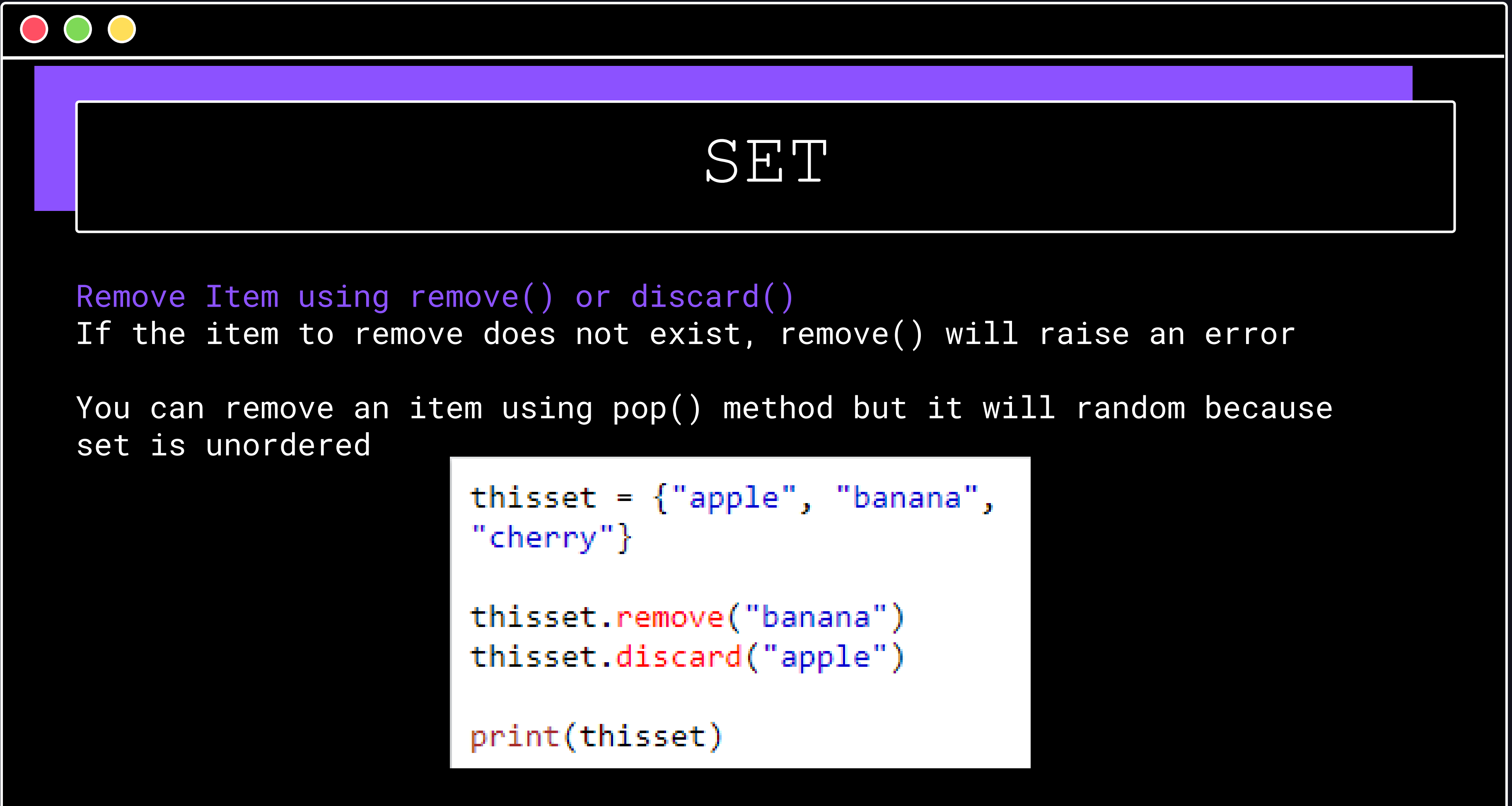



SET

Add Set Items `update()` Method

Once a set is created, you cannot change its items, but you can add new items. It can be any iterable object (tuples, list, dictionaries)

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}  
  
thisset.update(tropical)  
  
print(thisset)
```



SET

Remove Item using `remove()` or `discard()`

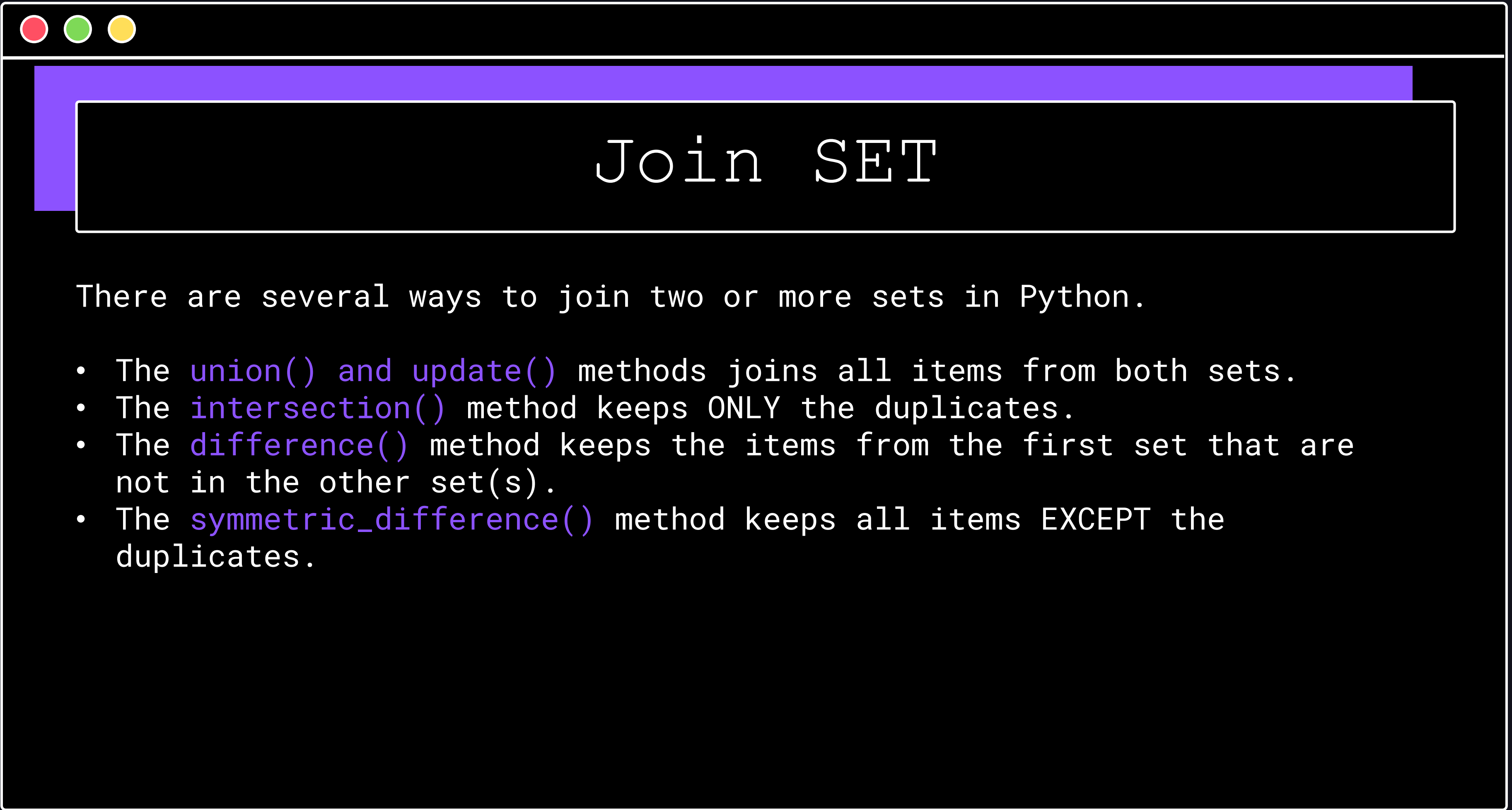
If the item to remove does not exist, `remove()` will raise an error

You can remove an item using `pop()` method but it will random because set is unordered

```
thisset = {"apple", "banana",  
"cherry"}
```

```
thisset.remove("banana")  
thisset.discard("apple")
```

```
print(thisset)
```



Join SET

There are several ways to join two or more sets in Python.

- The `union()` and `update()` methods joins all items from both sets.
- The `intersection()` method keeps ONLY the duplicates.
- The `difference()` method keeps the items from the first set that are not in the other set(s).
- The `symmetric_difference()` method keeps all items EXCEPT the duplicates.

Join SET

The `union()` method returns a new set with all items from both sets.

```
set1 = {"a", "b", "c"}  
set2 = {1, 2, 3}  
  
set3 = set1.union(set2)  
set3 = set1 | set2 #Symbol of Union  
print(set3)
```

Join SET

Join a set with a tuple

```
x = {"a", "b", "c"}  
y = (1, 2, 3)  
  
z = x.union(y)  
print(z)
```


Join SET

The `intersection()` method will return a new set, that only contains the items that are present in both sets. '&' operator consider the same

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"google", "microsoft", "apple"}  
  
set3 = set1.intersection(set2)  
print(set3)
```

Join SET

The `difference()` method will return a new set that will contain only the items from the first set that are not present in the other set.
'-' operator works the same

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"google", "microsoft", "apple"}  
  
set3 = set1.difference(set2)  
  
print(set3)
```

Join SET

The `symmetric_difference()` method will keep only the elements that are NOT present in both sets. '^' operator works the same

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"google", "microsoft", "apple"}  
  
set3 = set1.symmetric_difference(set2)  
  
print(set3)
```

LAB 24

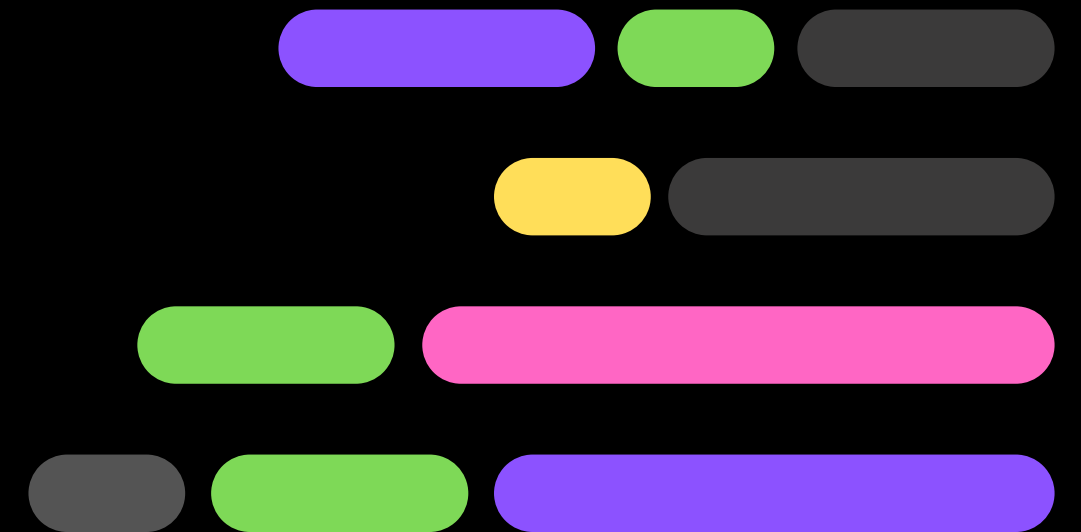
Set

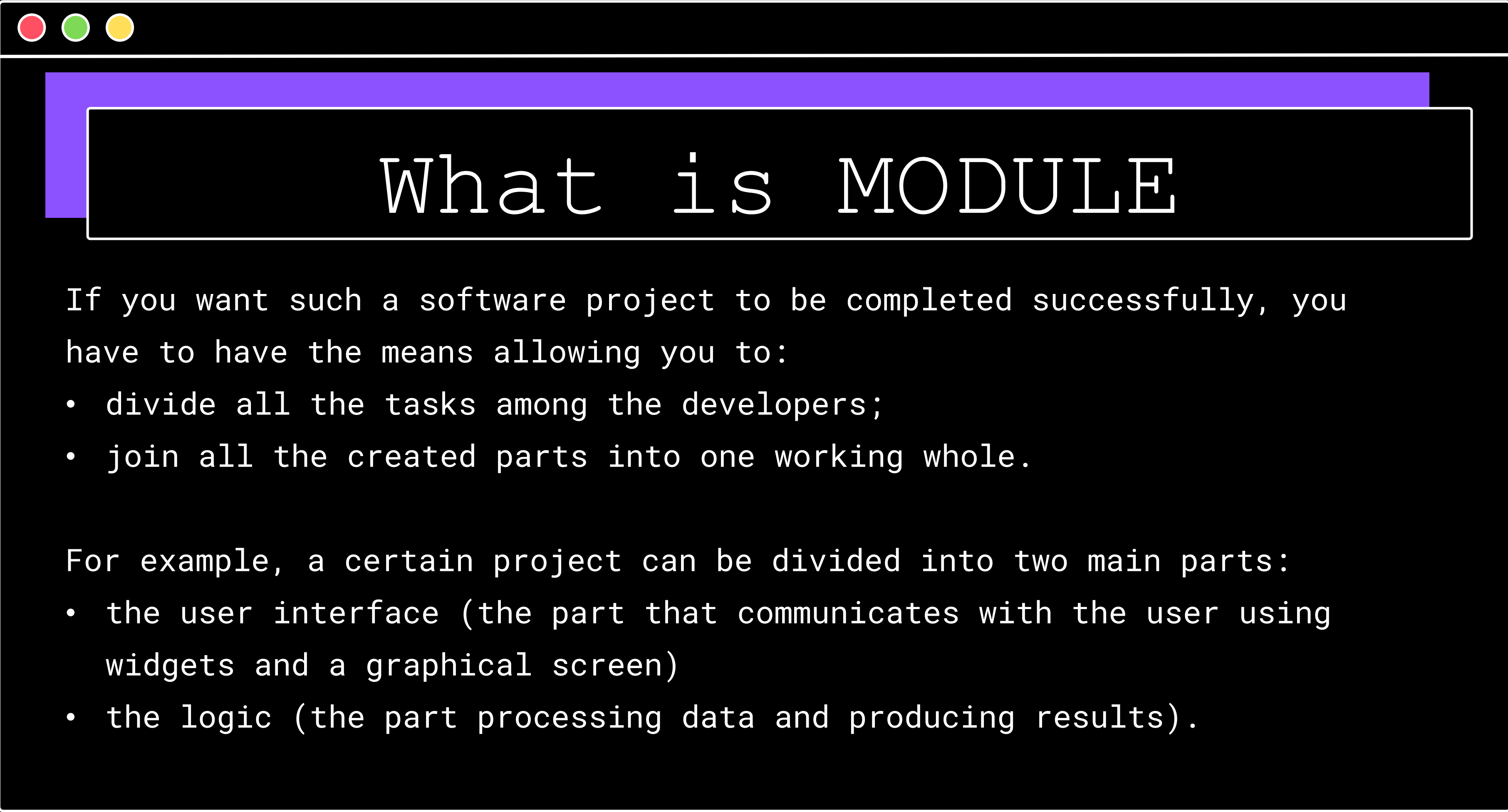
LAB 26

Exception



PYTHON MODULE AND PACKAGE





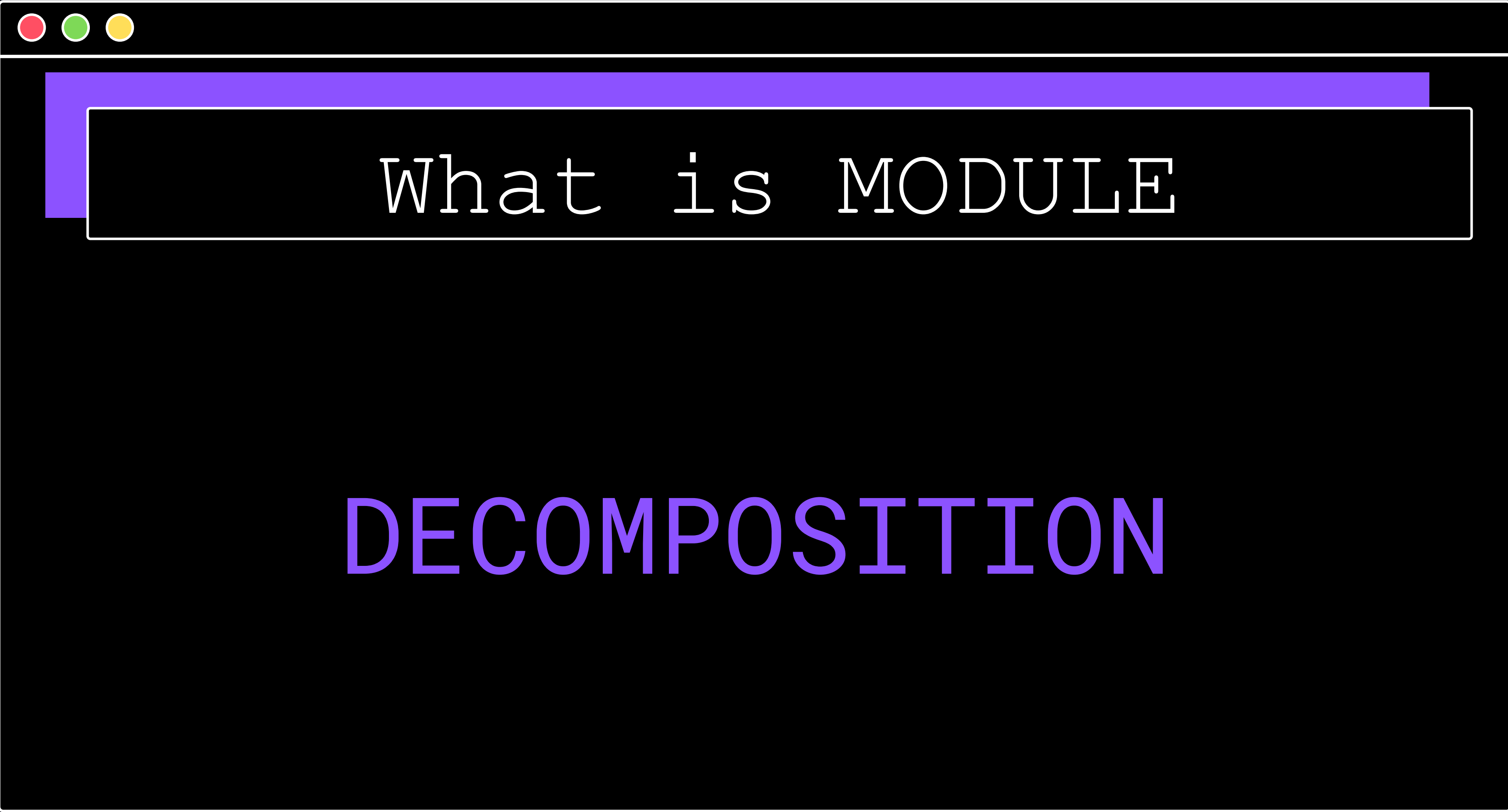
What is MODULE

If you want such a software project to be completed successfully, you have to have the means allowing you to:

- divide all the tasks among the developers;
- join all the created parts into one working whole.

For example, a certain project can be divided into two main parts:

- the user interface (the part that communicates with the user using widgets and a graphical screen)
- the logic (the part processing data and producing results).

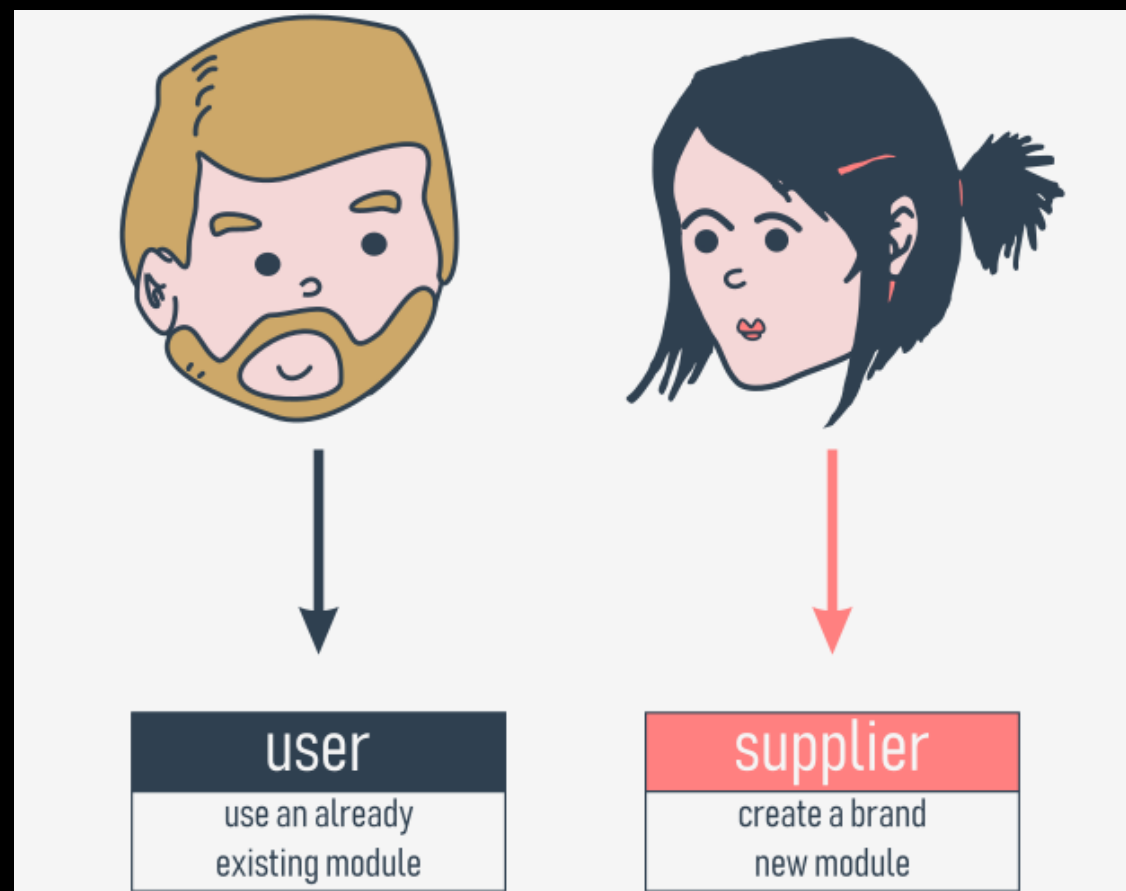
A stylized window with a title bar at the top containing three colored circles (red, green, yellow). Below the title bar is a purple header bar. The main content area of the window has a black background. The text "What is MODULE" is centered in the upper part of the main area, and "DECOMPOSITION" is centered below it.

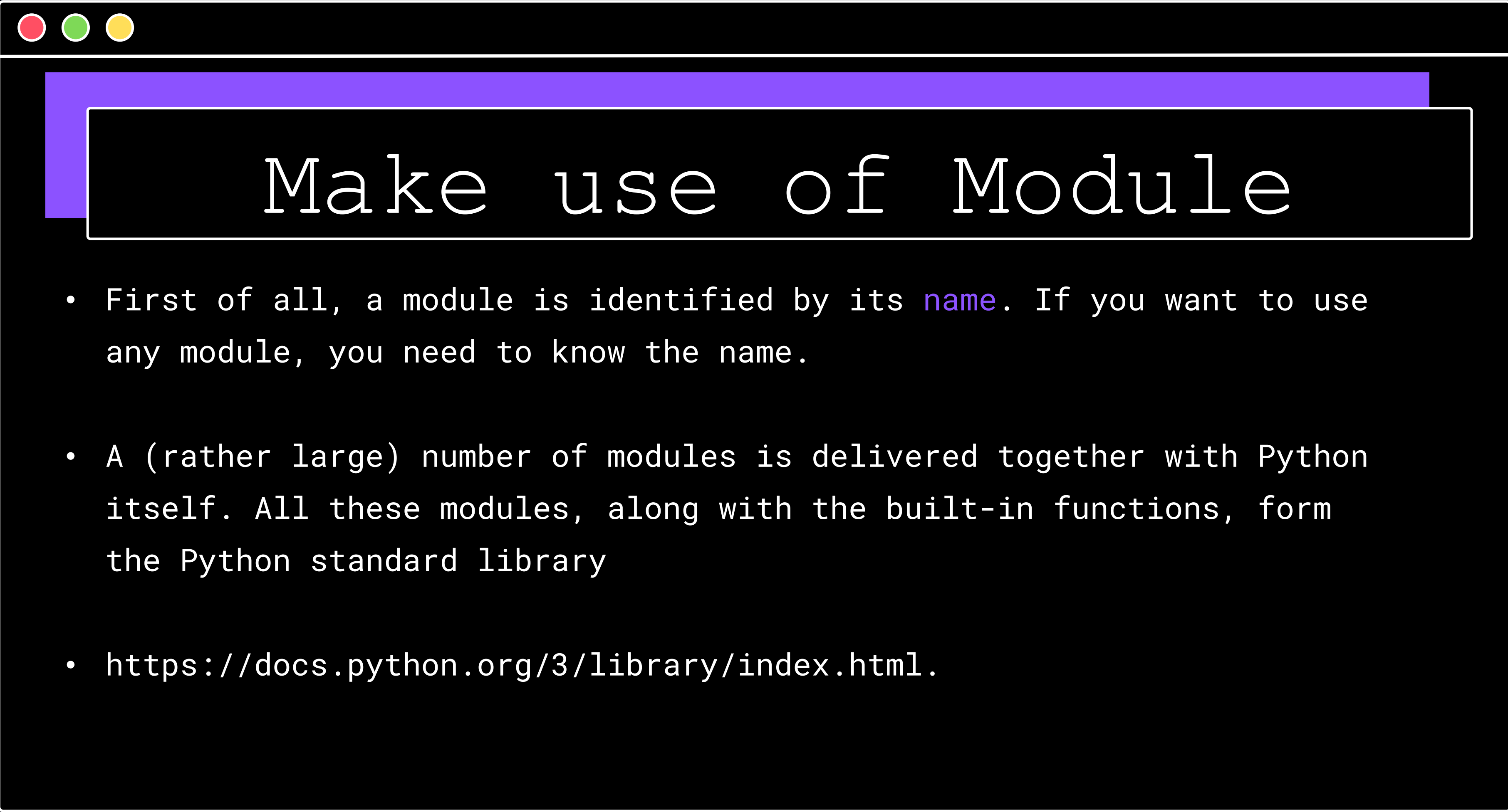
What is MODULE

DECOMPOSITION

Make use of Module

defines it as a `file containing Python definitions and statements`, which can be later imported and used when necessary.



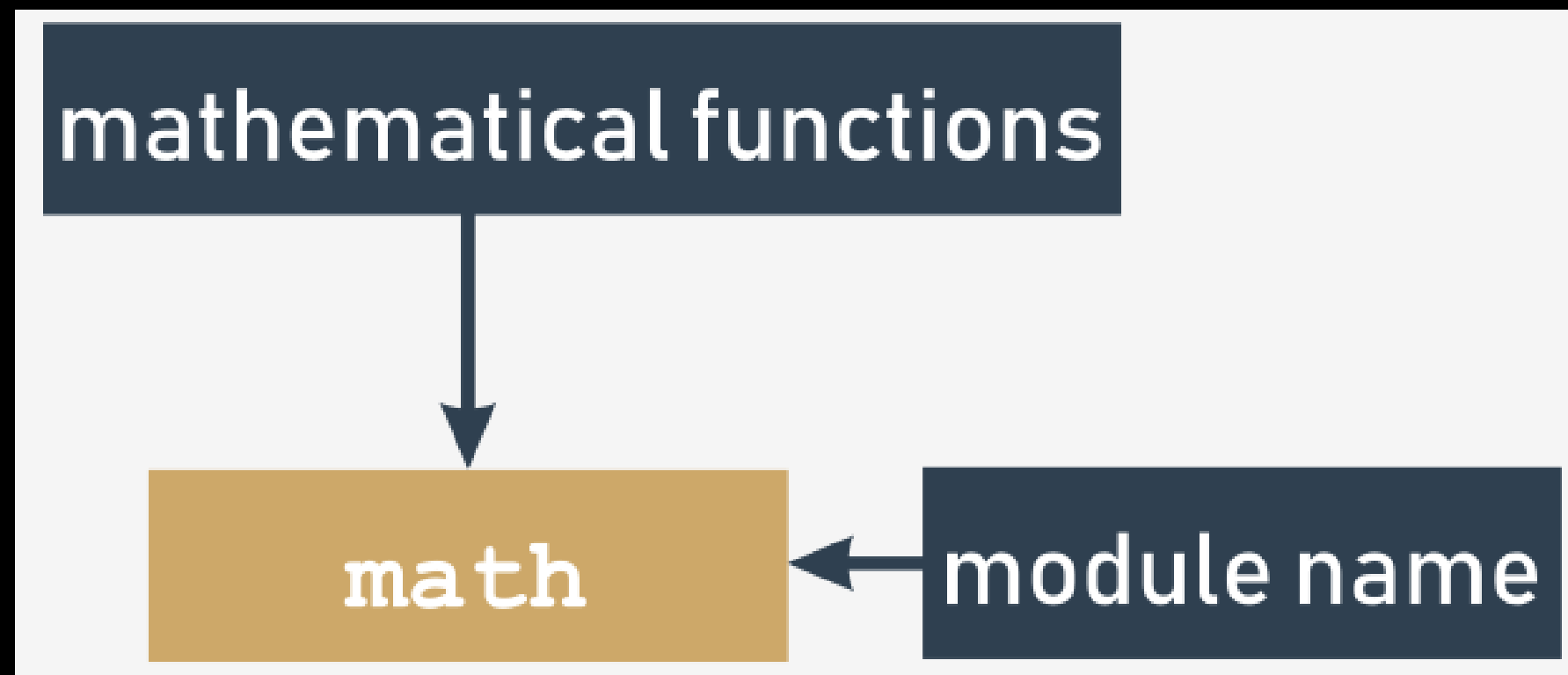


Make use of Module

- First of all, a module is identified by its `name`. If you want to use any module, you need to know the name.
- A (rather large) number of modules is delivered together with Python itself. All these modules, along with the built-in functions, form the Python standard library
- <https://docs.python.org/3/library/index.html>.

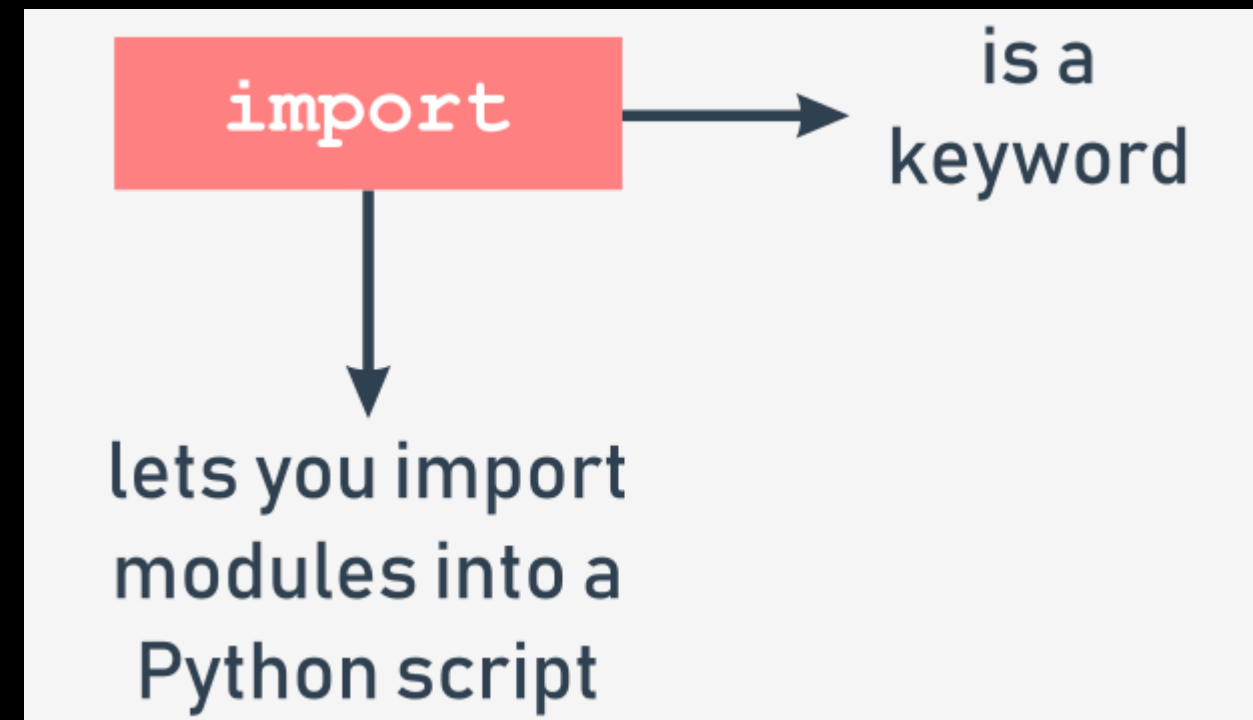
Make use of Module

- Each module **consists of entities** (like a book consists of chapters). These entities can be functions, variables, constants, classes, and objects.



Importing Module

- To make a module usable, you must `import` it (think of it like of taking a book off the shelf). Importing a module is done by an instruction named `import`



Importing Module

- The simplest way to import a particular module is to use the import instruction as follows:
- The instruction may be located anywhere in your code, but it must be placed **before the first use of any of the module's entities**.

```
import math  
import sys
```

```
import math, sys
```

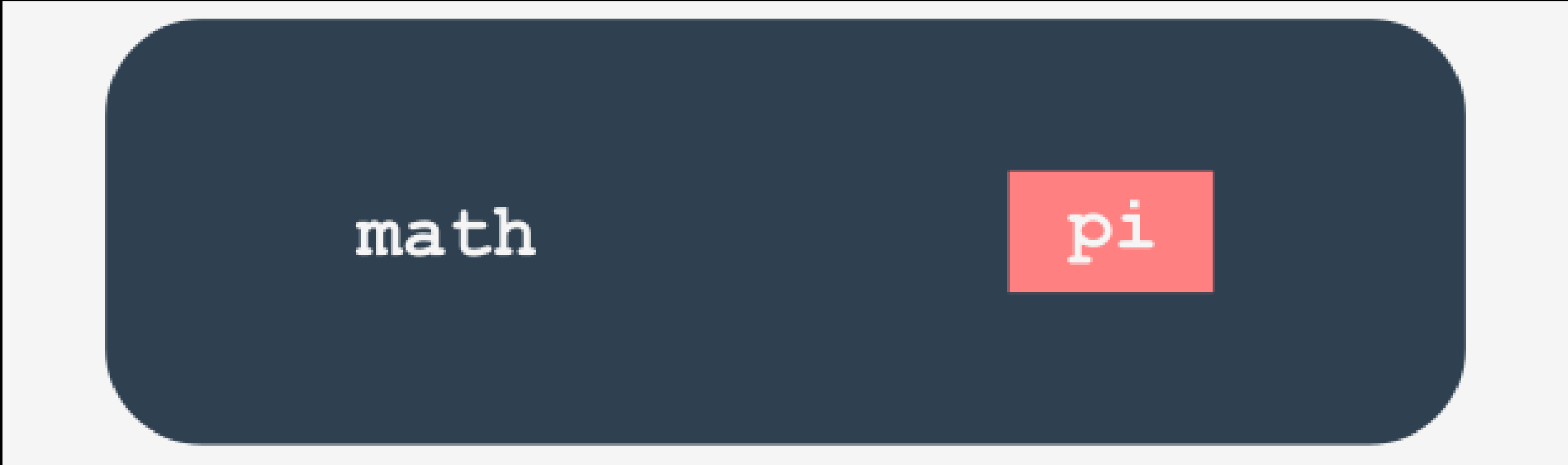


Importing Module

- A `namespace` is a space (understood in a non-physical context) in which some names exist and the names don't conflict with each other
- `Inside a certain namespace, each name must remain unique`
- If the module of a specified name `exists and is accessible` (a module is in fact a Python source file), Python imports its contents, i.e., `all the names defined in the module become known`, but they don't enter your code's namespace.

Importing Module

- This means that you can have your own entities named `sin` or `pi` and they won't be affected by the import in any way.



math

pi



Importing Module

- the name of the module (e.g., math)
- a dot (i.e., .)
- the name of the entity (e.g., pi)

```
import math  
print (math.sin (math.pi/2) )
```

1.0

Importing Module

- how the two namespaces (yours and the module's one) can coexist.

```
import math

def sin(x):
    if 2 * x == pi:
        return 0.999999999
    else:
        return None

pi = 3.14

print(sin(pi/2))
print(math.sin(math.pi/2))
```



Importing Module

- In the second method, the import's syntax precisely points out which module's entity (or entities) are acceptable in the code:

```
from math import pi
```

- the listed entities (and only those ones) are imported from the indicated module;
- the names of the imported entities are accessible without qualification.



Importing Module

- Note: no other entities are imported. Moreover, you cannot import additional entities using a qualification - a line like this one:
- will cause an error (e is Euler's number: 2.71828...)

```
print(math.e)
```

Importing Module

- The output should be the same as previously, as in fact we've used the same entities as before: 1.0.

```
from math import sin, pi  
  
print(sin(pi/2))
```


Importing Module

- In the third method, the import's syntax is a more aggressive form of the previously presented one
- The name of an entity (or the list of entities' names) is replaced with a single asterisk (*).
- Such an instruction imports all entities from the indicated module.

```
from module import *
```

Importing Module

- If you use the import module variant and you don't like a particular module's name you can give it any name you like - this is called **aliasing**.
- Note: after successful execution of an aliased import, the **original module name becomes inaccessible** and must not be used

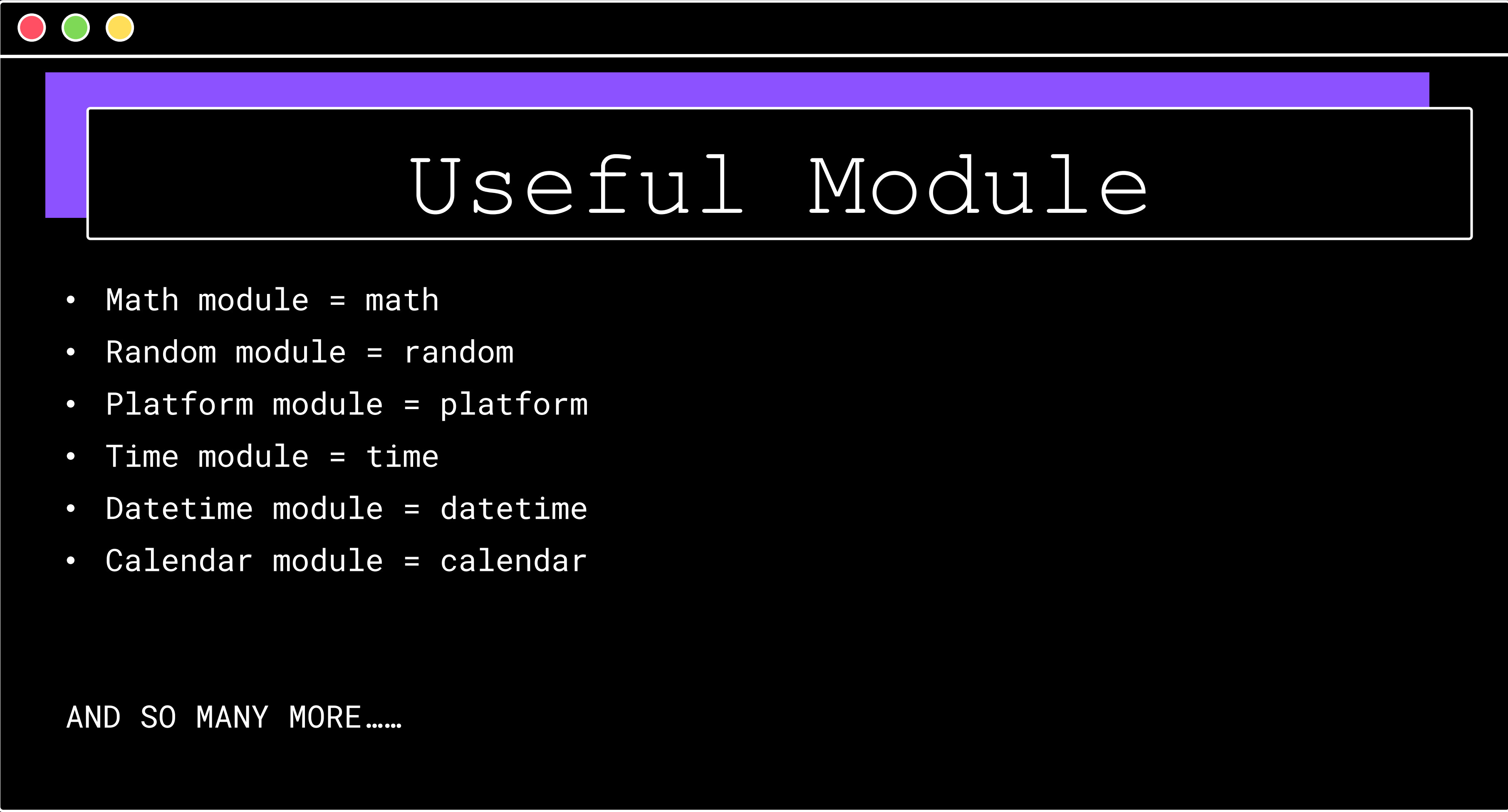
```
import module as alias
```

Useful Module

- Function returning all entities name available in module. If the module has been aliased, it must use the alias name `dir(module)`

```
import math

for name in dir(math):
    print(name, end="\t")
```

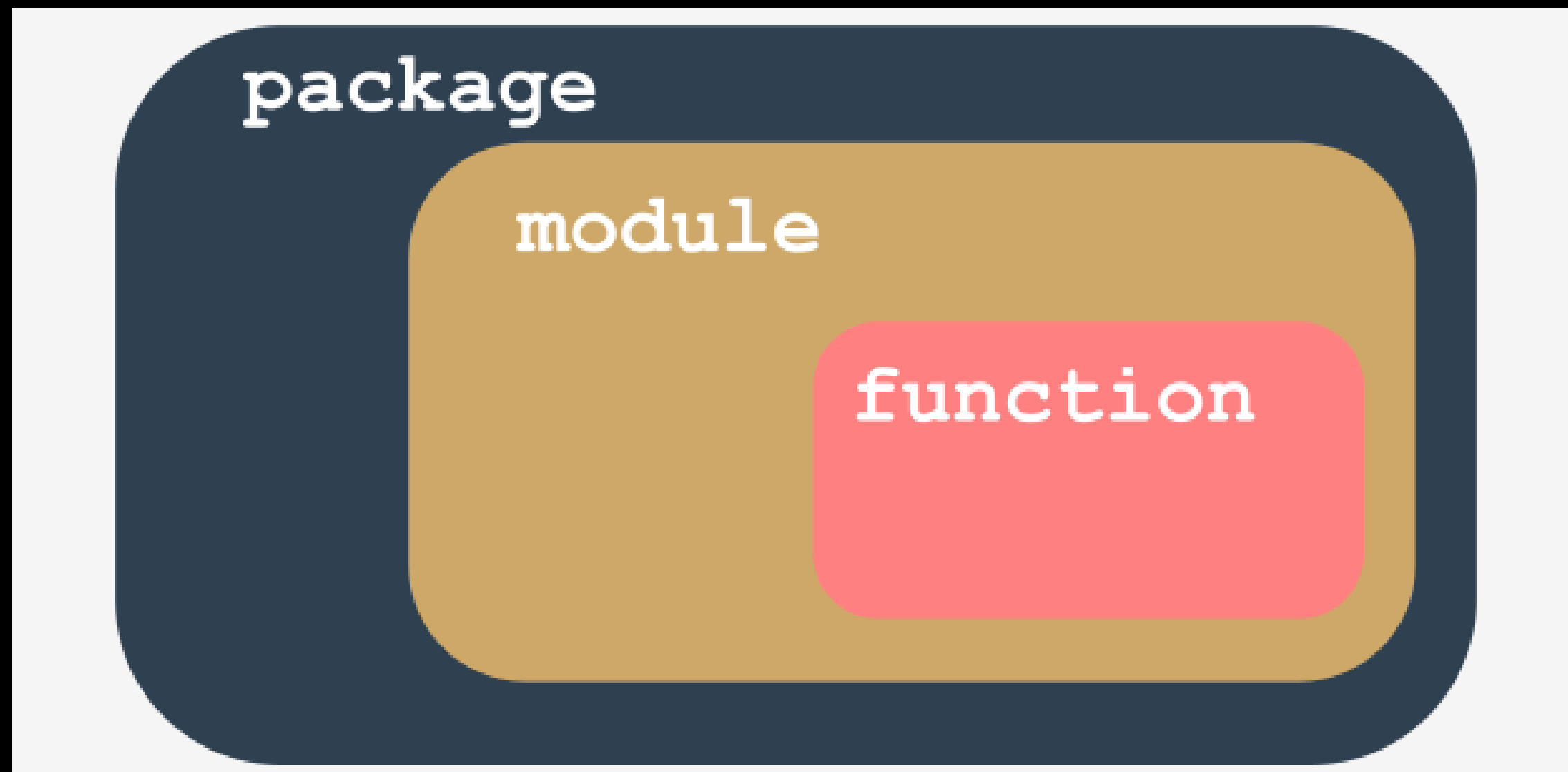


Useful Module

- `Math module = math`
- `Random module = random`
- `Platform module = platform`
- `Time module = time`
- `Datetime module = datetime`
- `Calendar module = calendar`

AND SO MANY MORE.....

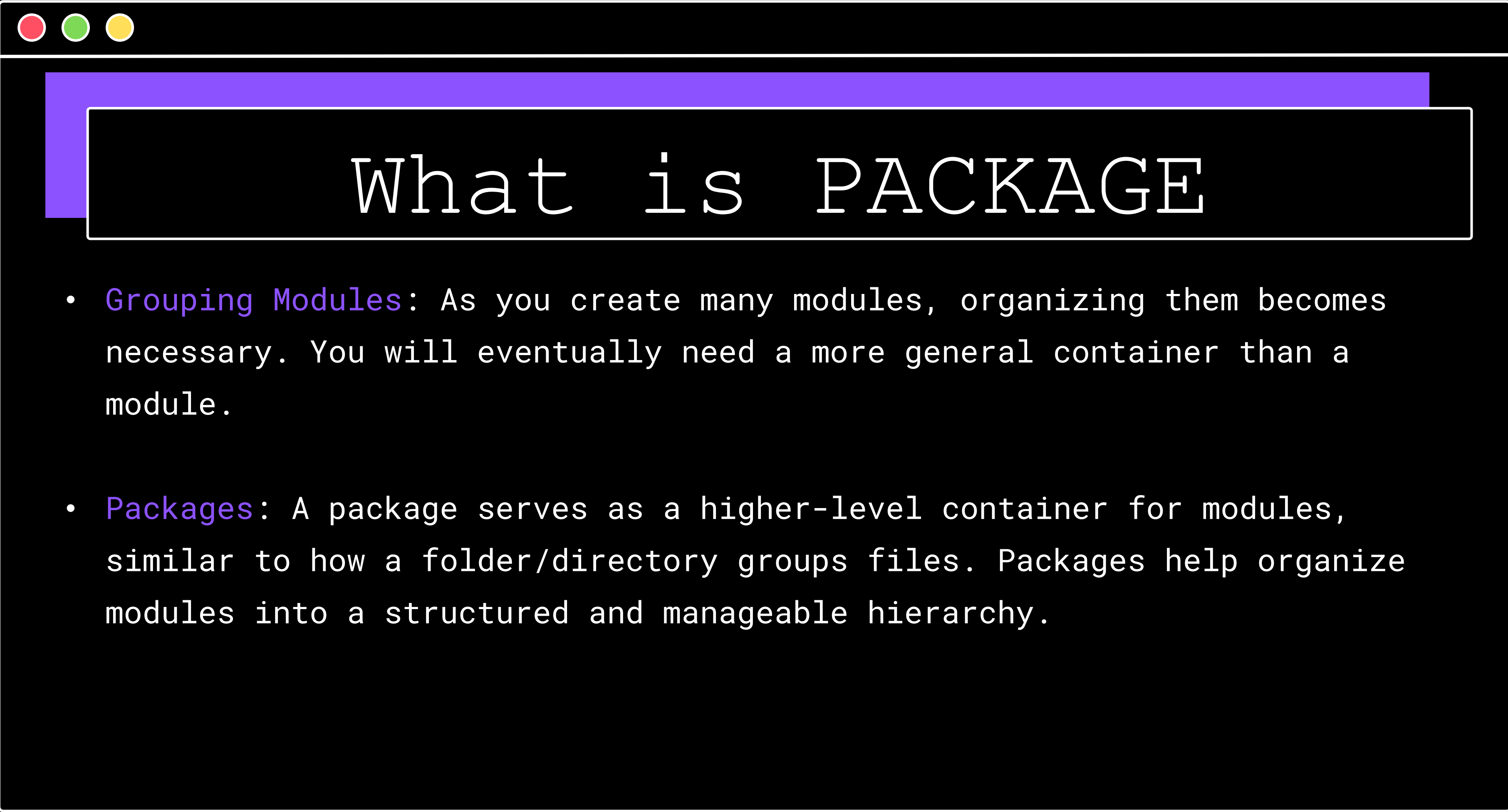
What is PACKAGE





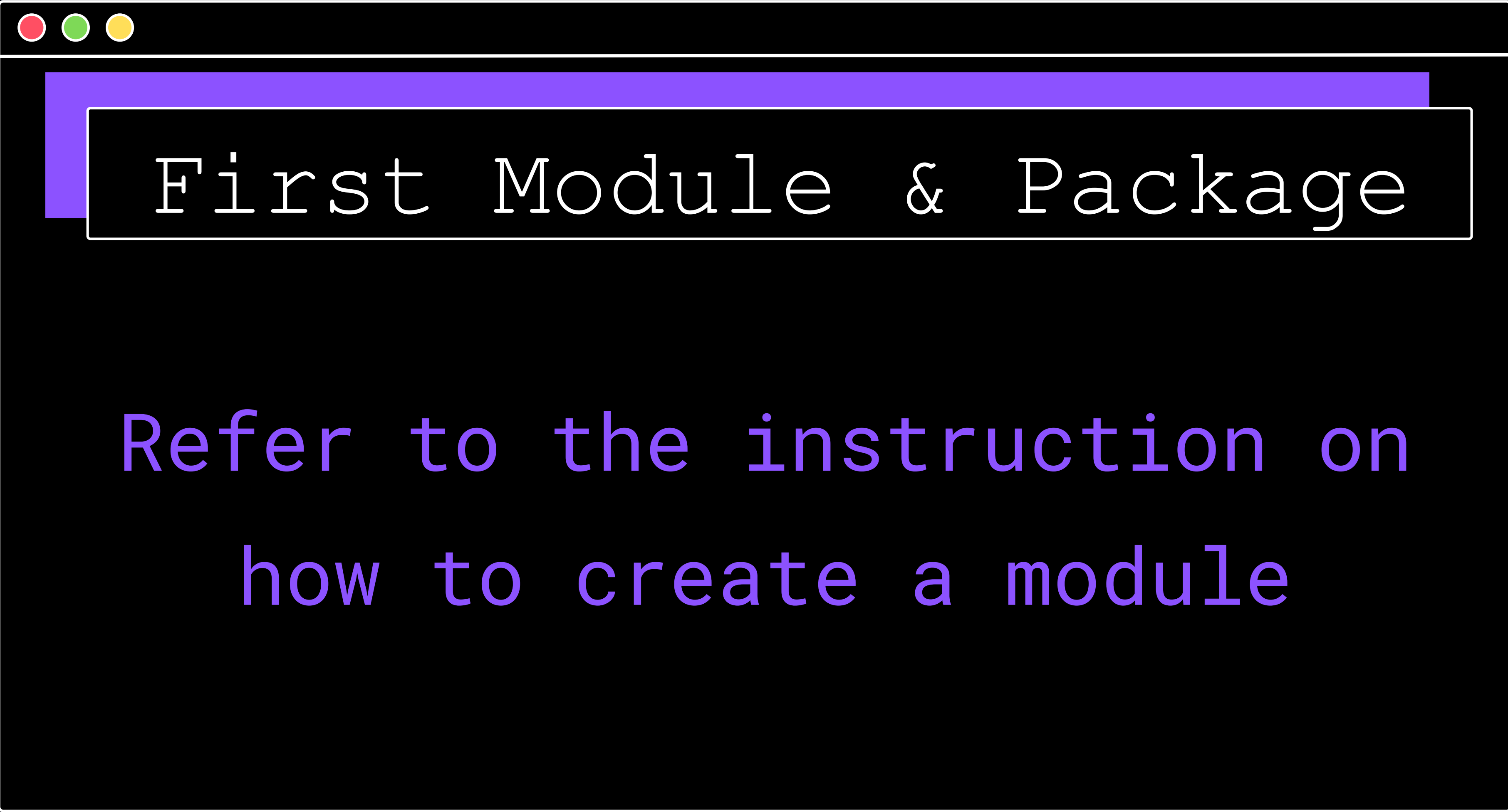
What is PACKAGE

- Modules as Containers: A `module` is a container filled with functions, allowing you to pack multiple functions into one module and distribute it globally.
- Organizing Functions: It's important to group related functions within the same module, similar to how a library organizes books. For example, don't place hard disk partitioning functions in a module named `arcade_games`.



What is PACKAGE

- **Grouping Modules:** As you create many modules, organizing them becomes necessary. You will eventually need a more general container than a module.
- **Packages:** A package serves as a higher-level container for modules, similar to how a folder/directory groups files. Packages help organize modules into a structured and manageable hierarchy.



First Module & Package

Refer to the instruction on
how to create a module

Python Ecosystem



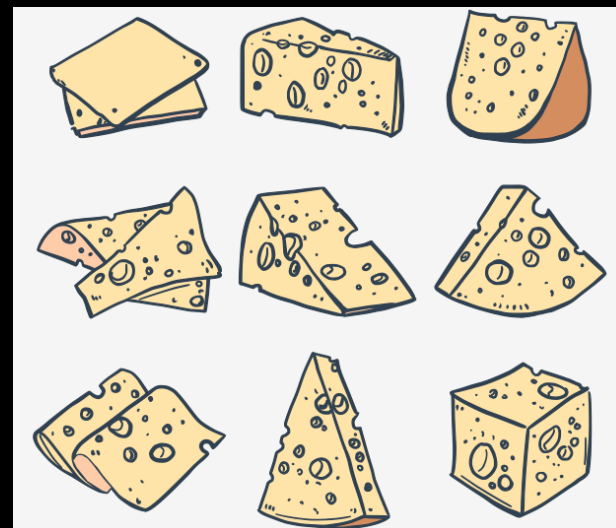
Python Ecosystem

- The repository (or repo for short) we mentioned before is named **PyPI** (it's short for Python Package Index)
- PyPI was home to 237,515 projects, consisting of 2,877,545 files managed by 427,487 users.



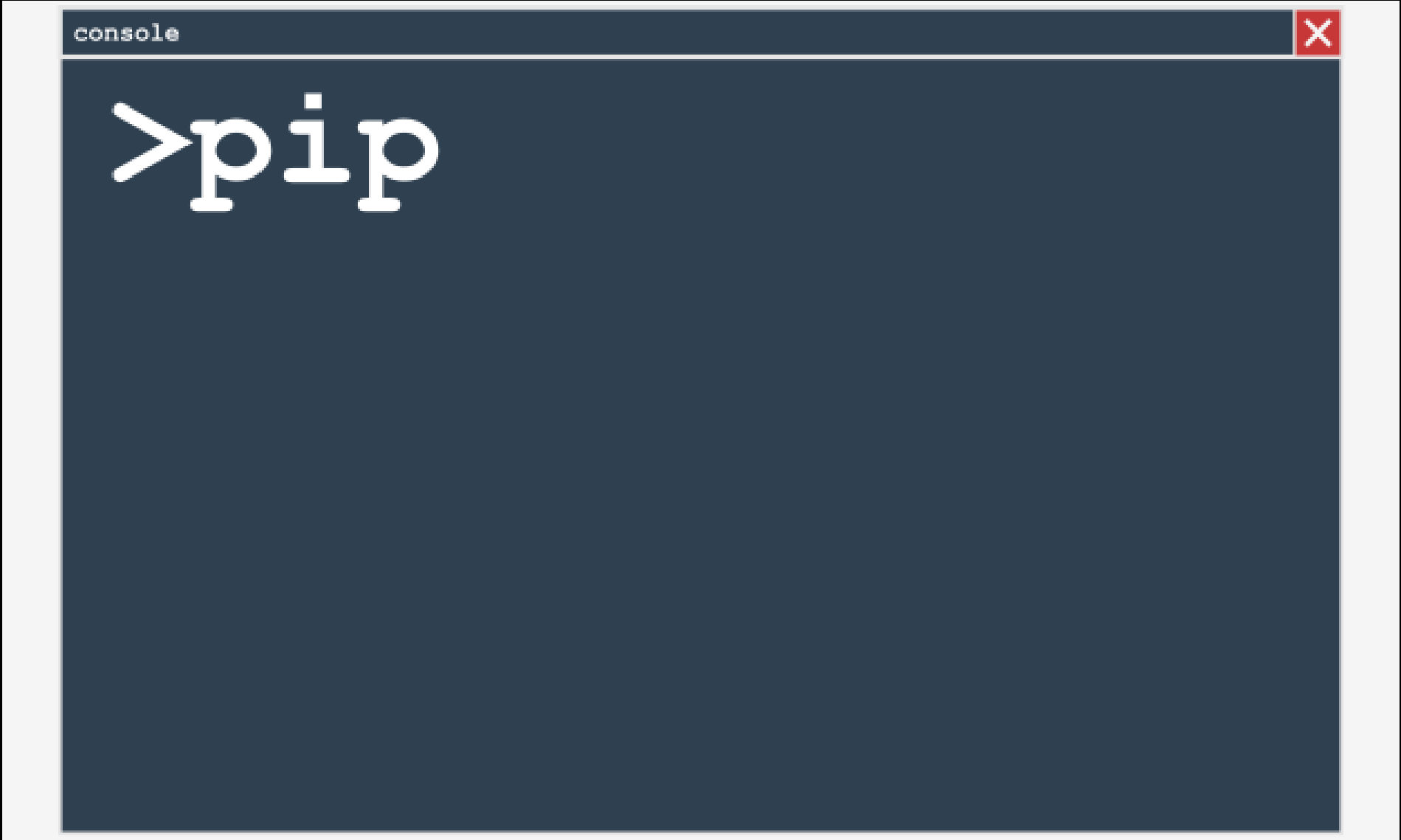
Python Ecosystem

- The PyPI repo is sometimes referred to as the Cheese Shop.
- PyPI is completely free, and you can just pick a code and use it
- PyPI has lots of software in stock and it's available 24/7.
- So if you want to make your own digital cheeseburger by using the goods offered by the PyPI Shop, you'll need a free tool named `pip`.





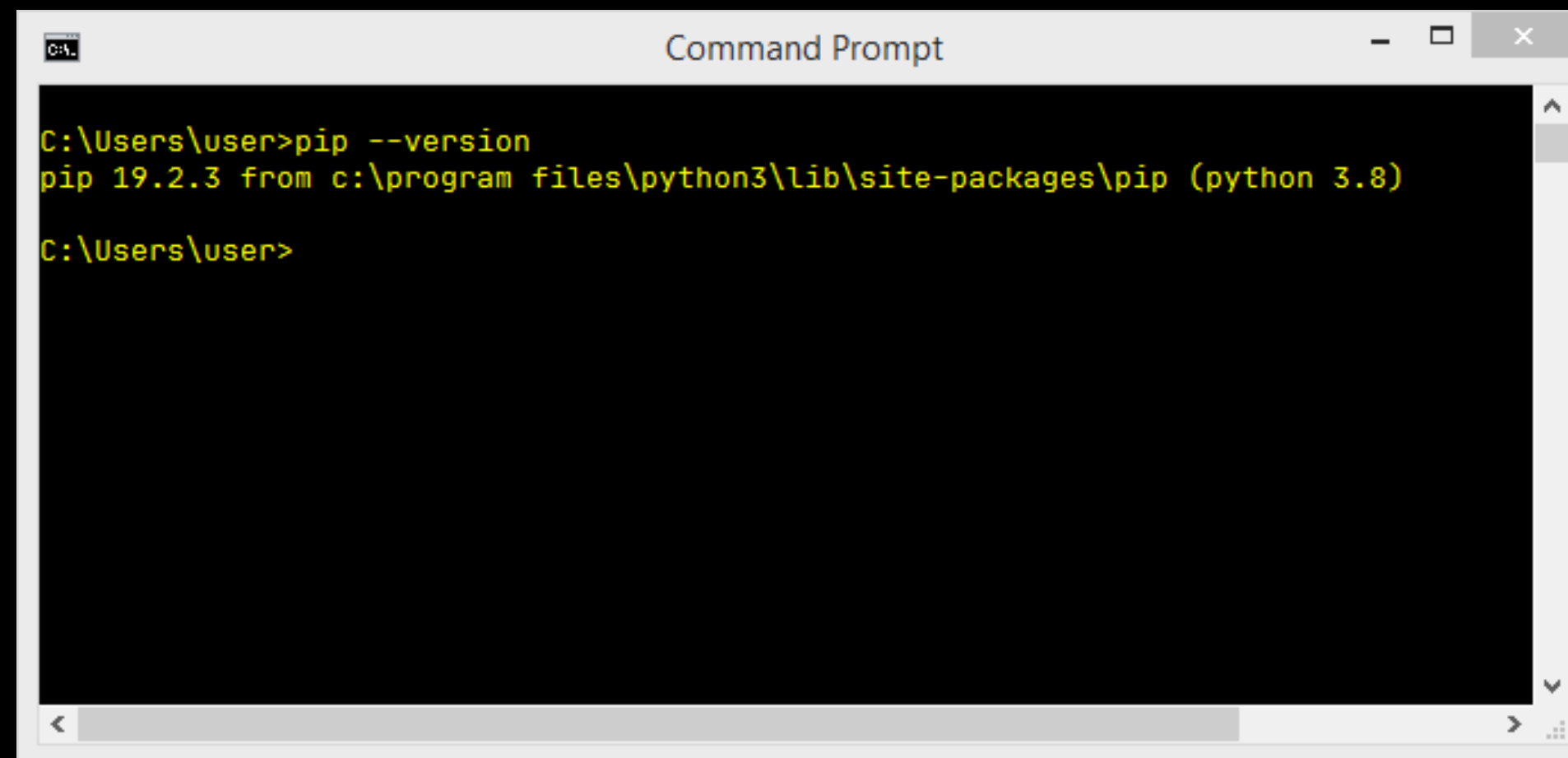
How to install PIP



```
>pip
```

How to install PIP

- On CMD - `pip --version`



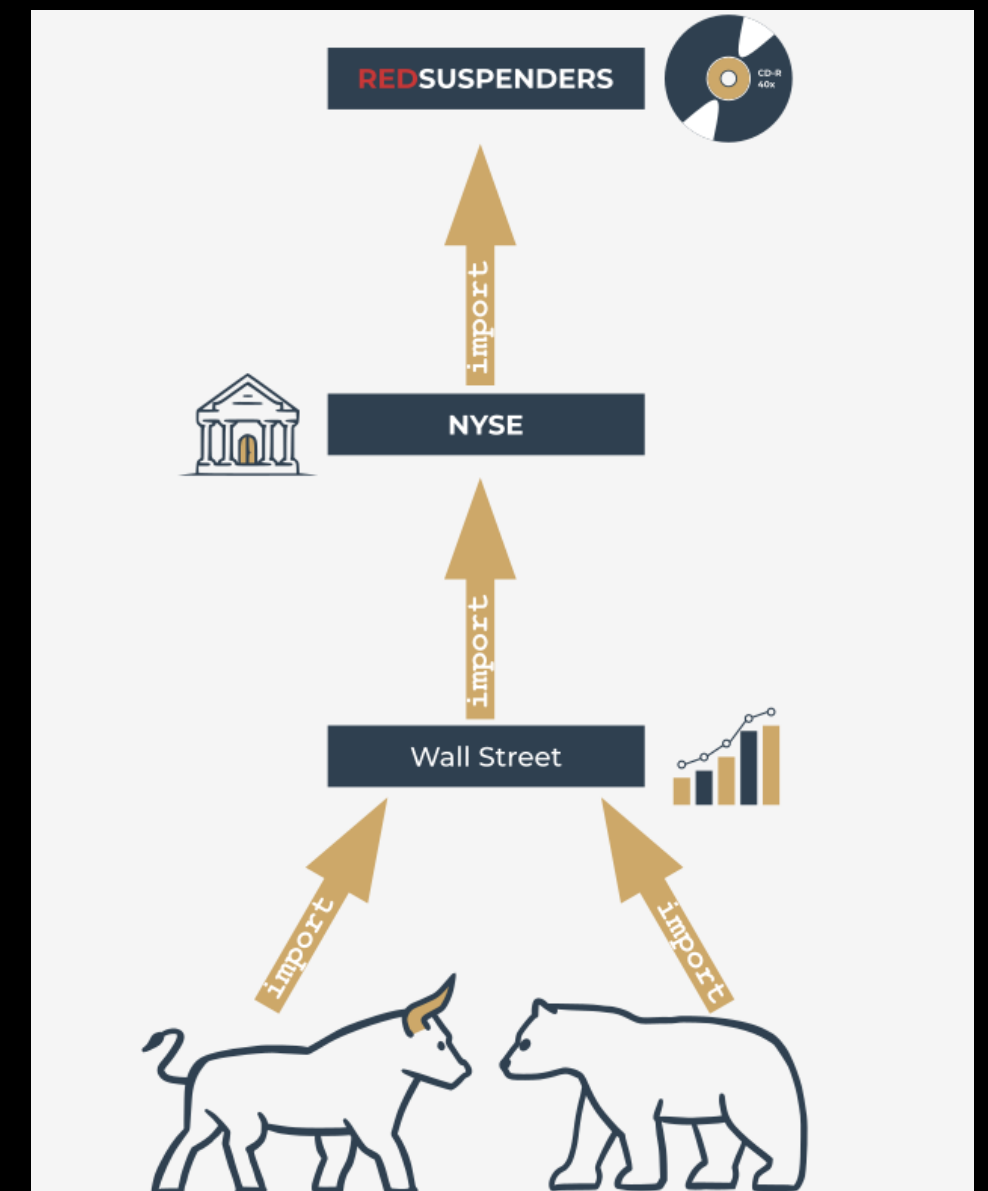
```
Command Prompt

C:\Users\user>pip --version
pip 19.2.3 from c:\program files\python3\lib\site-packages\pip (python 3.8)

C:\Users\user>
```

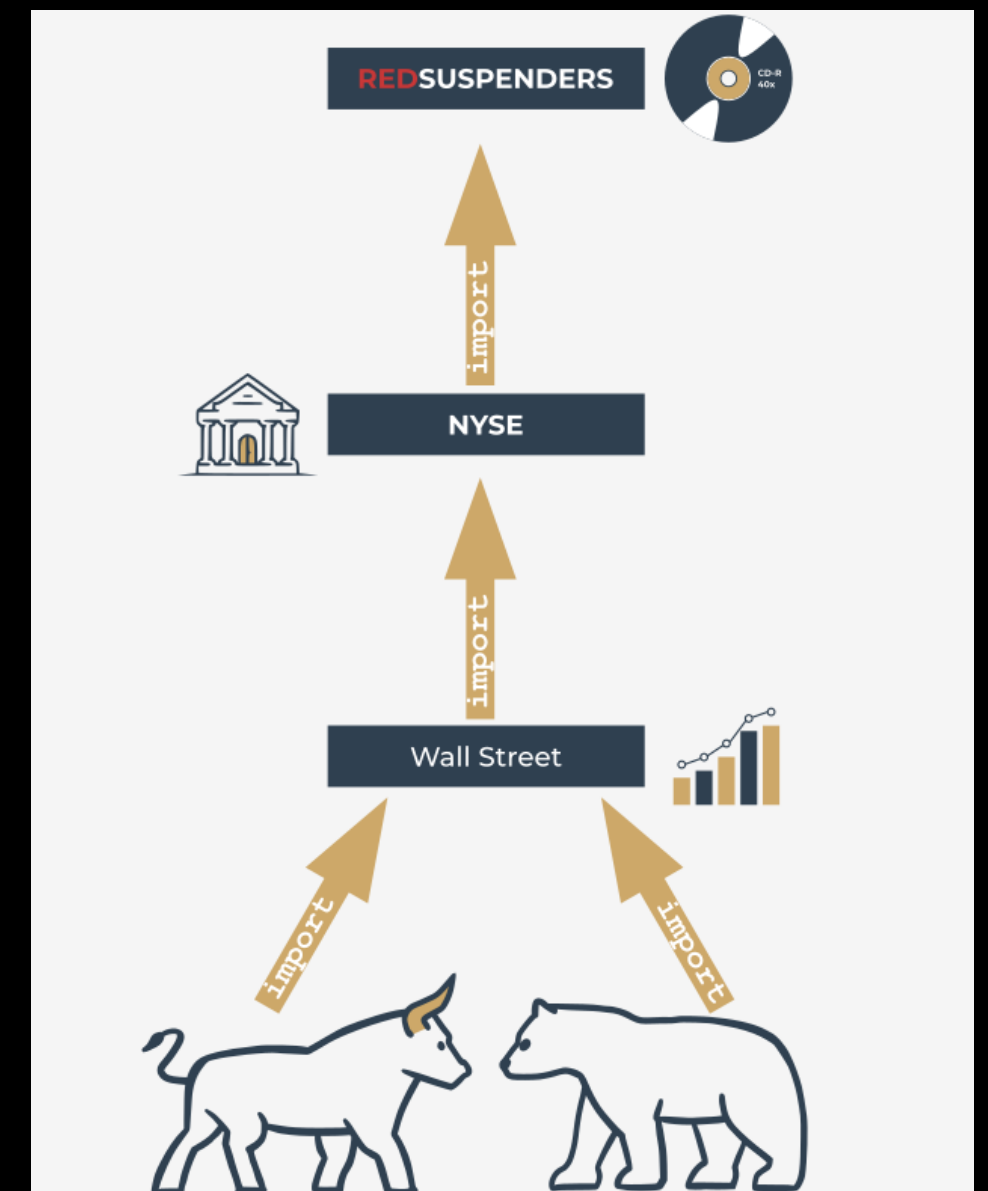
Dependencies

- if you develop an application like "redsuspenders" for stock prediction, which relies on packages like "nyse," "wallstreet," "bull," and "bear," users must ensure all these dependencies are installed. Manually tracking and installing each dependency can lead to "dependency hell."



Dependencies

- However, tools like pip can handle dependency management efficiently by automatically discovering, identifying, and resolving dependencies. This process is done cleverly, avoiding unnecessary downloads and reinstalls, thus simplifying the user experience.

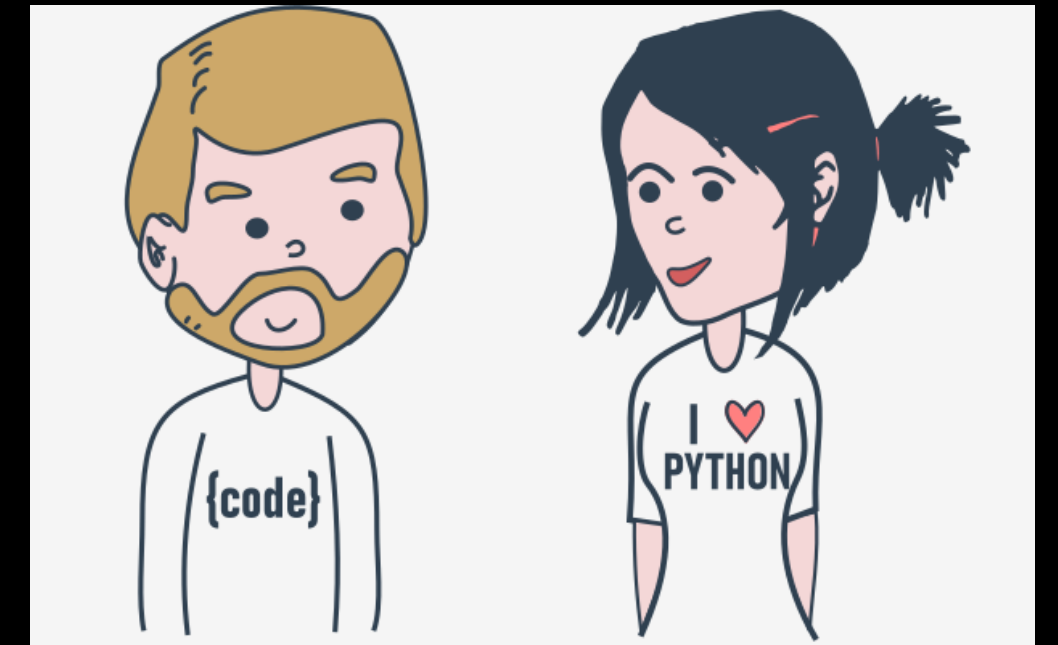


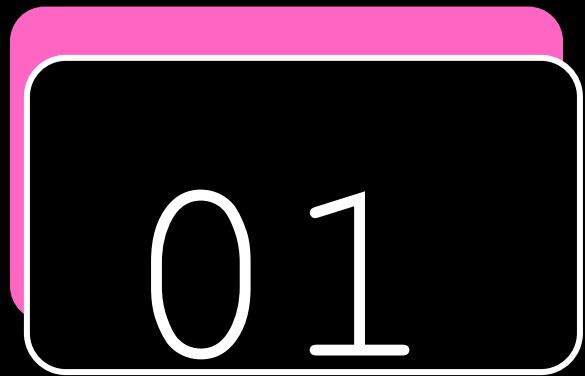
PIP

Installation

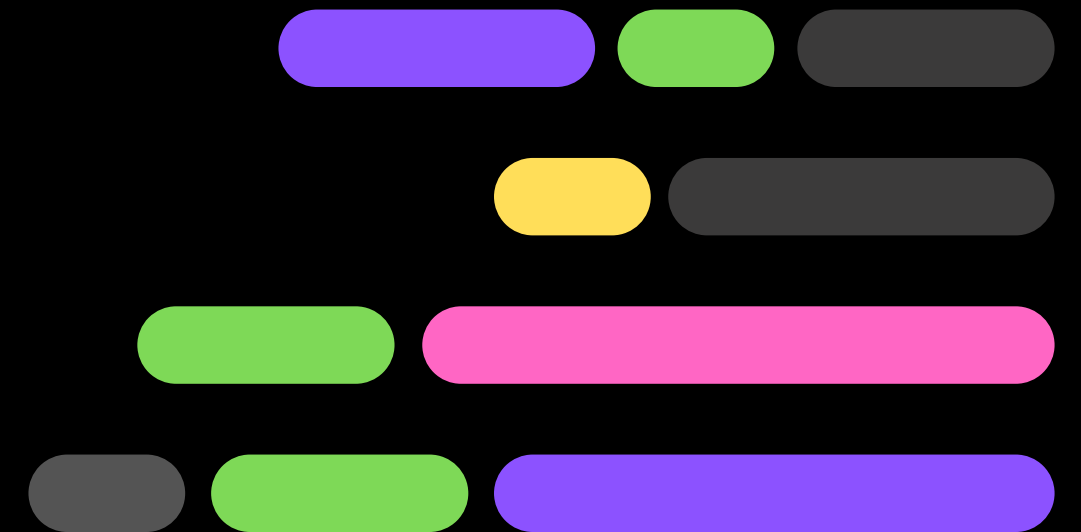
Dependencies

- Pip's capabilities don't end here, but the command set we've presented to you is enough to start successfully managing packages that aren't a part of the regular Python installation.



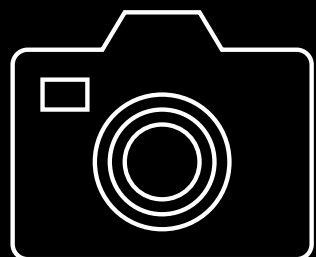
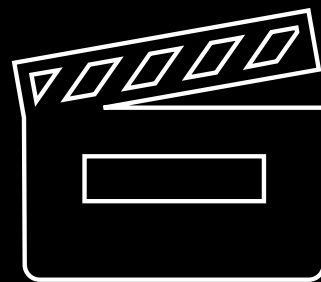
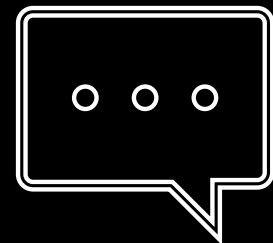
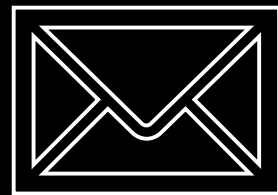


DATA ANALYSIS WORKFLOW

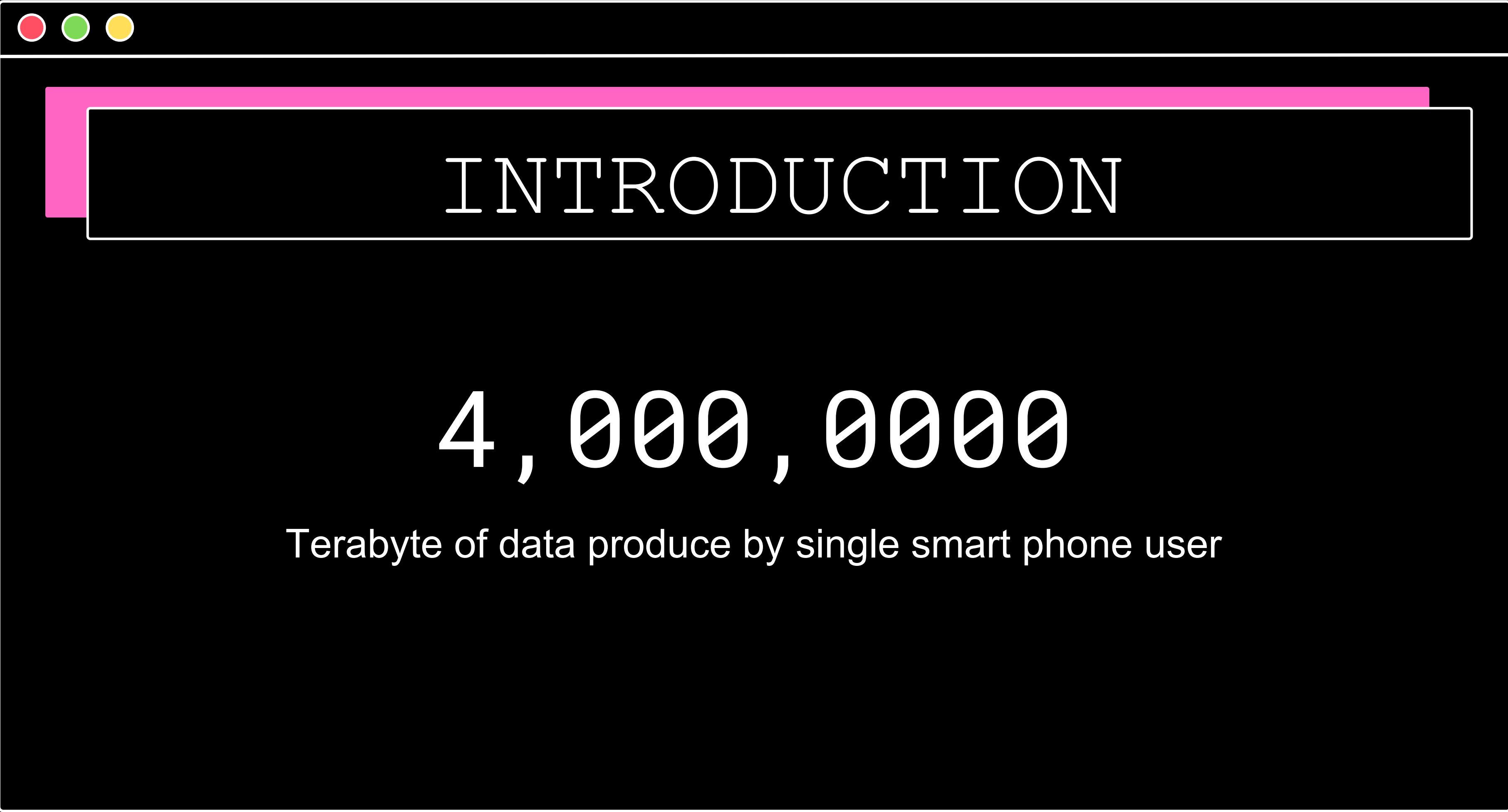




INTRODUCTION



40 Exabyte / Month

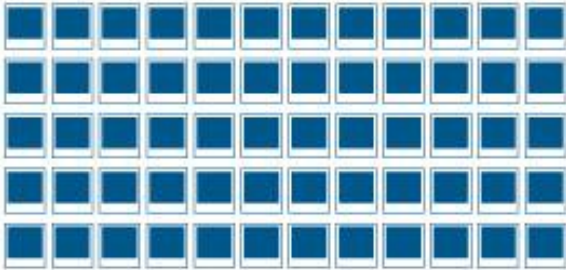


INTRODUCTION

4,000,000

Terabyte of data produce by single smart phone user

INTRODUCTION

Global Data Generated Every Day	Storage Capacity on a Single Drive	Digital Data Stored Worldwide
 294 billion emails	 1956: 2 digital photos 	2018: ~4.4 zettabytes 
 230 million tweets	2020: 12 million digital photos 	2020: ~44 zettabytes 
 1+ billion google searches		



INTRODUCTION

How big is a **Yottabyte?**

TERABYTE

Will fit 200,000 photos or mp3 songs on a single 1 terabyte hard drive.



PETABYTE

Will fit on 16 Backblaze storage pods racked in two datacenter cabinets.



EXABYTE

Will fit in 2,000 cabinets and fill a 4 story datacenter that takes up a city block.



ZETTABYTE

Will fill 1,000 datacenters or about 20% of Manhattan, New York.



YOTTABYTE

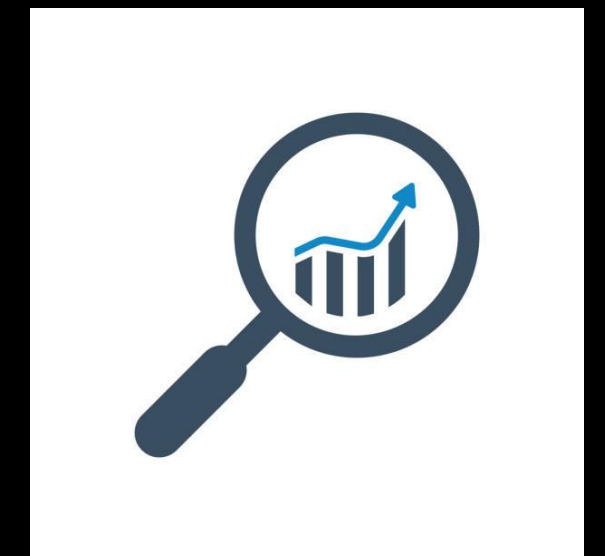
Will fill the states of Delaware and Rhode Island with a million datacenters.





INTRODUCTION

Data analysis is a broad term that covers a wide range of techniques that enable you to reveal any insights and relationships that may exist within raw data. As you might expect, Python lends itself readily to data analysis.



PROCESS

THE DATA ANALYSIS PROCESS

Step 1:

Define the question

Step 2:

Collect the data

Step 3:

Clean the data

Step 4:

Analyze the data

Step 5:

Visualize and share
your findings

PROCESS

STEP 1: Defining The Questions

- Defining your objective means coming up with a hypothesis and figuring how to test it.



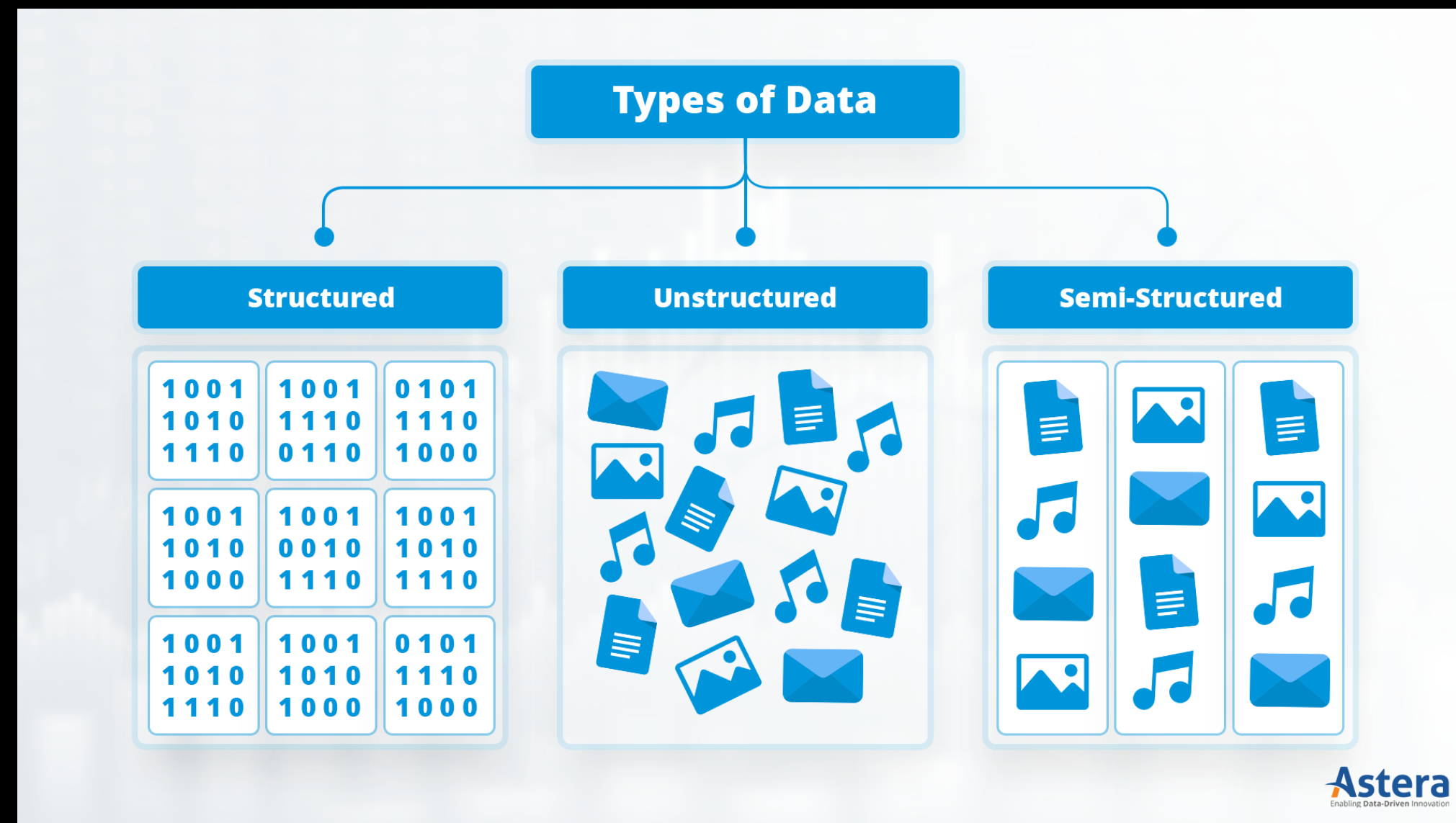


PROCESS

STEP 2: Collecting Data

- Once you've established your objective, you'll need to create a strategy for collecting and aggregating the appropriate data. A key part of this is determining which data you need. This might be quantitative (numeric) data, e.g. sales figures, or qualitative (descriptive) data, such as customer reviews.

PROCESS



PROCESS

Unstructured vs Structured Data



Structured Data

Often numbers or labels, stored in a structured framework of columns and rows relating to pre-set parameters.

ID CODES IN DATABASES

NUMERICAL DATA GOOGLE SHEETS

STAR RATINGS



Semi-unstructured Data

Loosely organized into categories using meta tags

EMAILS BY INBOX, SENT, DRAFT

TWEETS ORGANIZED BY HASHTAGS

FOLDERS ORGANIZED BY TOPIC



Unstructured Data

Text-heavy information that's not organized in a clearly defined framework or model.

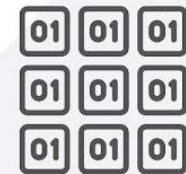
MEDIA POSTS, EMAILS, ONLINE REVIEWS

VIDEOS, IMAGES

SPEECH, SOUNDS

PROCESS

Structured data



Characteristics

Predefined data models
Easy to search
Text-based
Shows what's happening

Resides in

Relational databases
Data warehouses

Stored in

Rows and columns

Examples

Dates, phone numbers, social security numbers, customer names, transaction info

Unstructured data



Characteristics

No predefined data models
Difficult to search
Text, pdf, images, video
Shows the why

Resides in

Applications
Data warehouses and lakes

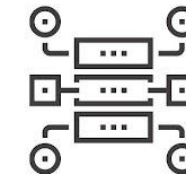
Stored in

Various forms

Examples

Documents, emails and messages, conversation transcripts, image files, open-ended survey answers

Semi-structured data



Characteristics

Loosely organized
Meta-level structure that can contain unstructured data
HTML, XML, JSON

Resides in

Relational databases
Tagged-text format

Stored in

Abstracts & figures

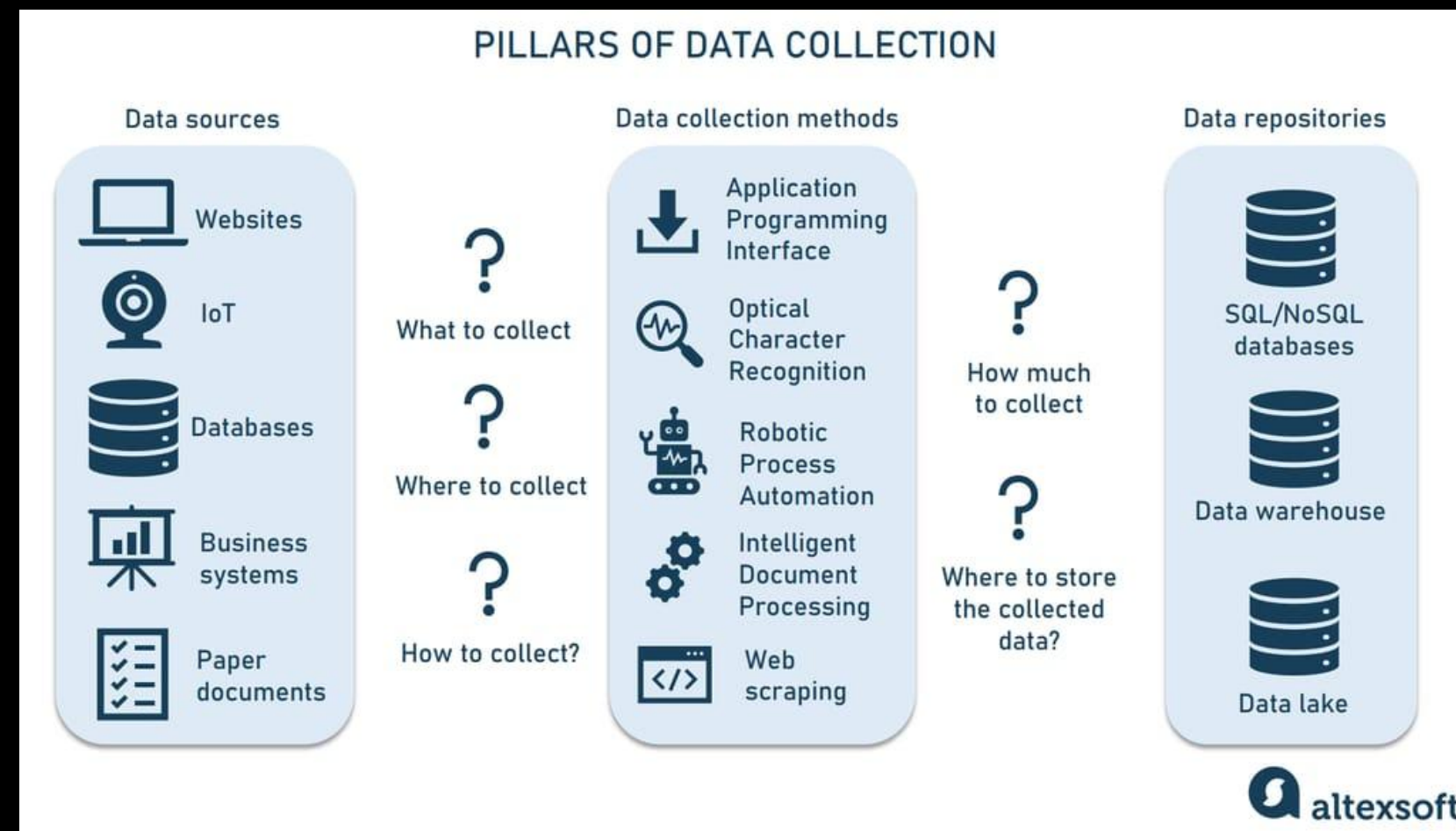
Examples

Server logs, tweets organized by hashtags, emails sorting by folders (inbox; sent; draft)

PROCESS

STEP 2: Collecting Data

The Fundamental Data Collection



Database

Vs

Data Warehouse

Vs

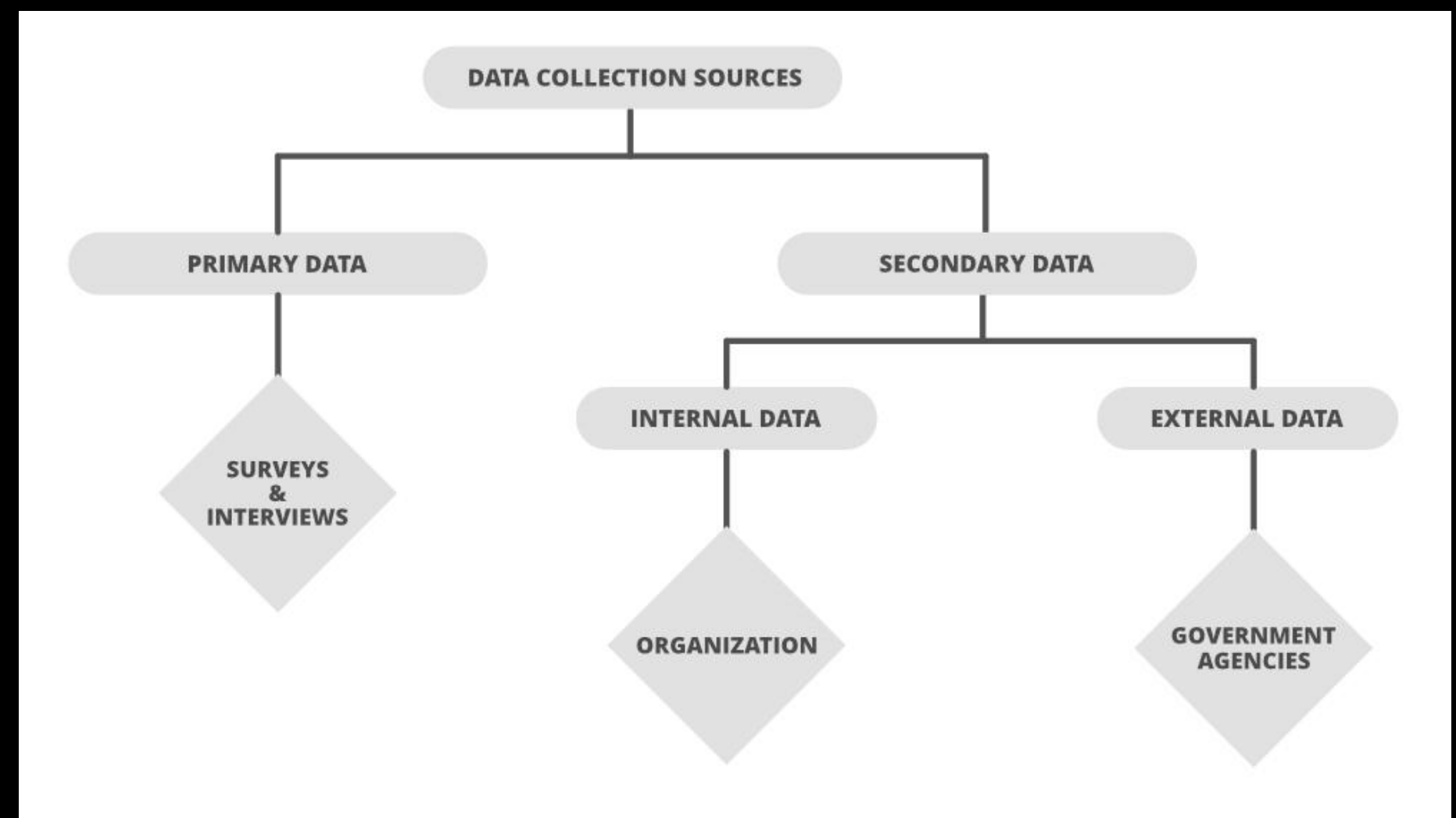
Data Lake



PROCESS

STEP 2: Collecting Data

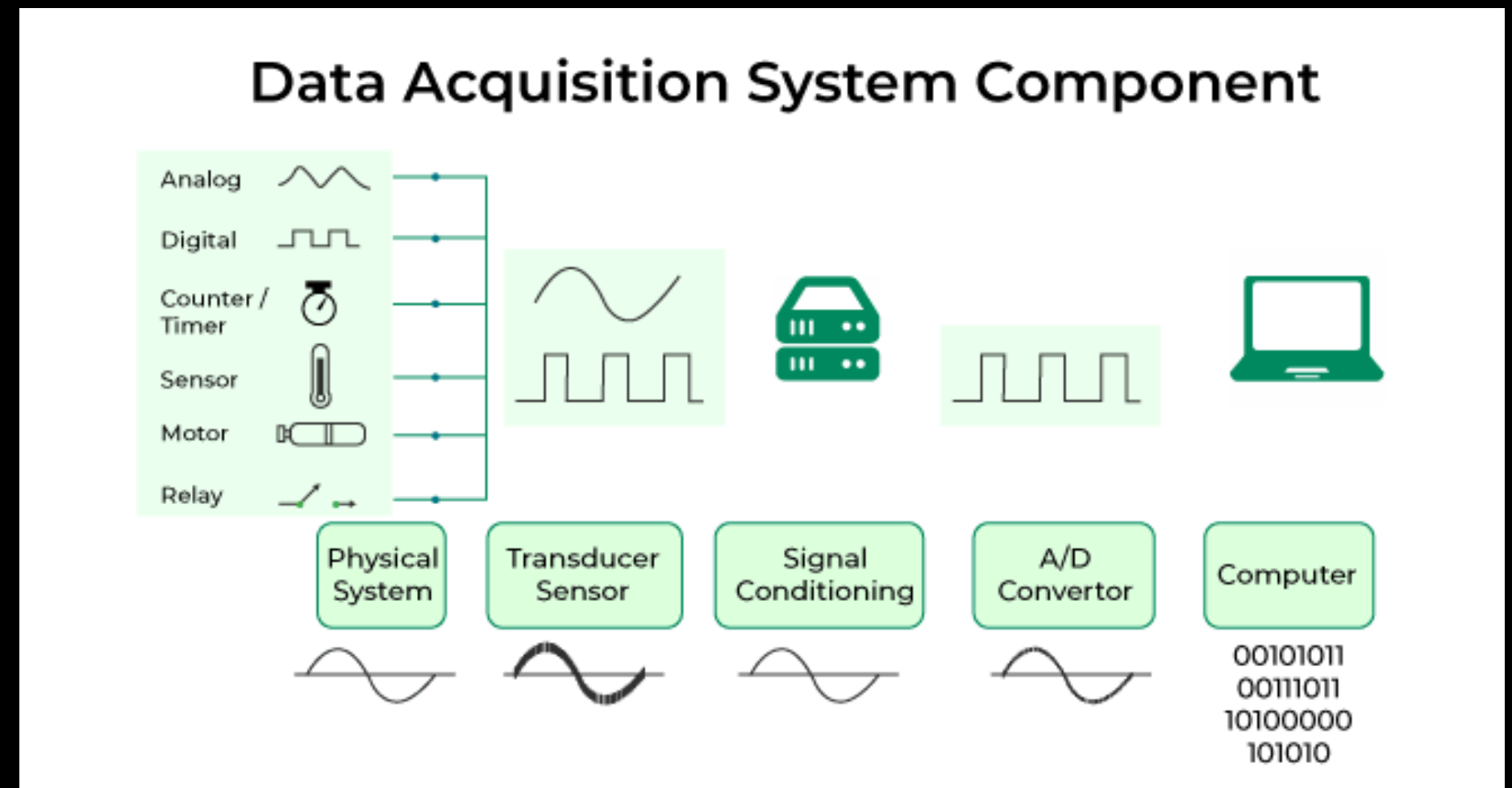
Source of Data



PROCESS

STEP 2: Collecting Data

Data Acquisition Methods



PROCESS

STEP 2: Collecting Data

Data Acquisition Methods

Direct Data Collection	Indirect Data Collection	Technological Data Collection
1. Field Studies 2. Lab Experiments 3. Field Experiments 4. Ethnography	1. Survey and Questionnaires 2. Interviews 3. Documents Analysis 4. Content Analysis	1. Sensor and IoT devices 2. Web Scraping 3. Social Media Monitoring 4. Mobile Apps and Wearable

PROCESS

STEP 3: Clean Data

Once you've collected your data, the next step is to get it ready for analysis. This means **cleaning**, or '**scrubbing**' it, and is crucial in making sure that you're working with high-quality data. Key data cleaning tasks include:

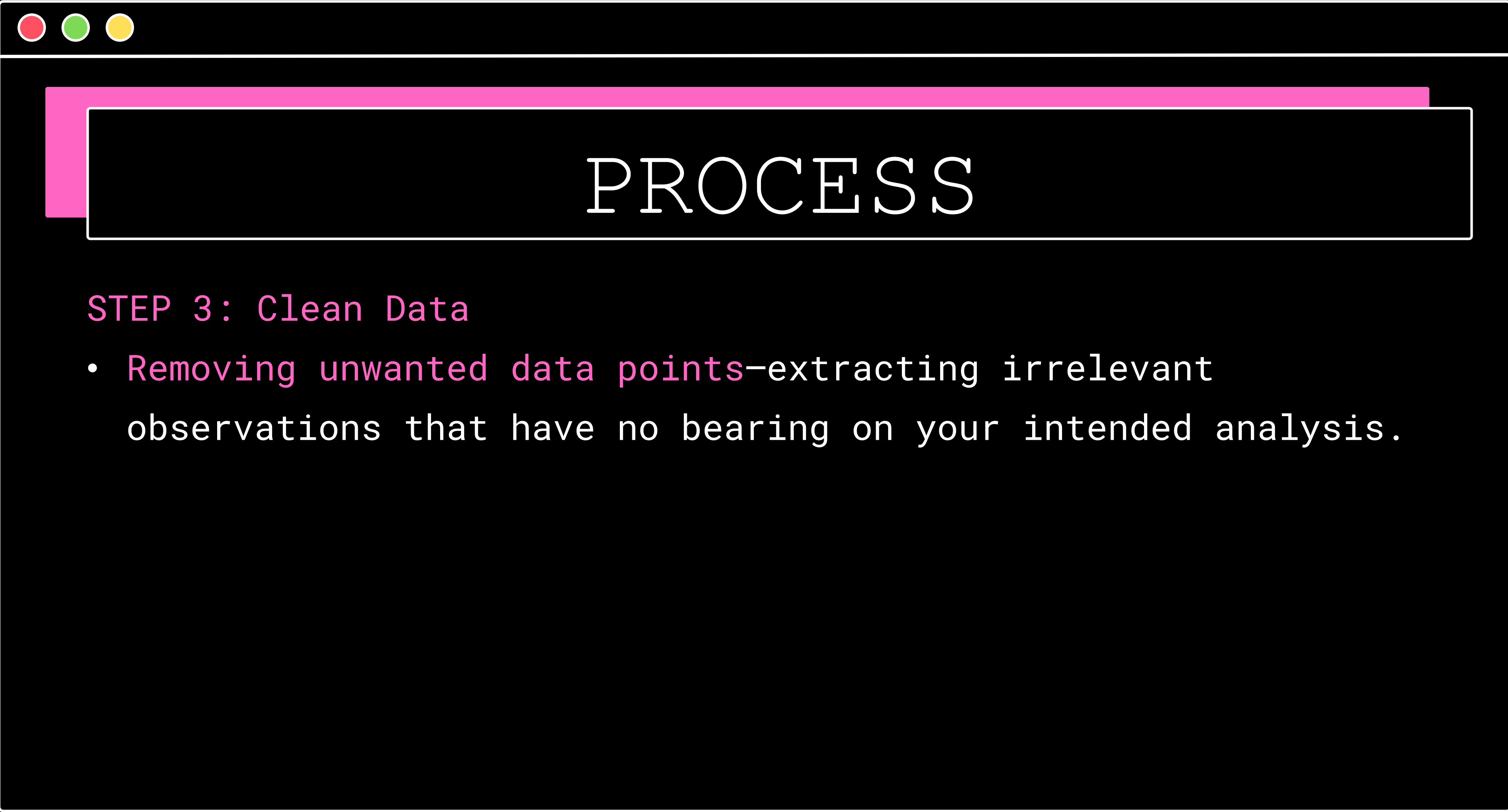




PROCESS

STEP 3: Clean Data

- Removing major errors, duplicates, and outliers—all of which are inevitable problems when aggregating data from numerous sources.



PROCESS

STEP 3: Clean Data

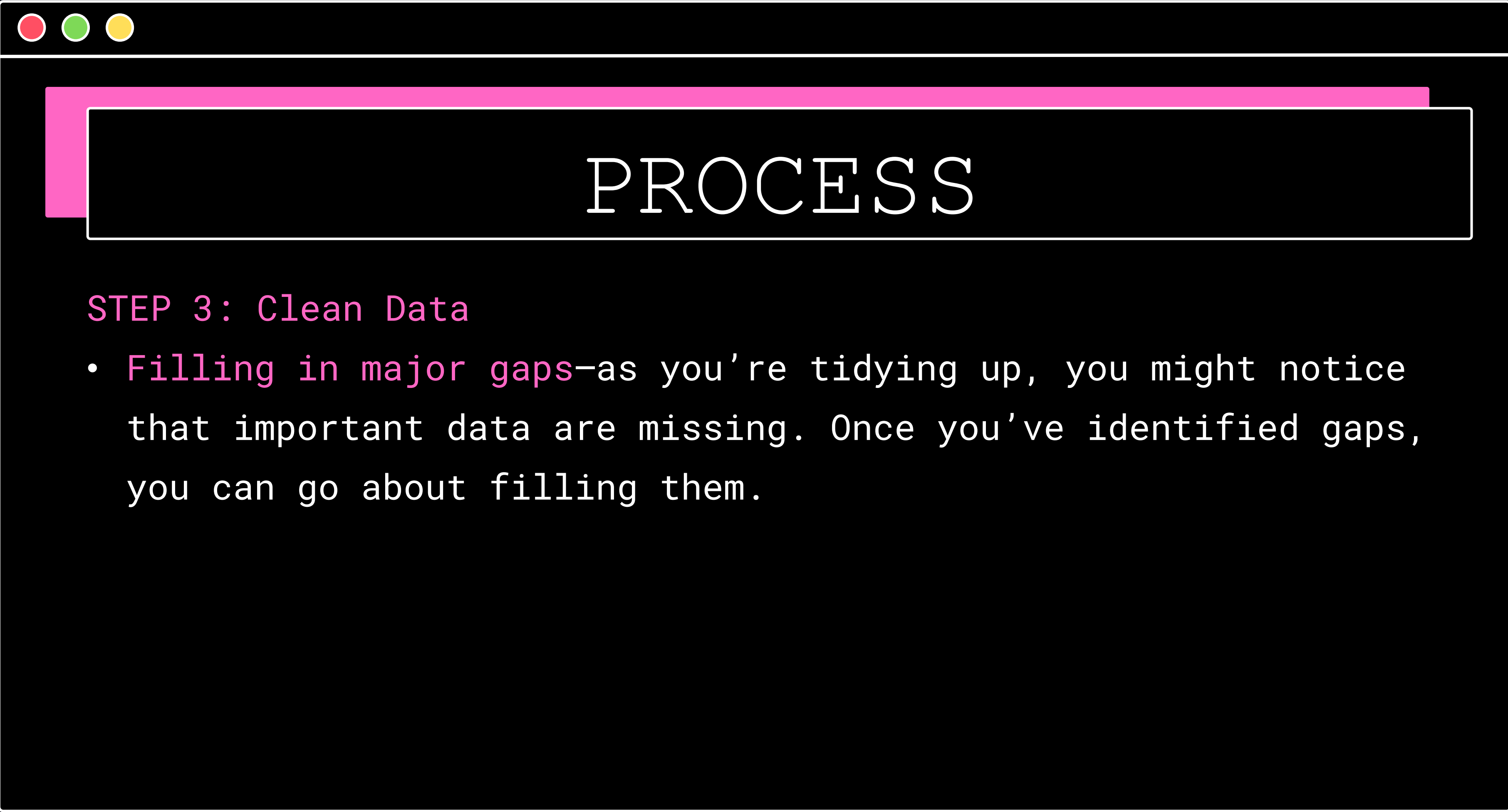
- Removing unwanted data points—extracting irrelevant observations that have no bearing on your intended analysis.



PROCESS

STEP 3: Clean Data

- Bringing structure to your data—general ‘housekeeping’, i.e. fixing typos or layout issues, which will help you map and manipulate your data more easily.



PROCESS

STEP 3: Clean Data

- **Filling in major gaps**—as you're tidying up, you might notice that important data are missing. Once you've identified gaps, you can go about filling them.

AN INTRO TO

DATA CLEANING



CF





PROCESS

STEP 3: Clean Data

- Another thing many data analysts do (alongside cleaning data) is to carry out an **exploratory analysis**. This helps identify initial trends and characteristics, and can even refine your hypothesis.



PROCESS

STEP 3: Clean Data

- There is a lot of tools available in python for cleaning the data. There are library that being used a lot which is Pandas



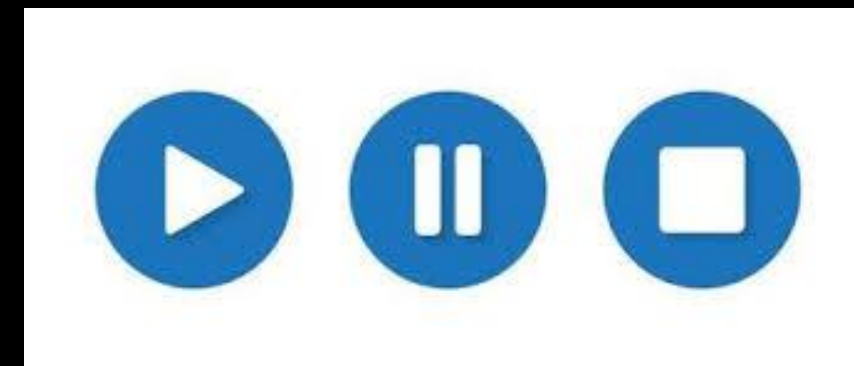
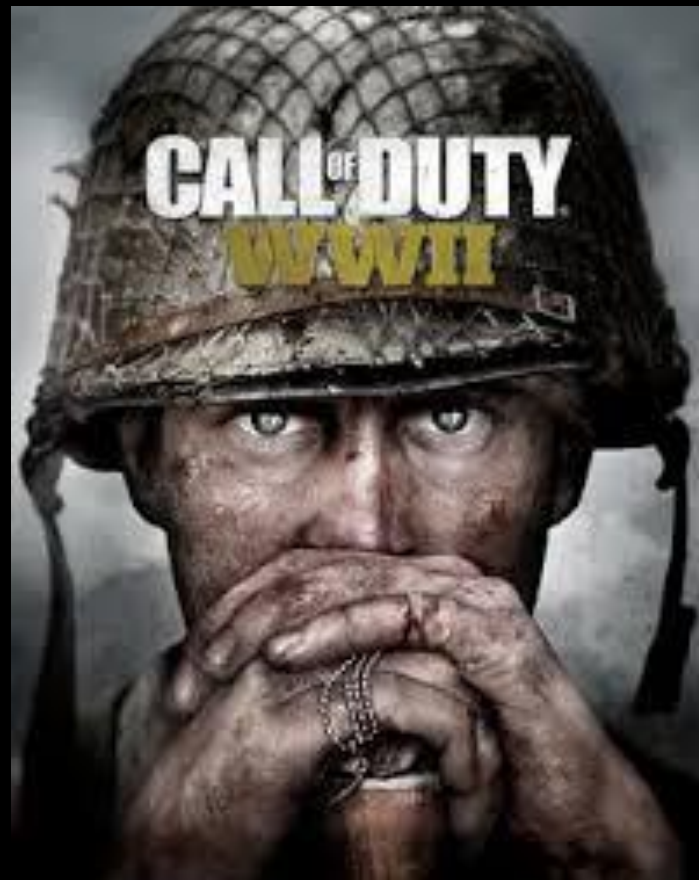
PROCESS

STEP 4: Analyzing the Data

- Encompass a wide range of methodologies and tools used to extract actionable insights from large and complex datasets. where organizations have access to vast amounts of information, effective data analysis is essential for making informed decisions, identifying trends, and discovering valuable patterns within the data.

PROCESS

STEP 4: Analyzing the Data



PROCESS

STEP 4: Analyzing the Data

Predict the hurricane five days in advance with the usage of big data. Sandy Hurricane 2012



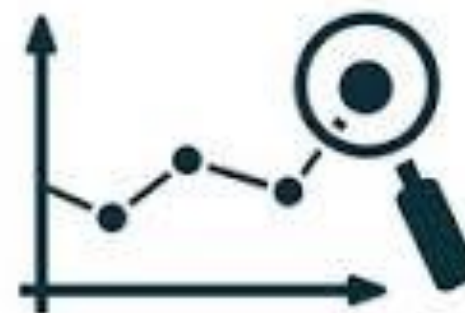
Predictive Analytics

What is it? How does it work? What are its benefits?



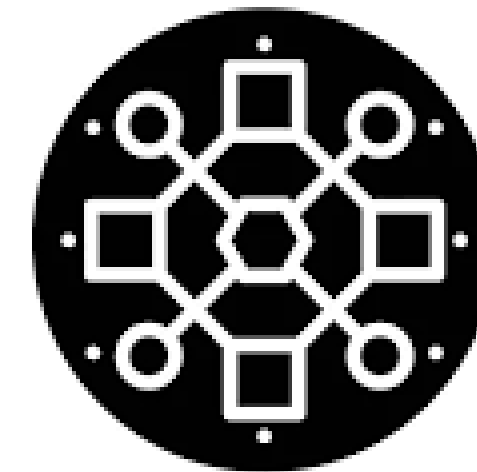
PROCESS

STEP 4: Analyzing the Data



PREDICTIVE ANALYTICS

shutterstock.com · 1440357755



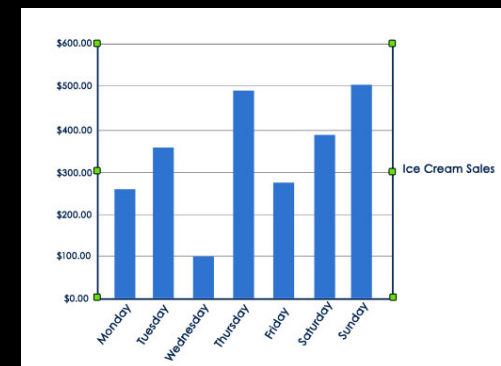
Pattern Recognition

PREDICTIVE ANALYSIS

PROCESS

STEP 5: Visualize and Share your Findings

- The final step of the data analytics process is to share these insights with the wider world (or at least with your organization's stakeholders!)

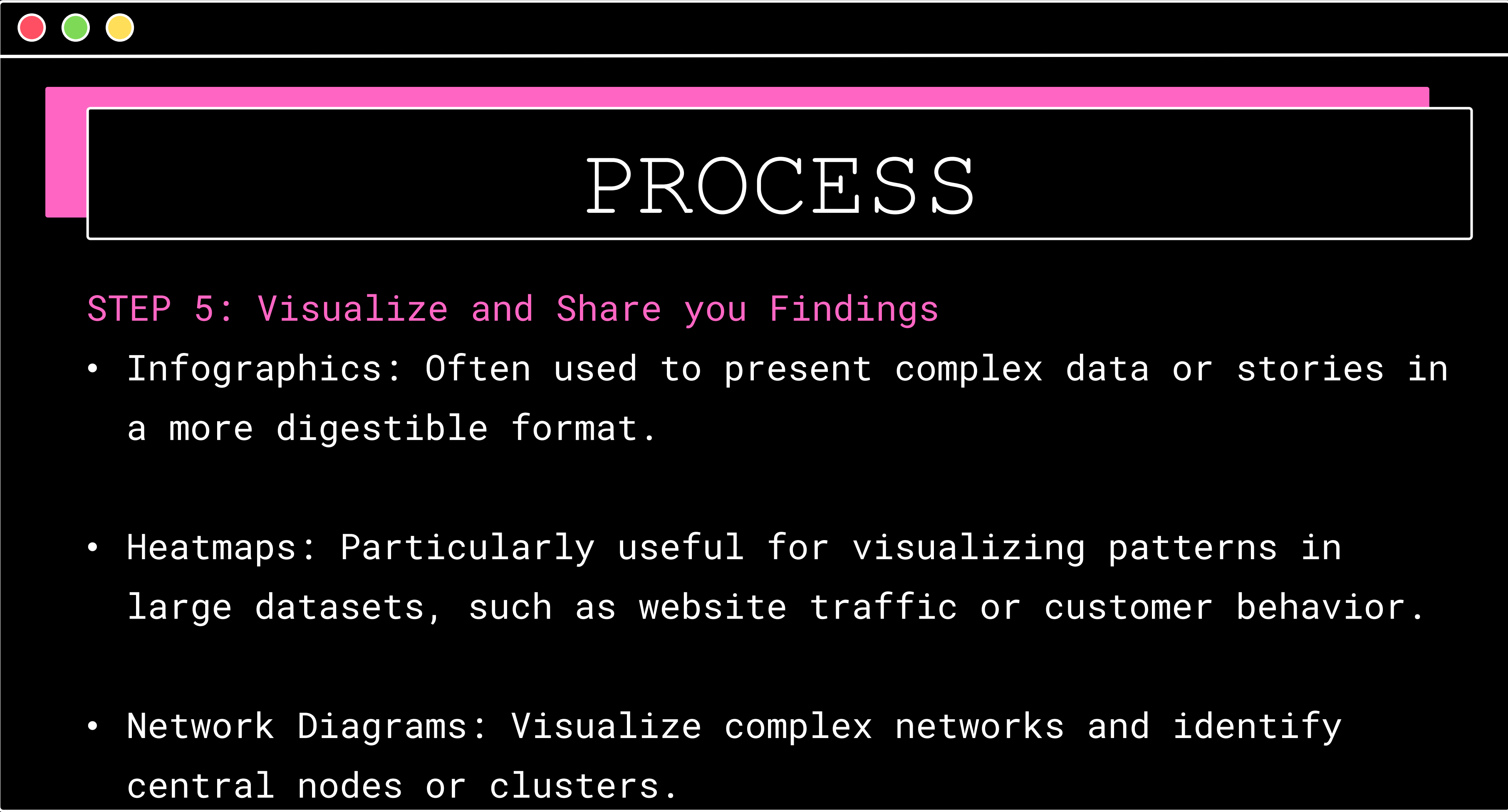




PROCESS

STEP 5: Visualize and Share your Findings

- Charts and Graphs: Effective for showing relationships between variables, trends over time, and proportions within a dataset.
- Maps: Useful for understanding spatial patterns, such as distribution of population, resources, or the spread of diseases



PROCESS

STEP 5: Visualize and Share your Findings

- Infographics: Often used to present complex data or stories in a more digestible format.
- Heatmaps: Particularly useful for visualizing patterns in large datasets, such as website traffic or customer behavior.
- Network Diagrams: Visualize complex networks and identify central nodes or clusters.

PROCESS

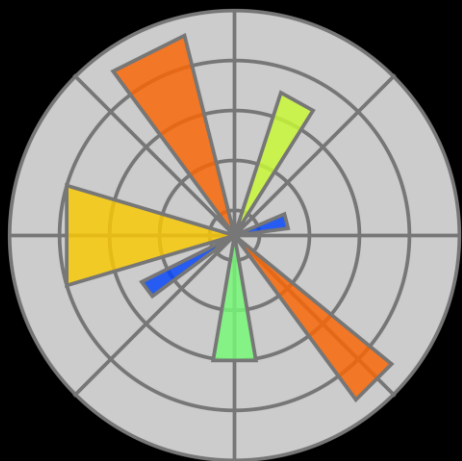
STEP 6: Share result to the world!



PROCESS

STEP 6: Share result to the world!

There are tons of data visualization tools available



PROCESS

THE DATA ANALYSIS PROCESS

Step 1:

Define the question

Step 2:

Collect the data

Step 3:

Clean the data

Step 4:

Analyze the data

Step 5:

Visualize and share
your findings

What is IPython



INTRODUCTION

IPython stands for **Interactive Python**. It's an enhanced version of the default Python shell that offers a much more powerful and user-friendly interactive experience.



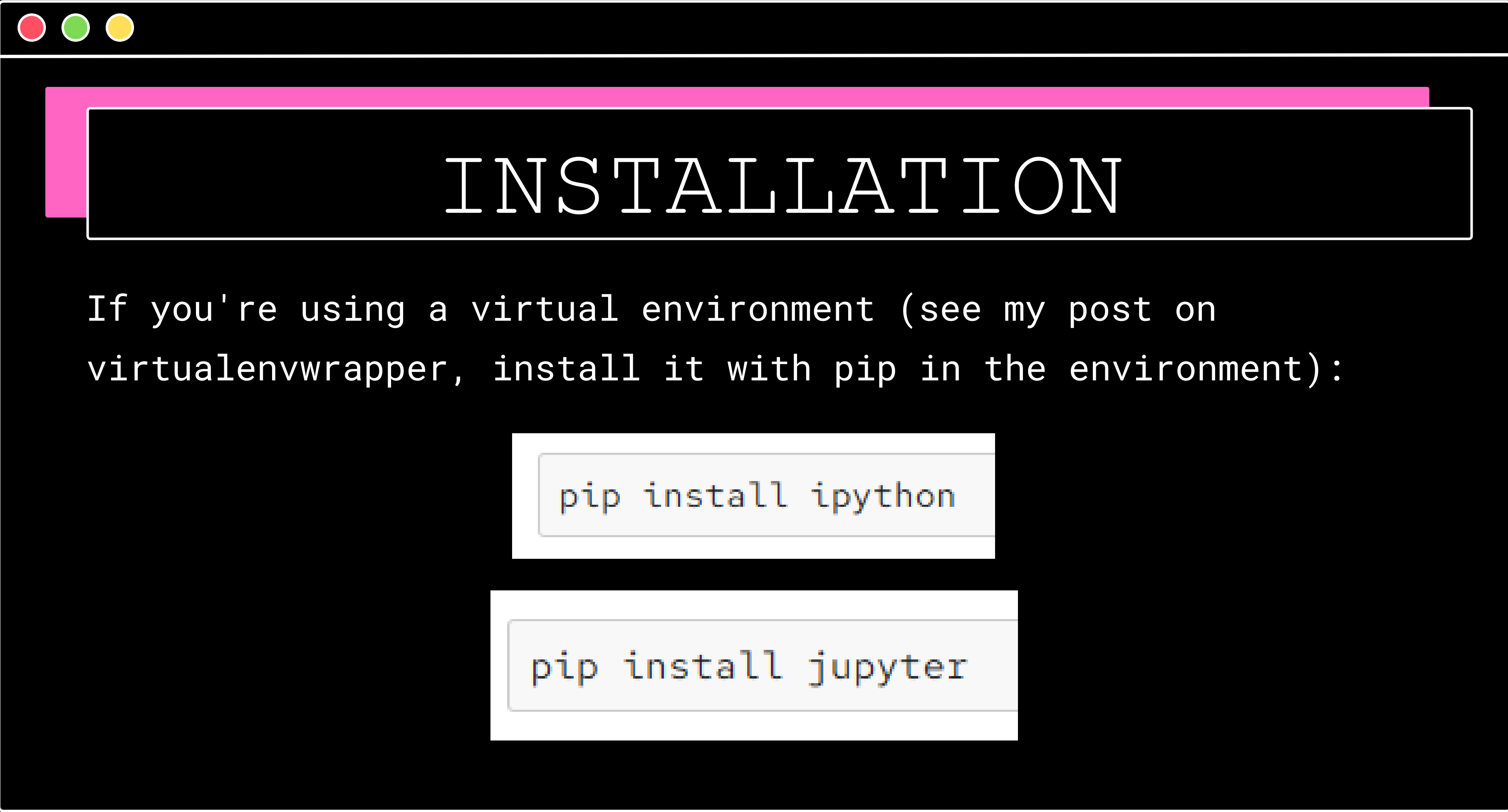
IP[y]:

INTRODUCTION

Jupyter Notebook is a web-based interactive environment that allows you to create documents containing:

- Live code
- Equations
- Visualizations
- Narrative text (Markdown)



A terminal window with a pink title bar and three window control buttons (red, green, yellow) in the top-left corner. The main content area is dark blue. A pink rectangular highlight is on the left side of the terminal. The word "INSTALLATION" is centered in a white, monospaced font.

INSTALLATION

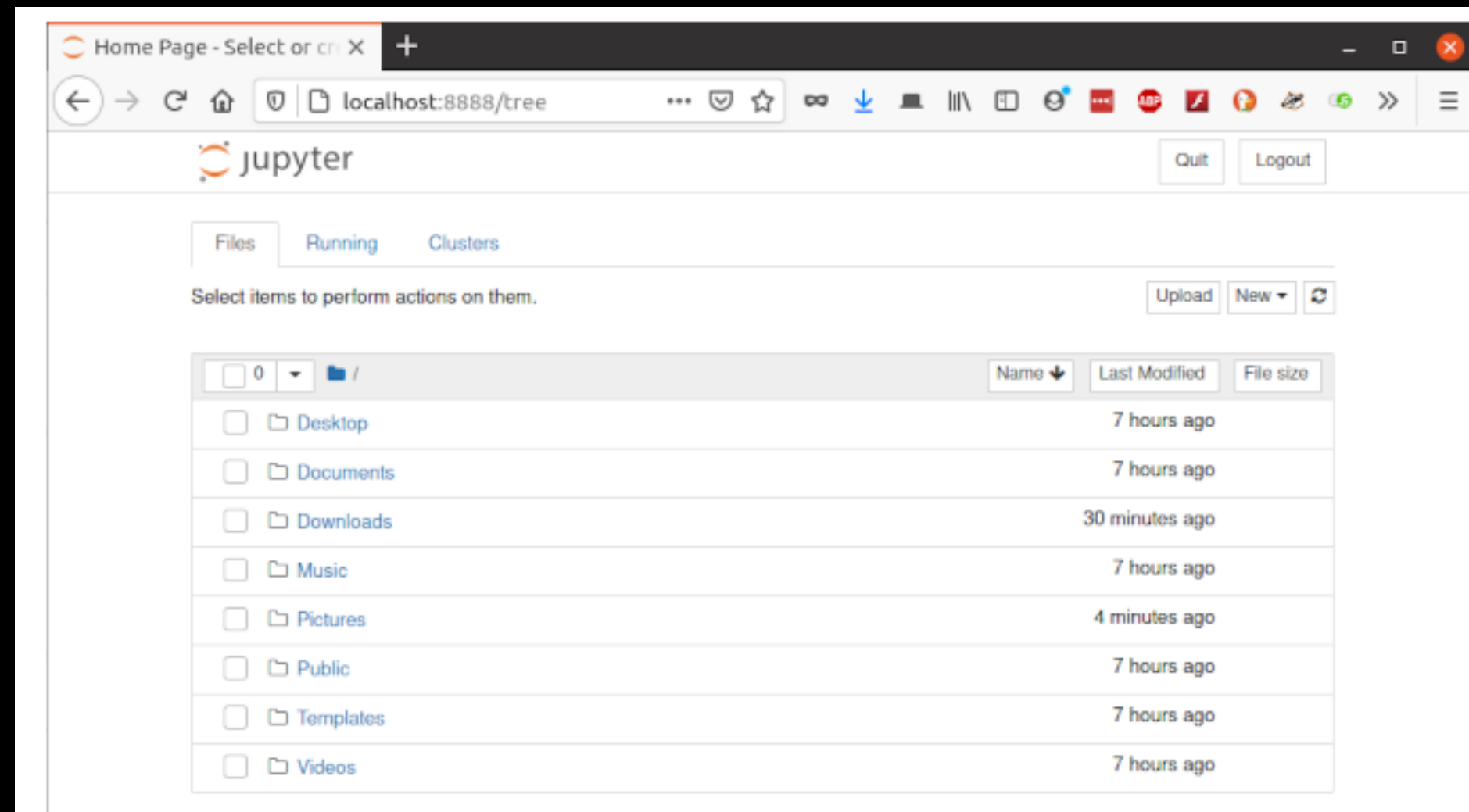
If you're using a virtual environment (see my post on [virtualenvwrapper](#), install it with pip in the environment):

```
pip install ipython
```

```
pip install jupyter
```

INSTALLATION

Launch the notebook with *jupyter notebook*



INSTALLATION

Now you can write and execute commands in the In[] fields.
Use Enter for a newline within the block and Shift+Enter to execute:



The screenshot shows a Jupyter Notebook interface. At the top, there are three colored window control buttons (red, green, yellow). Below them is a large pink rectangular header area. The main content area contains a code cell. The code cell has a prompt 'In [1]:' followed by a Python for loop: `for i in range(10):` on the first line and `print(i ** 2)` on the second line. Below the code, the output of the loop is displayed as a vertical list of numbers: 0, 1, 4, 9, 16, 25, 36, 49, 64, and 81. At the bottom of the code cell, there is an input field with the prompt 'In []:' and a blue cursor, indicating where to enter new code.

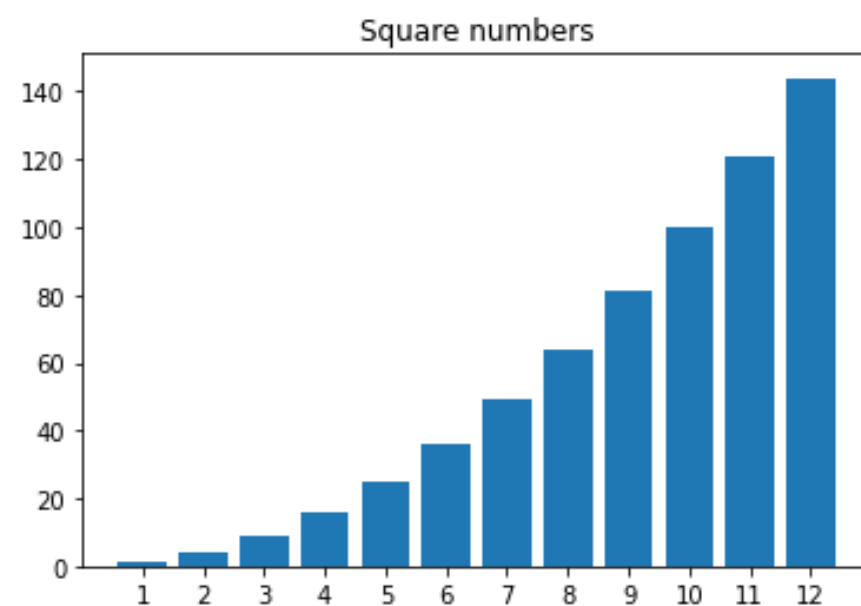
```
In [1]: for i in range(10):  
        print(i ** 2)  
  
0  
1  
4  
9  
16  
25  
36  
49  
64  
81  
  
In [ ]:
```


INSTALLATION

```
In [1]: import matplotlib.pyplot as plt
```

```
In [2]: x = range(1, 13)  
y = [i**2 for i in x]
```

```
In [3]: plt.bar(range(len(x)), y)  
plt.title("Square numbers")  
plt.xticks(range(len(x)), x)  
  
plt.show()
```



```
In [ ]:
```



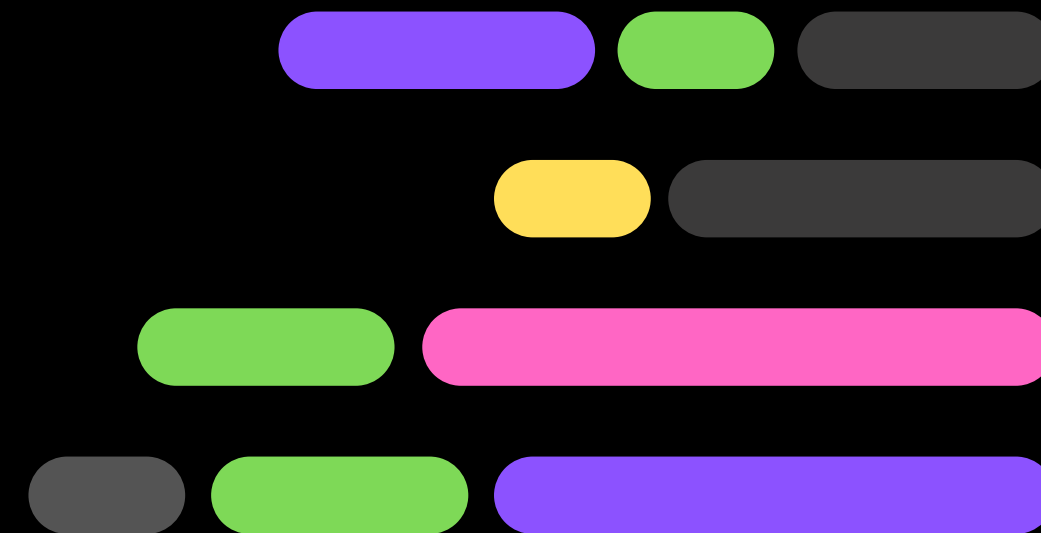
INFORMATION

MORE INFO:

<https://ipython.org/>



PYTHON NUMPY



INTRODUCTION

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices.

NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely. NumPy stands for **Numerical Python**.





INTRODUCTION

- Handles homogeneous data types (all elements in an array must be of the same type, such as all integers or all floats).
- Generally faster for performing mathematical operations due to its lower-level operations on arrays.





INTRODUCTION

- **Purpose:** NumPy is a library for numerical computing and efficient array handling.
- **Features:** Provides support for arrays, matrices, and mathematical functions to operate on these data structures.
- **Common Use:** Essential for handling large datasets and performing mathematical operations, especially in machine learning and scientific computing.
- **Strength:** Fast, efficient manipulation of large arrays and matrices, and forms the backbone of many other libraries like Pandas.

WHY NUMPY?

- The array object in NumPy is called `ndarray`, A powerful `n-dimensional array` object for handling data in an efficient manner.
- Arrays are very frequently used in data science, where speed and resources are very important.





INSTALLATION

If you have Python and PIP already installed on a system

```
C:\Users\Your Name>pip install numpy
```



NUMPY

- NumPy is usually imported under the `np` alias.
- NumPy is used to work with arrays. The array object in NumPy is called `ndarray`. We can create a NumPy ndarray object by using the `array()` function.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

NUMPY

- NumPy is usually imported under the `np` alias.
- NumPy is used to work with arrays. The array object in NumPy is called `ndarray`. We can create a NumPy ndarray object by using the `array()` function.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

```
import numpy as np

arr = np.array((1, 2, 3, 4, 5))

print(arr)
```


NUMPY-DIMENSION

0-d Arrays (Scalar)

- 0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

```
import numpy as np

arr = np.array(42)

print(arr)
```



NUMPY-DIMENSION

1-D Arrays

- An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array. These are the most common and basic arrays.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

NUMPY-INDEXING

Array indexing is the same as accessing an array element. You can access an array element by referring to its index number.

```
import numpy as np

d1A = np.array([1, 2, 3, 4])
print(d1A[0])
print(d1A[2] + d1A[3])

d2A = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print('2nd element on 1st row: ', d2A[0, 1])

d3A = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(d3A[1, 0, 2])
```



NUMPY-DATA TYPES

NumPy has some extra data types, and refer to data types with one character, like **i** for integers, **u** for unsigned integers etc.

- i** - integer
- b** - boolean
- u** - unsigned integer
- f** - float
- c** - complex float

- S** - string
- U** - unicode string
- V** - fixed chunk of memory for other type (void)

NUMPY-DATA TYPES

The NumPy array object has a property called `dtype` that returns the data type of the array:

```
import numpy as np

iarr = np.array([1, 2, 3, 4])
sarr = np.array(['apple', 'banana', 'cherry'])

print(iarr.dtype)
print(sarr.dtype)
```


NUMPY-DATA TYPES

We use the `array()` function to create arrays, this function can take an optional argument: `dtype` that allows us to define the expected data type of the array elements:

```
# Defined Datatype
arr = np.array(object: [1, 2, 3, 4], dtype='S')
print(arr)
print(arr.dtype)
💡

#Define Size
arr2 = np.array(object: [1, 2, 3, 4], dtype='i4')
print(arr2)
print(arr2.dtype)
```

NUMPY-DATA TYPES

The best way to change the data type of an **existing array**, is to make a copy of the array with the **astype()** method.

The **astype()** function creates a copy of the array, and allows you to specify the data type as a parameter.

```
import numpy as np

arr = np.array([1.1, 2.1, 3.1])

newarr = arr.astype('i')

print(newarr)
print(newarr.dtype)
```

NUMPY-ARITHMETIC

NumPy supports vectorized operations (no loops needed).

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
a + b    # [5 7 9]  
a * b    # [4 10 18]  
a ** 2   # [1 4 9]
```

Scalar operations

python

```
a + 10    # [11 12 13]
```

NUMPY-COPY AND VIEW

Make a copy, change the original array, and display both arrays:.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42

print(arr)
print(x)
```

NUMPY-COPY AND VIEW

Make a copy, change the original array, and display both arrays. The copy **SHOULD NOT** be affected by the changes made to the original array.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42

print(arr)
print(x)
```




NUMPY-COPY AND VIEW

Make a copy, change the original array, and display both arrays. The view **SHOULD** be affected by the changes made to the original array.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42

print(arr)
print(x)
```



NUMPY-ARRAY SHAPE

The shape of an array is the number of elements in each dimension.

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

print(arr.shape)
```



NUMPY-ARRAY SHAPE

Reshaping means changing the shape of an array. The shape of an array is the number of elements in each dimension.

By reshaping we can add or remove dimensions or change number of elements in each dimension.

NUMPY-ARRAY SHAPE

Reshape From 1-D → 2-D and 3-D

```
import numpy as np

d1tod2 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr1 = d1tod2.reshape(4, 3)
print(newarr1)

d1tod3 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr2 = d1tod3.reshape(2, 3, 2)
print(newarr2)
```

NUMPY-ARRAY SHAPE

you do not have to specify an exact number for one of the dimensions in the reshape method. Pass -1 as the value, and NumPy will calculate this number for you.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(2, 2, -1)

print(newarr)
```


NUMPY-ARRAY SHAPE

Flattening array means converting a multidimensional array into a 1D array.

We can use `reshape(-1)` to do this.

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)
```

NUMPY-ITERATING

If we iterate on a n-D array it will go through n-1th dimension one by one. The function `nditer()` solves some basic issues which we face in iteration

```
import numpy as np

arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

for x in np.nditer(arr):
    print(x)
```

NUMPY-ITERATING

Sometimes we require corresponding index of the element while iterating, the `ndenumerate()` method can be used for those usecases.

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

NUMPY-JOINING

Joining means putting contents of two or more arrays in a single array. We pass a sequence of arrays that we want to join to the `concatenate()` function, along with the axis. If axis is not explicitly passed, it is taken as 0.

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.concatenate((arr1, arr2))

print(arr)
```



NUMPY-JOINING

Join two 2-D arrays along rows (axis=1)

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=1)

print(arr)
```


NUMPY-JOINING

Joining Arrays Using Stack Functions

- Stacking is same as concatenation, the only difference is that stacking is done along a new axis.

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.stack((arr1, arr2), axis=1)

print(arr)
```


NUMPY-JOINING

Joining Arrays Using Stack Functions

- `hstack()` to stack along rows.
- `vstack()` to stack along columns.

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

arrrows = np.hstack((arr1, arr2))
arrcolumn = np.vstack((arr1, arr2))
print(arrrows)
print(arrcolumn)
```



NUMPY-SPLITTING

We use `array_split()` for splitting arrays, we pass it the array we want to split and the number of splits.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr)
```



NUMPY-SPLITTING

The return value of the `array_split()` method is an array containing each of the split as an array. If you split an array into 3 arrays, you can access them from the result just like any array element:

```
newarr = np.array_split(arr, 3)

print(newarr[0])
print(newarr[1])
print(newarr[2])
```



NUMPY-SPLITTING

Splitting 2-D Arrays

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]])

newarr = np.array_split(arr, 3)

print(newarr)
```

NUMPY-SEARCHING

You can search an array for a certain value, and return the indexes that get a match. To search an array, use the `where()` method.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])

x = np.where(arr == 4)

print(x)
```


NUMPY-SEARCHING

There is a method called `searchsorted()` which performs a binary search in the array, and returns the index where the specified value would be inserted to maintain the search order.

```
import numpy as np

arr = np.array([1, 3, 5, 7])

x = np.searchsorted(arr, [2, 4, 6])

print(x)
```




NUMPY-SORTING

Ordered sequence is any sequence that has an order corresponding to elements, like numeric or alphabetical, ascending or descending. The NumPy ndarray object has a function called `sort()`, that will sort a specified array.

```
import numpy as np

arr = np.array([3, 2, 0, 1])

print(np.sort(arr))
```



NUMPY-SORTING

If you use the `sort()` method on a 2-D array, both arrays will be sorted:

```
import numpy as np

arr = np.array([[3, 2, 4], [5, 0, 1]])

print(np.sort(arr))
```



NUMPY-FILTERING

Getting some elements out of an existing array and creating a new array out of them is called filtering. In NumPy, you filter an array using a **boolean index** list.



NUMPY-FILTERING

Create an array from the elements on index 0 and 2

```
import numpy as np

arr = np.array([41, 42, 43, 44])

x = [True, False, True, False]

newarr = arr[x]

print(newarr)
```



NUMPY-AGGREGATING

You can do some aggregating by using method from the numpy such as, sum, mean, mode, median, max, min and etc.

```
arr1 = np.array([1, 2, 3, 2, 6, 8, 2, 6])  
sum = np.sum(arr1)  
print(sum)
```


NUMPY-AGGREGATING

NumPy provides fast math and statistics functions.

Math Functions

```
np.sqrt(arr)
np.abs(arr)
np.round(arr)
np.power(arr, 2)
```

Statistical Functions

```
np.sum(arr)
np.mean(arr)
np.median(arr)
np.std(arr)
np.var(arr)
np.min(arr)
np.max(arr)
```




ARRAY-ORIENTED

Array-oriented programming means:

- You **think in terms of arrays**, not loops
- Replace for loops with **vectorized operations**
- Code becomes **shorter, faster, and clearer**



ARRAY-ORIENTED

```
result = []  
for x in arr:  
    result.append(x * 2)
```

VS

```
result = arr * 2
```

ARRAY-ORIENTED

NumPy allows conditions to work on whole arrays.

```
arr = np.array([10, 20, 30, 40])
```

```
arr > 25
```

```
[False False  True  True]
```

ARRAY-ORIENTED

NumPy allows conditions to work on whole arrays.

```
arr = np.array([10, 20, 30, 40])  
  
arr[arr > 25]
```

```
30, 40])
```



ARRAY-ORIENTED

Using `np.where()` (if-else logic)

```
np.where(arr > 25, "Pass", "Fail")
```




ARRAY-ORIENTED

Unique values

```
arr = np.array([1, 2, 2, 3, 3, 3])  
np.unique(arr)
```




ARRAY-ORIENTED

NumPy supports set-like operations on arrays.

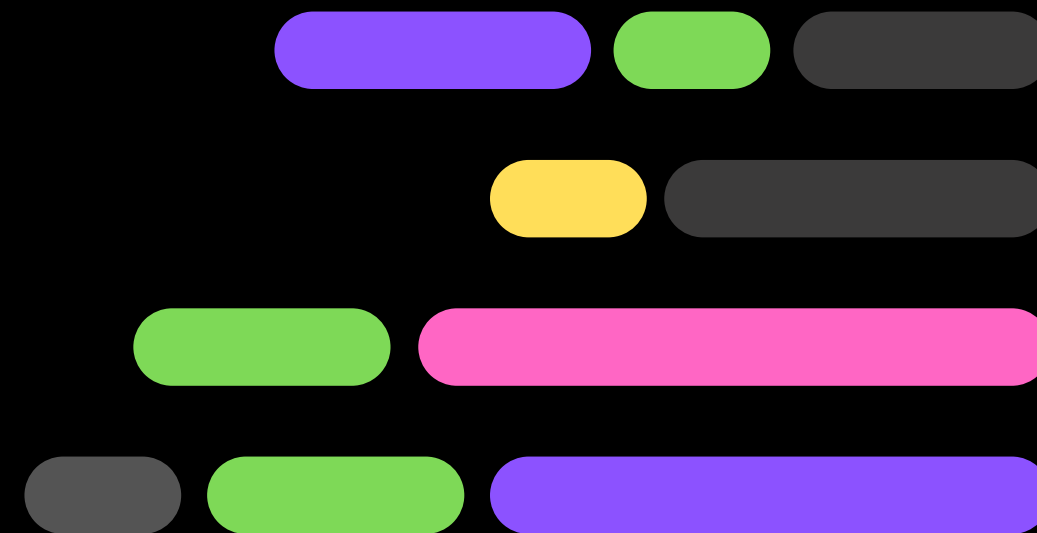
```
a = np.array([1, 2, 3])  
b = np.array([3, 4, 5])  
  
np.intersect1d(a, b)    # common elements  
np.union1d(a, b)        # all unique elements  
np.setdiff1d(a, b)      # in a but not in b  
np.in1d(a, b)           # membership test
```

LAB 1

COFFEE SHOP



PYTHON PANDAS





INTRODUCTION

- Built on top of NumPy, it is designed for data manipulation and analysis, providing more complex data structures (like DataFrames) and tools to handle tabular or time series data efficiently.
- Can handle heterogeneous data types within a DataFrame (each column can be of a different type, such as integers, floats, strings, etc.).

INTRODUCTION

Pandas is a powerful Python library that is specifically designed to work on data frames that have "relational" or "labeled" data.





INTRODUCTION

- **Purpose:** Pandas is a data manipulation and analysis library.
- **Features:** It provides data structures like DataFrames for handling tabular and time-series data, and many built-in functions for cleaning, filtering, transforming, and analyzing data.
- **Common Use:** Primarily used for loading, cleaning, and manipulating datasets before visualization or further analysis.
- **Strength:** Simplifies the process of working with structured data, like CSV or SQL table data, with intuitive methods for data analysis.

INTRODUCTION

pandas DataFrames are data structures that contain:

- Data organized in two dimensions, rows and columns
- Labels that correspond to the rows and columns





DS – SERIES

If you have Python and PIP already installed on a system, then installation of Pandas is very easy.

Install it using this command:

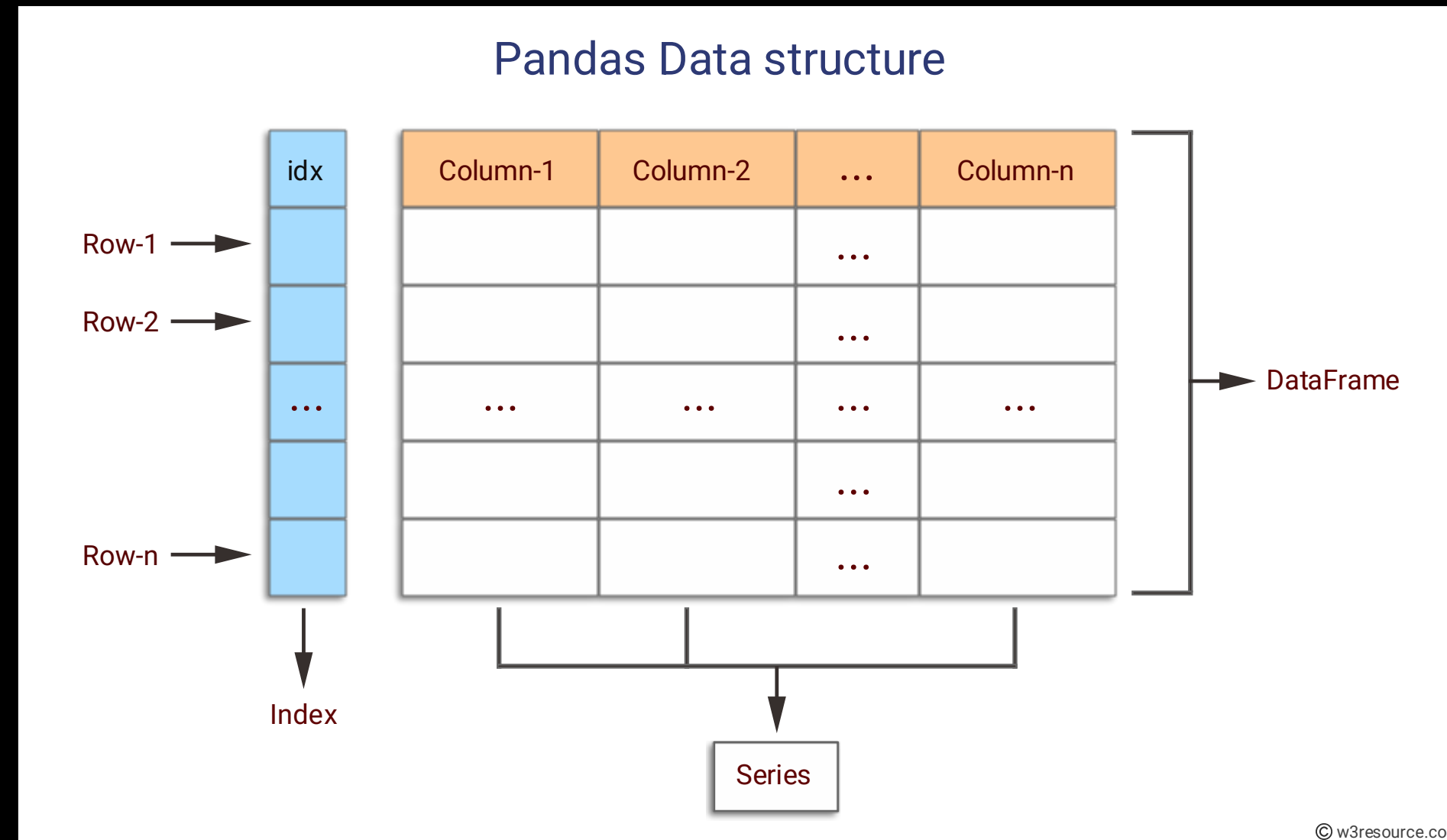
```
C:\Users\Your Name>pip install pandas
```

DATA STRUCTURE

Pandas deals with the following three data structures -

Data Structure	Dimensions	Description
Series	1	1D labeled homogeneous array, sizeimmutable.
Data Frames	2	General 2D labeled, size-mutable tabular structure with potentially heterogeneously typed columns.
Panel	3	General 3D labeled, size-mutable array.

DS – DATAFRAME





DS – SERIES

Series is a one-dimensional array like structure with **homogeneous** data. For example, the following series is a collection of integers 10, 23, 56, ...

10	23	56	17	52	61	73	90	26	72
----	----	----	----	----	----	----	----	----	----

DS – SERIES

A pandas Series can be created using the following constructor

```
pandas.Series( data, index, dtype, copy)
```

Sr.No	Parameter & Description
1	data data takes various forms like ndarray, list, constants
2	index Index values must be unique and hashable, same length as data. Default np.arange(n) if no index is passed.
3	dtype dtype is for data type. If None, data type will be inferred
4	copy Copy data. Default False



DS – SERIES

A Pandas Series is like a column in a table.

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar)
```



DS – SERIES

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```



DS – SERIES

Key/Value Objects as Series

You can also use a key/value object, like a dictionary, when creating a Series.

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

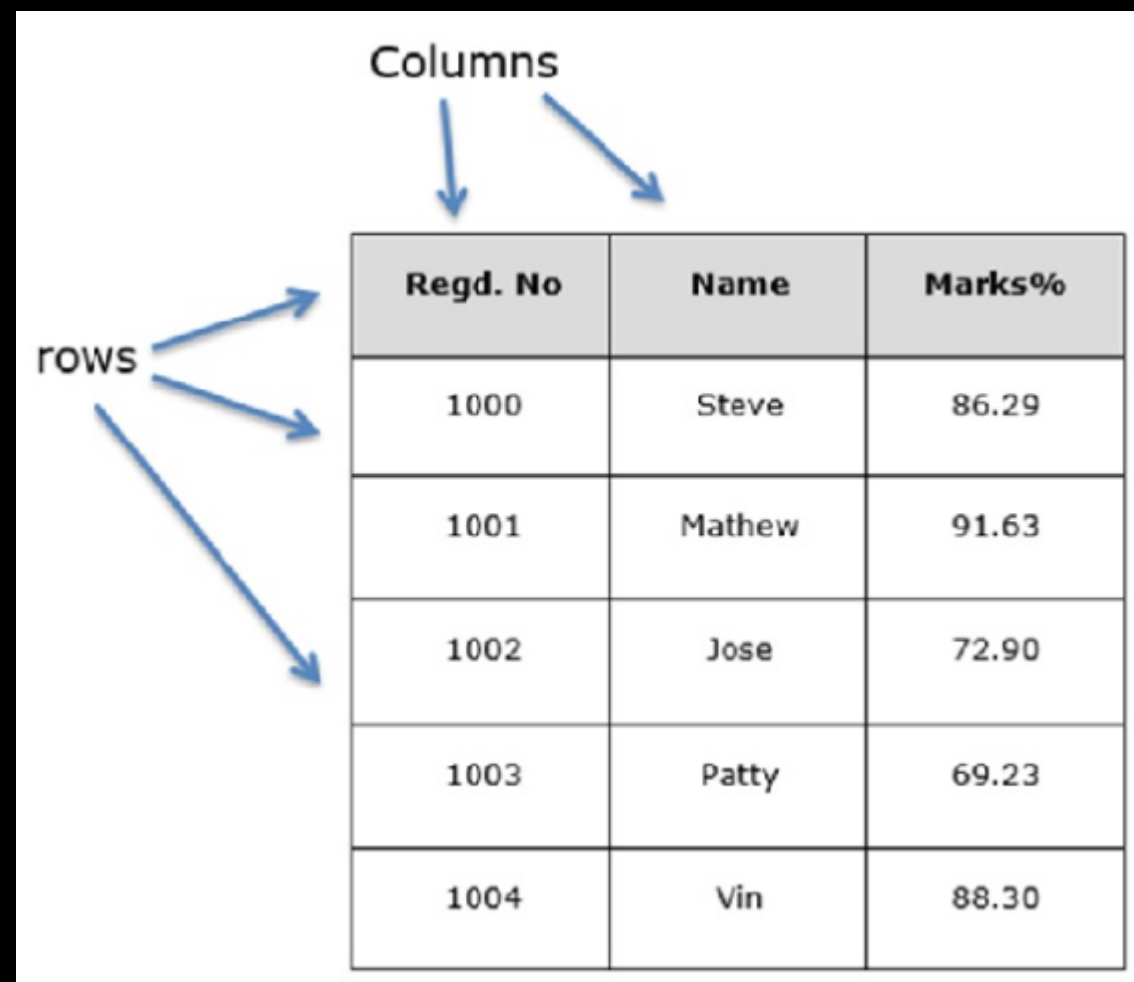
DS – DATAFRAME

DataFrame is a two-dimensional array with heterogeneous data. For example,

STRING	INTEGER	STRING	FLOAT
Name	Age	Gender	Rating
Steve	32	Male	3.45
Lia	28	Female	4.6
Vin	45	Male	3.9
Katie	38	Female	2.78

DS – DATAFRAME

You can think of it as an SQL table or a spreadsheet data representation.



The diagram shows a table with three columns and five rows. The columns are labeled 'Regd. No', 'Name', and 'Marks%'. The rows are labeled with registration numbers 1000 through 1004. Blue arrows point from the word 'Columns' to the column headers, and from the word 'rows' to the row headers.

Regd. No	Name	Marks%
1000	Steve	86.29
1001	Mathew	91.63
1002	Jose	72.90
1003	Patty	69.23
1004	Vin	88.30

DS – DATAFRAME

pandas DataFrame can be created using the following constructor

```
pandas.DataFrame( data, index, columns, dtype, copy)
```

Sr.No	Parameter & Description
1	data data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame.
2	index For the row labels, the Index to be used for the resulting frame is Optional Default np.arange(n) if no index is passed
3	columns For column labels, the optional default syntax is - np.arange(n). This is only true if no index is passed.
4	dtype Data type of each column.
5	copy This command (or whatever it is) is used for copying of data, if the default is False.



DS – DATAFRAME

Data sets in Pandas are usually multi-dimensional tables, called DataFrames. Series is like a column, a DataFrame is the whole table.

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```



DS – DATAFRAME

Locate Row

As you can see from the result above, the DataFrame is like a table with rows and columns. Pandas use the loc attribute to return one or more specified row(s)

```
#refer to the row index:  
print(df.loc[0])
```

DS – DATAFRAME

Locate Row

As you can see from the result above, the DataFrame is like a table with rows and columns. Pandas use the loc attribute to return one or more specified row(s)

```
#refer to the row index:  
print(df.loc[0])
```

```
#use a list of indexes:  
print(df.loc[[0, 1]])
```



DS – DATAFRAME

Reindexing

With the `index` argument, you can name your own indexes.

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```



DS – DATAFRAME

Reindexing

fill_value sets default for missing entries

```
s2 = s.reindex(['a', 'b', 'c', 'd'], fill_value=0)
print(s2)
```




DS – DATAFRAME

Locate Named Indexes

Use the named index in the loc attribute to return the specified row(s).

```
#refer to the named index:  
print(df.loc["day2"])
```


PANDAS

```
import pandas as pd
# Create a dictionary in the specified format
formatted_data = {
    "Name": ["Steve", "Lia", "Vin", "Katie"],
    "Age": [32, 28, 45, 38],
    "Gender": ["Male", "Female", "Male", "Female"],
    "Rating": [3.45, 4.6, 3.9, 2.78]
}
```



DS — ACCESSING

You can access a column by referring to it by name. Returns the 'Name' column as a Series

```
print(rating["Name"])
```



DS — ACCESSING

Use indexing to select and print specific columns. If you use a single bracket with a column name (e.g., `rating['Name']`), Pandas returns a Series object, which is essentially a single column of data.

```
#Accessing Specific Columns  
print(rating[['Name', 'Rating']])
```



DS — ADDING COLUMN

You can add new column by the same method as to create a new dictionary but this depend on your datatype.

```
rating['Occupation'] = ['Teacher', 'Programmer', 'Teacher', 'Lecturer']  
print(rating)
```



DS – FILTERING

Use Boolean indexing to filter rating that are over desired number

```
#Filter Data
~~~~~
filtered_rating = rating[rating['Rating'] > 3.8]
print(filtered_rating)
```




DS – SORTING

Sort the DataFrame by RATING to identify the lowest and highest rating.

```
#Sorting Data
sorted_rating = rating.sort_values(by='Rating')
print(sorted_rating)
```




DS — GROUPING

`groupby()` returns a `GroupBy` object rather than the actual grouped data itself. This object is a reference to the groups, but it doesn't immediately show the contents of those groups.

```
group_gender = rating.groupby('Gender')  
print(group_gender)
```



DS — GROUPING

To see the results of the grouping, you can use aggregation methods such as `sum()`, `mean()`, or `size()` to get summary statistics for each group.

```
group_gender = rating.groupby('Gender').size()
print(group_gender)
```



DS — GROUPING

You can also loop through the GroupBy object to access each group and its data.

```
for name, group in group_gender:  
    print(name) # Prints the group name (e.g., "male", "female")  
    print(group) # Prints the DataFrame for that group
```

DS — GROUPING

You can also loop through the GroupBy object to access each group and its data.

Female

	Name	Age	Gender	Rating	Occupation
1	Lia	28	Female	4.60	Programmer
3	Katie	38	Female	2.78	Lecturer

Male

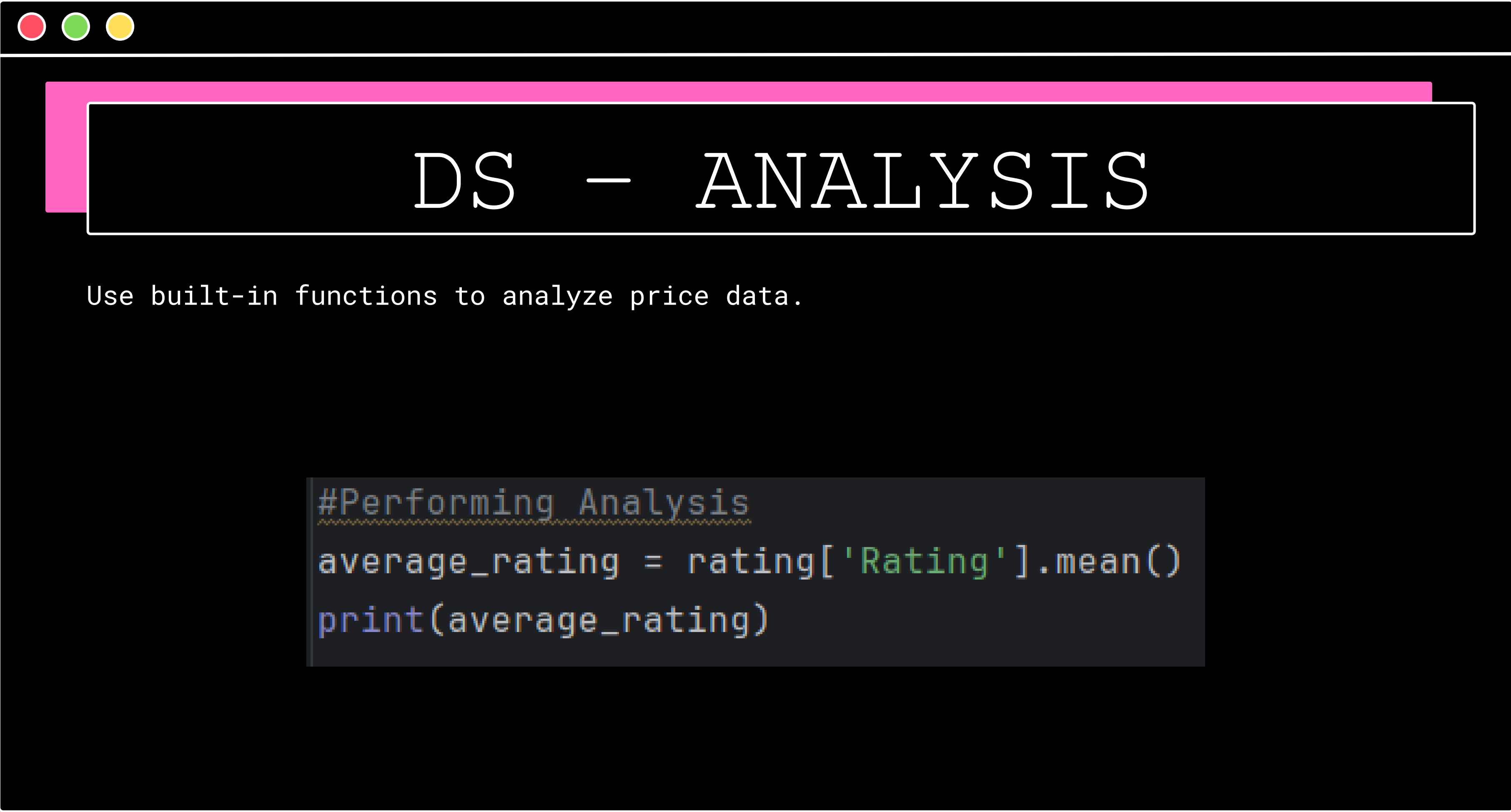
	Name	Age	Gender	Rating	Occupation
0	Steve	32	Male	3.45	Teacher
2	Vin	45	Male	3.90	Teacher



DS — GROUPING

If you want to convert the grouped data back to a DataFrame with a default integer index, you can use the `reset_index()` method:

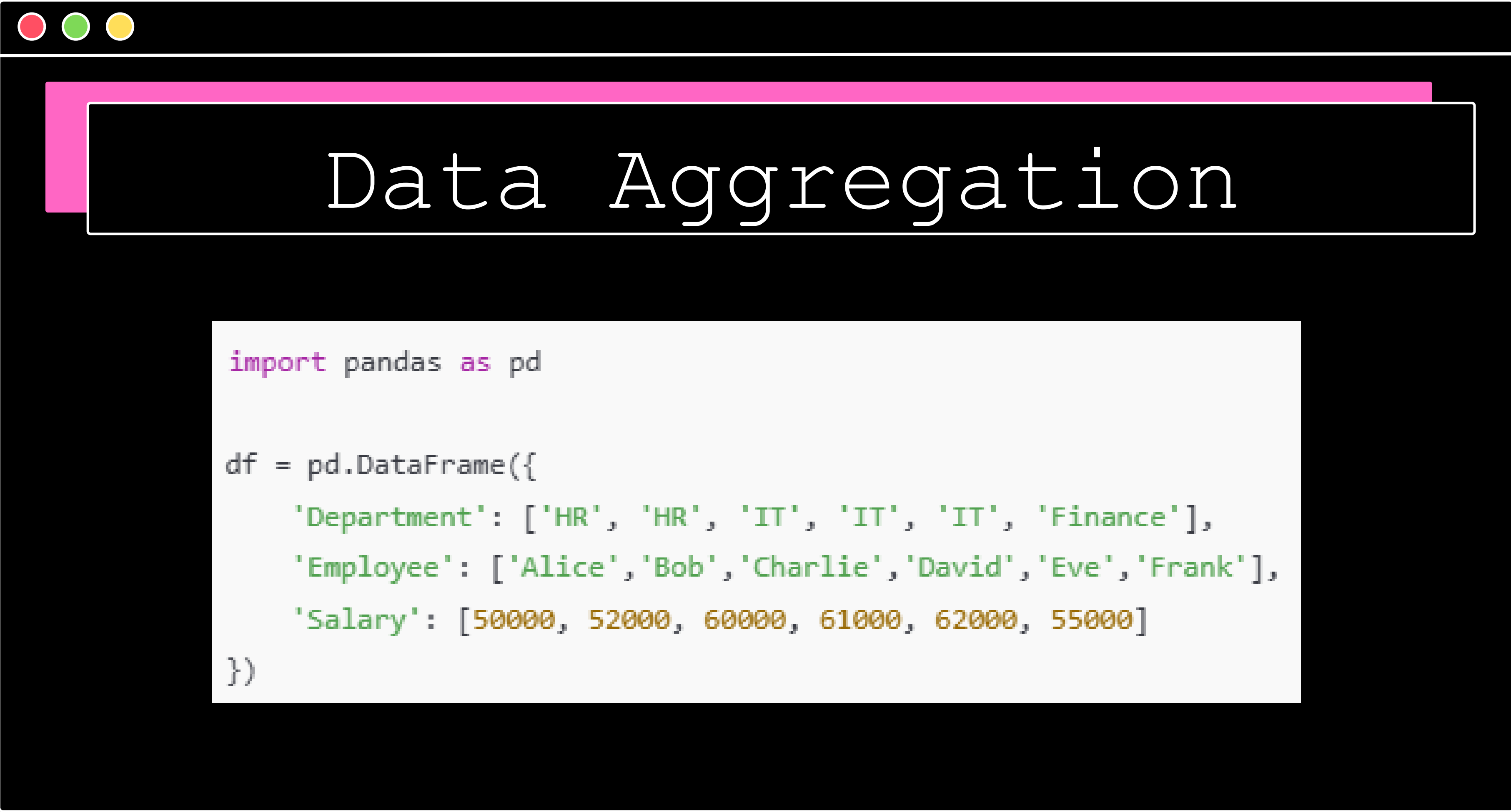
```
#Grouping Data  
group_rating = rating.groupby('Gender')['Rating'].sum().reset_index()  
print(group_rating)
```

DS — ANALYSIS

Use built-in functions to analyze price data.

```
#Performing Analysis
~~~~~
average_rating = rating['Rating'].mean()
print(average_rating)
```

Data Aggregation

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['HR', 'HR', 'IT', 'IT', 'IT', 'Finance'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Salary': [50000, 52000, 60000, 61000, 62000, 55000]
})
```



Data Aggregation

Column-wise aggregation

```
grouped['Salary'].agg('mean')
```



Data Aggregation

Multiple functions

```
grouped['Salary'].agg(['mean', 'max', 'min'])
```



Apply

apply allows custom operations on groups.

```
def range_salary(x):  
    return x.max() - x.min()  
  
grouped[ 'Salary' ].apply(range_salary)
```



Pivot Tables

index → rows

values → column to aggregate

aggfunc → aggregation function

```
aggfunc=['mean', 'max'])
```

```
pd.pivot_table(df, index='Department', values='Salary', aggfunc='mean')
```




Cross-Tabulation

a statistical method using a table (contingency table) to show the frequency distribution of two or more categorical variables, revealing relationships or patterns between them

```
data = pd.DataFrame({  
    'Gender': ['M', 'F', 'F', 'M', 'F', 'M'],  
    'Department': ['HR', 'HR', 'IT', 'IT', 'IT', 'Finance']  
})  
  
pd.crosstab(data['Department'], data['Gender'])
```


MultiIndex

Hierarchical indexing allows multiple levels of row or column indexes, which is useful for multi-dimensional data.

```
import pandas as pd
import numpy as np

data = pd.DataFrame({
    'Region': ['North', 'North', 'South', 'South'],
    'Department': ['HR', 'IT', 'HR', 'IT'],
    'Sales': [250, 400, 300, 500]
})

# Set multi-level index
df = data.set_index(['Region', 'Department'])
print(df)
```



MultiIndex

Hierarchical indexing allows multiple levels of row or column indexes, which is useful for multi-dimensional data.

```
# Swap index levels
df_swapped = df.swaplevel()
print(df_swapped)

# Sort by index
df_sorted = df.sort_index(level=1)
print(df_sorted)
```

Combining & Merging

SQL-like Join can be implement in Python

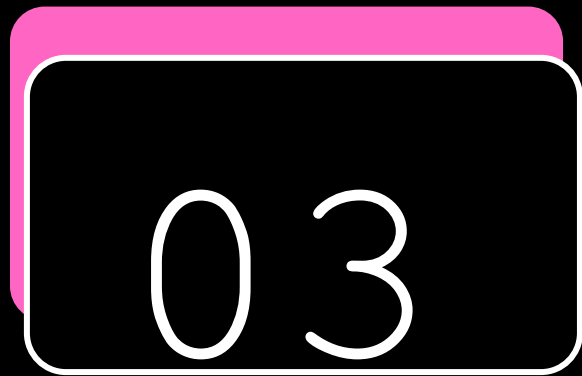
```
df1 = pd.DataFrame({'key': ['A', 'B', 'C'], 'value1': [1, 2, 3]})
df2 = pd.DataFrame({'key': ['B', 'C', 'D'], 'value2': [4, 5, 6]})

# Inner join (intersection)
pd.merge(df1, df2, on='key', how='inner')

# Left, right, outer joins also supported
```

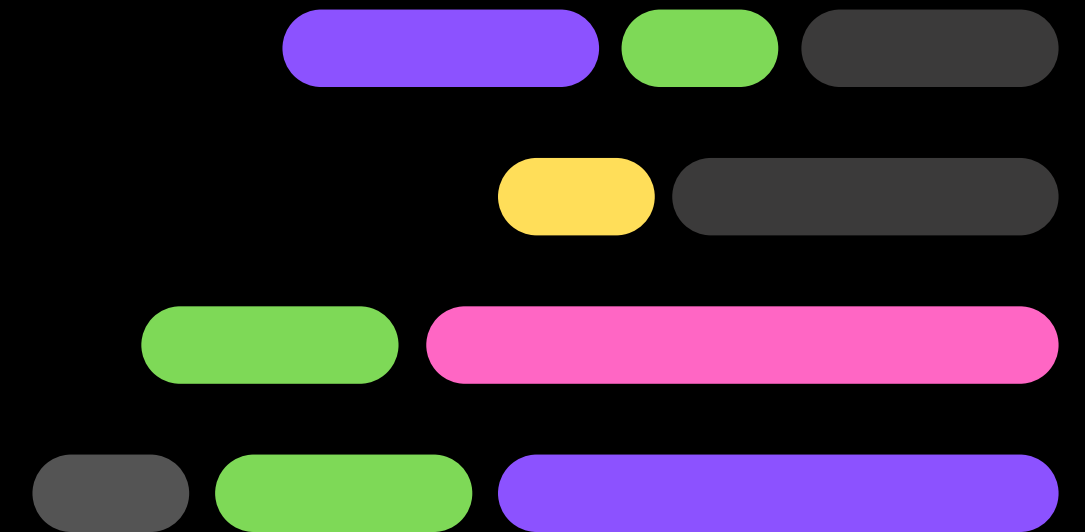
LAB 2

BOOKSTORE



PANDAS

I/O OPERATION





PANDAS LOAD FILE

If your data sets are stored in a file, Pandas can load them into a DataFrame.
Load a comma separated file (CSV file) into a DataFrame

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df)
```



PANDAS LOAD FILE

JSON (JavaScript Object Notation) is widely used in web applications to store and transmit data. Pandas provides functions to read and process JSON data:

```
import json

# Reading JSON data into a DataFrame
df_json = pd.read_json('data.json')
```




PANDAS LOAD FILE

Sometimes, data is presented on websites in HTML tables. Pandas makes it easy to extract this data:

```
# Reading HTML tables  
dfs = pd.read_html('https://example.com') # Returns a list of DataFrames  
df_html = dfs[0] # Access the first table
```



PANDAS LOAD FILE

If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows. Use `to_string()` to print the entire DataFrame.

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())
```



PANDAS LOAD FILE

The number of rows returned is defined in Pandas option settings.
You can check your system's maximum rows with the `pd.options.display.max_rows` statement.

```
import pandas as pd

print(pd.options.display.max_rows)
```



PANDAS LOAD FILE

One of the most used method for getting a quick overview of the DataFrame, is the `head()` method. The `head()` method returns the headers and a specified number of rows, starting from the top.

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```



PANDAS LOAD FILE

There is also a `tail()` method for viewing the last rows of the DataFrame. The `tail()` method returns the headers and a specified number of rows, starting from the bottom.

```
print(df.tail())
```




PANDAS LOAD FILE

Info About the Data

The DataFrames object has a method called `info()`, that gives you more information about the data set.

```
print(df.info())
```



PANDAS LOAD FILE

This gives a summary of numerical data, such as count, mean, min, max, standard deviation, etc., for columns like age, fare, and sibsp.

```
# Basic statistics for the numerical columns  
df.describe()
```




DATA GROUPBY

It allows you to group data based on one or more keys (columns) and perform operations on those groups.

```
data = {  
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],  
    'Values': [10, 20, 30, 40, 50, 60]  
}  
  
df = pd.DataFrame(data)  
print(df)
```

```
grouped = df.groupby('Category')
```



DATA GROUPBY

You can perform aggregation operations such as sum, mean, or count on the grouped data. For example, to calculate the sum of Values for each category:

```
sum_values = grouped['Values'].sum()  
print(sum_values)
```



DATA GROUPBY

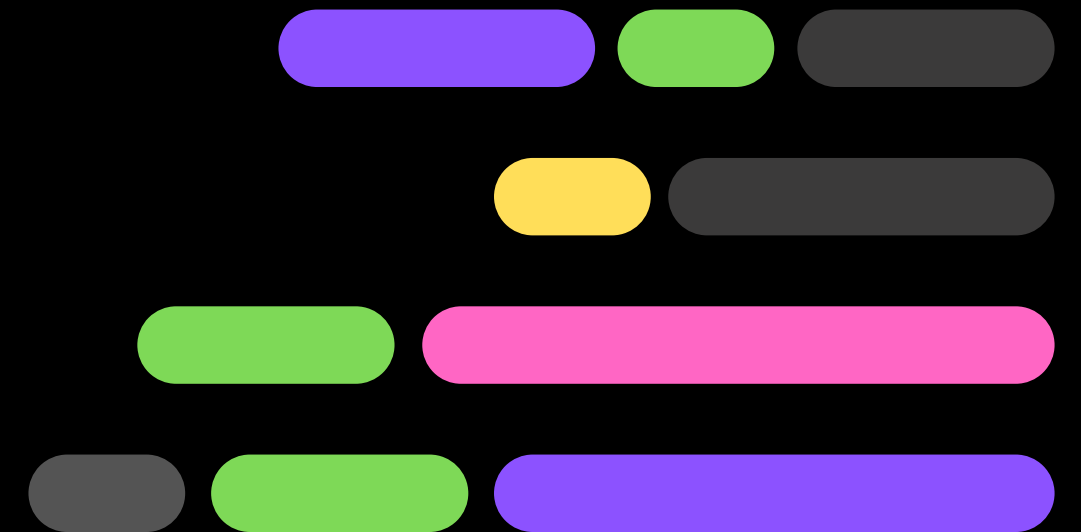
If you want to convert the grouped data back to a DataFrame with a default integer index, you can use the `reset_index()` method:

```
result = grouped['Values'].sum().reset_index()
print(result)
```



PANDAS

DATA CLEANING





PANDAS DATA CLEANING

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells

- Data in wrong format

- Wrong data

- Duplicates

PANDAS DATA CLEANING

Checking for Missing Values

Missing values in a dataset can lead to biased results or errors during analysis. Pandas provides simple methods to check for missing data

```
import pandas as pd
df = pd.read_csv('data.csv')
# Check for missing values
print(df.isnull().sum()) # Sum of missing values per column
print(df.isnull().any()) # Check if there are any missing values
```

PANDAS DATA CLEANING

Empty Cells - Remove Rows

One way to deal with empty cells is to remove rows that contain empty cells. This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result.

```
import pandas as pd

df = pd.read_csv('data.csv')

new_df = df.dropna()

print(new_df.to_string())
```


PANDAS DATA CLEANING

Replace Empty Values

Another way of dealing with empty cells is to insert a new value instead. This way you do not have to delete entire rows just because of some empty cells. The `fillna()` method allows us to replace empty cells with a value

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna(130, inplace = True)
```

```
# Removing rows with missing data
df_cleaned = df.dropna()

# Filling missing data with mean
df_filled = df.fillna(df.mean())
```

PANDAS DATA CLEANING

Replace Empty Values

The example above replaces all empty cells in the whole Data Frame.

To only replace empty values for one column, specify the column name for the DataFrame:

```
import pandas as pd

df = pd.read_csv('data.csv')

df["Calories"].fillna(130, inplace = True)
```

PANDAS DATA CLEANING

Replace Empty Values

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.

Pandas uses the `mean()` `median()` and `mode()` methods to calculate the respective values for a specified column:

```
import pandas as pd

df = pd.read_csv('data.csv')

x = df["Calories"].mean()

df["Calories"].fillna(x, inplace = True)
```

PANDAS DATA CLEANING

Data of Wrong Format

Cells with data of wrong format can make it difficult, or even impossible, to analyze data.

To fix it, you have two options: remove the rows, or convert all cells in the columns into the same format.

```
import pandas as pd

df = pd.read_csv('data.csv')

df['Date'] = pd.to_datetime(df['Date'])

print(df.to_string())
```



PANDAS DATA CLEANING

Removing Rows

The result from the converting in the example above gave us a NaT value, which can be handled as a NULL value, and we can remove the row by using the `dropna()` method.

```
df.dropna(subset=['Date'], inplace = True)
```


PANDAS DATA CLEANING

Wrong Data

"Wrong data" does not have to be "empty cells" or "wrong format", it can just be wrong, like if someone registered "199" instead of "1.99".

6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1



PANDAS DATA CLEANING

Replacing Values

One way to fix wrong values is to replace them with something else. In our example, it is most likely a typo, and the value should be "45" instead of "450", and we could just insert "45" in row 7

```
df.loc[7, 'Duration'] = 45
```


PANDAS DATA CLEANING

Replacing Values

For small data sets you might be able to replace the wrong data one by one, but not for big data sets.

To replace wrong data for larger data sets you can create some rules, e.g. set some boundaries for legal values, and replace any values that are outside of the boundaries.

```
for x in df.index:  
    if df.loc[x, "Duration"] > 120:  
        df.loc[x, "Duration"] = 120
```



PANDAS DATA CLEANING

Removing Rows

Another way of handling wrong data is to remove the rows that contains wrong data.

This way you do not have to find out what to replace them with, and there is a good chance you do not need them to do your analyses.

```
for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.drop(x, inplace = True)
```

PANDAS DATA CLEANING

Discovering Duplicates

Duplicate rows are rows that have been registered more than one time.

11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/12'	100	120	250.7

PANDAS DATA CLEANING

Discovering Duplicates

To discover duplicates, we can use the `uplicated()` method. The `uplicated()` method returns a Boolean values for each row

Returns True for every row that is a duplicate, otherwise False

```
print(df.duplicated())
```



PANDAS DATA CLEANING

Removing Duplicates

To remove duplicates, use the `drop_duplicates()` method.

```
df.drop_duplicates(inplace = True)
```


DATA CORRELATION

Finding Relationships

A great aspect of the Pandas module is the `corr()` method.

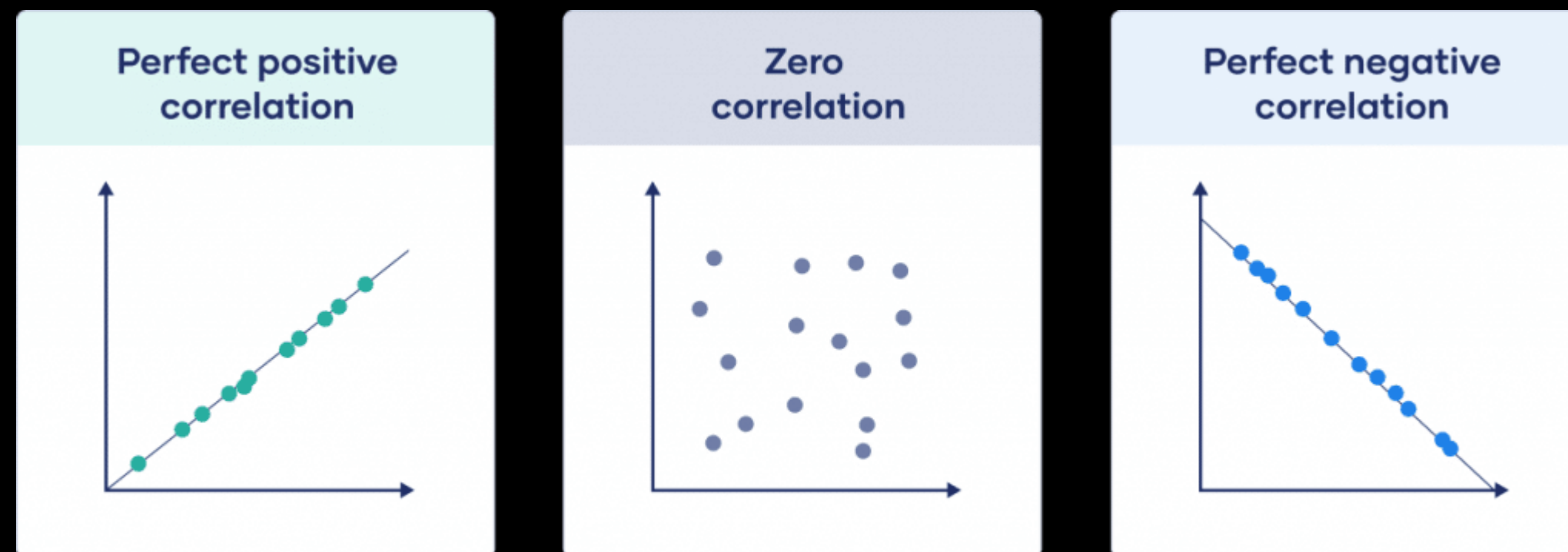
The `corr()` method calculates the relationship between each column in your data set.

```
df.corr()
```

	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922721
Pulse	-0.155408	1.000000	0.786535	0.025120
Maxpulse	0.009403	0.786535	1.000000	0.203814
Calories	0.922721	0.025120	0.203814	1.000000

DATA CORRELATION

In statistics, a perfect positive correlation is represented by the correlation coefficient value $+1.0$, while 0 indicates no correlation, and -1.0 indicates a perfect inverse (negative) correlation.





DATA CORRELATION

Perfect Correlation

We can see that "Duration" and "Duration" got the number 1.000000, which makes sense, each column always has a perfect relationship with itself.



DATA CORRELATION

Good Correlation

"Duration" and "Calories" got a 0.922721 correlation, which is a very good correlation, and we can predict that the longer you work out, the more calories you burn, and the other way around: if you burned a lot of calories, you probably had a long work out.



DATA CORRELATION

Bad Correlation

"Duration" and "Maxpulse" got a 0.009403 correlation, which is a very bad correlation, meaning that we can not predict the max pulse by just looking at the duration of the work out, and vice versa.



MERGING DATA

Often, you need to combine data from different sources. Pandas supports various types of joins, such as `merge()`, `concat()`, and `join()`. You can merge data based on one or more keys

```
# Example: Merging two dataframes
df1 = pd.read_csv('data1.csv')
df2 = pd.read_csv('data2.csv')

# Merging on a common column
merged_df = pd.merge(df1, df2, on='common_column', how='inner') # Inner
```



PLOTTING

Pandas uses the `plot()` method to create diagrams. We can use Pyplot, a submodule of the Matplotlib library to visualize the diagram on the screen.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')

df.plot()

plt.show()
```


DATA CORRELATION

Scatter Plot

Specify that you want a scatter plot with the kind argument
`kind = 'scatter'`

A scatter plot needs an x- and a y-axis.

Include the x and y arguments like this

`x = 'Duration', y = 'Calories'`

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')

df.plot(kind = 'scatter', x = 'Duration', y = 'Calories')

plt.show()
```

DATA CORRELATION

Scatter Plot

Let's create another scatterplot, where there is a bad relationship between the columns, like "Duration" and "Maxpulse", with the correlation 0.009403:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')

df.plot(kind = 'scatter', x = 'Duration', y = 'Maxpulse')

plt.show()
```


DATA CORRELATION

Histogram

A histogram needs only one column. A histogram shows us the frequency of each interval, e.g. how many workouts lasted between 50 and 60 minutes?

```
df["Duration"].plot(kind = 'hist')
```

DATA CORRELATION

Histogram

A histogram needs only one column. A histogram shows us the frequency of each interval, e.g. how many workouts lasted between 50 and 60 minutes?

```
df["Duration"].plot(kind = 'hist')
```

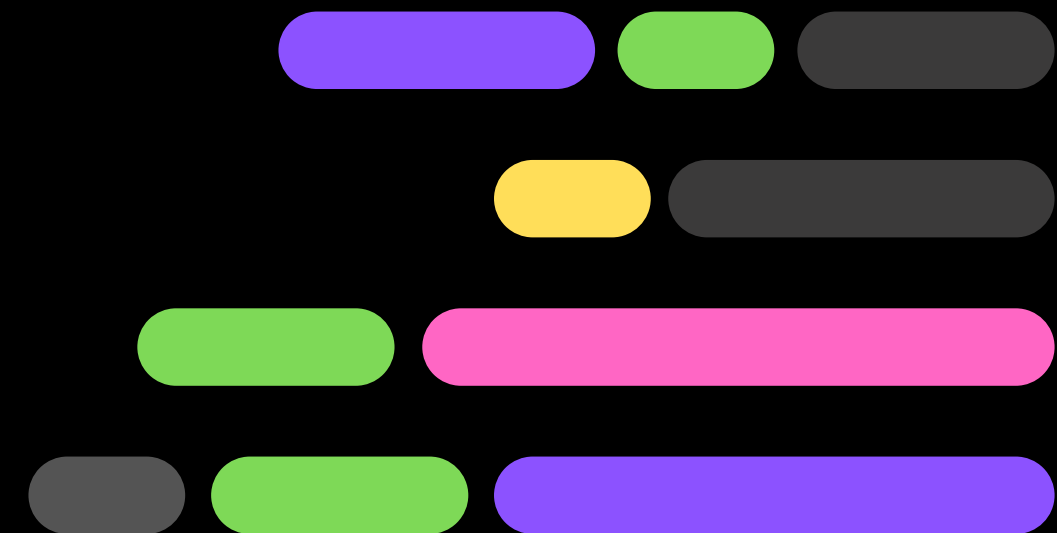
LAB 3

CRIME RATE



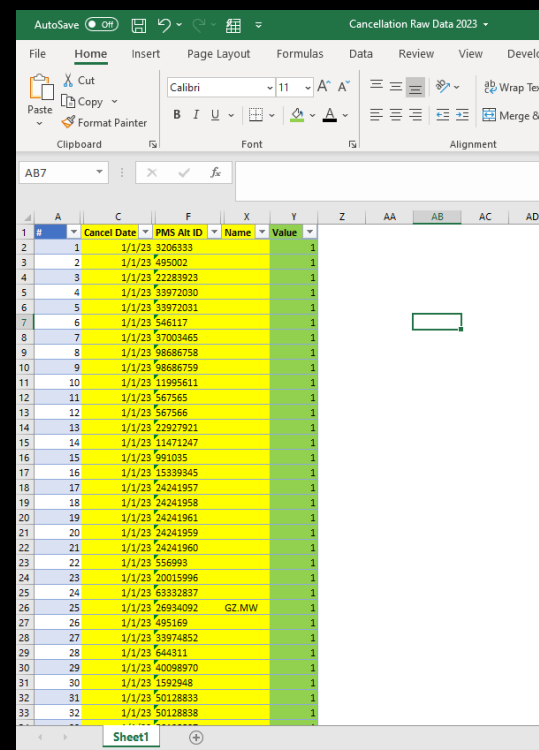
DATA

VISUALIZATION



INTRODUCTION

Data visualization plays a key role in transforming raw data into actionable information, helping decision-makers see patterns, trends, and outliers quickly.





INTRODUCTION

One of the key components to Netflix's success is the ability to leverage its data to inform decision-making. The company has invested heavily in the development of its own data visualization tool to provide relevant information, in real-time





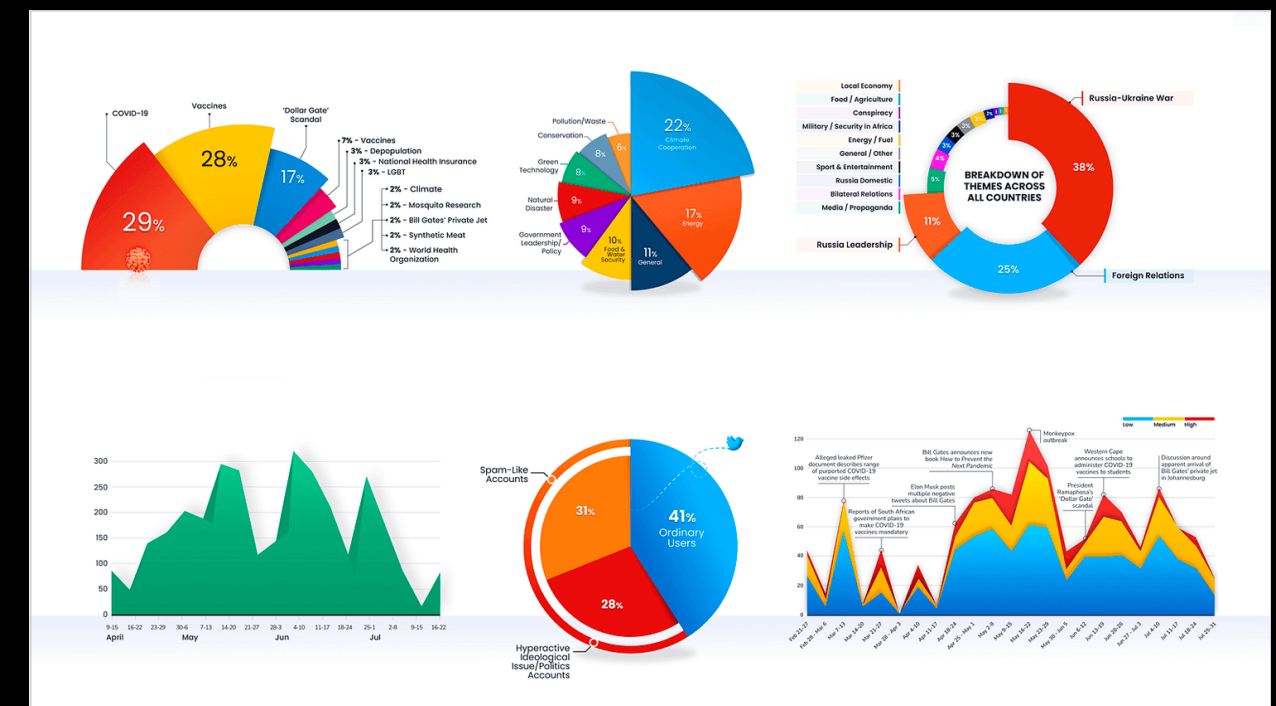
INTRODUCTION

The future of data visualization involves integrating AI and machine learning to automate and enhance visual analytics. Advanced visual tools will enable predictive analytics and real-time insights,



INTRODUCTION

Various techniques are used in data visualization, such as bar charts, line graphs, scatter plots, heat maps, and dashboards. Each technique serves different purposes depending on the type of data and the insights needed.



INTRODUCTION

Matplotlib is a powerful plotting library in Python used for creating static, animated, and interactive visualizations.





INTRODUCTION

Key Features:

- Versatility
- Customization
- Integration with Numpy
- Publication Quality
- Extensible
- Cross-Platform
- Interactive Plot

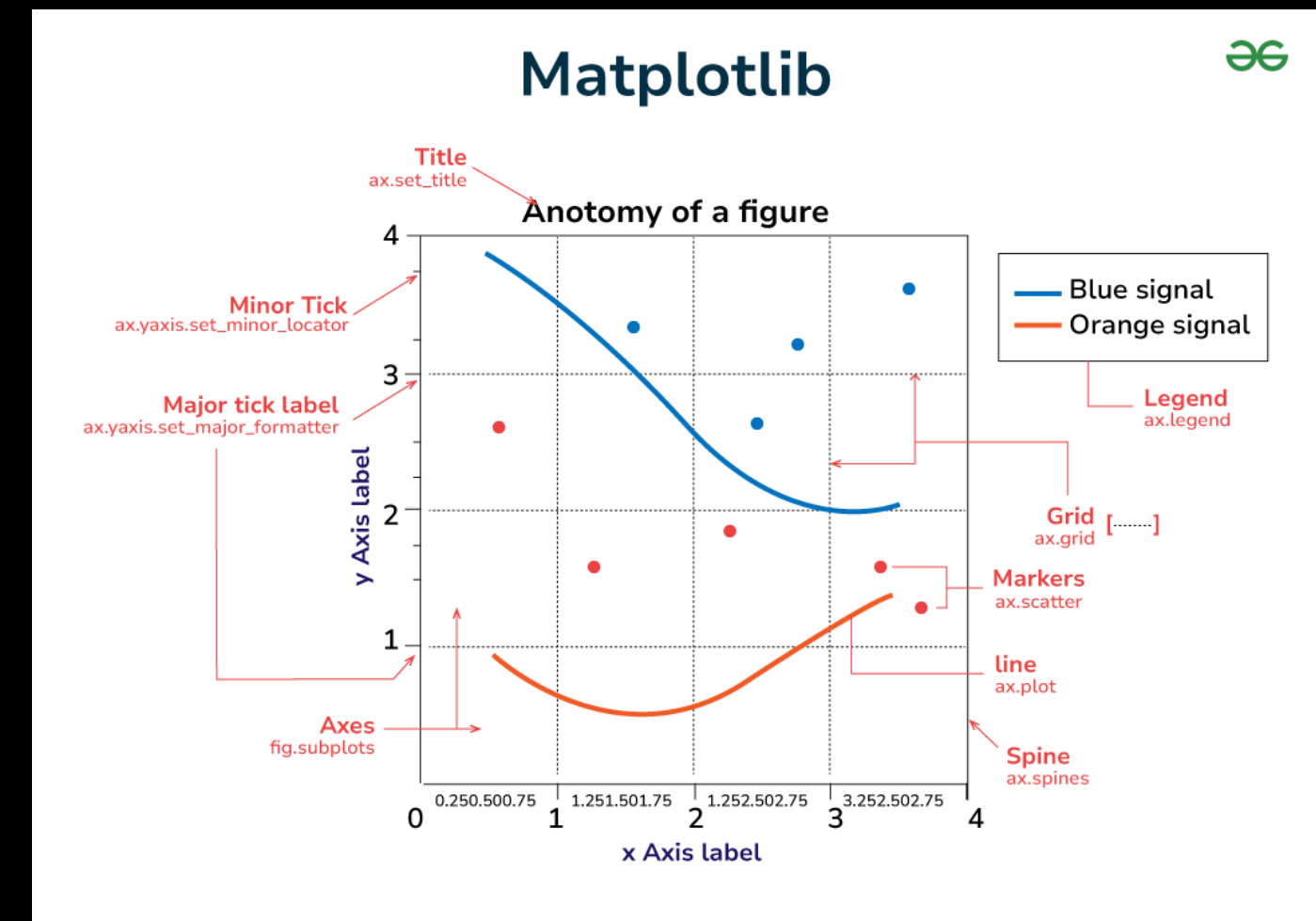


INTRODUCTION

- **Purpose:** Matplotlib is a widely-used library for static, animated, and interactive plots.
- **Features:** It's highly customizable but can require a lot of manual effort to create more complex plots.
- **Common Use:** Often used for creating simple 2D plots, like line plots, bar charts, histograms, etc.
- **Strength:** It's highly flexible, supports detailed customizations, and forms the base of other visualization libraries like Seaborn and Pandas plotting functions.

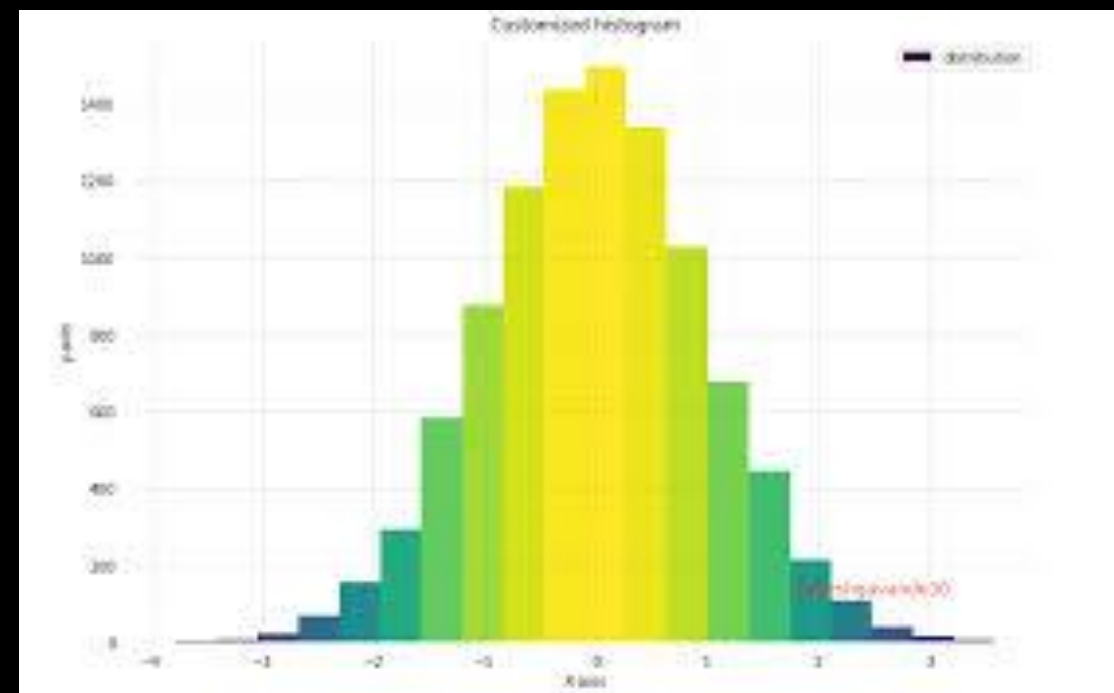
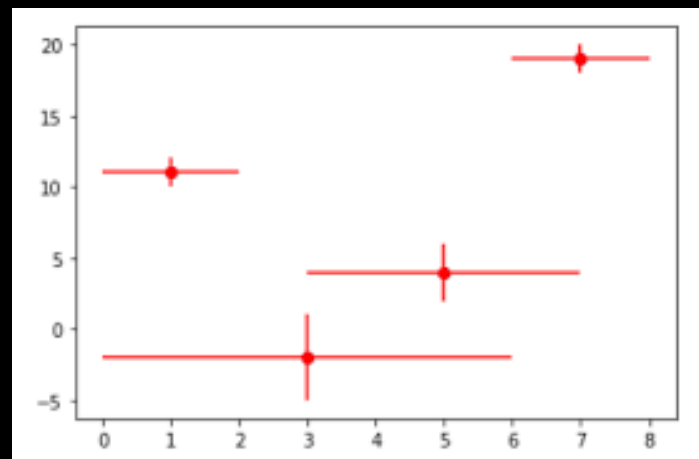
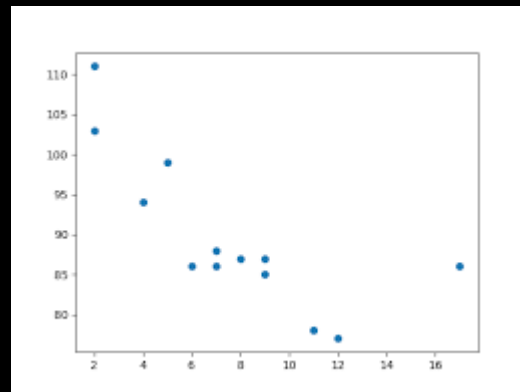
INTRODUCTION

In Matplotlib, a **figure** is the top-level container that holds all the elements of a plot.



INTRODUCTION

Different Types of Plot in Matplotlib





INTRODUCTION

Most of the Matplotlib utilities lies under the `pyplot` submodule, and are usually imported under the `plt` alias:

```
import matplotlib.pyplot as plt
```

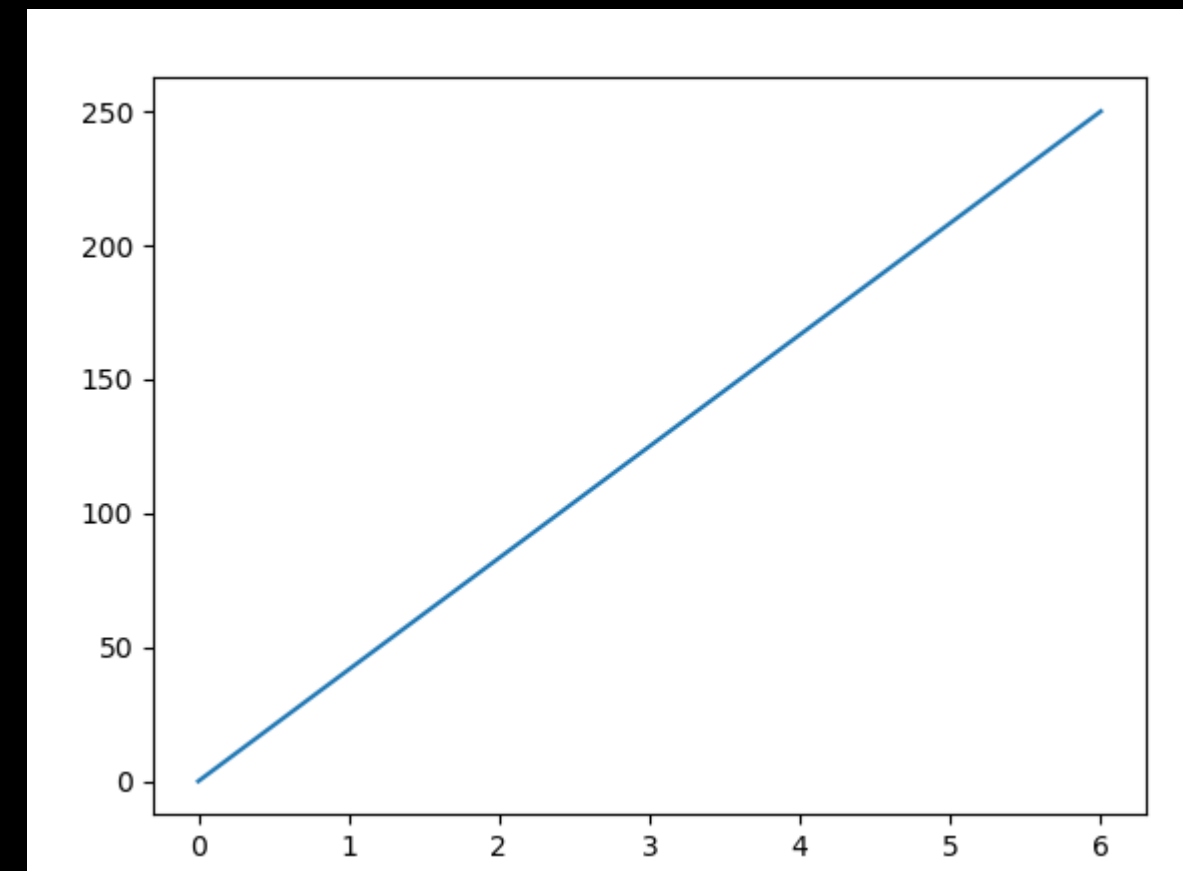

INTRODUCTION

Getting Started:

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

plt.plot(xpoints, ypoints)
plt.show()
```



INTRODUCTION

By default, the `plot()` function draws a line from point to point. The function takes parameters for specifying points in the diagram.

- Parameter 1 is an array containing the points on the x-axis.
- Parameter 2 is an array containing the points on the y-axis.

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

plt.plot(xpoints, ypoints)
plt.show()
```

MATPLOTLIB

With Pyplot, you can use the `xlabel()` and `ylabel()` functions to set a label for the x- and y-axis.

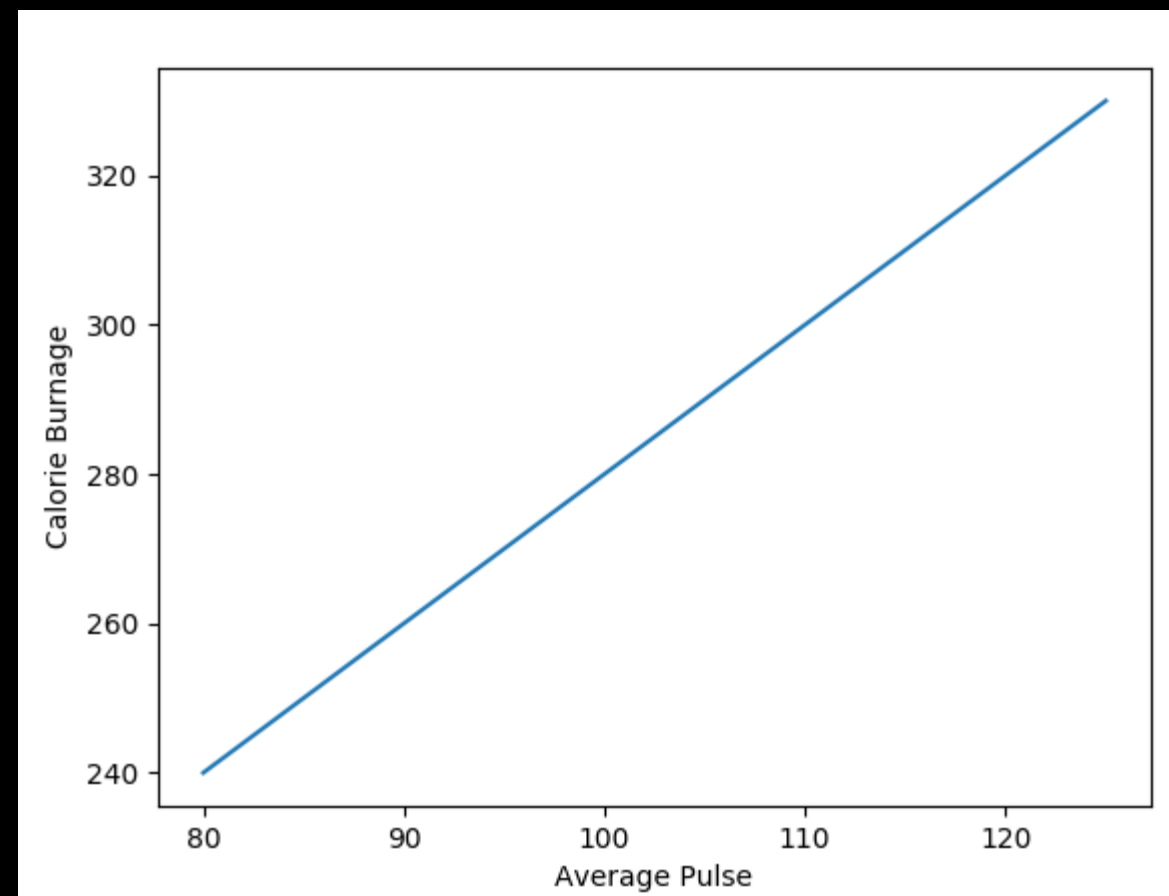
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```



MATPLOTLIB

With Pyplot, you can use the `title()` function to set a title for the plot.

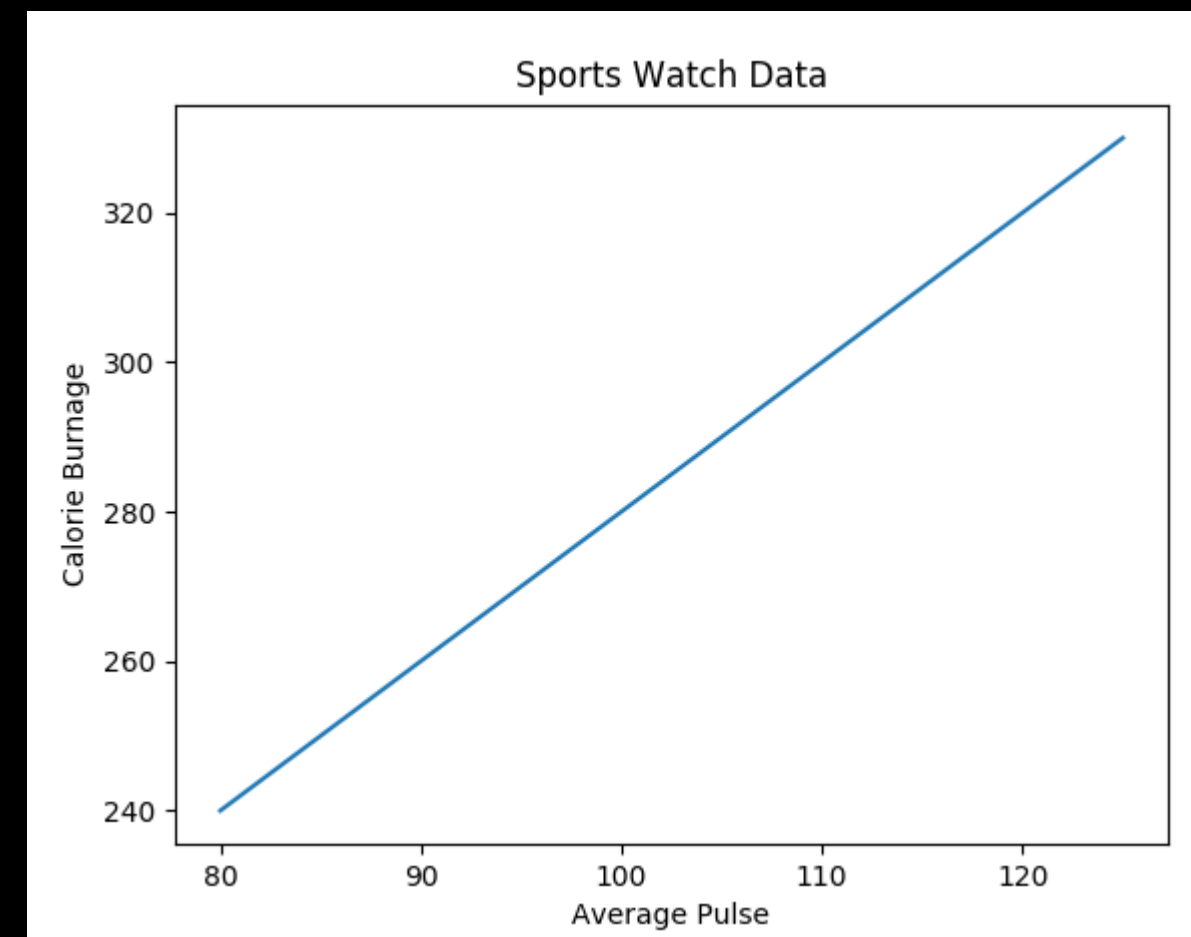
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```





MATPLOTLIB

The `subplot()` function takes three arguments that describes the layout of the figure. The layout is organized in rows and columns, which are represented by the first and second argument. The third argument represents the index of the current plot.

```
plt.subplot(1, 2, 1)  
#the figure has 1 row, 2 columns, and this plot is the first plot.
```


MATPLOTLIB

if we want a figure with 2 rows and 1 column (meaning that the two plots will be displayed on top of each other instead of side-by-side)

```
import matplotlib.pyplot as plt
import numpy as np

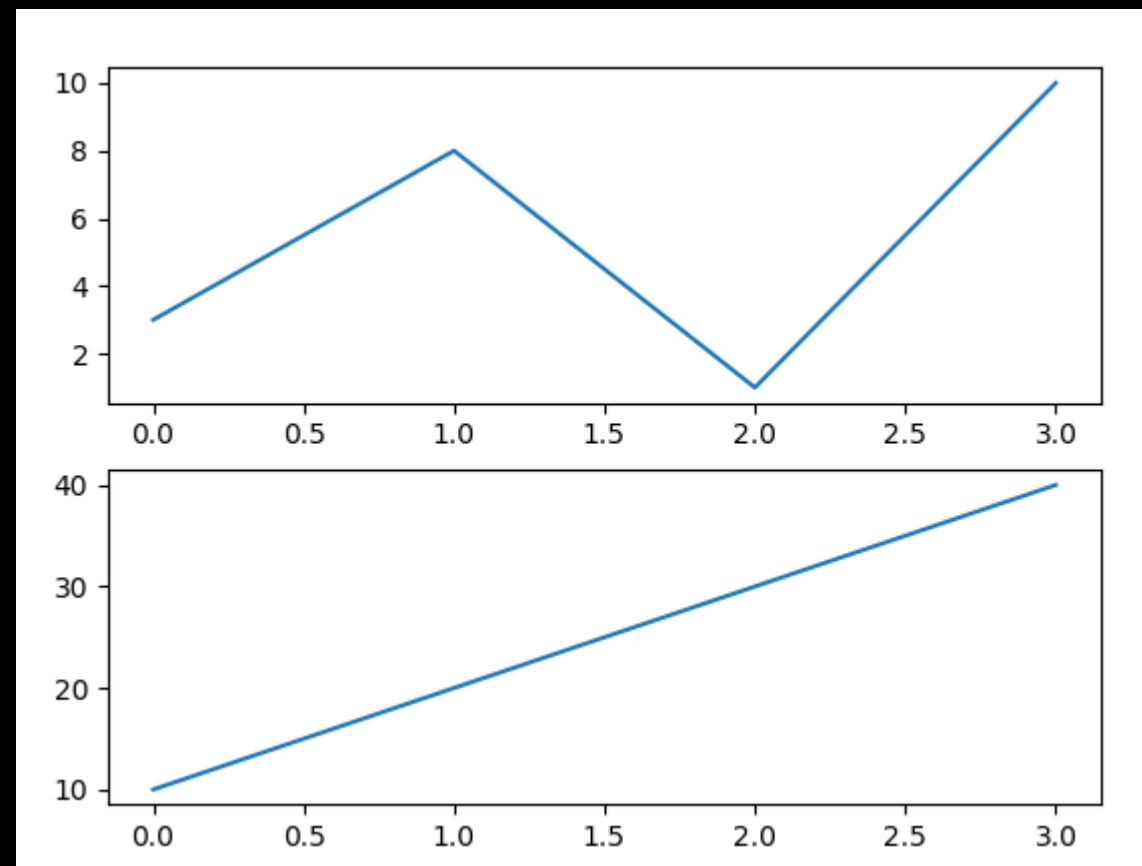
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 1, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 1, 2)
plt.plot(x,y)

plt.show()
```



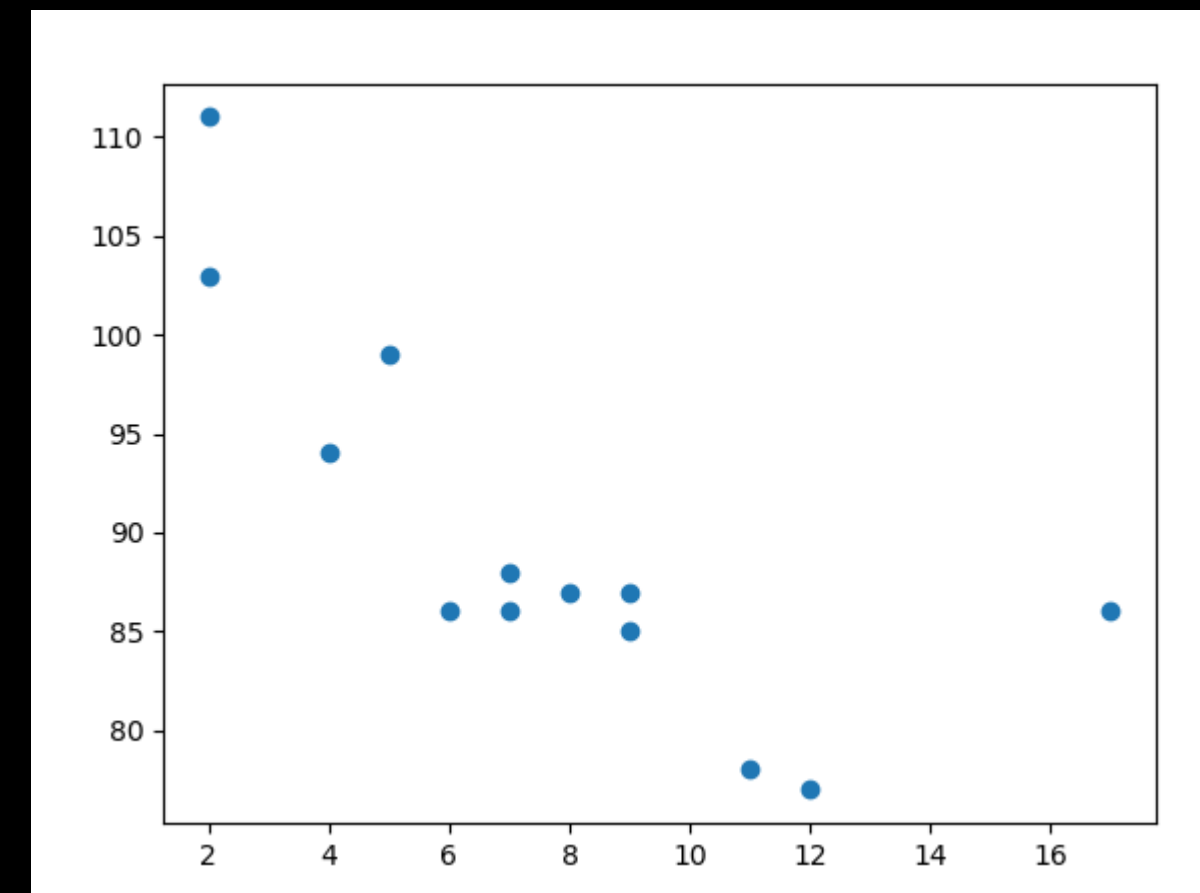
MATPLOTLIB

With Pyplot, you can use the `scatter()` function to draw a scatter plot.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y)
plt.show()
```



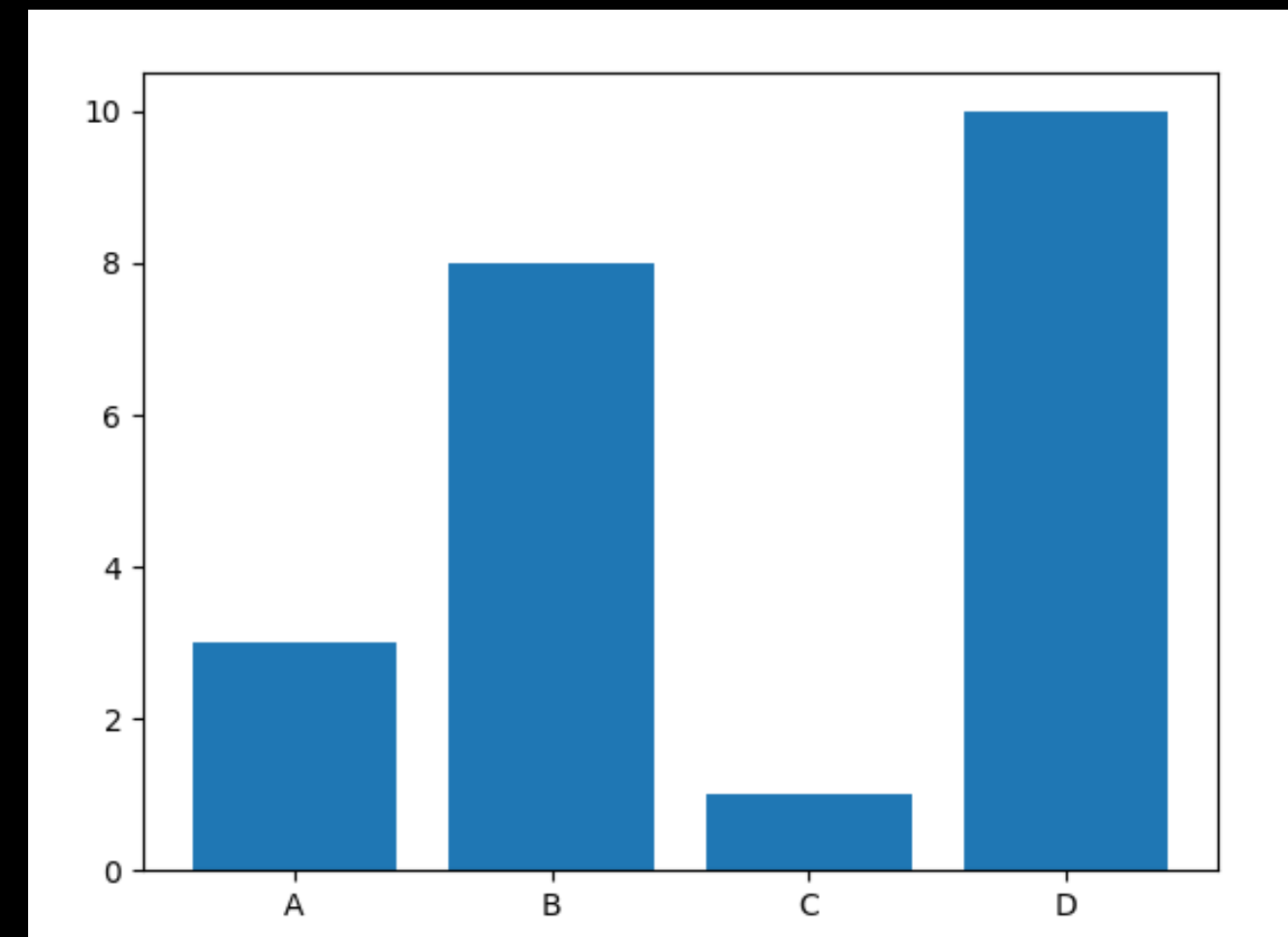
MATPLOTLIB

With Pyplot, you can use the `bar()` function to draw bar graphs

```
import matplotlib.pyplot as plt
import numpy as np

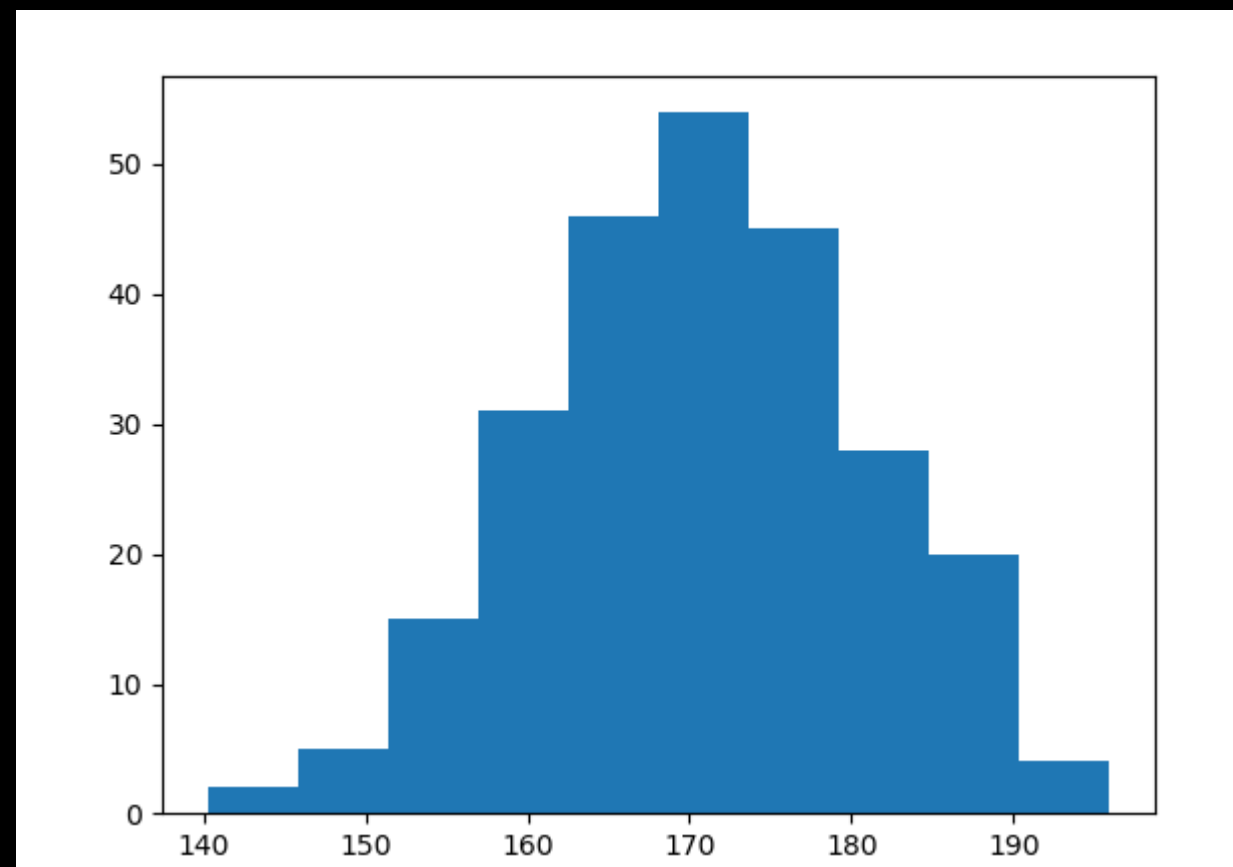
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```



MATPLOTLIB

A **histogram** is a graph showing frequency distributions.
It is a graph showing the number of observations within each given interval.





MATPLOTLIB

In Matplotlib, we use the `hist()` function to create histograms. The `hist()` function will use an array of numbers to create a histogram, the array is sent into the function as an argument.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x)
plt.show()
```

MATPLOTLIB

With Pyplot, you can use the `pie()` function to draw pie charts

```
import matplotlib.pyplot as plt
import numpy as np

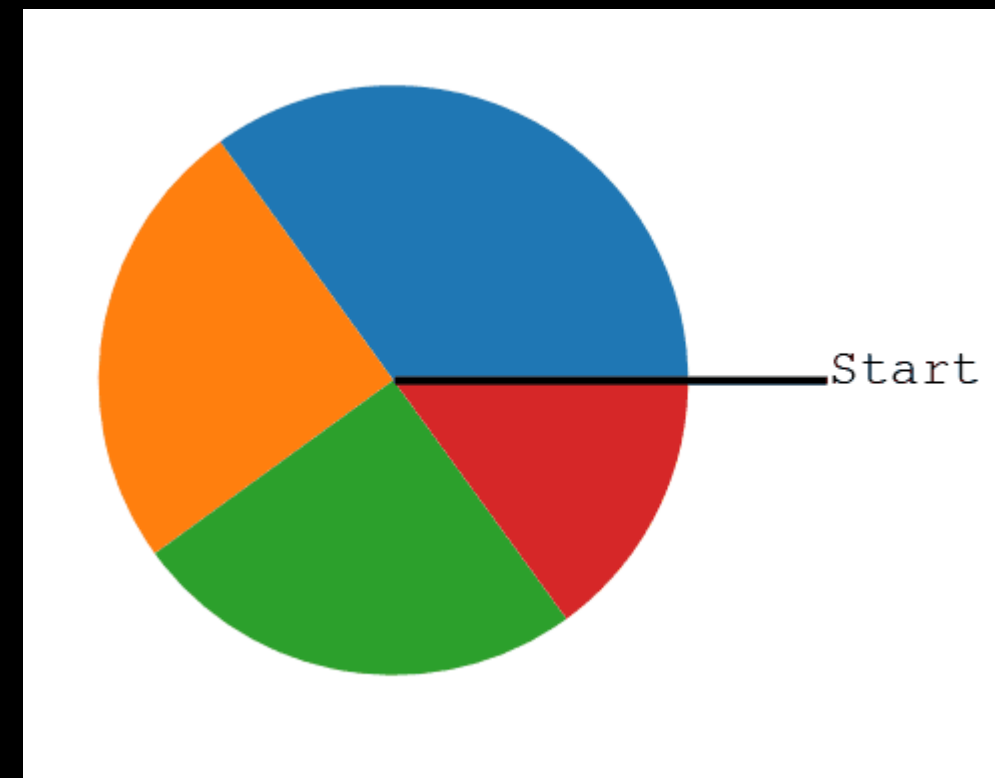
y = np.array([35, 25, 25, 15])

plt.pie(y)
plt.show()
```



MATPLOTLIB

As you can see the pie chart draws one piece (called a wedge) for each value in the array (in this case `[35, 25, 25, 15]`).
By default the plotting of the first wedge starts from the x-axis and moves counterclockwise:



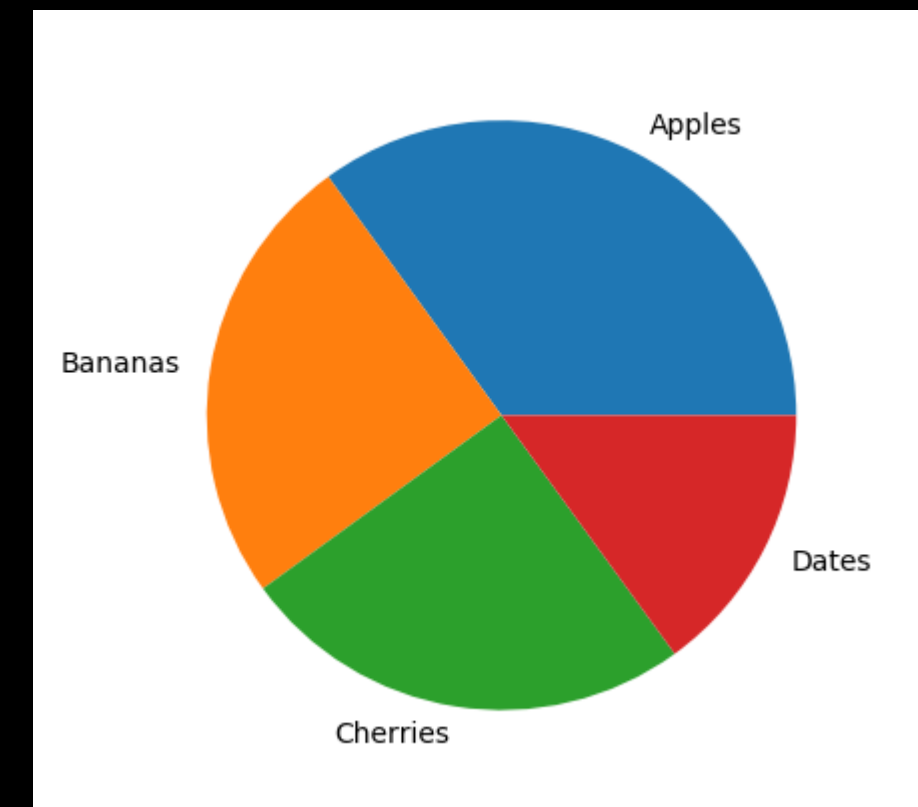
MATPLOTLIB

Add labels to the pie chart with the labels parameter.
The labels parameter must be an array with one label for each wedge

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.show()
```



The background of the slide is a dark blue/black field filled with out-of-focus, multi-colored text that resembles computer code. The colors include yellow, green, orange, and pink. A central black rectangle with a pink border contains the title text.

LAB 4

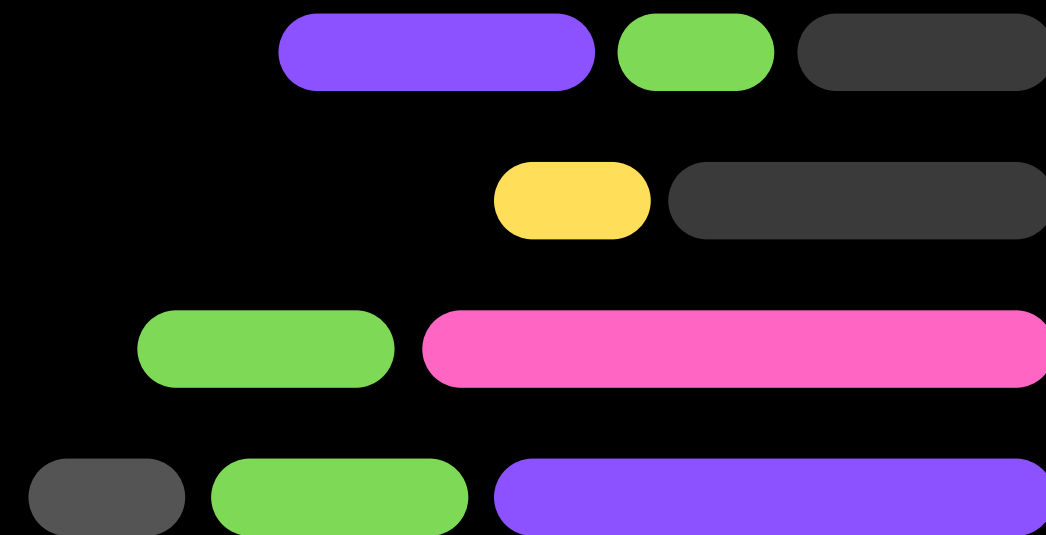
CRIME RATE

VISUALIZATION



PYTHON

SEABORN



SEABORN

It is built on top matplotlib library and is also closely integrated with the data structures from pandas.



seaborn

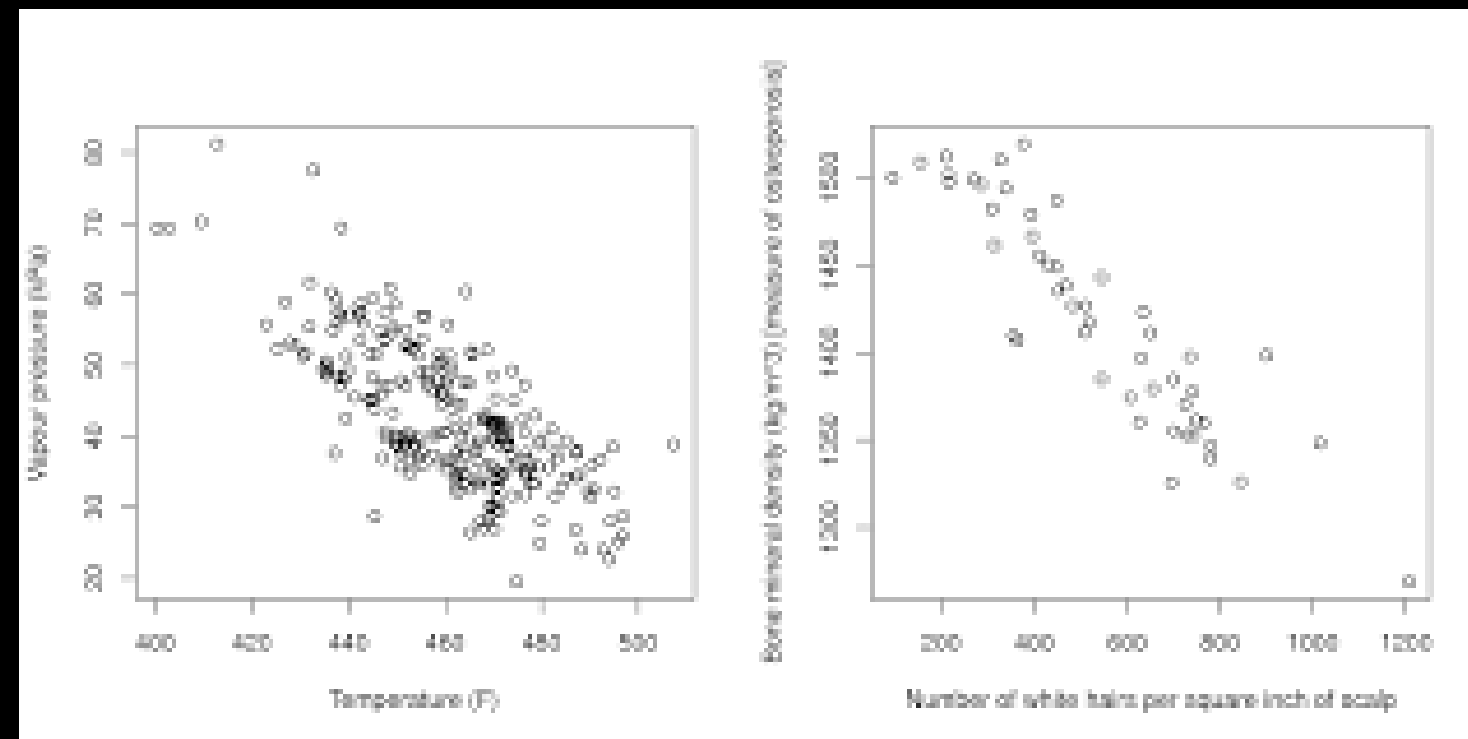


PLOT IN SEABORN

Plots are basically used for visualizing the relationship between variables. Those variables can be either completely numerical or a category like a group, class, or div

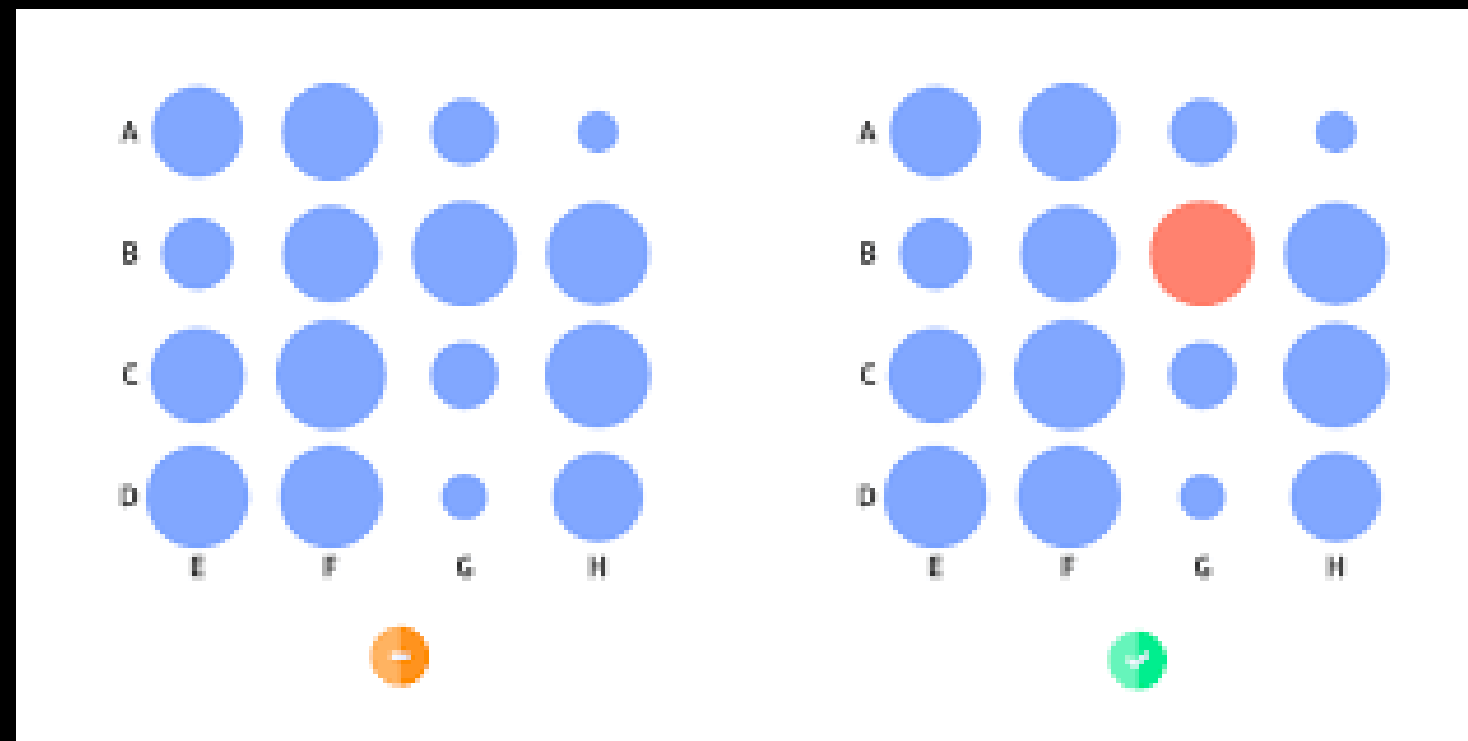
PLOT IN SEABORN

Relational plots: This plot is used to understand the relation between two variables.



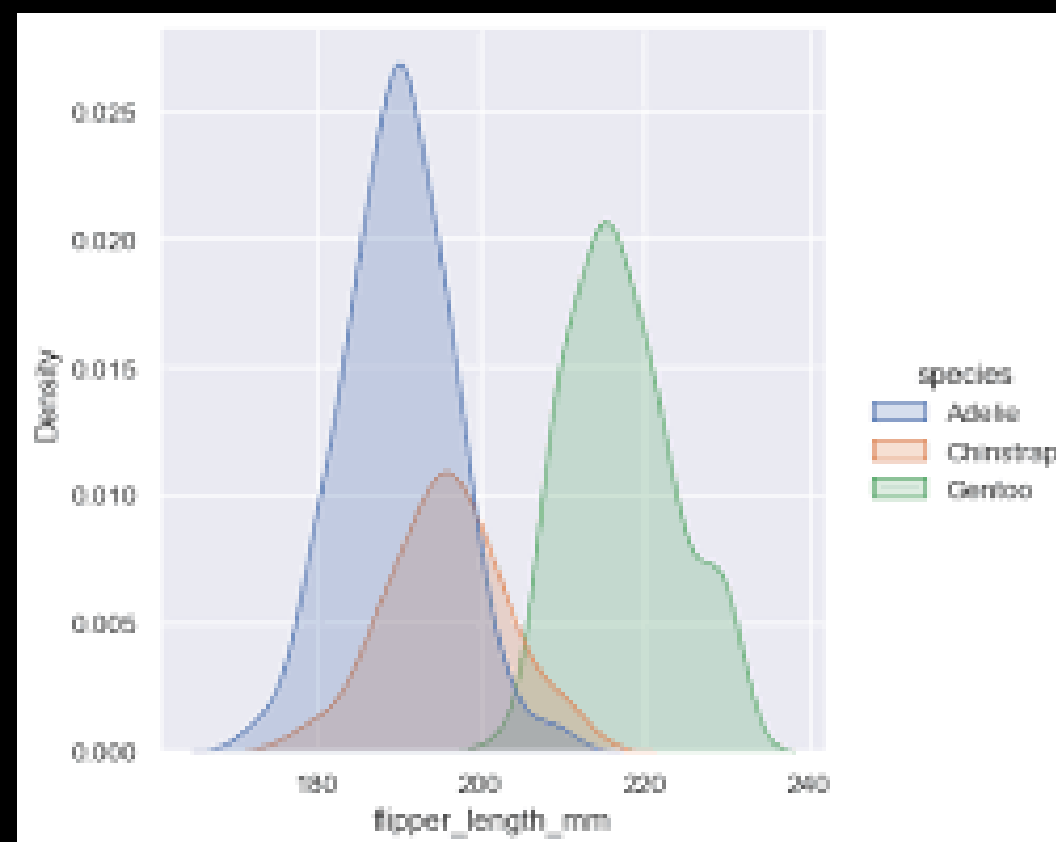
PLOT IN SEABORN

Categorical plots: This plot deals with categorical variables and how they can be visualized.



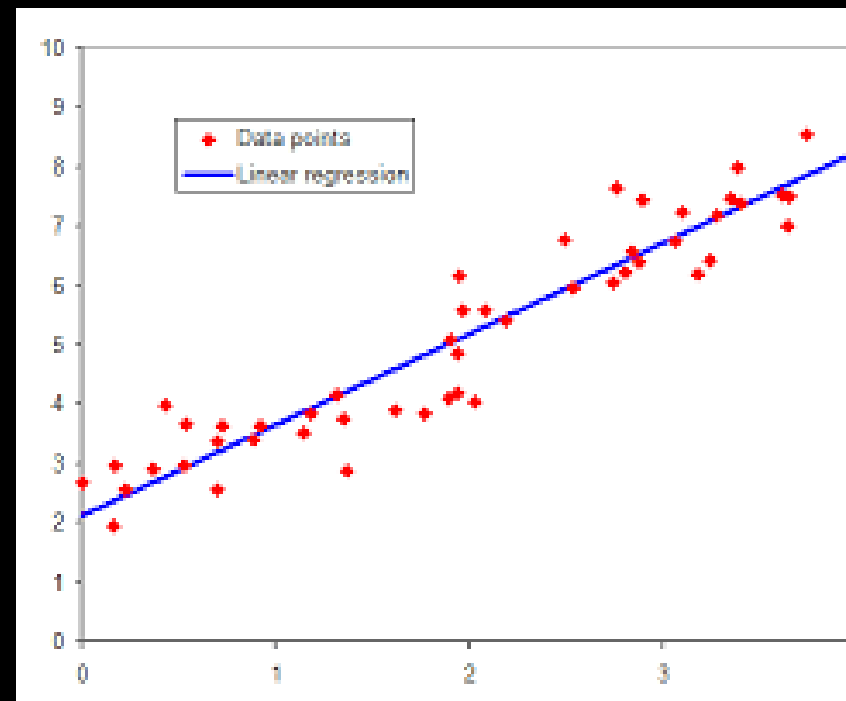
PLOT IN SEABORN

Distribution plots: This plot is used for examining univariate and bivariate distributions



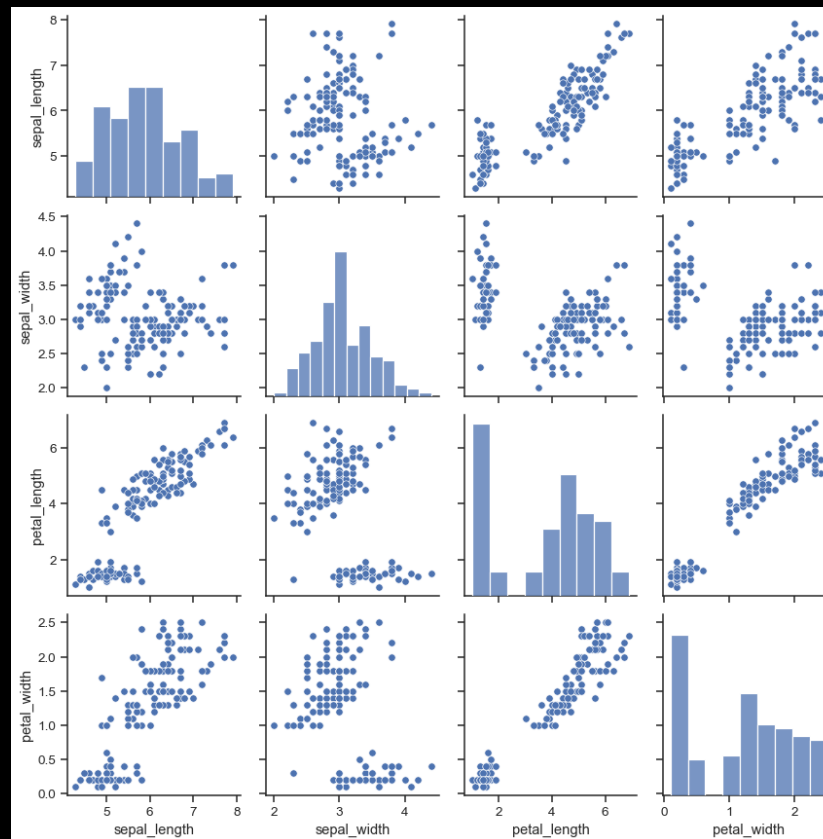
PLOT IN SEABORN

Regression plots: The regression plots in Seaborn are primarily intended to add a visual guide that helps to emphasize patterns in a dataset during exploratory data analyses.



PLOT IN SEABORN

Multi-plot grids: It is a useful approach to draw multiple instances of the same plot on different subsets of the dataset.



A terminal window with a dark background and a white border. The title bar at the top contains three colored circles (red, green, yellow). A thick pink horizontal bar is positioned above the main content area. The word "INSTALLATION" is displayed in a large, white, monospaced font within a white-bordered box.

INSTALLATION

For Python environment :

```
pip install seaborn
```



BASIC

- You can use dataset from seaborn by using `sns.load_dataset()`
- Plotting with
`Sns.scatterplot()`
`Sns.lineplot()`
`Sns.histplot()`

```
tips = sns.load_dataset('tips')  
sns.scatterplot(x='total_bill', y='tip', data=tips)  
plt.show()
```



CUSTOMIZING

- You enhance visualizations with color, themes, labels, and titles

```
sns.set_style('darkgrid')
sns.scatterplot(x='total_bill', y='tip', data=tips, hue='sex', palette='coolwarm')
plt.title('Total Bill vs Tip')
plt.show()
```



ADVANCED PLOTS

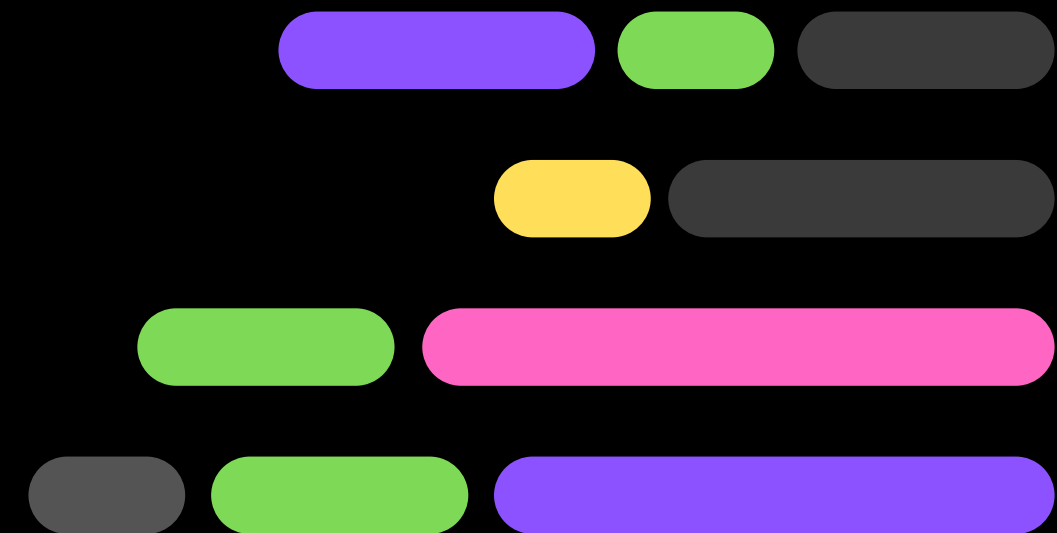
- `pairplot()`: Show correlations across multiple variables.
- `heatmap()`: Visualize relationships in a matrix format.
- `violinplot()`: Combination of boxplot and KDE.

```
sns.pairplot(tips, hue='sex')  
plt.show()  
  
sns.heatmap(tips.corr(), annot=True)  
plt.show()
```




PYTHON

PLOTLY





PLOTLY

Plotly is an open-source graphing library that makes it easy to create interactive and visually appealing plots.





PLOTLY

- **Purpose:** Plotly is a library for creating interactive, web-based visualizations.
- **Features:** It allows for interactive, zoomable, and clickable plots that can be embedded in web applications or dashboards.
- **Common Use:** Ideal for creating interactive charts, maps, and dashboards (especially when combined with Dash for web-based apps).
- **Strength:** Offers 3D plotting and interactive graphs out-of-the-box.



PLOTLY

Advantage of using Plotly:

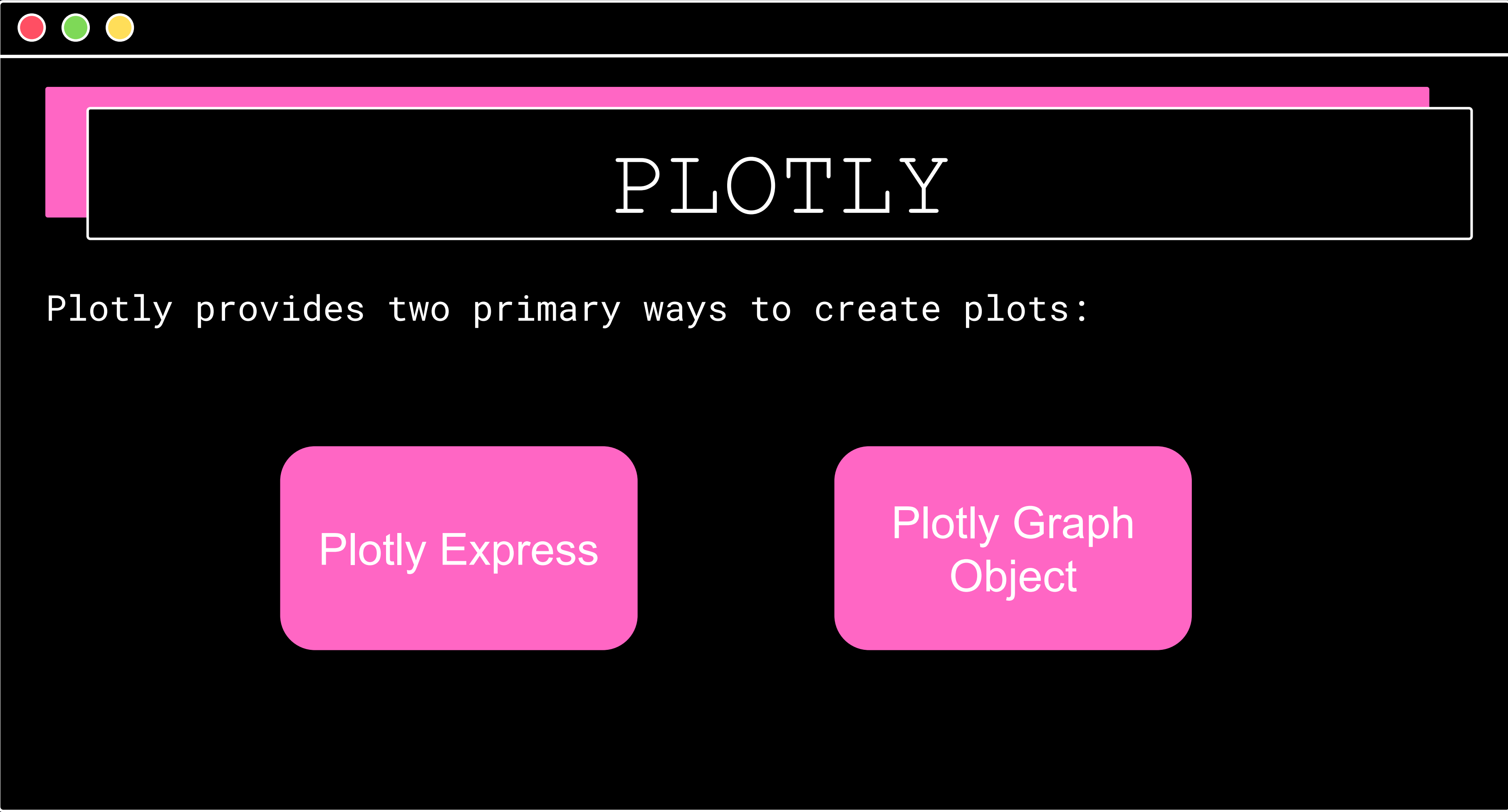
- **Interactivity**: Plotly allows for highly interactive charts,
- Aesthetics: The plots created with Plotly are visually appealing
- **Easy Integration**: Plotly integrates smoothly with many frameworks like Dash.
- **Wide Range of Chart Types**: From simple line plots to complex 3D surface plots and geospatial visualizations
- **Web-based Sharing**: Plotly charts can be easily embedded in websites or shared through Plotly's cloud service,



PLOTLY

You can install Plotly using Python's package manager, pip. To install it, run the following command in your terminal or command prompt:

```
pip install plotly
```



PLOTLY

Plotly provides two primary ways to create plots:

Plotly Express

Plotly Graph
Object



PLOTLY

In Plotly, every plot or chart is represented by a Figure object.

A figure consists of two main component

- **Data**: This contains the traces (or series of data) that define what is plotted
- **Layout**: This defines the overall appearance and styling of the figure



PLOTLY

Here, the data contains a line plot created with `Scatter()`, and the layout includes the title and axis labels.

```
import plotly.graph_objects as go

# Create a Figure with data and layout
fig = go.Figure(
    data=[go.Scatter(x=[1, 2, 3], y=[4, 5, 6], mode='lines+markers')],
    layout=go.Layout(title="Basic Line Plot", xaxis_title="X-Axis", yaxis_title="Y-Axis")
)

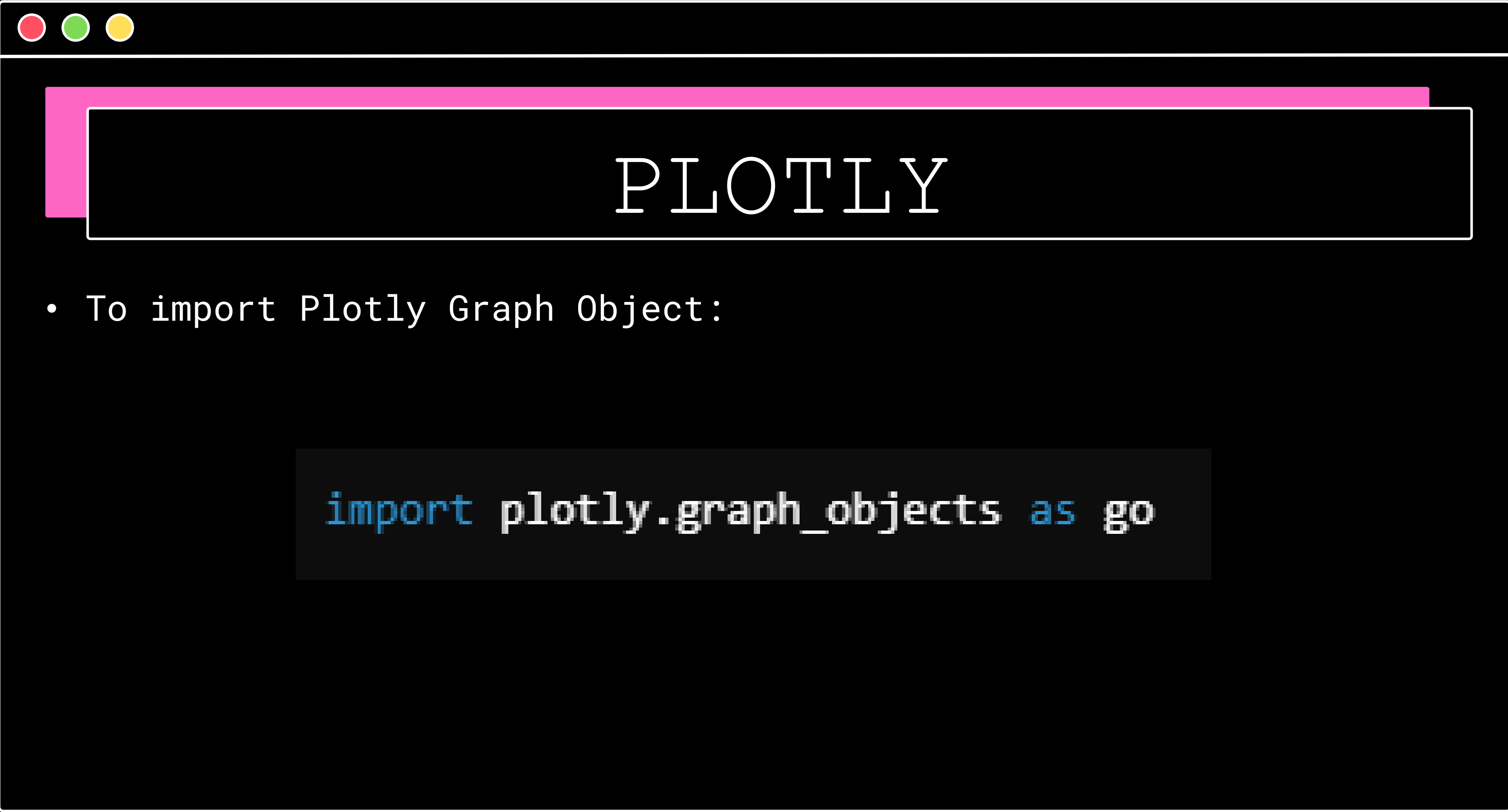
# Show the figure
fig.show()
```



PLOTLY

Plotly Graph Object

- **Low-level API** offering more granular control over every aspect of a plot.
- Ideal for advanced users who need fine-tuned customizations.
- Requires more code to build the same visualizations compared to Plotly Express but offers flexibility in controlling traces, layout, and interactivity.
- Complex figures with multiple subplots or mixed chart types are easier to build.



PLOTLY

- To import Plotly Graph Object:

```
import plotly.graph_objects as go
```



PLOTLY

Plotly Express

- **High-level API** for quick, simple visualizations.
- Best suited for beginners or when you need to create charts rapidly.
- Automatically handles a lot of the layout and styling, requiring fewer lines of code.
- Simplified syntax for creating common charts like scatter plots, bar charts, etc.



PLOTLY

- To import Plotly Express:

```
import plotly.express as px
```



PLOTLY

- Here is an anatomy of a basic plot in Plotly Express:

```
import plotly.express as px

fig = px.plotting_function(
    dataframe,
    x='column-for-xaxis',
    y='column-for-yaxis',
    title='Title For the Plot',
    width=width_in_pixels,
    height=height_in_pixels
)

fig.show()
```



PLOTLY

- To start creating plots, we need a dataset. we're using a **Diamonds dataset**. It contains a nice combination of numeric and categorical features :

```
#STEP 1: OPEN CSV
~~~~~
diamonds = pd.read_csv("diamonds.csv")
diamonds.head()
```


PLOTLY - HISTOGRAM

- A histogram is probably the very first visual people learn. It orders the values of a distribution and puts them into bins. Then, bars are used to represent how many values fall into each bin:

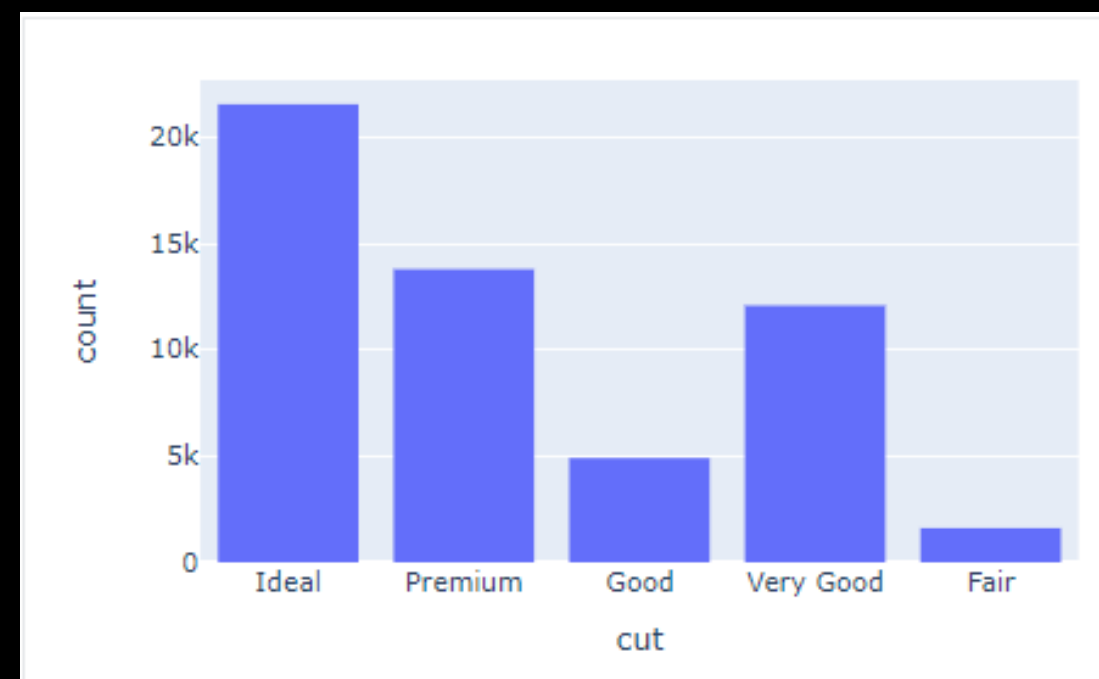
```
fig = px.histogram(  
    diamonds,  
    x="price",  
    title="Histogram of diamond prices",  
    width=600,  
    height=400,  
)
```



PLOTLY – BAR CHARTS

- To analyze categorical data, we turn to bar charts. passing the categorical cut column to the histogram function, we get a type of bar chart called countplot.

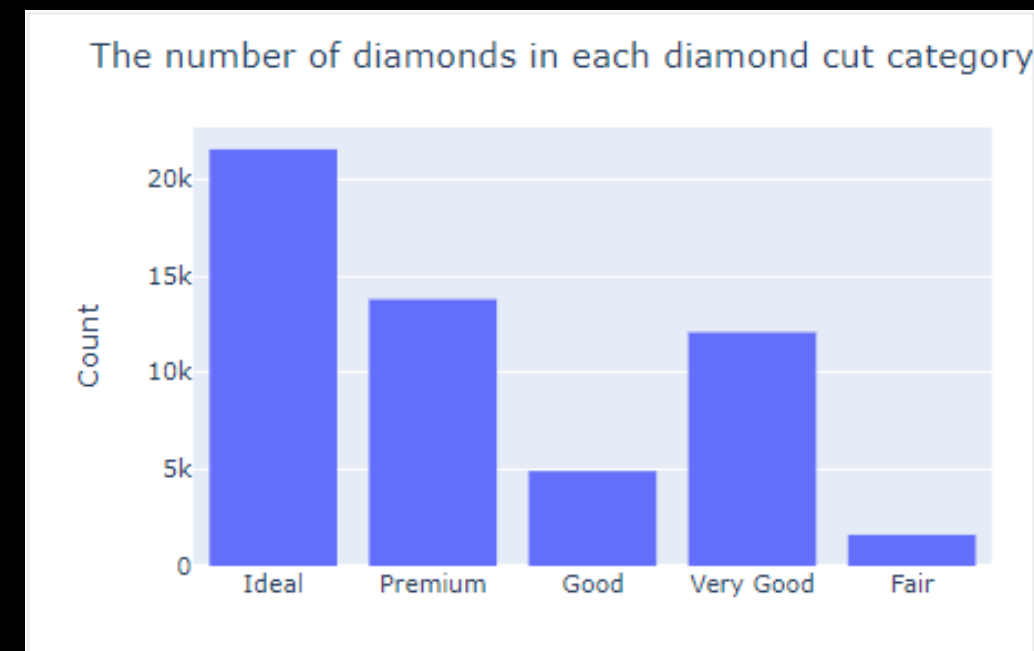
```
fig = px.histogram(diamonds, x="cut")
```



PLOTLY – BAR CHARTS

- we didn't put any labels on the plot! Let's fix that using the `update_layout` function, which can modify many aspects of a figure after it has been created..

```
fig = px.histogram(  
    diamonds,  
    x="price",  
    title="Histogram of diamond prices",  
    width=600,  
    height=400,  
)
```



PLOTLY – BAR CHARTS

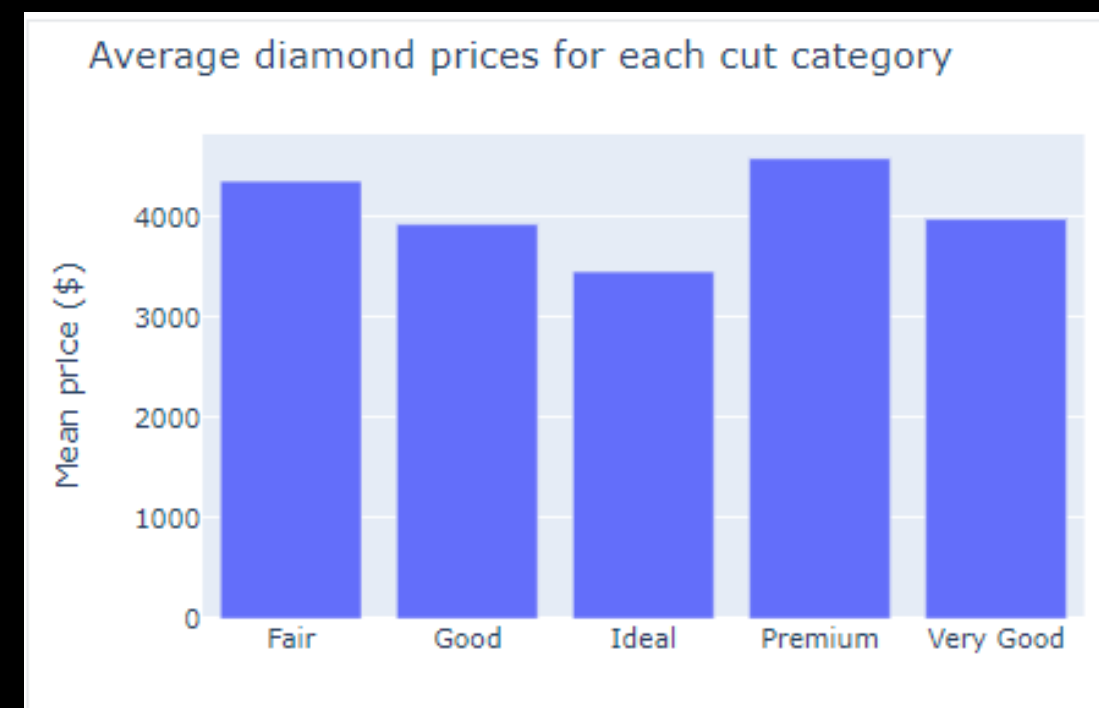
- Now, let's see the mean price for each diamond cut category.
First, we use pandas groupby

```
mean_prices = (
    diamonds.groupby("cut")["price"].mean().reset_index()
)

print(mean_prices)
fig = px.bar(mean_prices, x="cut", y="price")

fig.update_layout(
    title="Average diamond prices for each cut category",
    xaxis_title="",
    yaxis_title="Mean price ($)",
)

fig.show()
```



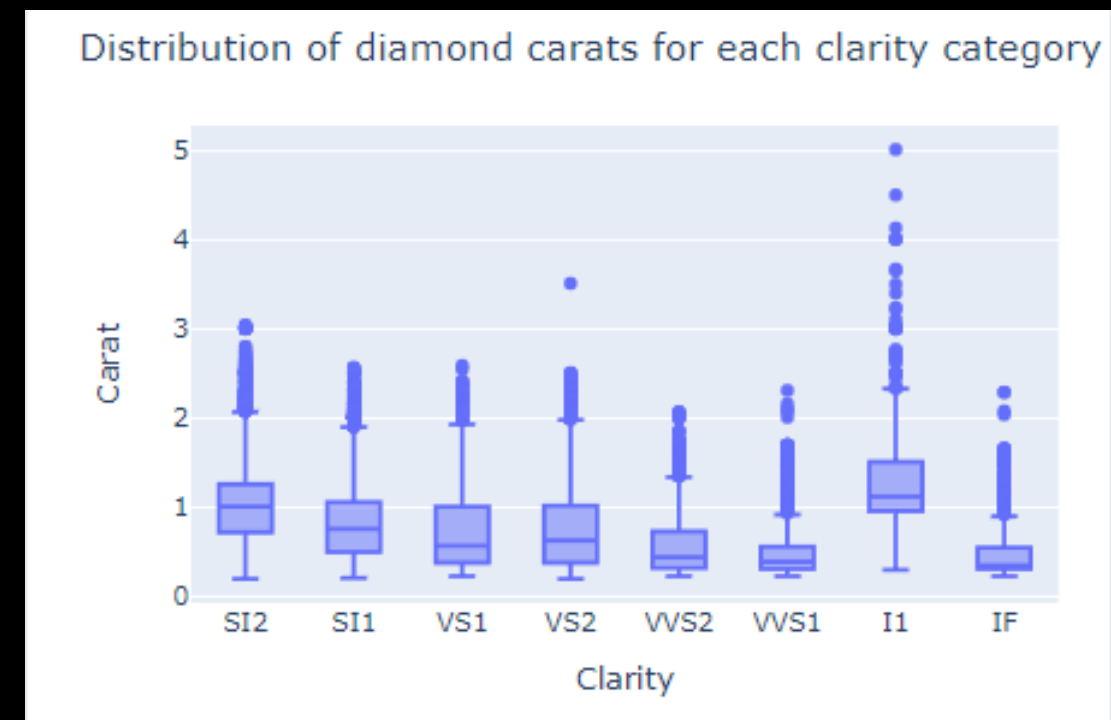
PLOTLY – BOX PLOT

- Another common plot for analyzing distributions is a boxplot, created with `box` in Plotly Express

```
fig = px.box(diamonds, x="clarity", y="carat")

fig.update_layout(
    title="Distribution of diamond carats for each clarity category",
    xaxis_title="Clarity",
    yaxis_title="Carat",
)

fig.show()
```



PLOTLY – VIOLIN PLOT

- Another fun chart to explore distributions is a violin plot. Violin plots are basically box plots with a couple of extra flourishes

```
fig = px.violin(diamonds, x="cut", y="price", box=True)  
fig.show()
```





PLOTLY

- Visualizing relationships between features on Plotly Express

You start to see new patterns and insights when you plot features that are closely related. For example, if we assume that the carat (weight) of the diamond is the main factor determining its price, let's explore that relationship.

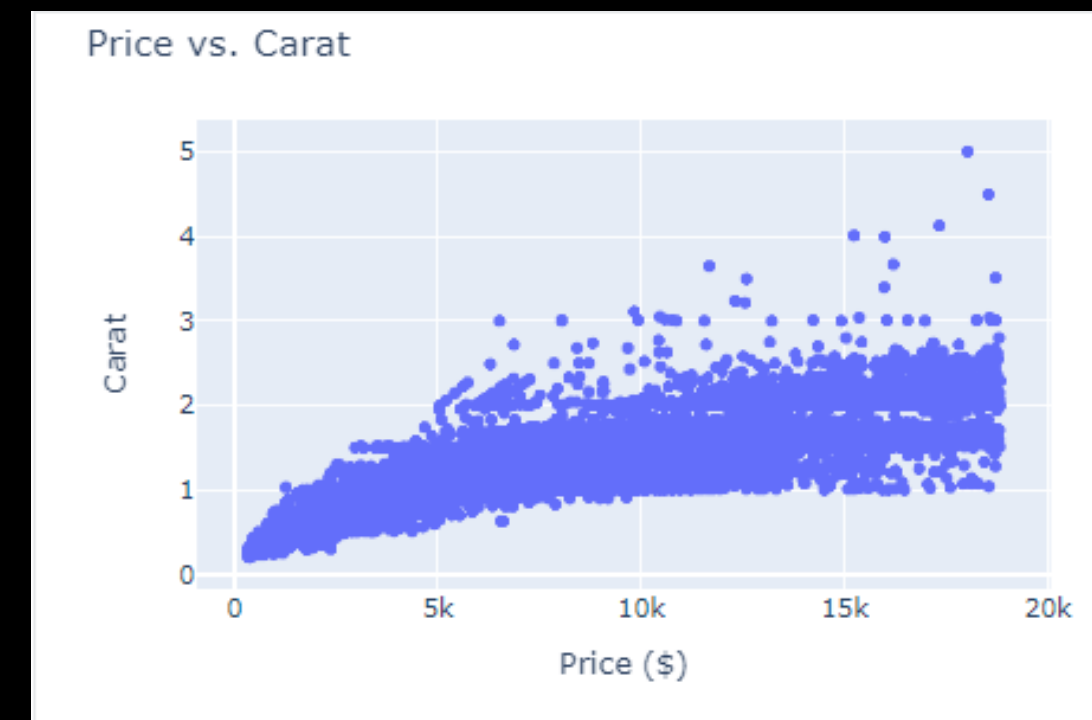
PLOTLY – SCATTERPLOT

- We can use a scatterplot (`px.scatter`) which plots all diamonds in the dataset as dots. The position of the dots is determined by their corresponding price and carat values

```
fig = px.scatter(diamonds, x="price", y="carat")

fig.update_layout(
    title="Price vs. Carat",
    xaxis_title="Price ($)",
    yaxis_title="Carat",
)

fig.show()
```



PLOTLY – SCATTERPLOT

- We can fix the **overplotting** by plotting only ~10% of the dataset, which will be enough to reveal any existing patterns:

```
fig = px.scatter(  
    diamonds.sample(5000), x="price", y="carat"  
)
```




PLOTLY — HEATMAPS

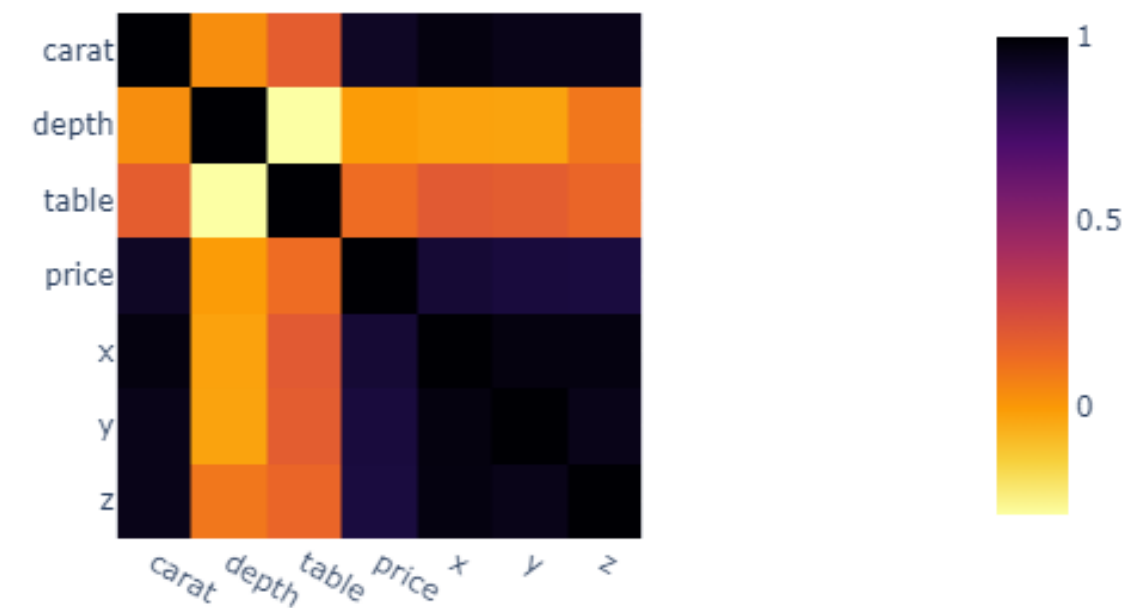
- Heatmaps are great for visualizing complex datasets, such as a large matrix where it is difficult to see patterns from a horde of numbers in cells.
- You can calculate the correlation matrix of any dataset using the `corr` method of pandas, which returns a DataFrame. If you pass it to the `imshow` function of Express, you get a heatmap:

PLOTLY – HEATMAPS

```
import plotly.express as px

# Create heatmap using Plotly Express
fig = px.imshow(
    diamonds.corr(),
    color_continuous_scale="Inferno_r",
)

# Show plot
fig.show()
```





PLOTLY CUSTOMIZATION

- One of the greatest features of the Plotly Express API is the easy customization of plots.
- The `update_layout` function contains many parameters you can use to modify almost any aspect of your charts.

PLOTLY CUSTOMIZATION

Template

- One of the first things to choose is the plot theme. Themes in Plotly are default chart configurations that control almost all components.

```
sample = diamonds.sample(3000)

fig = px.scatter(
    sample, x="price", y="carat", template="ggplot2"
)

fig.show()
```

PLOTLY CUSTOMIZATION

Colors

- We can color each dot in the plot based on which category the diamond belongs to. To do this, you only have to pass cut to color parameter

```
sample = diamonds.sample(3000)

fig = px.scatter(sample, x="price", y="carat", color="cut")

fig.show()
```


PLOTLY CUSTOMIZATION

Marker Size

- By setting the size parameter to carat, we get differently sized dots based on diamond carat:

```
fig = px.scatter(sample, x="price", y="x", size="carat")  
  
fig.show()
```

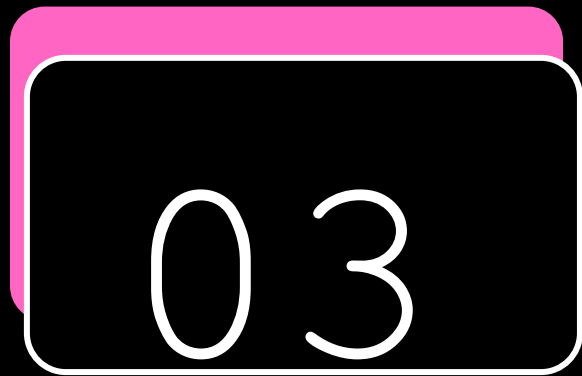

LAB 5

CRIME RATE

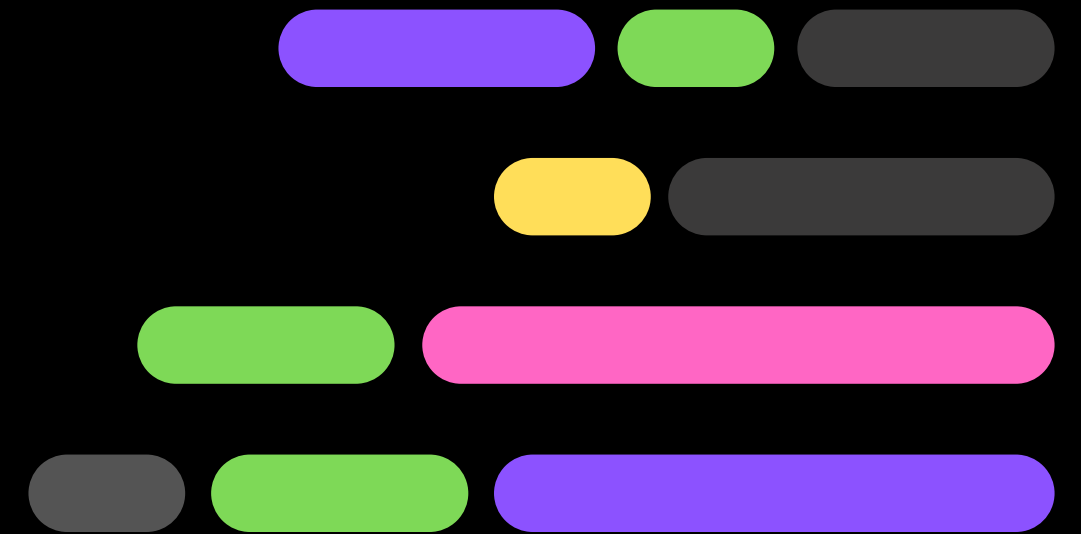
PLOTLY

LAB 6

TITANIC



INTRODUCTION TO AI

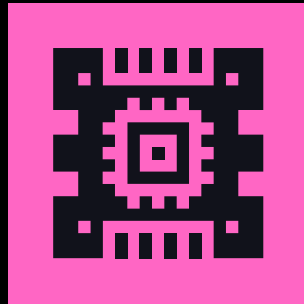


INTRODUCTION

- Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. It encompasses various techniques aimed at enabling computers to perform tasks that typically require human intelligence, such as learning, problem-solving, understanding natural language, and recognizing patterns.

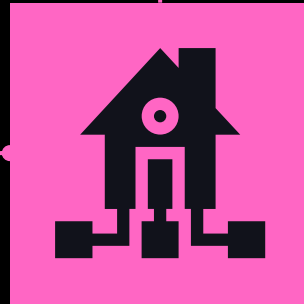


EVOLUTION OF AI



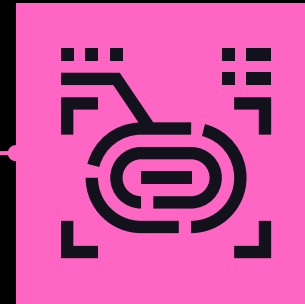
1950-1960

The birth of AI as an academic discipline,



1970-1980

AI faced significant challenges, known as the "AI winter



1990-2000

Revival of interest in AI with advancements in machine learning algorithms



**2010-
Present**

Explosive growth in AI capabilities driven by big data

AI APPROACH – ML

Supervised Learning

Definition

training a model on labeled data where the input data is paired with the correct output.

Process

The model learns to map input data to the correct output

Application

Used in tasks such as image classification, speech recognition, and predicting housing prices.

AI APPROACH – ML

Unsupervised Learning

Definition

The goal is to find hidden patterns or intrinsic structures in the data.

Process

Algorithms cluster data points based on similarities or reduce the dimensionality of the data for further analysis.

Application

Commonly used for customer segmentation, anomaly detection, and recommendation systems.

AI APPROACH – ML

Reinforcement Learning

Definition

involves an agent learning to make decisions in an environment to achieve a specific goal through trial and error.

Process

learns by receiving feedback in the form of rewards or penalties for actions taken.

Application

Used in robotics, game playing (e.g., AlphaGo), and autonomous vehicle navigation.



AI APPROACH – DL

Deep Learning is a subset of Machine Learning that utilizes neural networks with multiple layers to model and extract patterns from complex data.

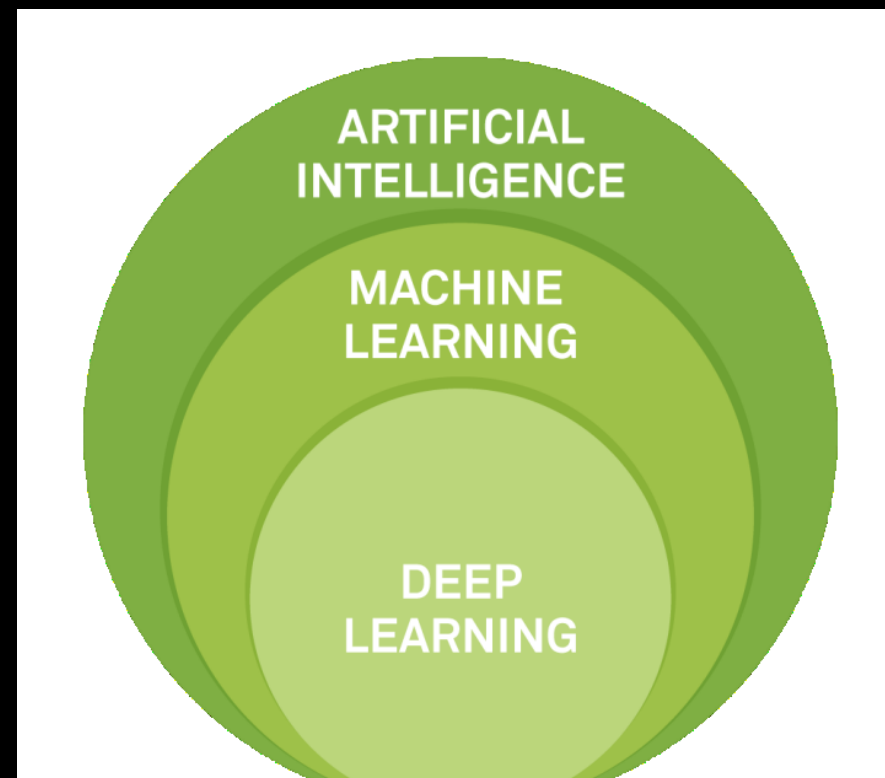
Neural Network

CNN

RNN

AI APPROACH – DL

Deep Learning is a subset of Machine Learning that utilizes neural networks with multiple layers to model and extract patterns from complex data.



Why Python Is The Go-To Language for AI?



SIMPLE & READABLE

Python's clean, English-like syntax makes it incredibly easy to read and write.

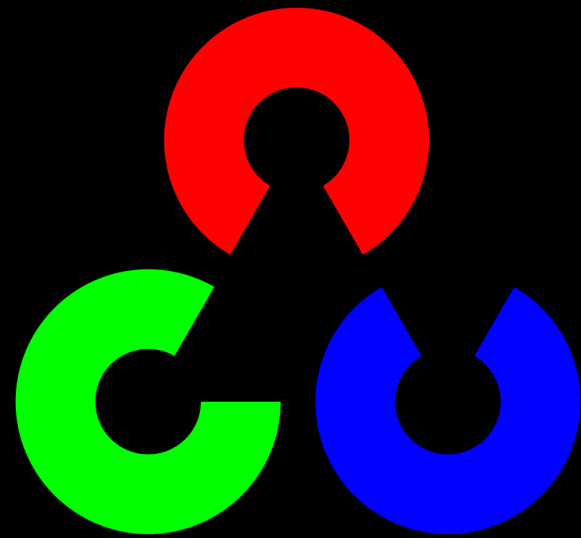
This allows AI developers to prototype faster and collaborate across teams (e.g., data scientists, ML engineers, domain experts).

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()  
model.fit(X_train, y_train)
```

MASSIVE LIBRARY



TensorFlow



OpenCV





INTERGRATED

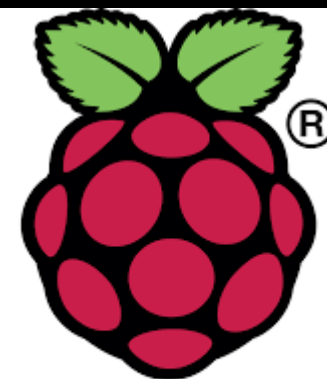
- Integrates smoothly with C/C++ for performance boosts.
- Works with web frameworks like Flask and FastAPI to deploy AI models as services.
- Compatible with cloud platforms (AWS, Azure, GCP) which are essential for scalable AI.

VERSATILE

- Python runs on:



MacOS

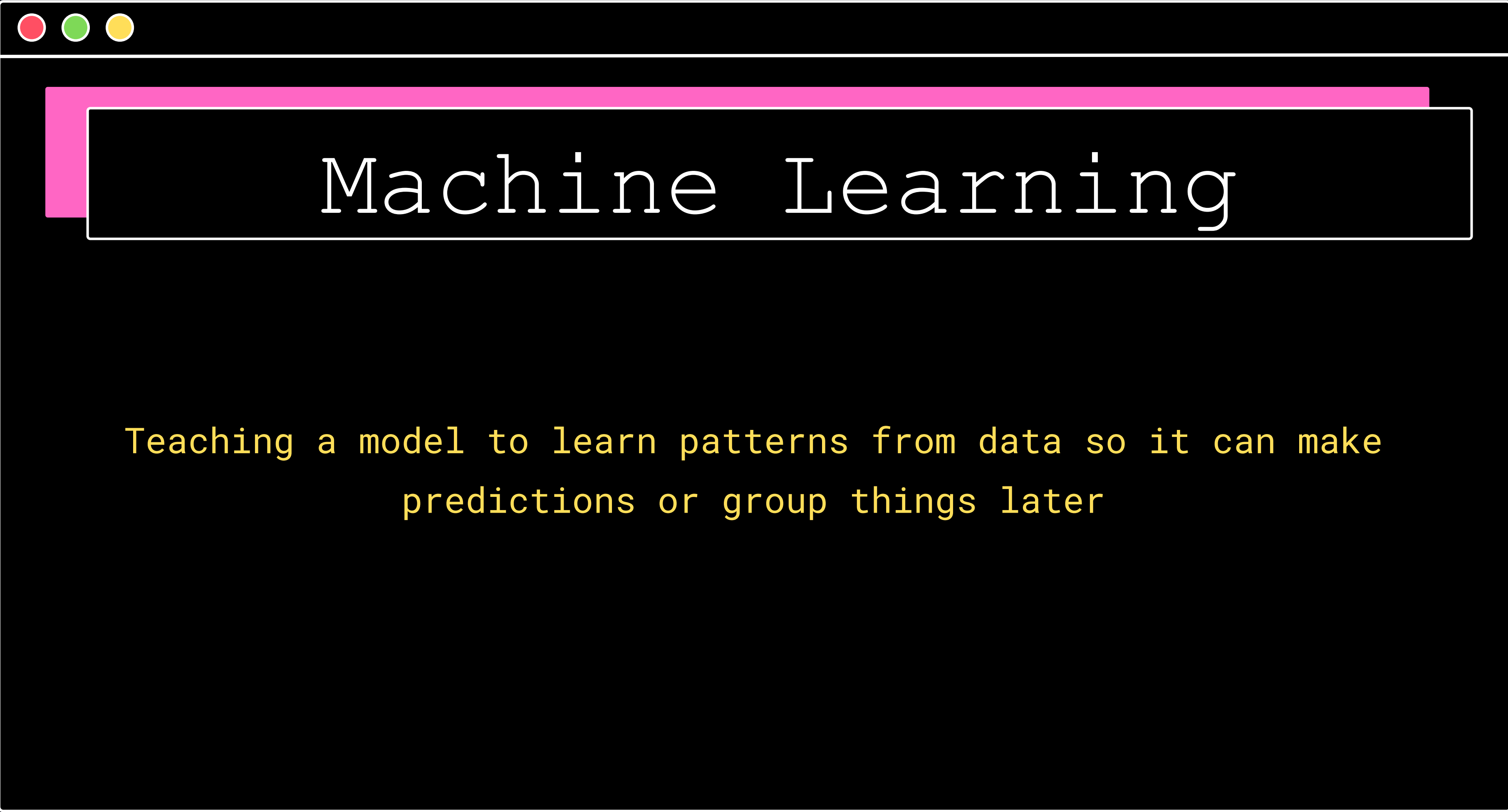


Raspberry Pi



RESEARCH FRIENDLY

- Data preprocessing with `Pandas`, `NumPy`
- Model training with `scikit-learn`, `PyTorch`
- Hyperparameter tuning with `Optuna`, `Ray Tune`
- Deployment with `Flask`, `FastAPI`, or `TensorFlow Serving`
- Monitoring with tools like `MLflow`, `Weights & Biases`

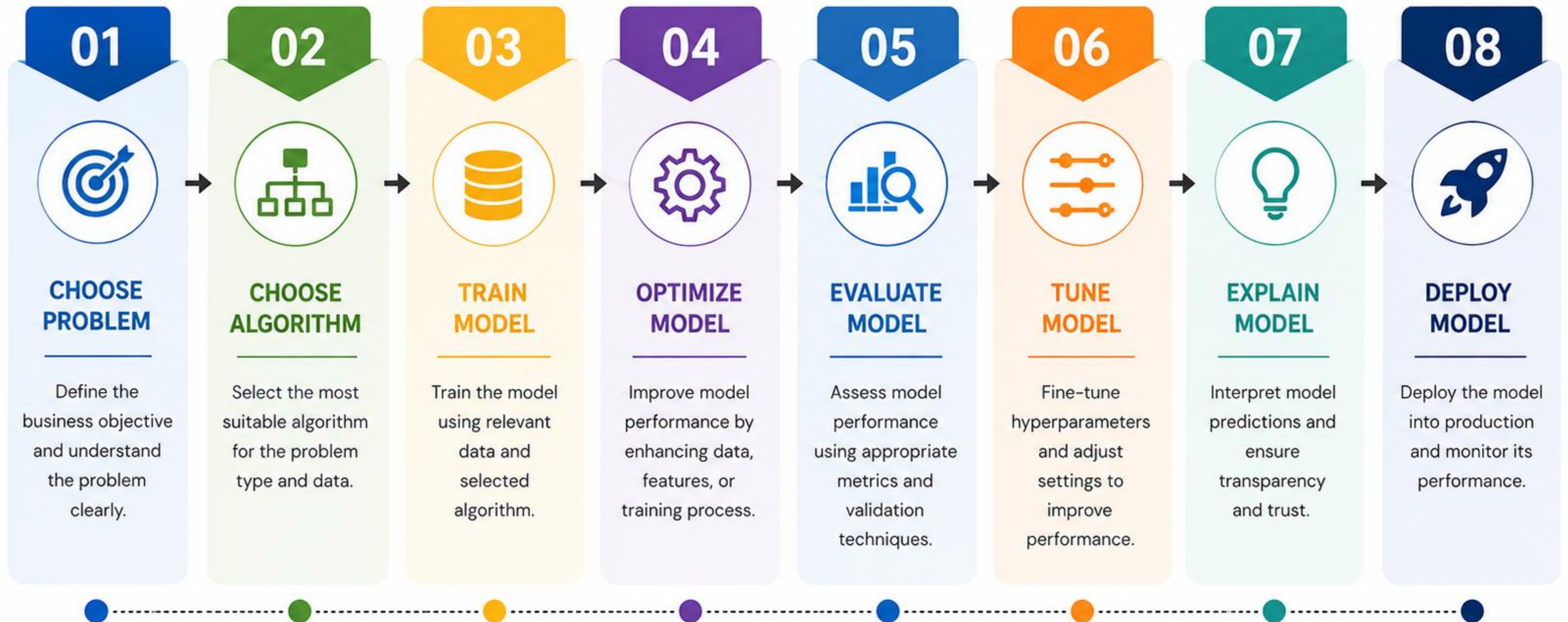


Machine Learning

Teaching a model to learn patterns from data so it can make
predictions or group things later

MACHINE LEARNING WORKFLOW

FROM PROBLEM TO DEPLOYMENT



An iterative process: Insights from later stages can inform earlier decisions for continuous improvement.





Data

This is where Pandas shines.

- Load data

- Understand columns

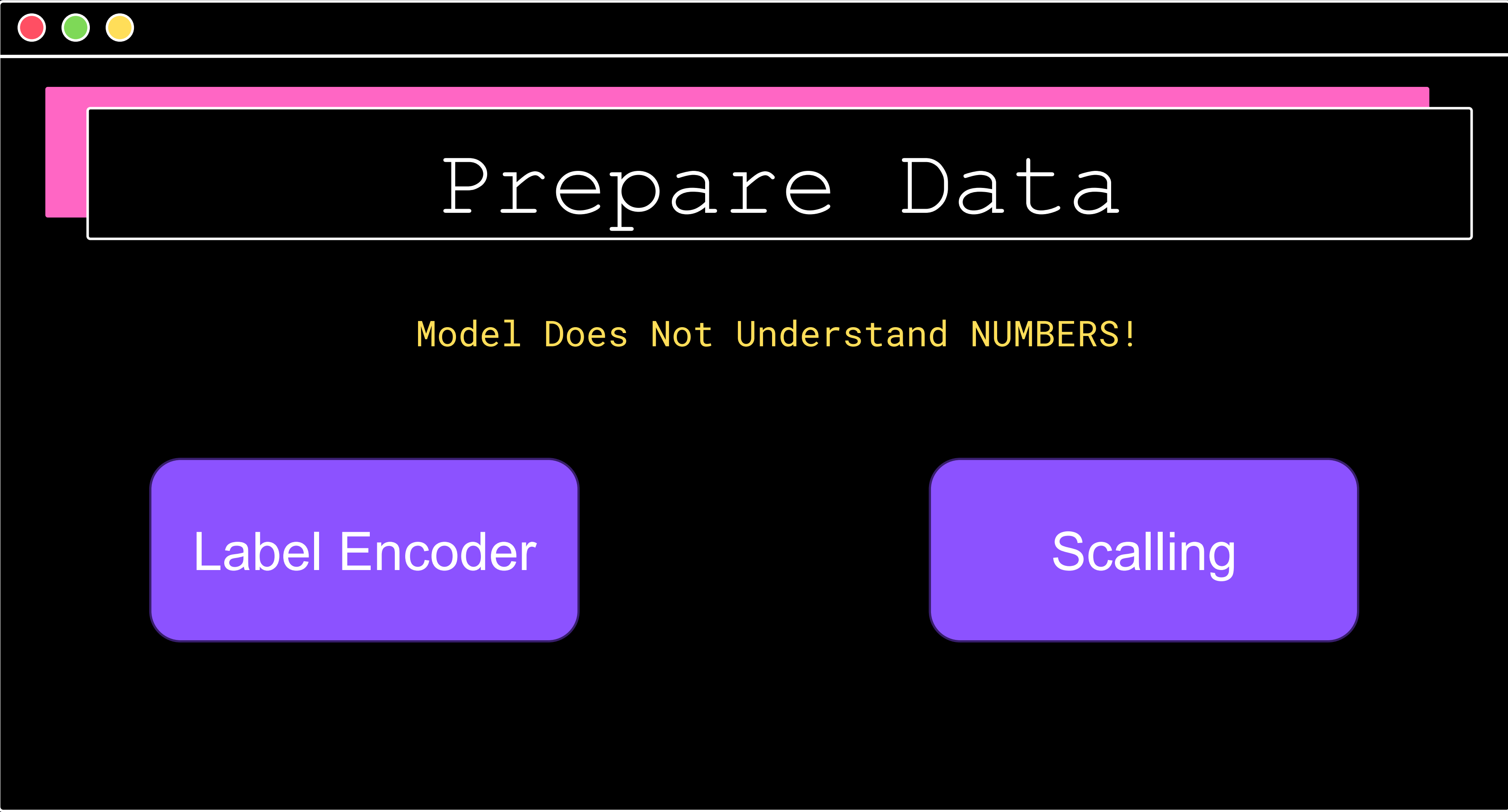
- Handle missing values

- Select features (X)

- Select target (y)

```
X = df[['age', 'salary', 'experience']]
```

```
y = df['purchased']
```



Prepare Data

Model Does Not Understand NUMBERS!

Label Encoder

Scalling



Feature Engineering

Transforming raw data into inputs that a model can understand and learn from.

```
Gender: Male  
Age: 25  
City: Kuala Lumpur
```

```
Gender_Male = 1  
Gender_Female = 0  
Age = 25  
City_KL = 1  
City_Penang = 0  
City_JB = 0
```

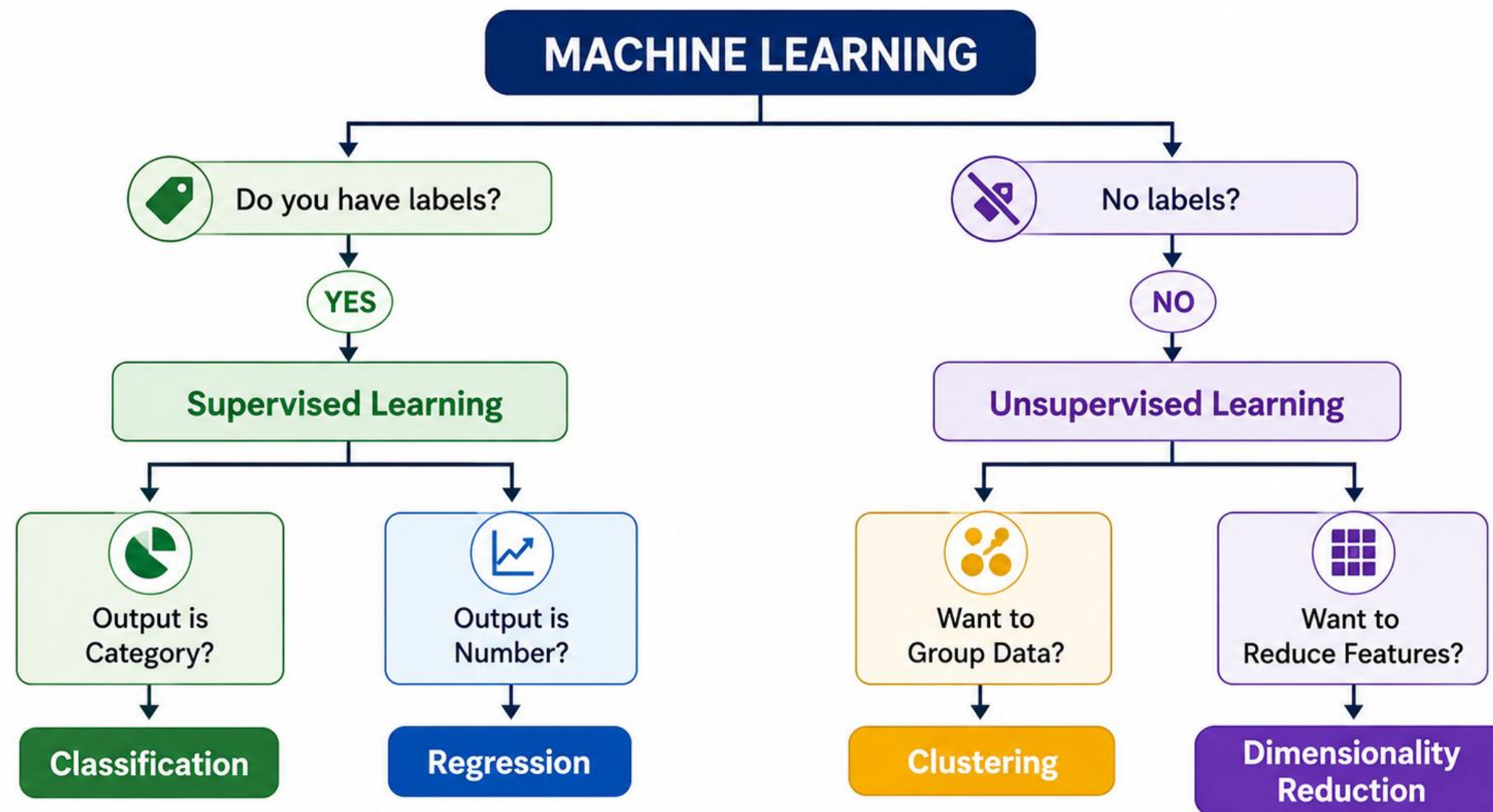
Feature Engineering

Common Feature Engineering Techniques

Technique	Purpose
Scaling	Make numbers comparable
Encoding	Convert categories to numbers
Feature creation	Create meaningful new features
Feature selection	Remove useless features

Problem Type

ANSWER THIS FIRST





Model Algorithm

Supervised Learning : You have inputs + correct answers

- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest

Used for:

- Prediction
- Classification



Model Algorithm

Unsupervised Learning (Clustering) : You have inputs with No Answer

- Kmean Clustering
- Hierarchical Clustering
- DBSCAN

Used for:

- Customer segmentation, grouping students by behavior, image segmentation



Model Algorithm

Unsupervised Learning (Dimensionality reduction) : You have inputs with No Answer

- PCA (Principal Component Analysis)
- t-SNE
- UMAP

Used for:

- Feature Compression, Data Visualization, Noise Reduction



Model Algorithm

Unsupervised Learning (Association) : You have inputs with No Answer

- Apriori
- FP-Growth

Used for:

- Used to find relationships between variables.



Model Algorithm

Unsupervised Learning (Anomaly/Outliers) : You have inputs with No Answer

- Isolation Forest
- One-Class SVM
- Local Outlier Factor (LOF)

Used for:

- Used to detect unusual or rare data points.



Creating Model

A mathematical rule that learns patterns from data so it can make predictions or decisions.

A model:

- Takes input data (features)
- Processes them using learned rules
- Produces an output (prediction)



Creating Model

Size (sqft)	Distance (km)	Price (RM)
800	10	200,000
1200	5	450,000
1500	3	600,000

- **Features (X)** → inputs (size, distance)
- **Target (y)** → output (price)

TRAINING & OPTIMIZATION (SUPERVISED)



1. LOSS FUNCTIONS

Loss function measures how far the predictions are from the true values.

MSE
(Mean Squared Error)

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

MAE
(Mean Absolute Error)

$$L = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

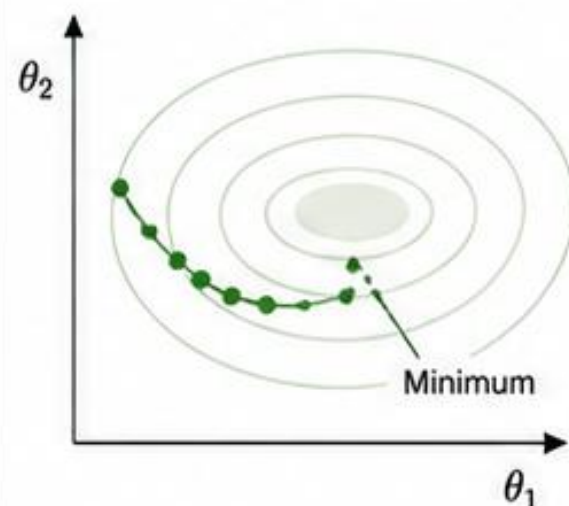
Cross-Entropy Loss
(Classification)

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$



2. GRADIENT DESCENT

An optimization algorithm used to minimize the loss function by updating parameters iteratively.



Update Rule:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} L(\theta^{(t)})$$

θ : parameters

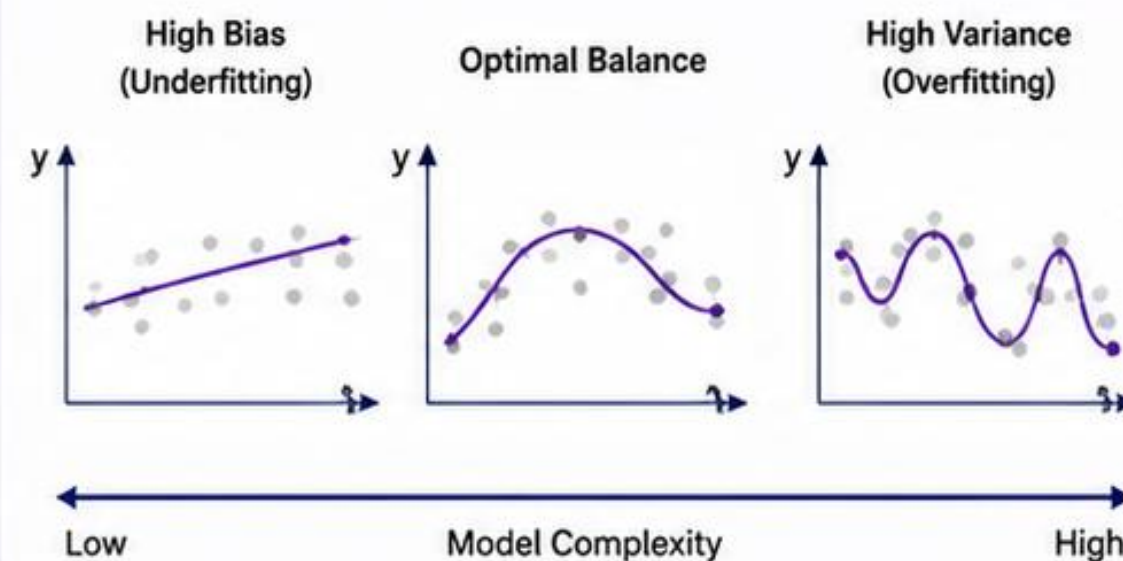
η : learning rate

$\nabla_{\theta} L$: gradient of loss



3. BIAS-VARIANCE TRADEOFF

Tradeoff between underfitting (high bias) and overfitting (high variance).



4. L1 REGULARIZATION

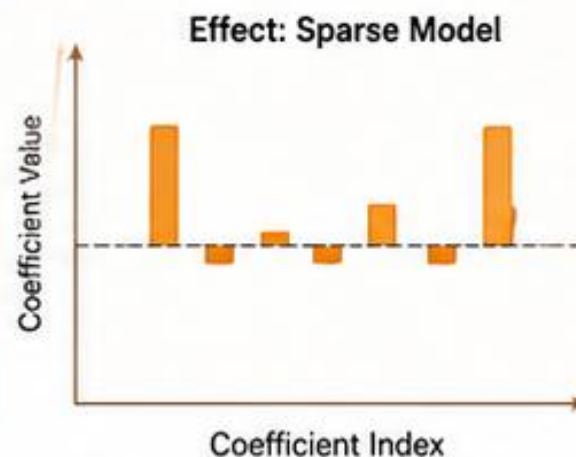
Adds the absolute value of coefficients as a penalty. Encourages sparsity (some coefficients become zero).

Cost Function:

$$J(\theta) = L(\theta) + \lambda \sum_{j=1}^p |\theta_j|$$

λ : regularization strength

θ_j : model coefficients



5. L2 REGULARIZATION

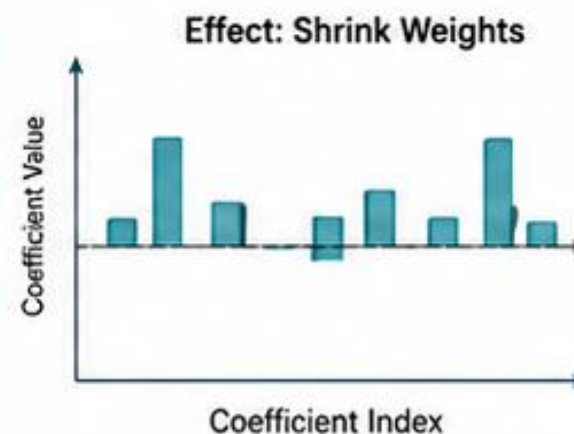
Adds the squared value of coefficients as a penalty. Encourages small weights but not exactly zero.

Cost Function:

$$J(\theta) = L(\theta) + \lambda \sum_{j=1}^p \theta_j^2$$

λ : regularization strength

θ_j : model coefficients



6. EARLY STOPPING

Stops training when performance on validation set stops improving to prevent overfitting.



Benefits:

- Prevents overfitting
- Saves training time
- Improves generalization



Key Takeaway: These techniques help improve model performance, generalization, and training efficiency in supervised learning.

CLUSTERING OPTIMIZATION

Key techniques and parameters to improve clustering performance and quality.



1. DISTANCE METRICS

Distance metrics define how similarity or dissimilarity between data points is measured.

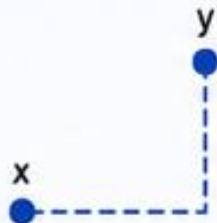
• Euclidean (L2)

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



• Manhattan (L1)

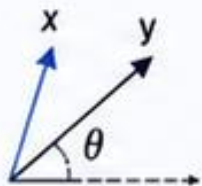
$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$



• Cosine Similarity

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

distance = $1 - \text{sim}(x, y)$



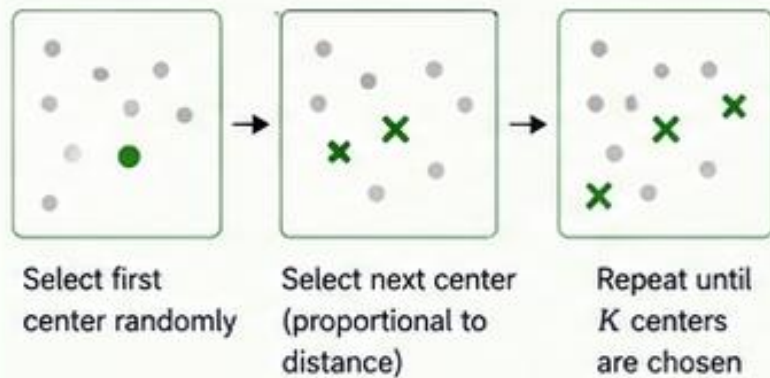
2. CLUSTER INITIALIZATION

How initial cluster centers are selected can impact convergence and results.

• Random Initialization



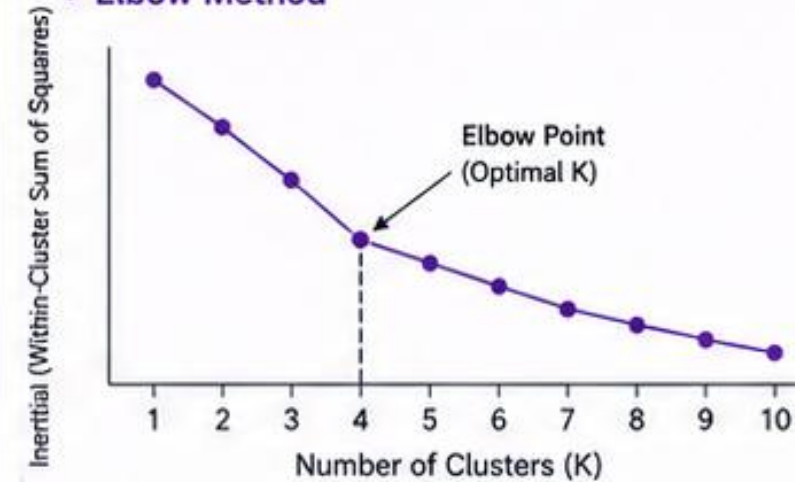
• K-Means++ Initialization



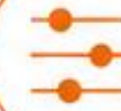
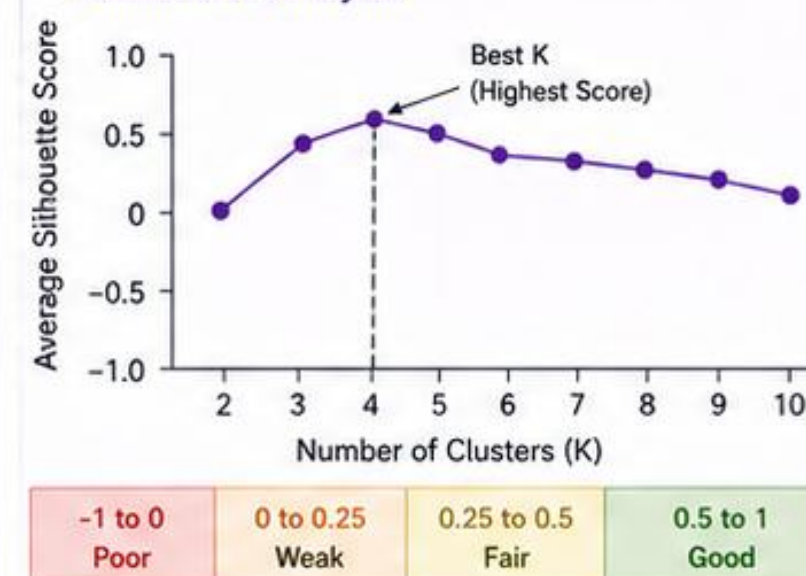
3. CHOOSING NUMBER OF CLUSTERS

Selecting the right number of clusters (K) is crucial for meaningful results.

• Elbow Method



• Silhouette Analysis

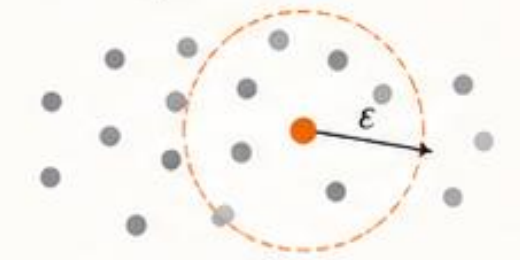


4. DENSITY PARAMETERS

Key parameters for density-based clustering (e.g., DBSCAN) that control neighborhood and cluster minimum size.

• Epsilon (ϵ)

Defines the radius of the neighborhood around a point.



Points within ϵ are neighbors

• Min Samples (minPts)

Minimum number of points required within ϵ -neighborhood to form a core point.



If neighbors $\geq$ minPts $\rightarrow$ Core Point



MODEL VALIDATION STRATEGY

Ensure the model generalizes well and performs reliably.



TRAIN-TEST SPLIT

- Split data into training and testing sets
- Simple and fast evaluation



K-FOLD CROSS VALIDATION

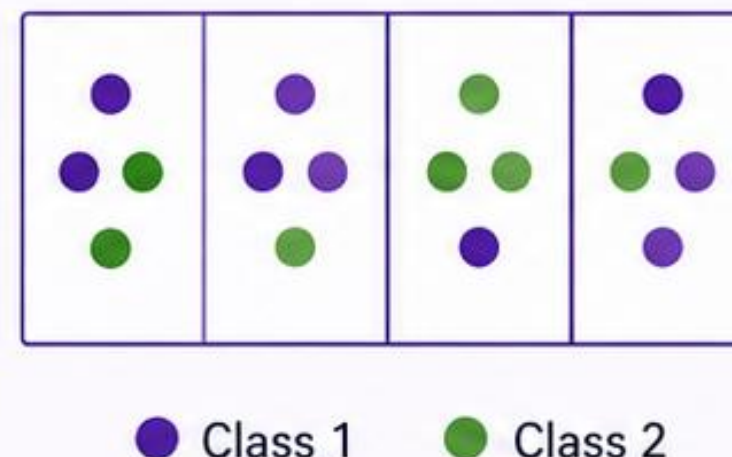
- Split data into K subsets (folds)
- Train on K-1 folds, validate on the remaining fold

Fold	1	2	3	4	5
Iteration 1	■	□	□	□	□
Iteration 2	□	■	□	□	□
Iteration 3	□	□	■	□	□
Iteration 4	□	□	□	■	□
Iteration 5	□	□	□	□	■



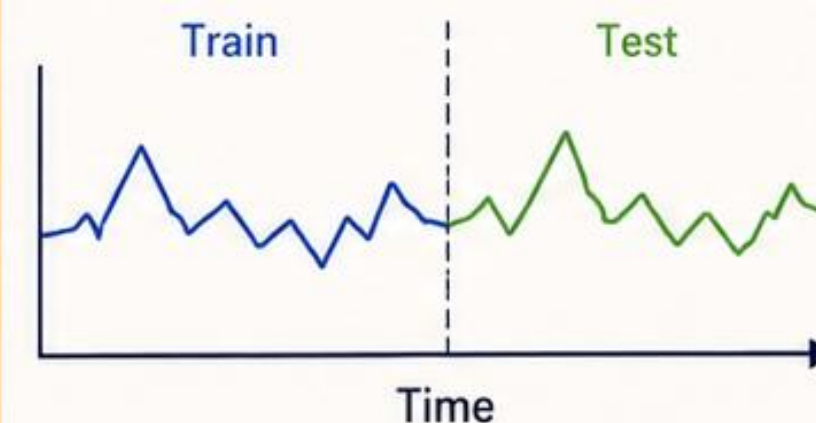
STRATIFIED K-FOLD

- Maintains class distribution in each fold
- Ideal for imbalanced datasets



TIME SERIES SPLIT

- Respect temporal order
- Train on past, validate on future
- Prevents data leakage





Model Validation

Data Splitting

Before training a model, we split the dataset so we can test how well the model generalizes with each metrics to the problem.

Common split:

- Training set ($\approx 70-80\%$) $\rightarrow$ used to learn patterns
- Testing set ($\approx 20-30\%$) $\rightarrow$ used to evaluate performance



Model Validation

Finding the best parameter values that minimize prediction error.

A model:

- After training: The model is ready to predict But only as good as the data it learned from

```
python
```

```
model.fit(X_train, y_train)
```



Model Validation

Train Test Split: Where we split the data into Training set and Test Set. It is to evaluate the performance of the model

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test =  
train_test_split(X, y, test_size=0.2, random_state=42)
```




EVALUATION METRICS

Measure model performance and effectiveness



CLASSIFICATION METRICS



Accuracy

Proportion of correct predictions (TP + TN) / Total



Precision

Proportion of true positives among predicted positives
 $TP / (TP + FP)$



Recall

Proportion of true positives among actual positives
 $TP / (TP + FN)$



F1 Score

Harmonic mean of precision and recall
 $2 * (Precision * Recall) / (Precision + Recall)$



ROC-AUC

Area Under the ROC Curve
Measures separability between classes



Confusion Matrix

Table showing TP, FP, FN, TN
Helps visualize classification performance



REGRESSION METRICS



MAE

Mean Absolute Error
Average of absolute differences
 $\frac{1}{n} \sum |y_i - \hat{y}_i|$



MSE

Mean Squared Error
Average of squared differences
 $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$



RMSE

Root Mean Squared Error
Square root of MSE
 $\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$



R²

Coefficient of Determination
Proportion of variance explained by the model
 $1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$



CLUSTERING METRICS



Inertia

Sum of squared distances of samples to their closest cluster center
Lower is better



Silhouette Score

Measures how similar an object is to its own cluster vs other clusters
Range: [-1, 1] (Higher is better)



Davies-Bouldin Index

Ratio of within-cluster scatter to between-cluster distance
Lower is better

HYPERPARAMETER TUNING



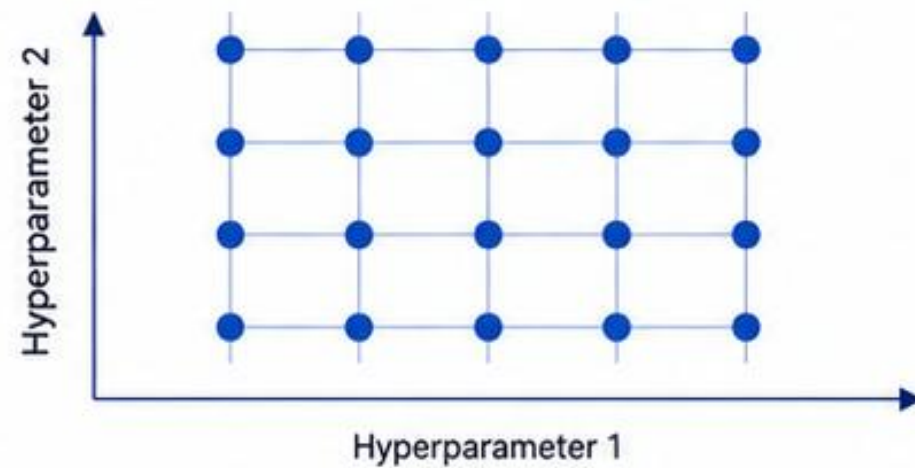
Hyperparameter tuning is the process of finding the best set of hyperparameters that maximizes model performance.



GRID SEARCH

Exhaustively searches through a specified grid of hyperparameter values.

HOW IT WORKS



PROS

- Simple and exhaustive
- Guarantees finding the best within the grid

CONS

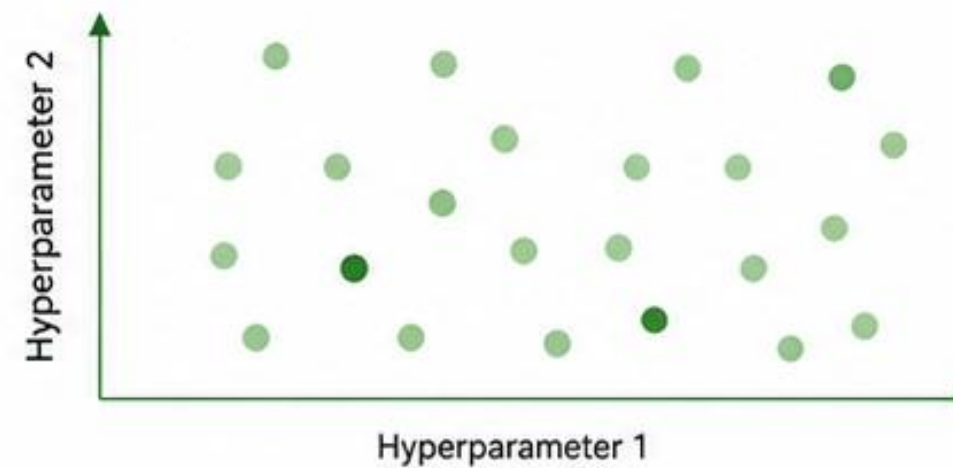
- Computationally expensive
- Not scalable to large search spaces



RANDOM SEARCH

Randomly samples hyperparameter combinations from the search space.

HOW IT WORKS



PROS

- More efficient than grid search
- Scales better to larger search spaces

CONS

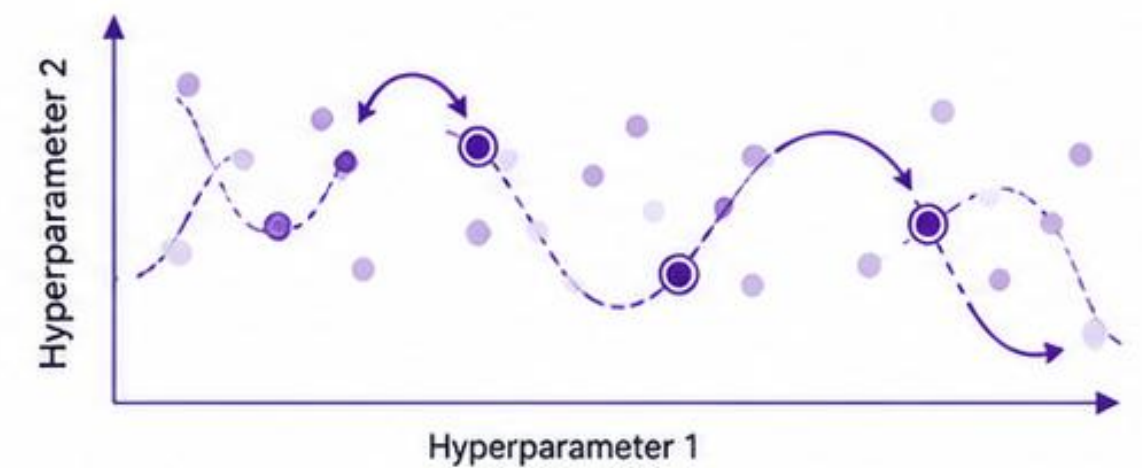
- May miss the best combination
- Results can vary



BAYESIAN OPTIMIZATION

Builds a probabilistic model to find the most promising hyperparameters iteratively.

HOW IT WORKS



PROS

- Very efficient
- Finds better results with fewer iterations
- Ideal for expensive models

CONS

- More complex to implement
- May require more hyperparameter settings



AI ALGORITHM

REFER TO ML ALGO
DOCUMENT



Linear Regression

Supervised Learning : You have inputs (X) + correct answers (y)

Type:

- Supervised Learning
- Regression (predicts numbers)

Used for:

- Predicting continuous values
- Finding relationships between variables
- Trend analysis

Linear Regression

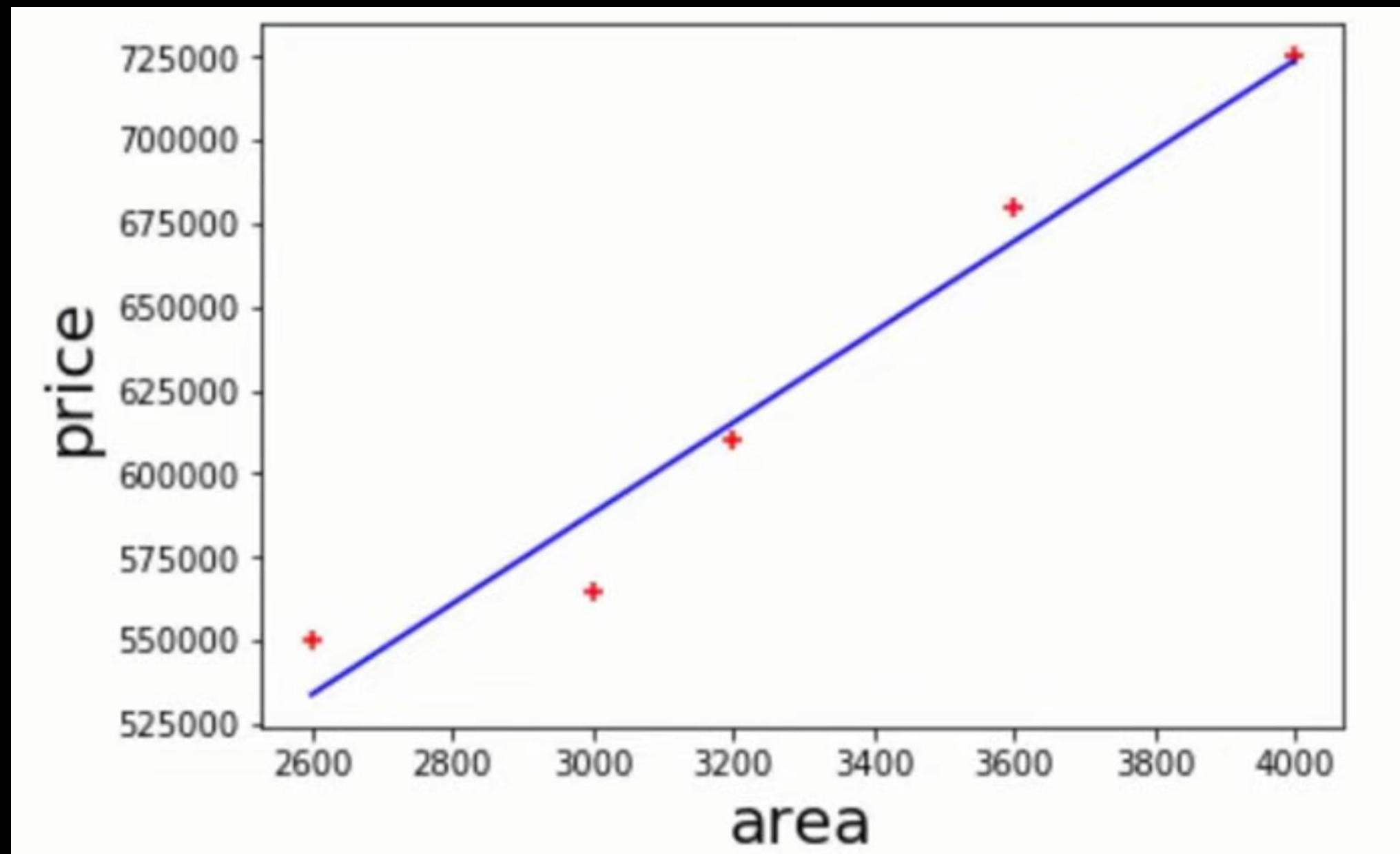
area	price
2600	550000
3000	565000
3200	610000
3600	680000
4000	725000

Given These home prices find out
prices of homes whose area is

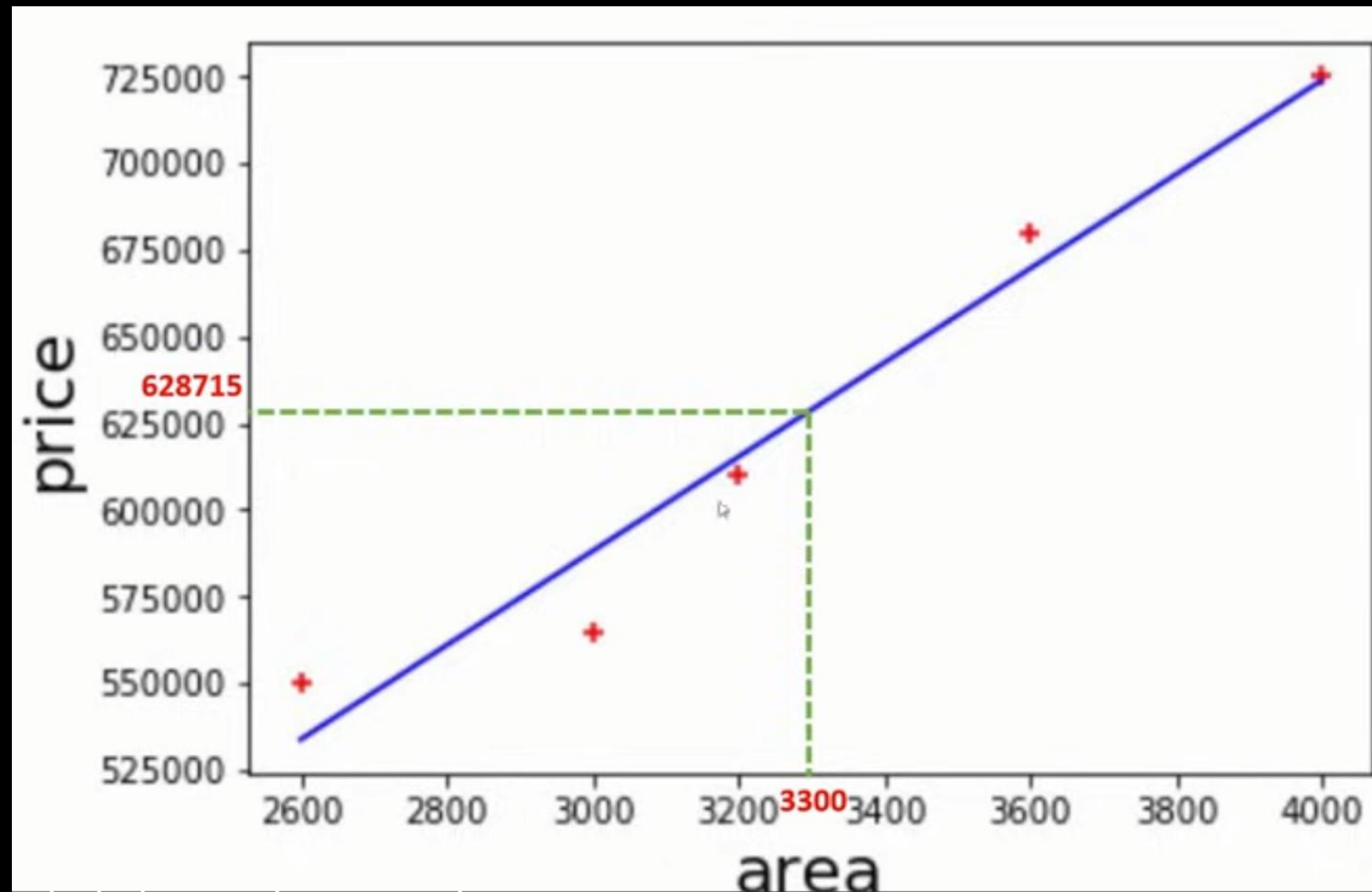
3300 square feet

5000 square feet

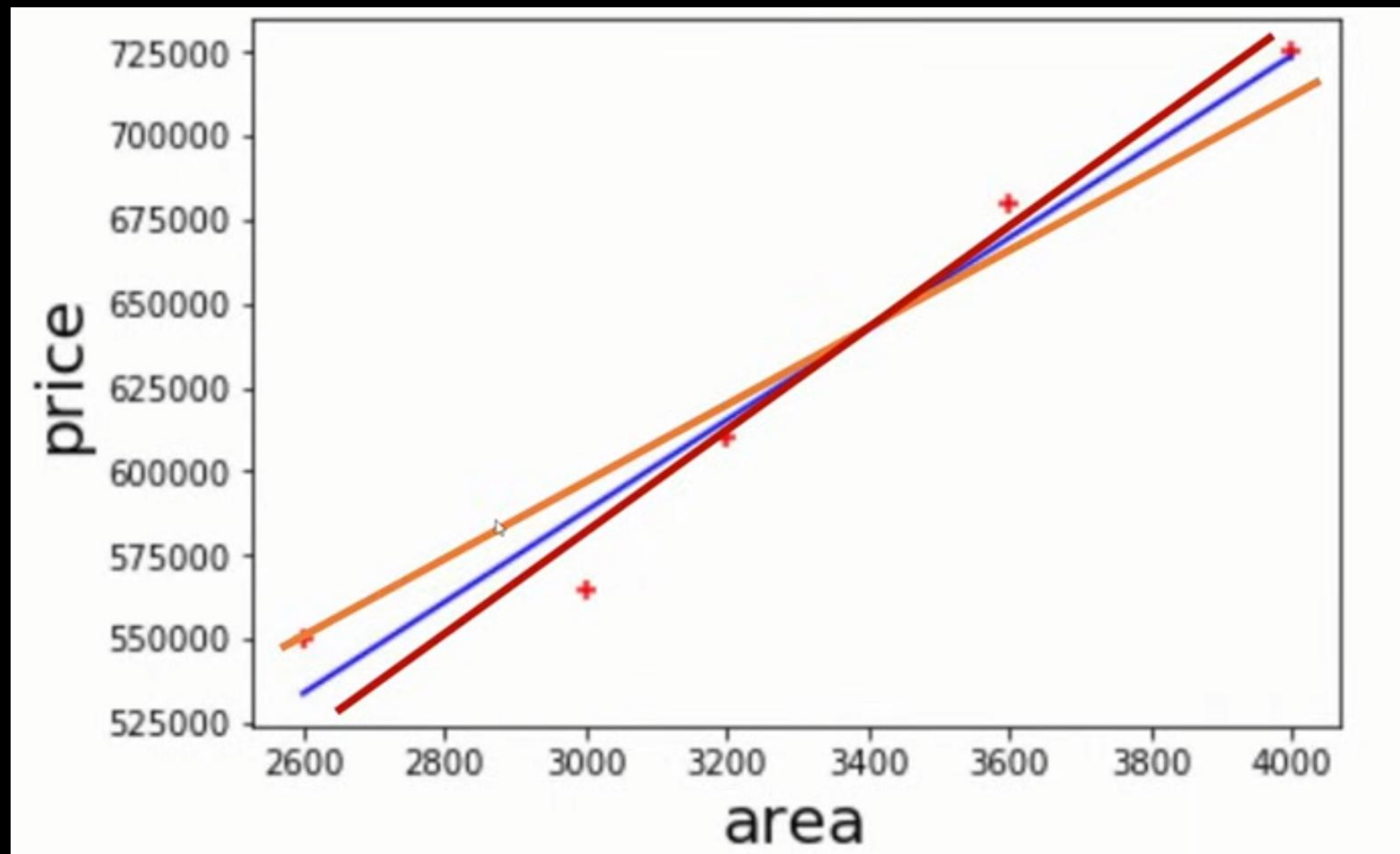
Linear Regression



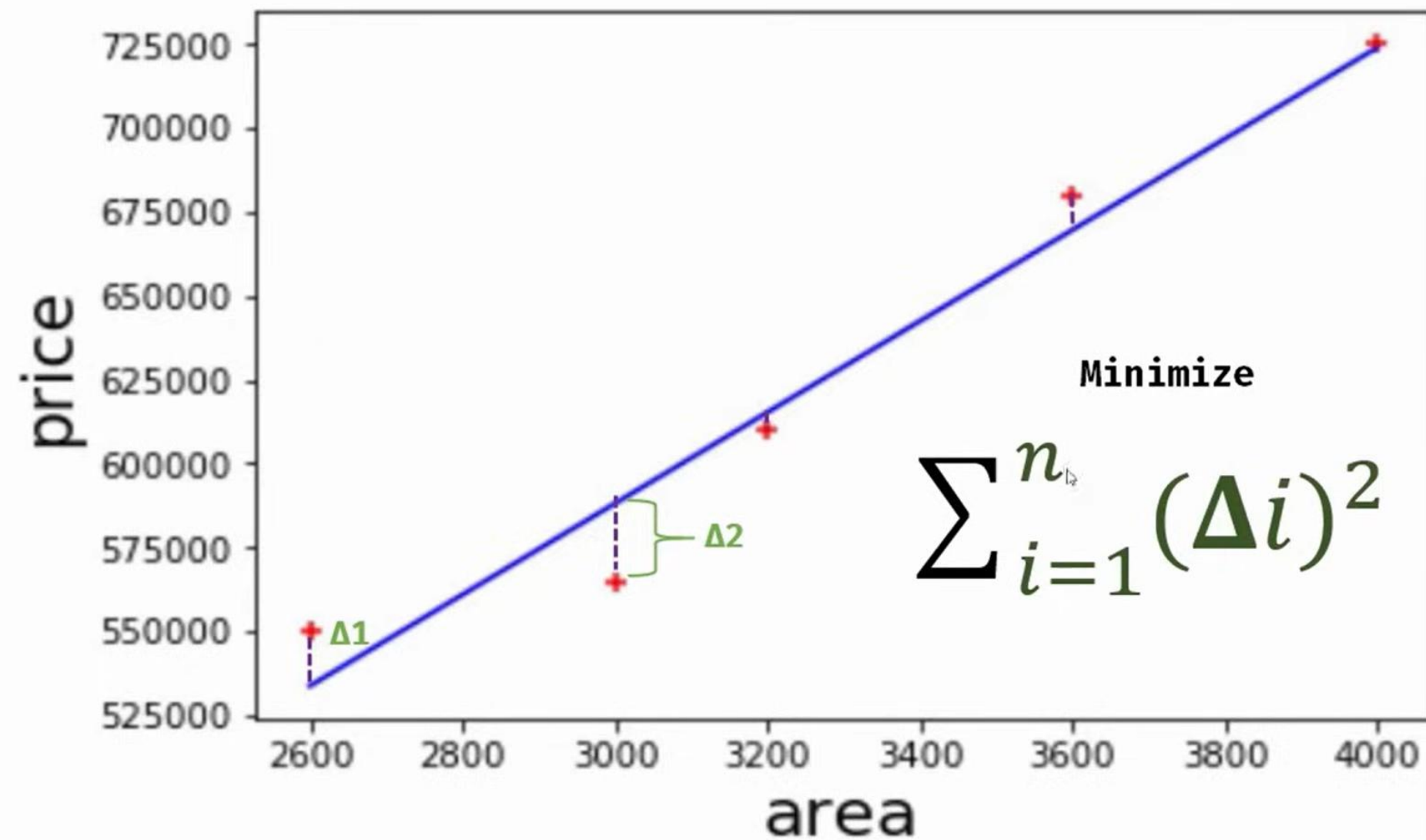
Linear Regression



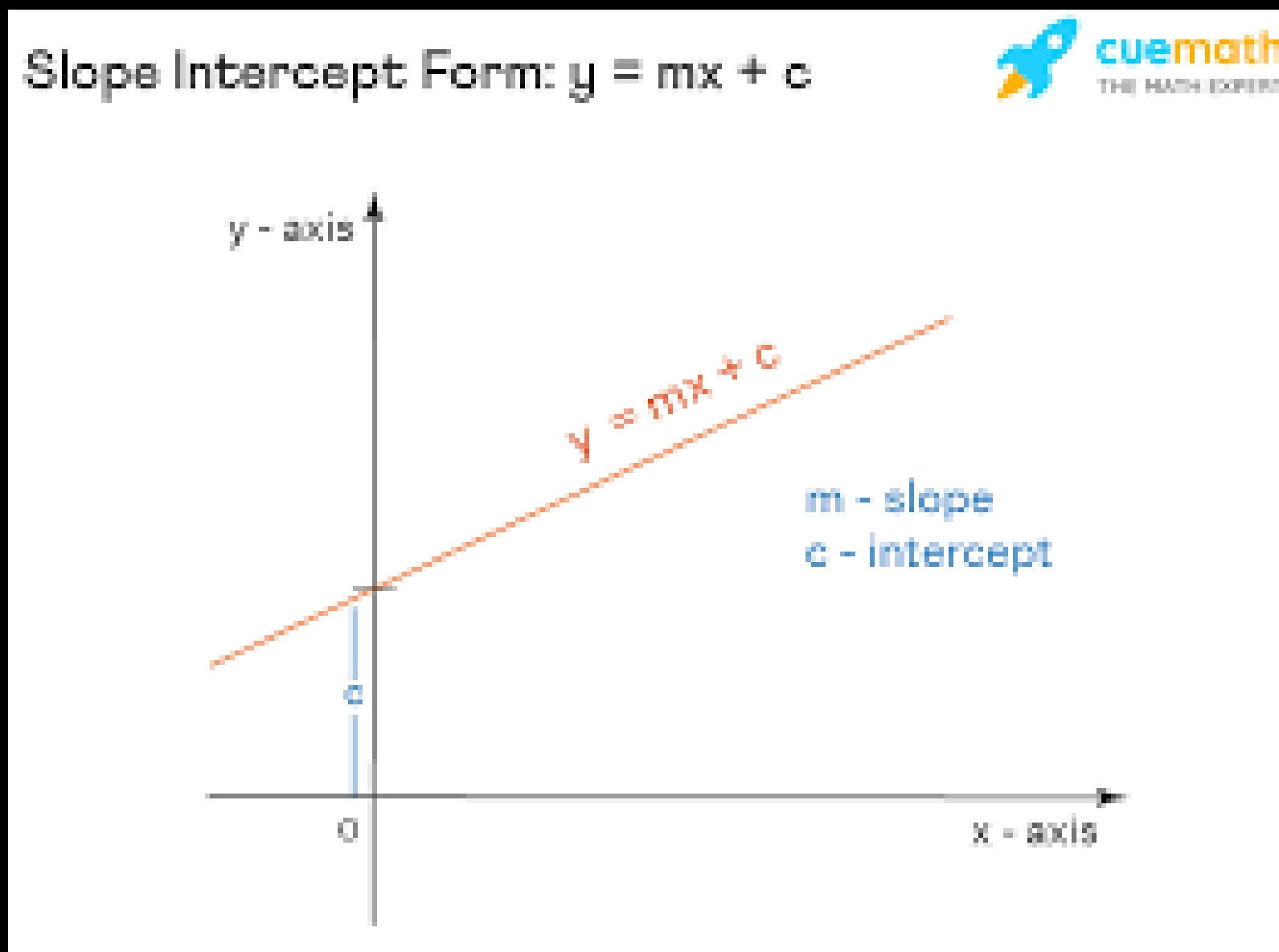
Linear Regression



Linear Regression



Linear Regression



$$\text{Price} = m * \text{area} + c$$

Dependent Variable

Independent Variable

Multivariate Regression

area	bedrooms	age	price
2600	3	20	550000
3000	4	15	565000
3200		18	610000
3600	3	30	595000
4000	5	8	760000

Given These home prices find out
prices of homes whose area is

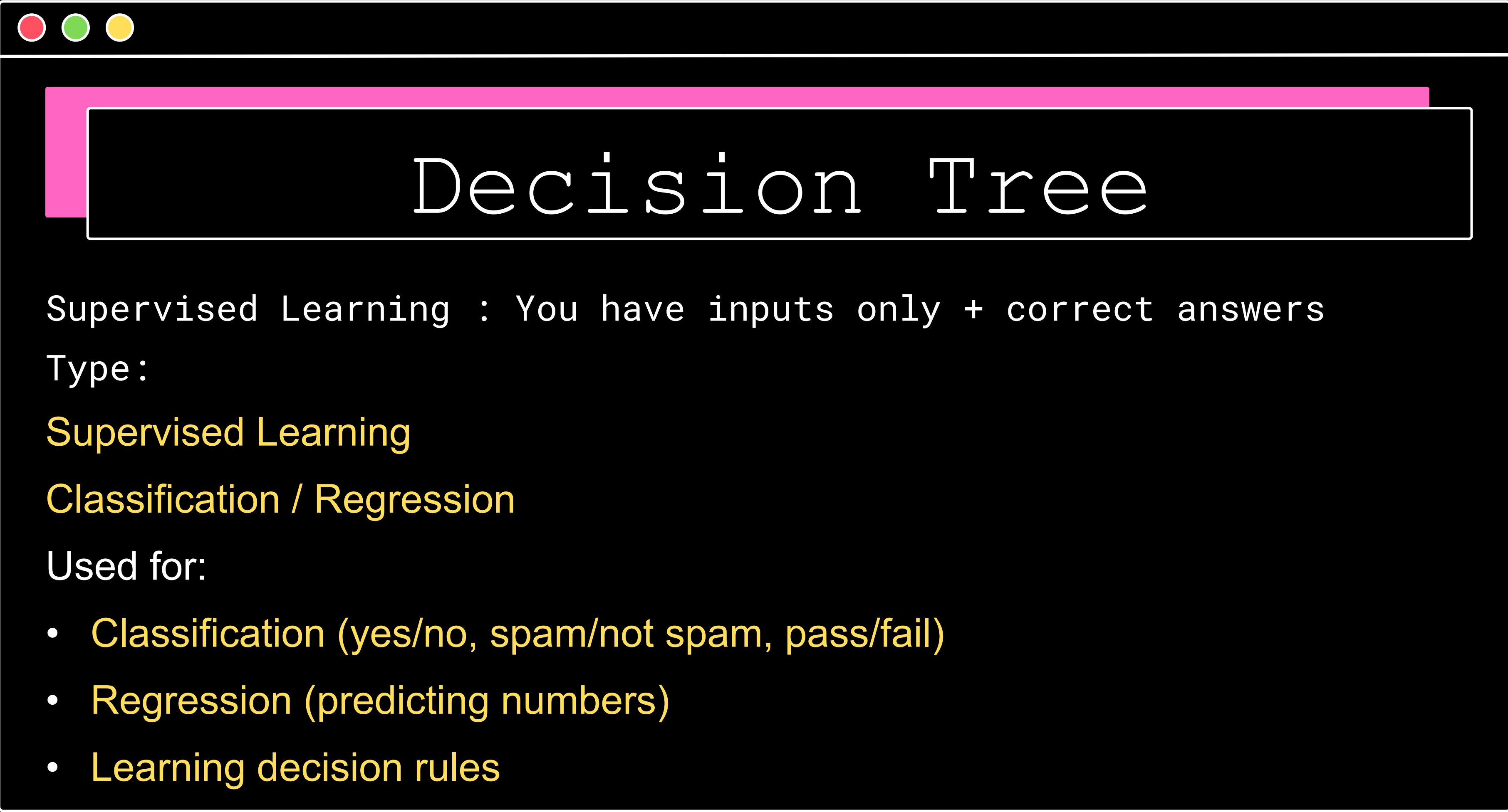
3000 sqrft, 3 bedroom, 40 years old

2500 sqrft, 4 bedroom, 5 years old

$$\text{Price} = m1 * \text{area} + m2 * \text{bedrooms} + m3 * \text{age} + b$$

LAB 2

Predicting Salary



Decision Tree

Supervised Learning : You have inputs only + correct answers

Type :

Supervised Learning

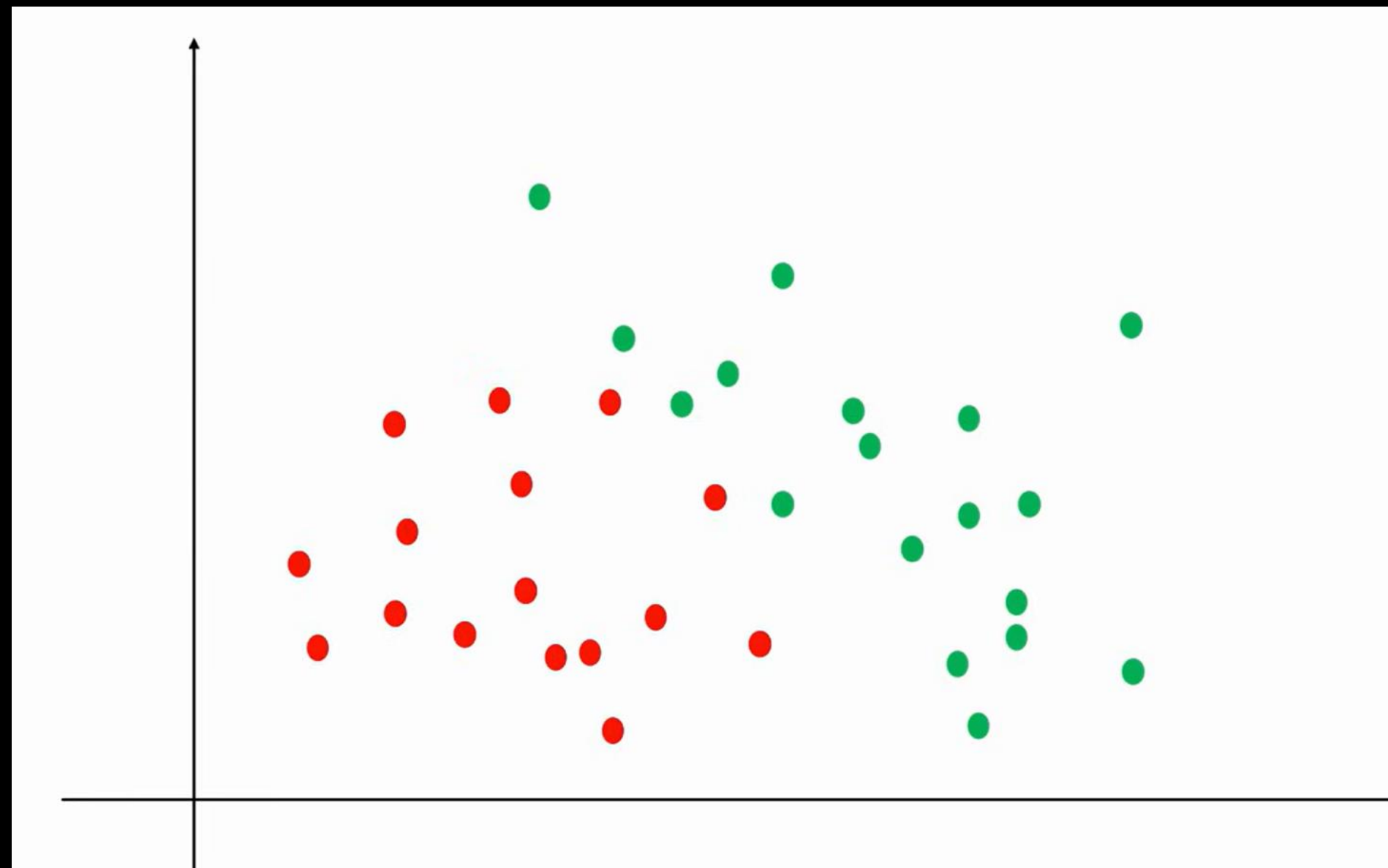
Classification / Regression

Used for:

- Classification (yes/no, spam/not spam, pass/fail)
- Regression (predicting numbers)
- Learning decision rules

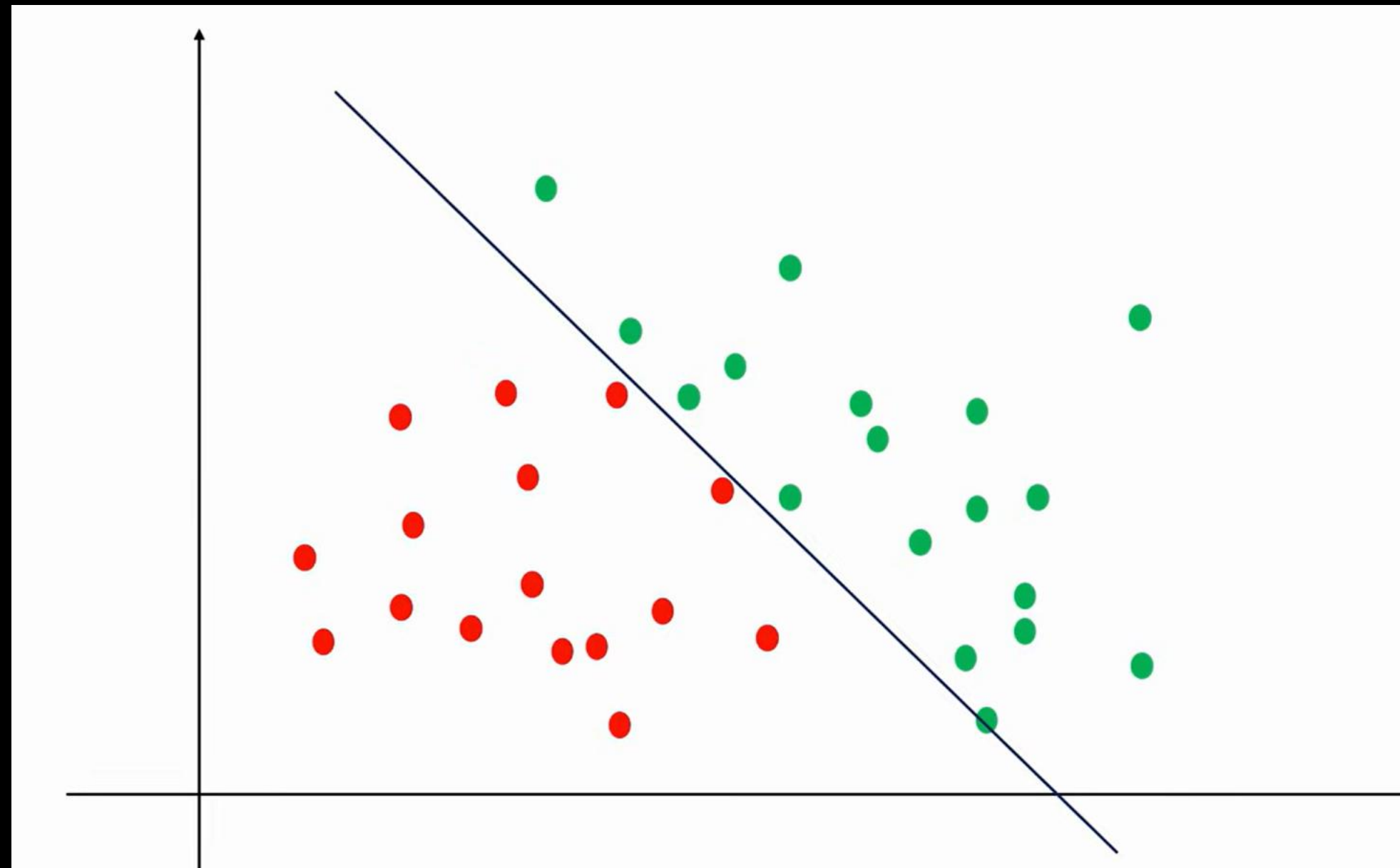


Decision Tree



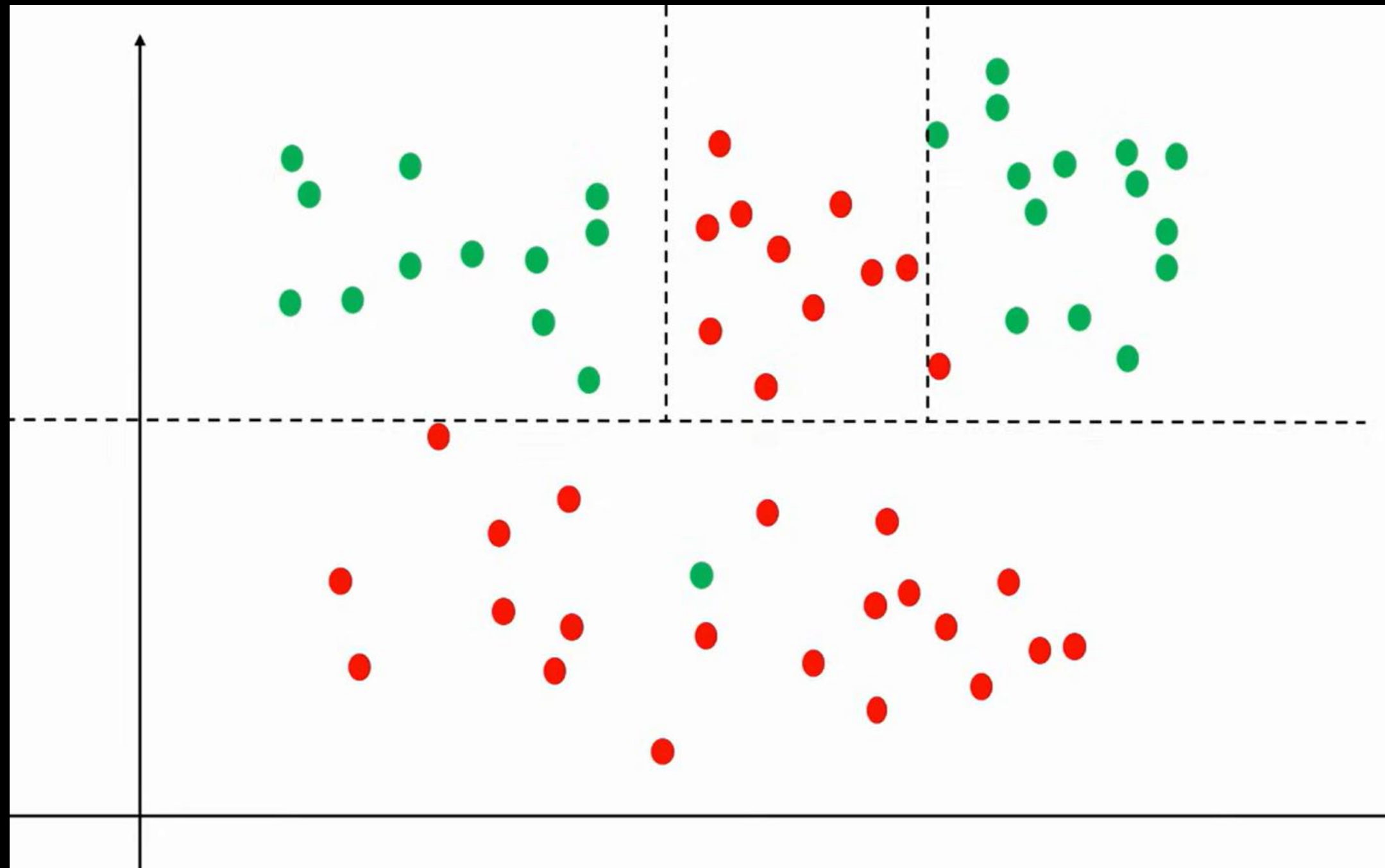


Decision Tree





Decision Tree

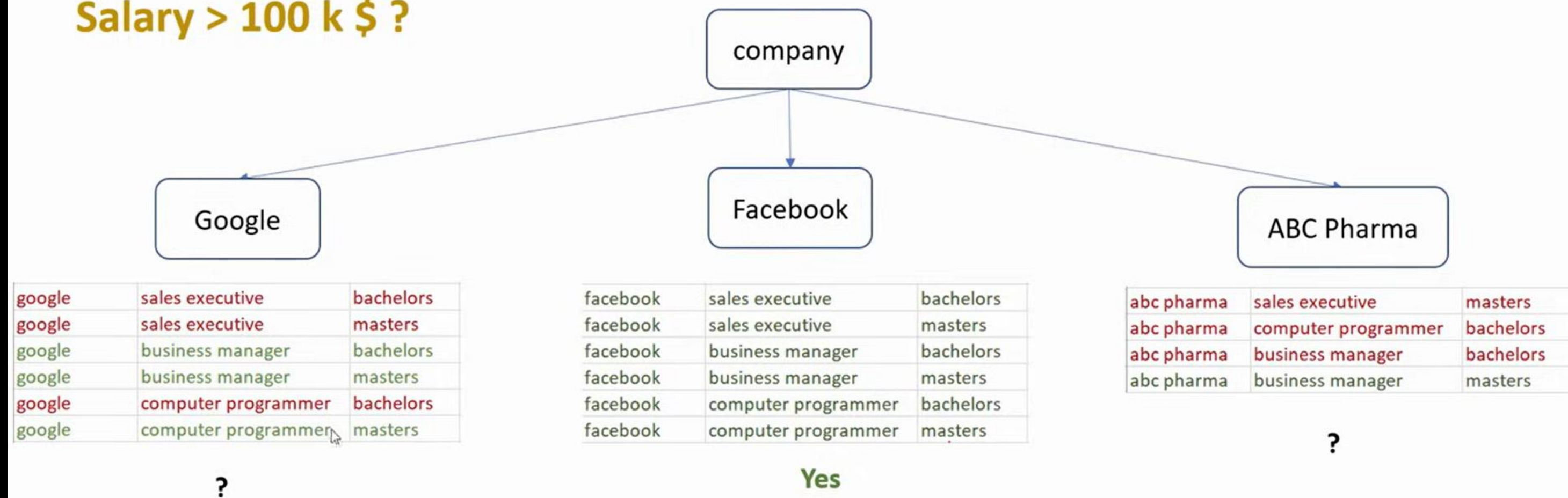


Decision Tree

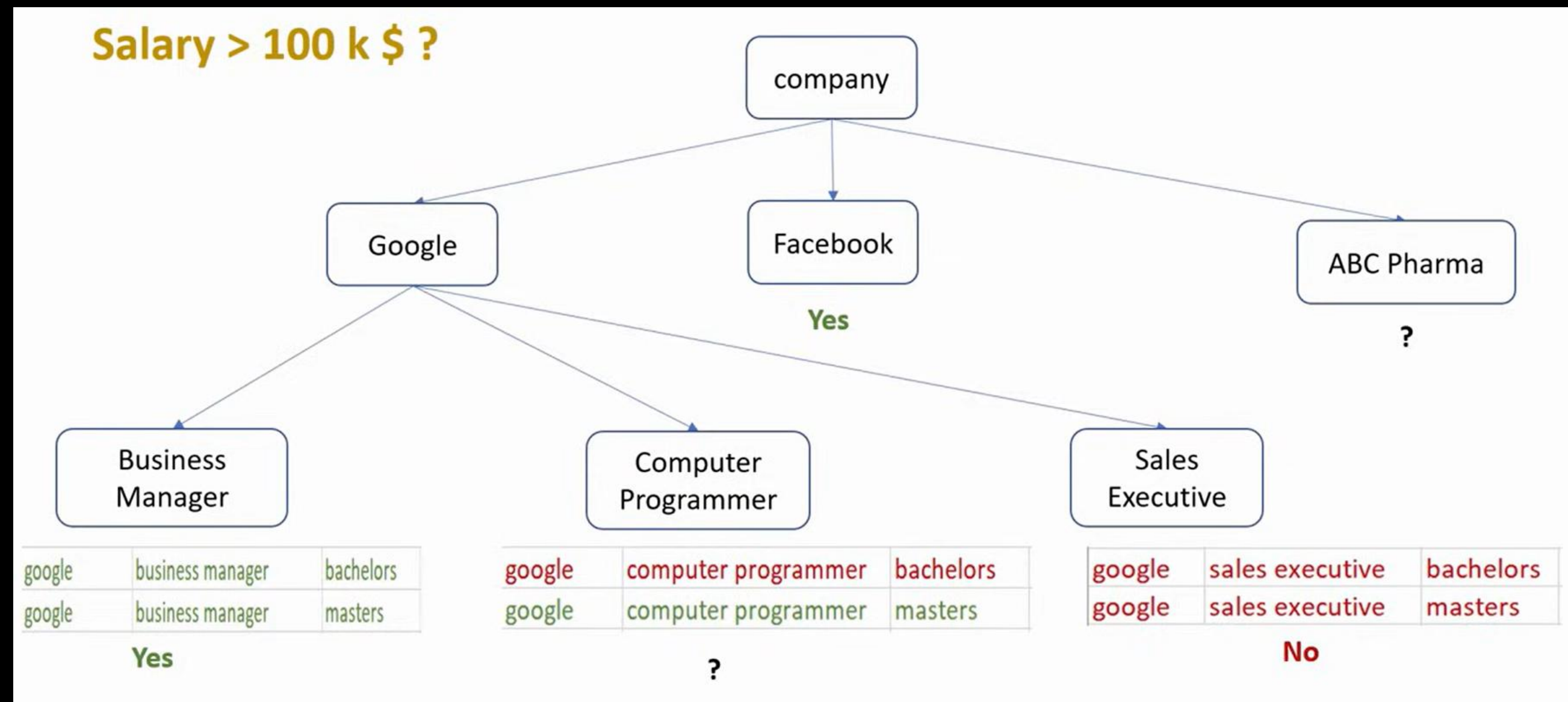
Company	Job	Degree	Salary_more_than_100k
google	sales executive	bachelors	0
google	sales executive	masters	0
google	business manager	bachelors	1
google	business manager	masters	1
google	computer programmer	bachelors	0
google	computer programmer	masters	1
abc pharma	sales executive	masters	0
abc pharma	computer programmer	bachelors	0
abc pharma	business manager	bachelors	0
abc pharma	business manager	masters	1
facebook	sales executive	bachelors	1
facebook	sales executive	masters	1
facebook	business manager	bachelors	1
facebook	business manager	masters	1
facebook	computer programmer	bachelors	1
facebook	computer programmer	masters	1

Decision Tree

Salary > 100 k \$?

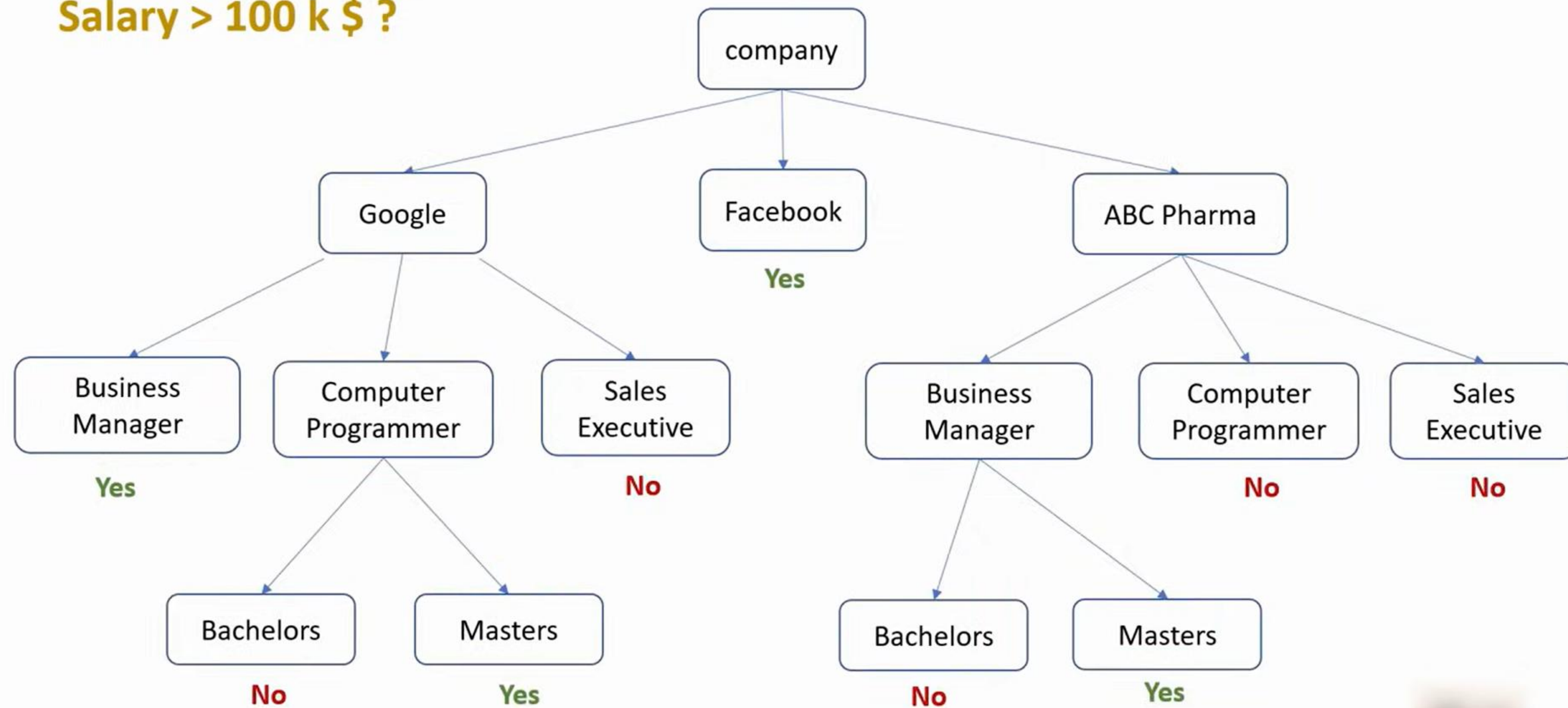


Decision Tree



Decision Tree

Salary > 100 k \$?

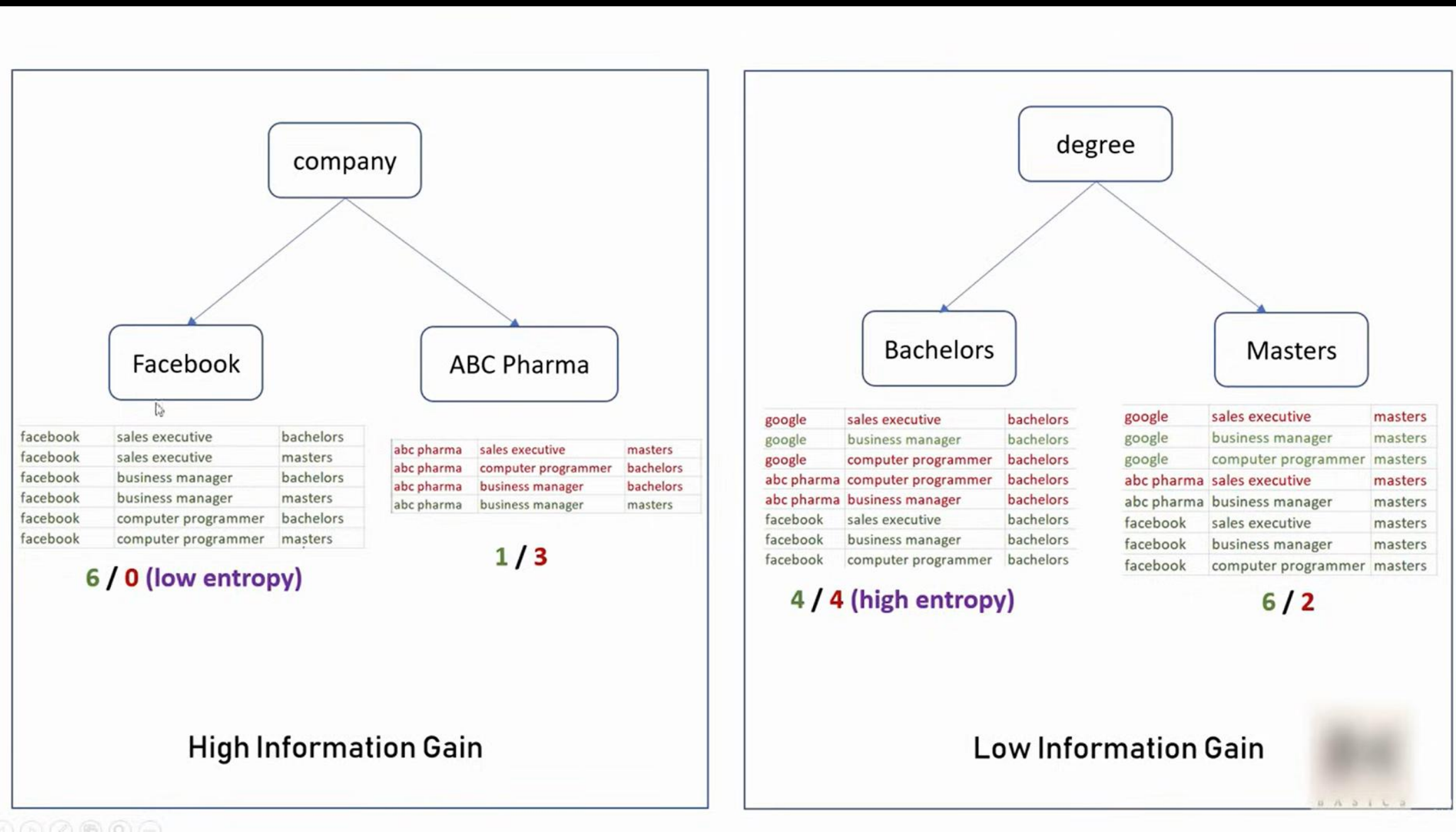




Decision Tree

How do you select
ordering of features?

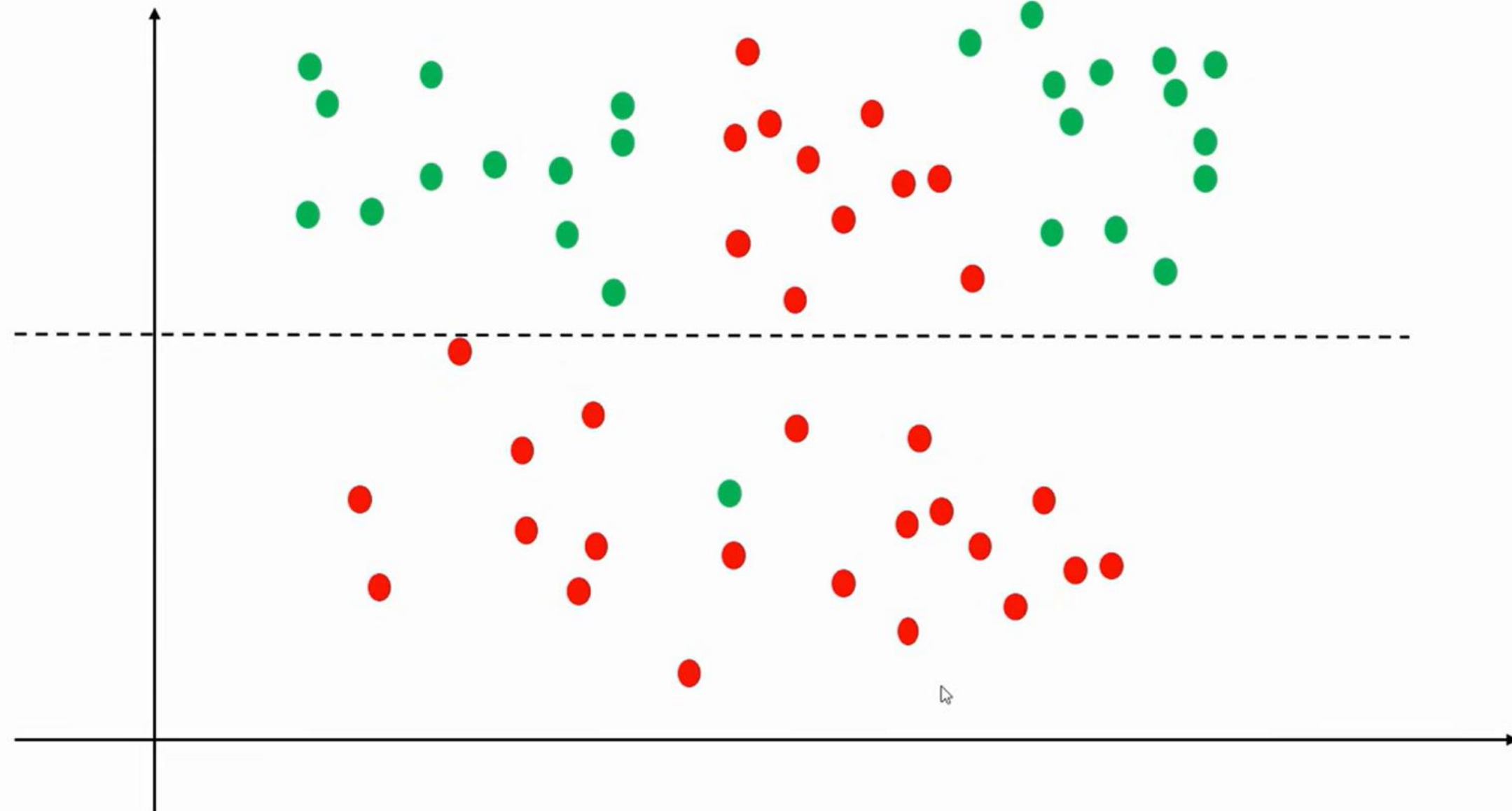
Decision Tree





Decision Tree

Gini Impurity



LAB 1

Predicting Purchase



Random Forest

Supervised Learning : You have inputs only + correct answers

Type :

Supervised Learning

Classification / Regression

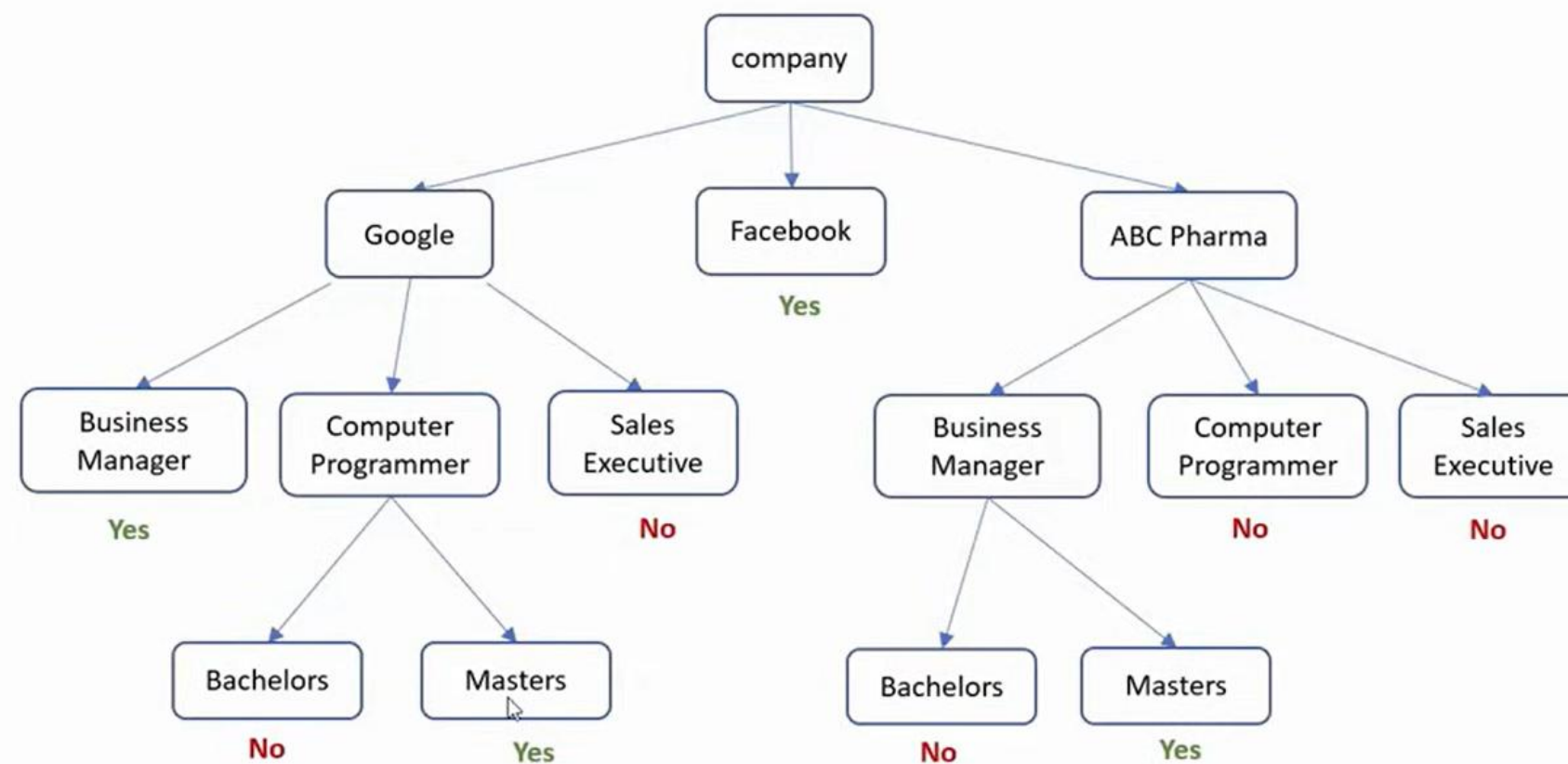
Used for:

- Classification (spam detection, fraud detection)
- Regression (price prediction, risk scoring)
- Improving accuracy over Decision Trees

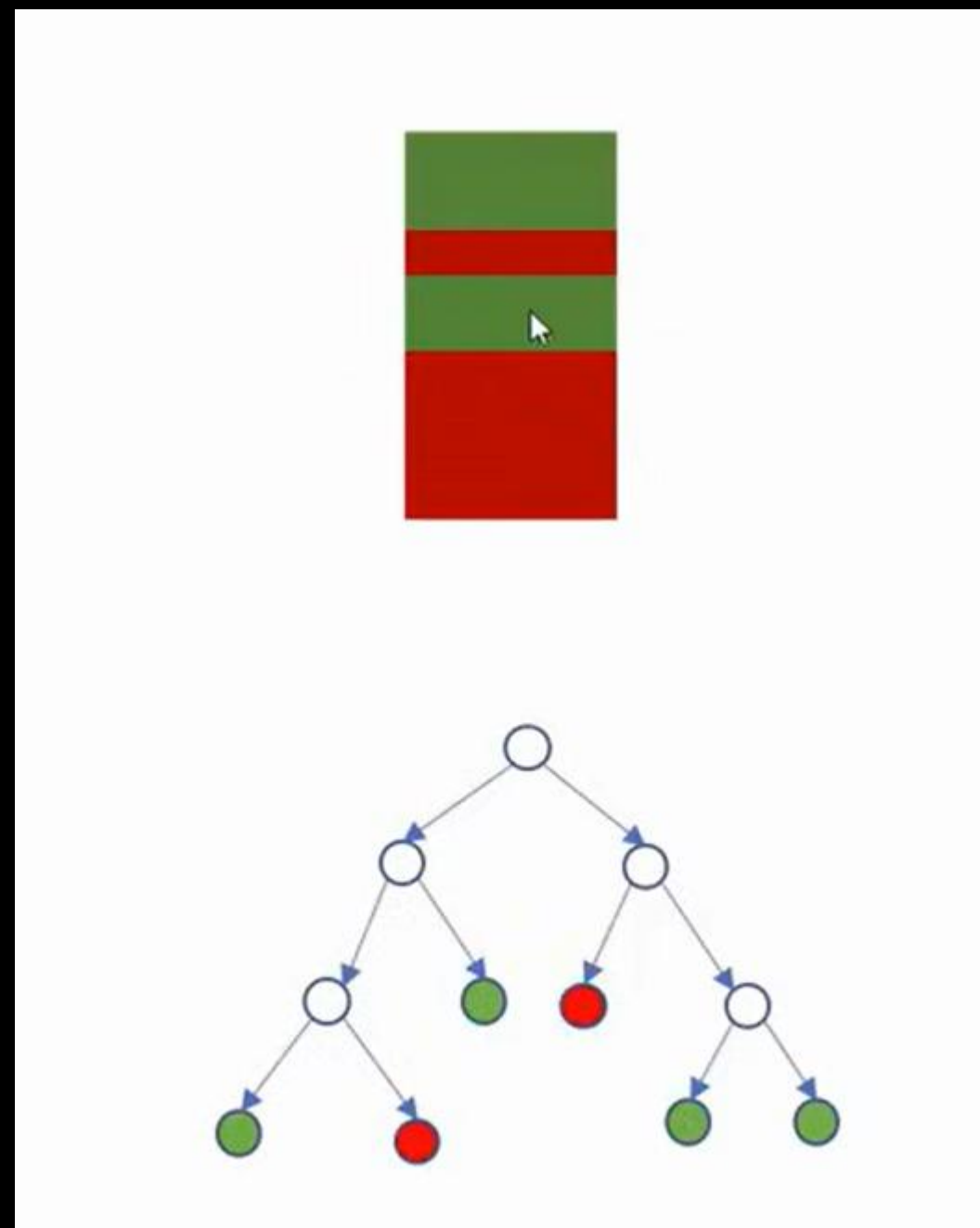
Random Forest

Salary > 100 k \$?

Company	Job	Degree	Salary_more_than_100k
google	sales executive	bachelors	0
google	sales executive	masters	0
google	business manager	bachelors	1
google	business manager	masters	1
google	computer programmer	bachelors	0
google	computer programmer	masters	1
abc pharma	sales executive	masters	0
abc pharma	computer programmer	bachelors	0
abc pharma	business manager	bachelors	0
abc pharma	business manager	masters	1
facebook	sales executive	bachelors	1
facebook	sales executive	masters	1
facebook	business manager	bachelors	1
facebook	business manager	masters	1
facebook	computer programmer	bachelors	1
facebook	computer programmer	masters	1

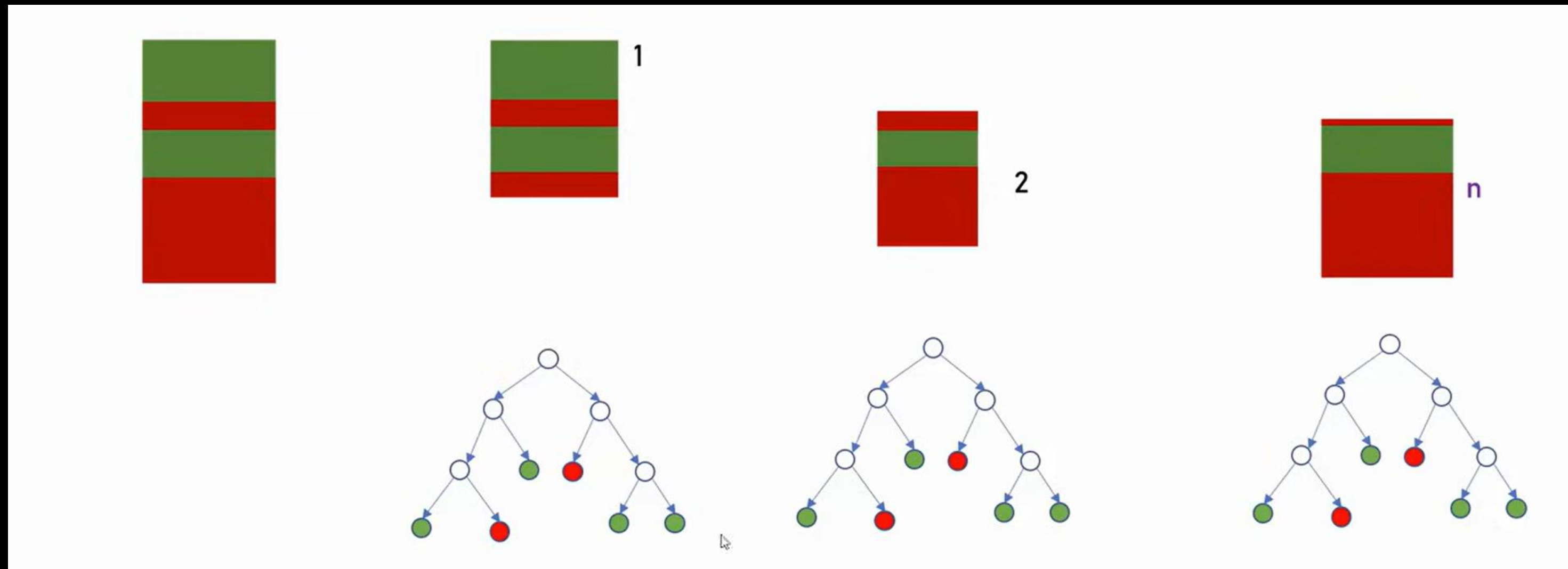


Random Forest

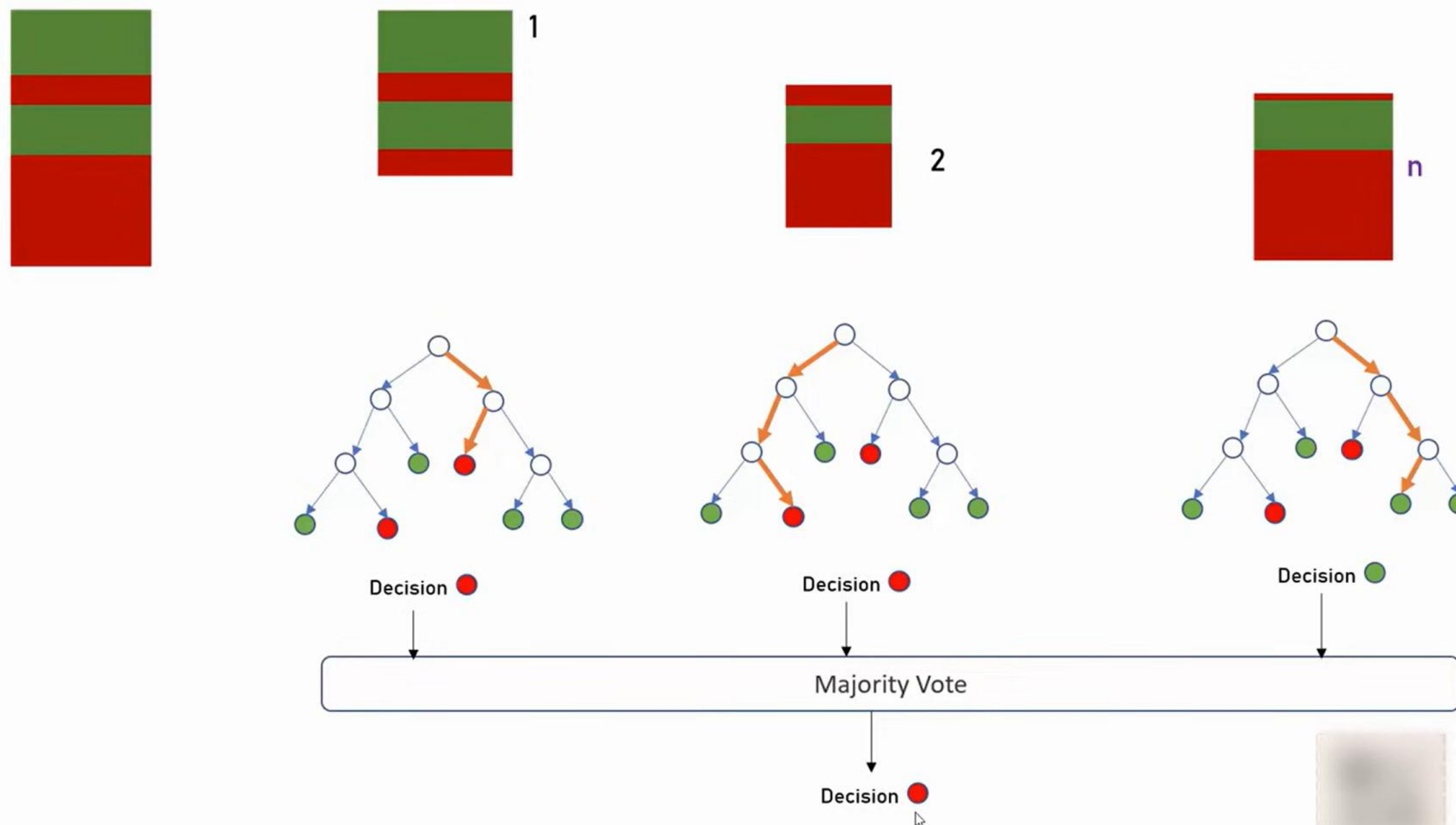




Random Forest

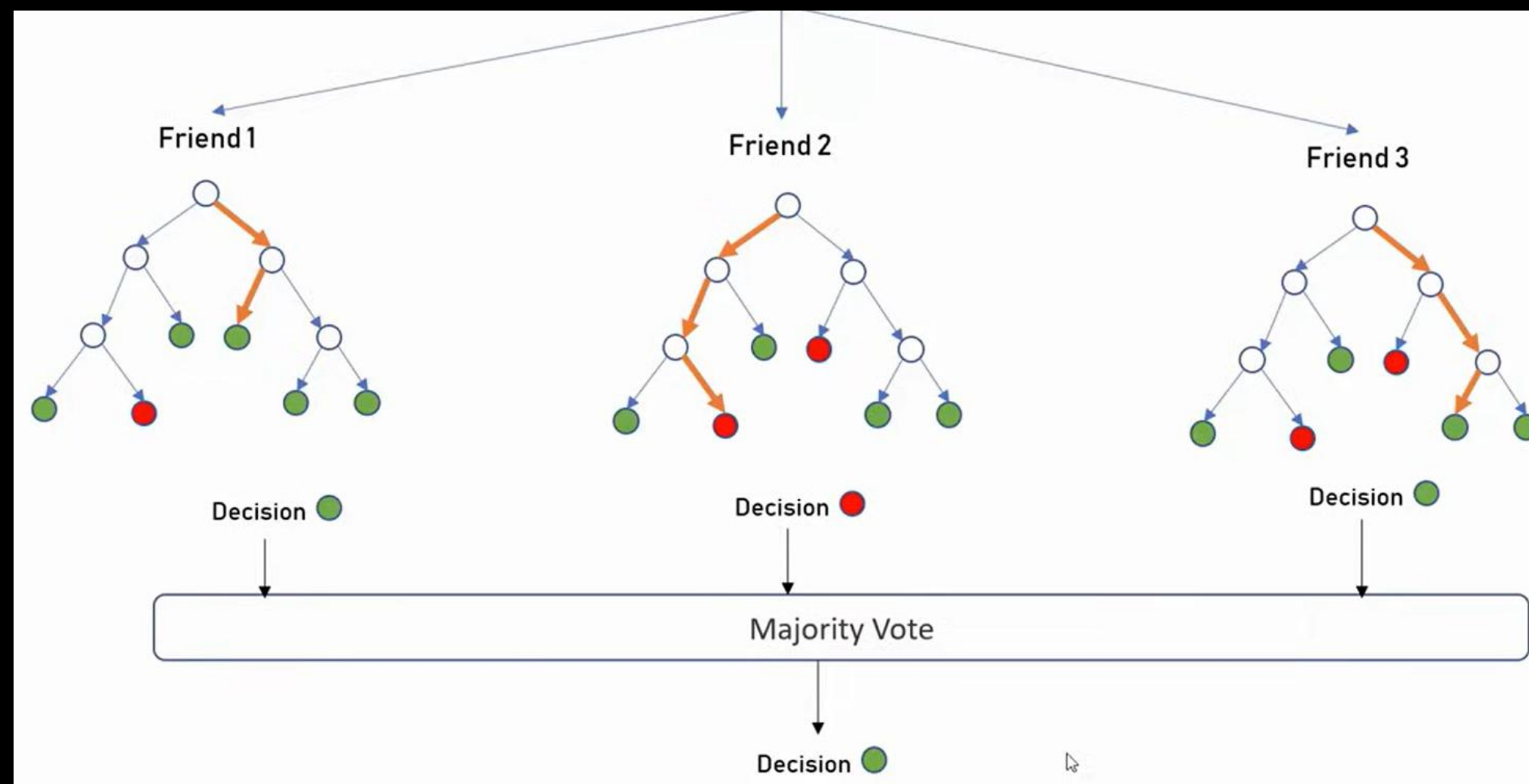


Random Forest



Random Forest

Should I buy PS5?



LAB 3

Flower

Classification



Naïve Bayes

Supervised Learning : Predicts probability of a class using Bayes Theorem

Type:

- Gaussian Naive Bayes
- Multinomial Naive Bayes
- Bernoulli Naive Bayes

Used for:

- Spam Detection
- Weather Prediction

Section 1: Basic Probability Theory



Coin Toss Example

When flipping a coin:

- $P(\text{Head}) = \frac{1}{2}$
- $P(\text{Tail}) = \frac{1}{2}$



There are only **two possible outcomes**.



Playing Card Example

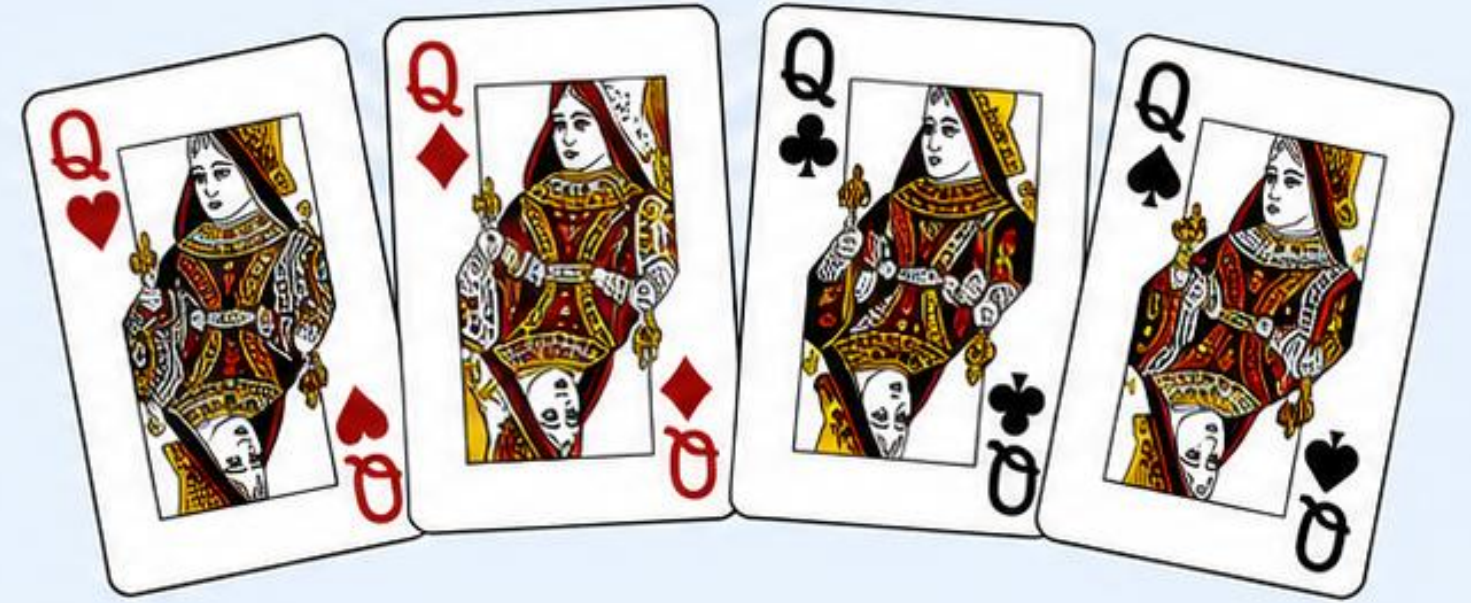


Probability of drawing a Queen:

$$\begin{aligned} P(\text{Queen}) &= \frac{4}{52} \\ &= \frac{1}{13} \end{aligned}$$



Since there are **4 Queens** in a deck of **52 cards**.



52
cards

Section 3: Conditional Probability



Question

If you already know the card is a **Diamond**, what is the probability it is a **Queen**?

Since:

- Total Diamonds = 13
- Diamond Queens = 1

$$P(\text{Queen} | \text{Diamond}) = \frac{1}{13}$$



This is called **Conditional Probability**.



Notation:

$$P(A | B)$$



Meaning:

Probability of event **A** occurring given that event **B** has already occurred.



Example:

$$P(\text{Queen} | \text{Diamond})$$

Probability of getting a **Queen** given the card is known to be a **Diamond**.

Section 4: Bayes Theorem



Thomas Bayes' Formula

Bayes Theorem allows us to calculate:

$$P(A|B)$$

using:

$$P(B|A)$$

along with the individual probabilities of **A** and **B**.



General formula:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



Why is Bayes Theorem Useful?

Sometimes:

- ✓ You know several probabilities.
- ✓ You need another probability that is difficult to calculate directly.



Bayes Theorem helps
derive the unknown probability
from the known ones.

Section 5: What Makes It "Naive"?



Naive Assumption

Naive Bayes assumes:



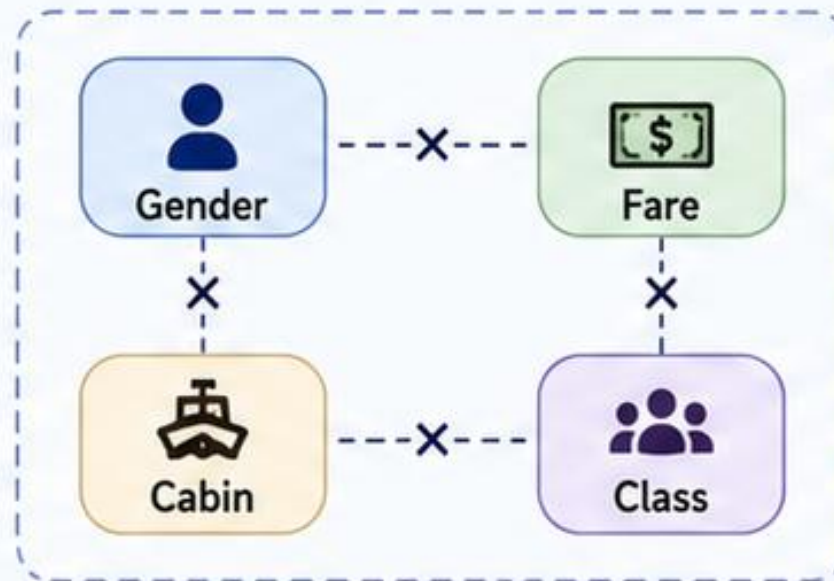
All features are **independent**.



For example:

Assume:

- **Gender** is independent of **Fare**
- **Fare** is independent of **Cabin**
- **Age** is independent of **Class**



Even though in reality some features may be related.



Example:

Fare and **Cabin** are often correlated.



Why Make This Assumption?

Because it:



Simplifies calculations



Reduces computational complexity



Makes the algorithm **fast**



Often produces surprisingly **good results**



This simplifying assumption is why it is called **Naive Bayes**.



Kmean Clustering

Unsupervised Learning : You have inputs (X) only – no correct answers

Type:

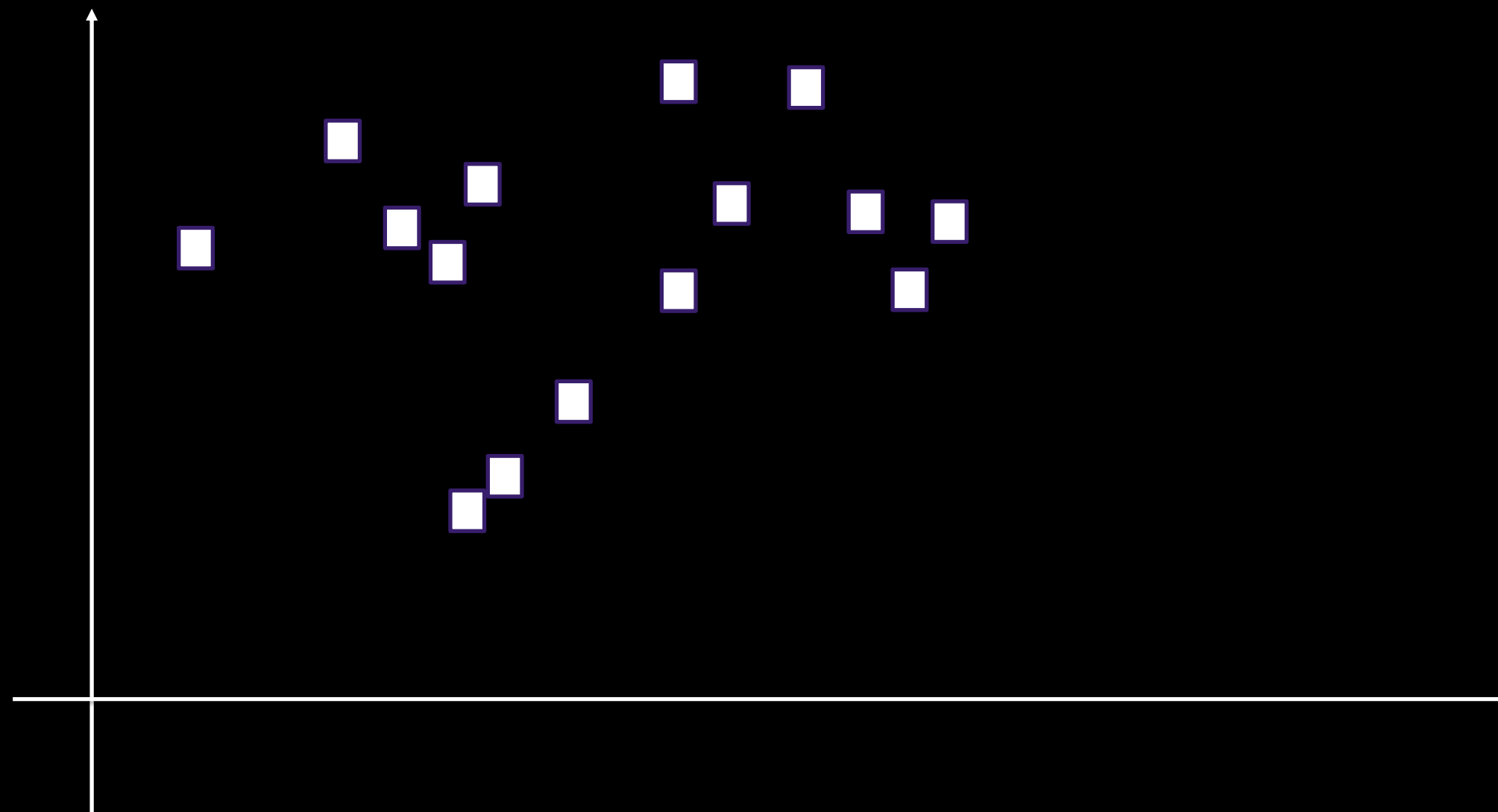
- Unsupervised Learning
- Clustering

Used for:

- Grouping similar data points
- Pattern discovery
- Data exploration

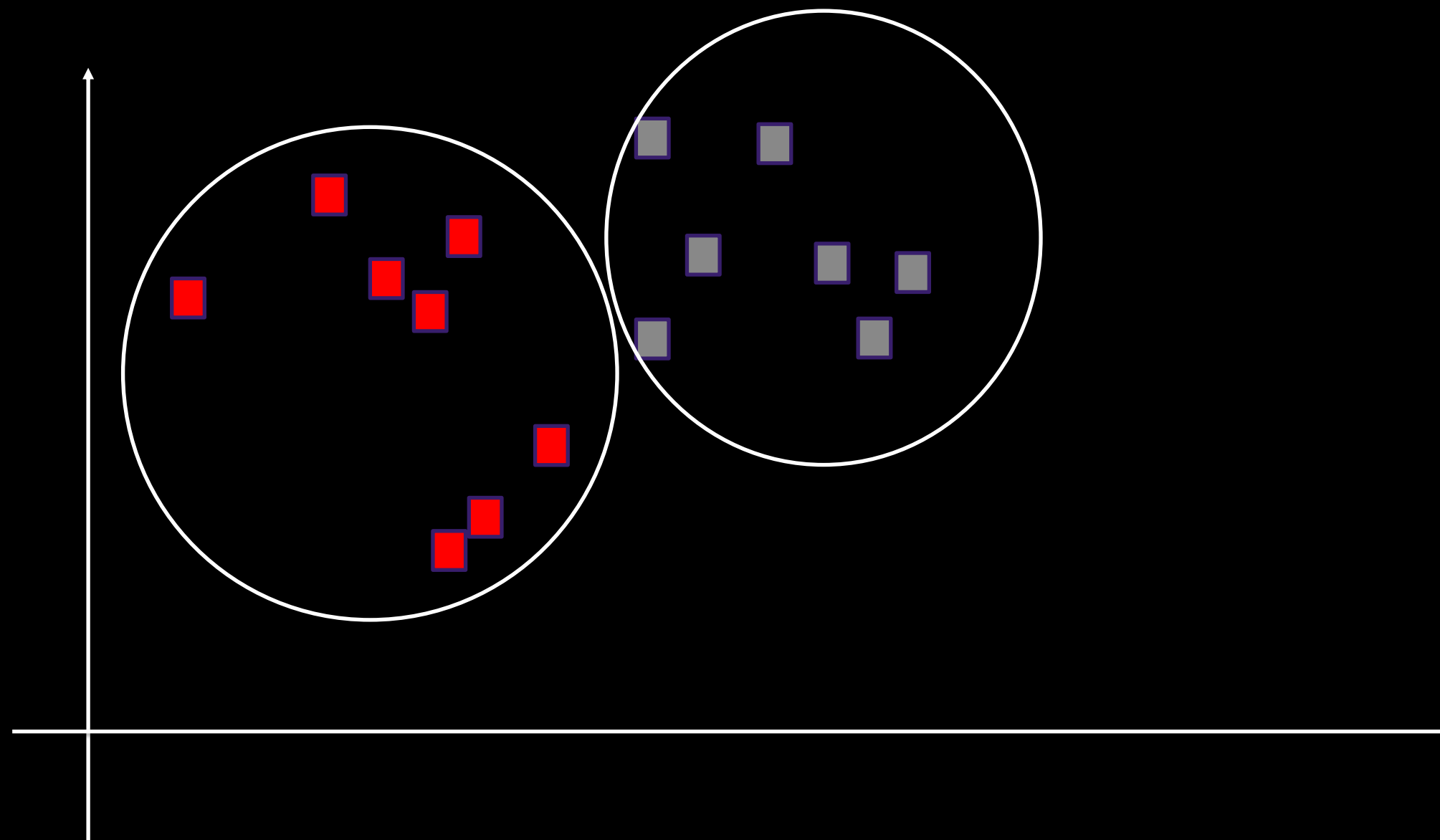


Kmean Clustering



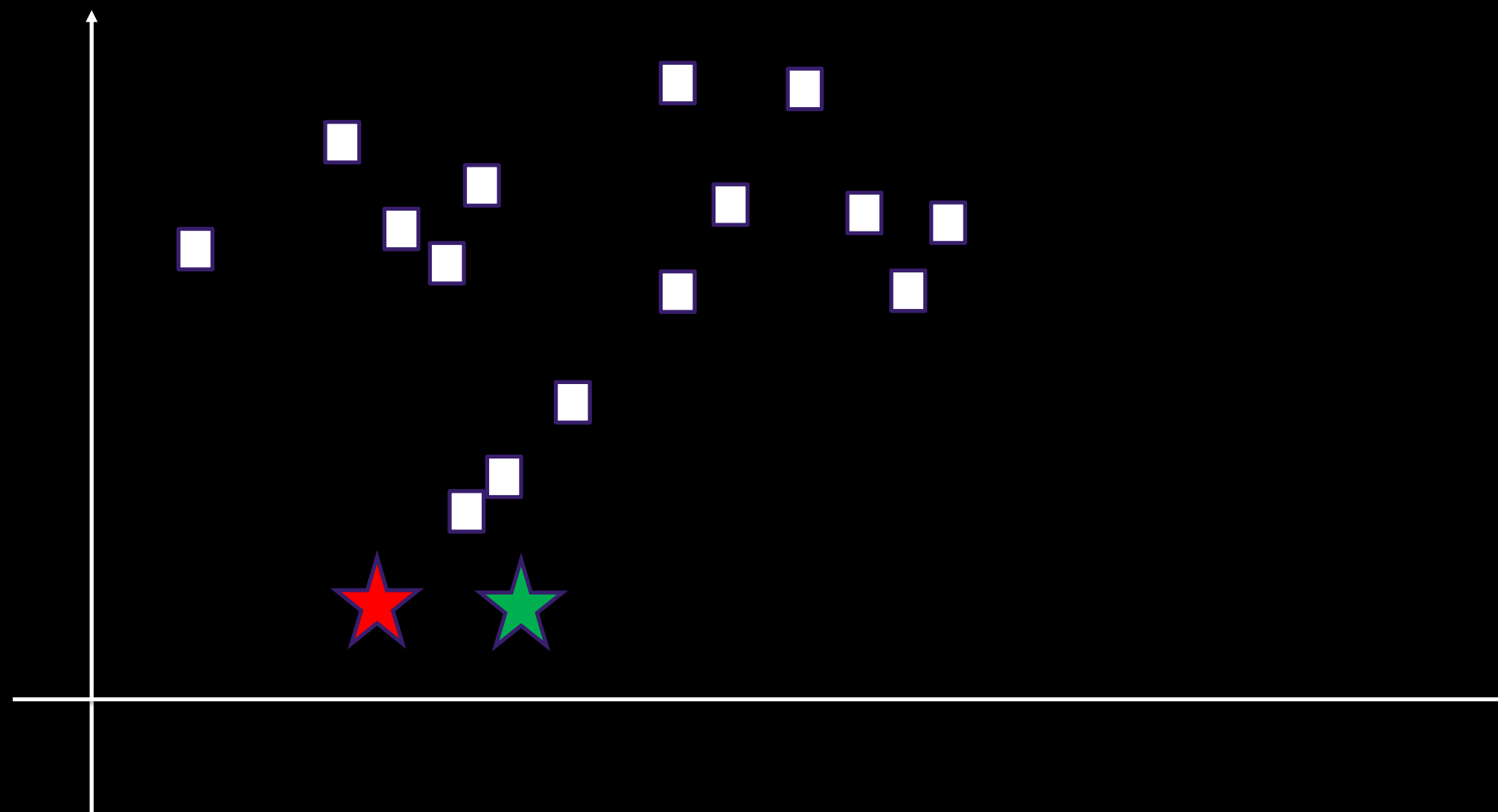


Kmean Clustering



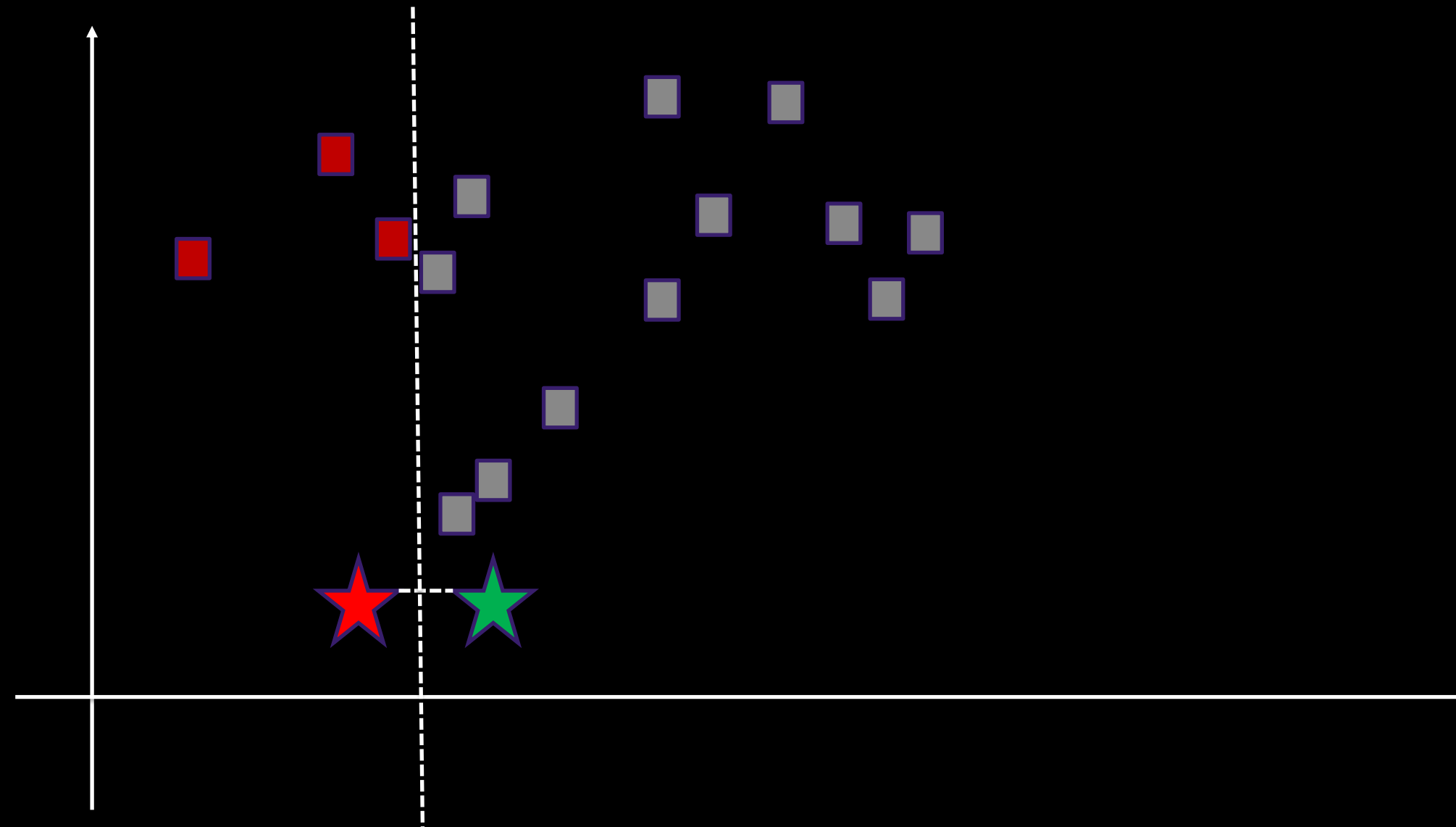


Kmean Clustering



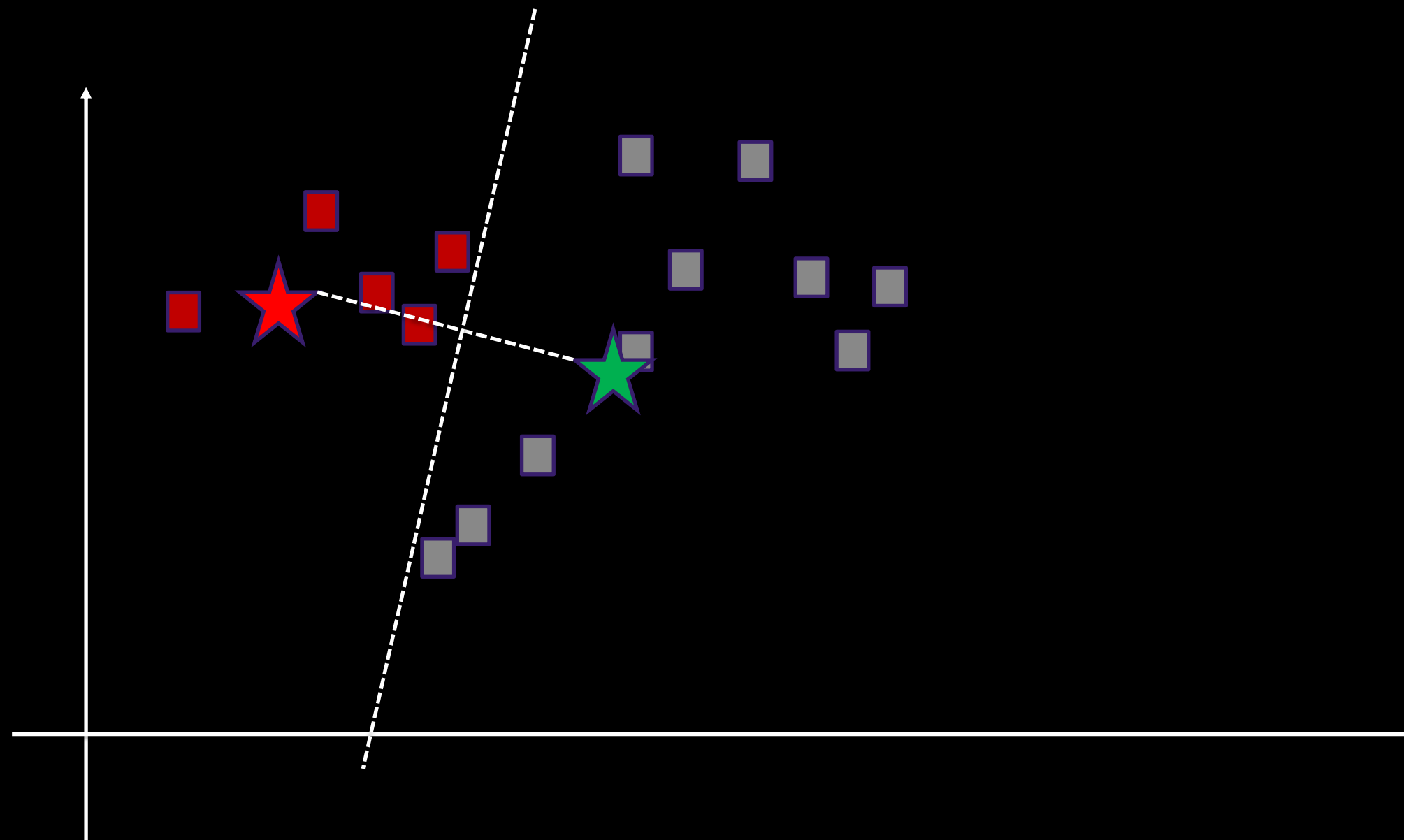


Kmean Clustering



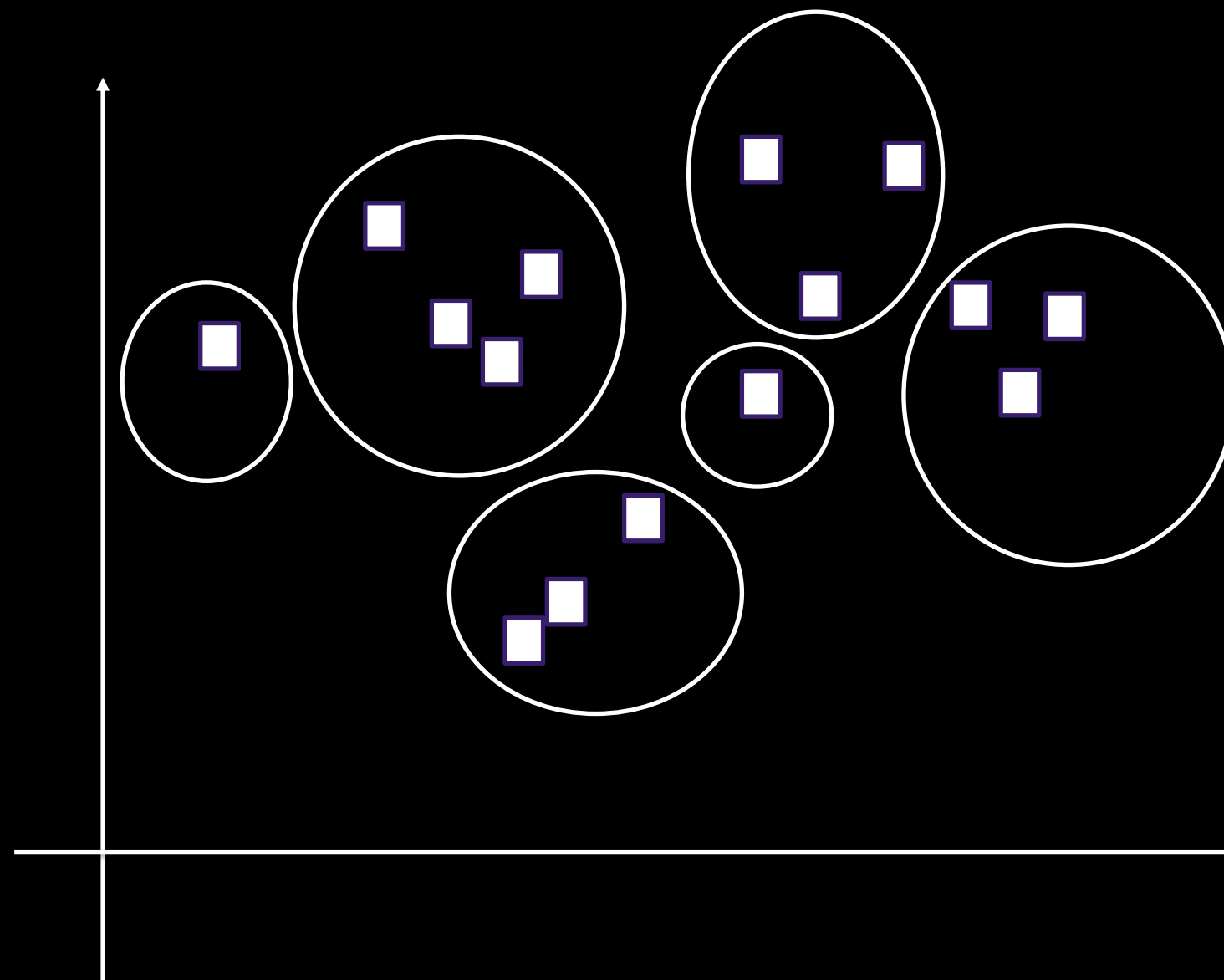


Kmean Clustering





Elbow Method

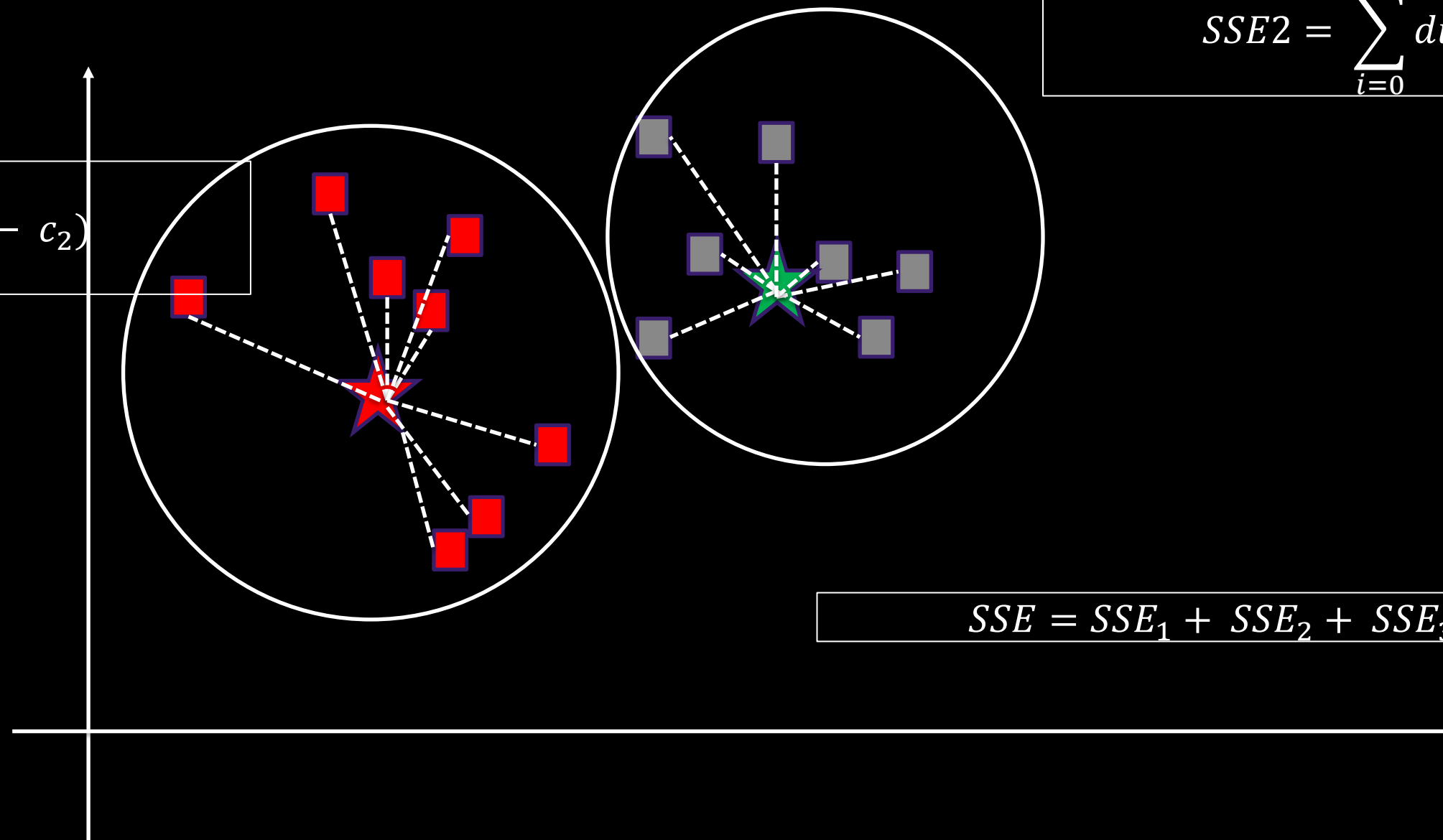


Elbow Method

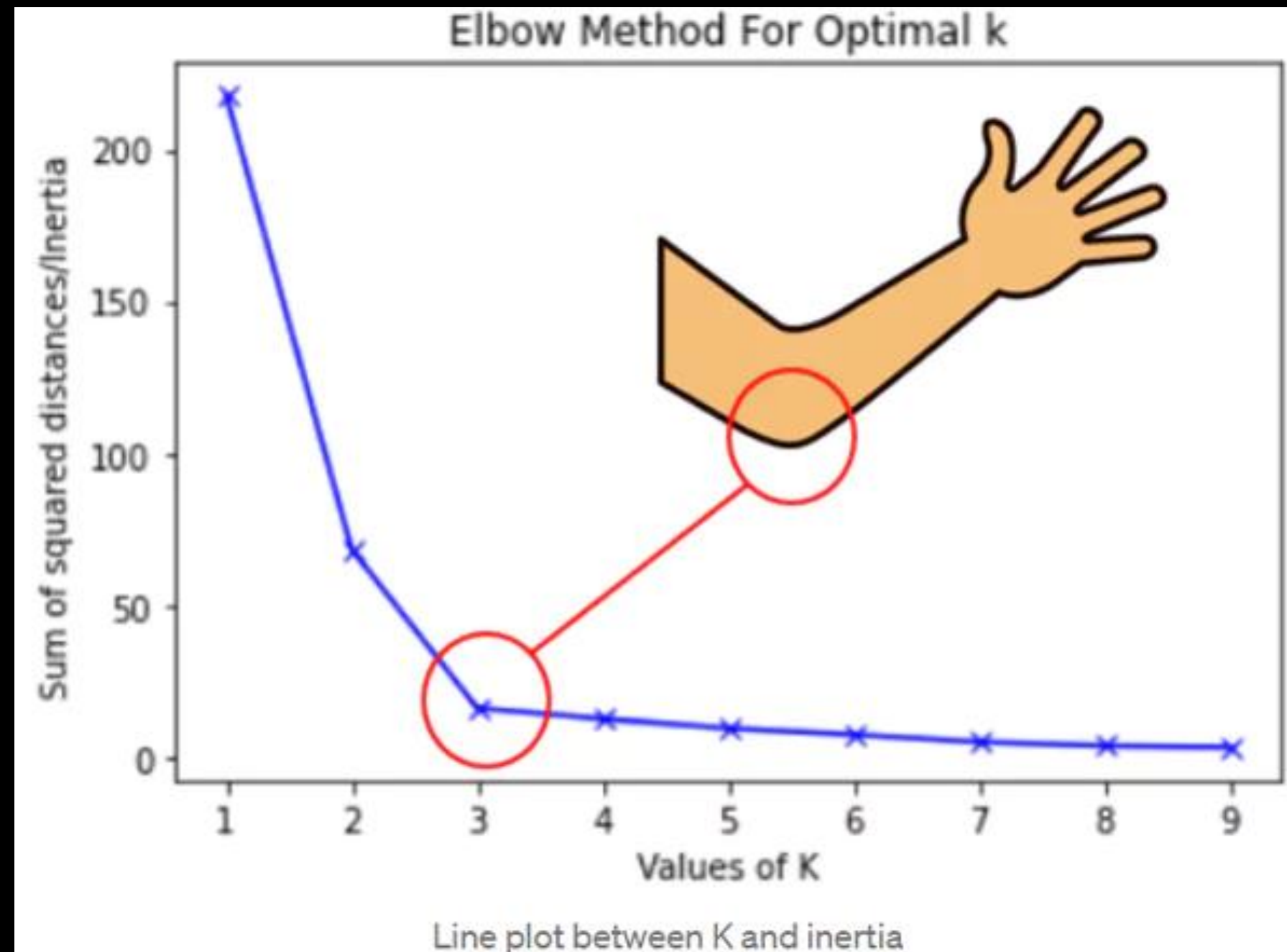
$$SSE1 = \sum_{i=0}^n dist(x_i - c_2)$$

$$SSE2 = \sum_{i=0}^n dist(x_i - c_2)$$

$$SSE = SSE_1 + SSE_2 + SSE_3 + \dots + SSE_k$$



Elbow Method



LAB 4

Flower

Segmentation



DimensionalityReduction

dimensionality reduction : Technique used to simplify a dataset while preserving as much information (variance) as possible.

Type:

- Unsupervised Learning

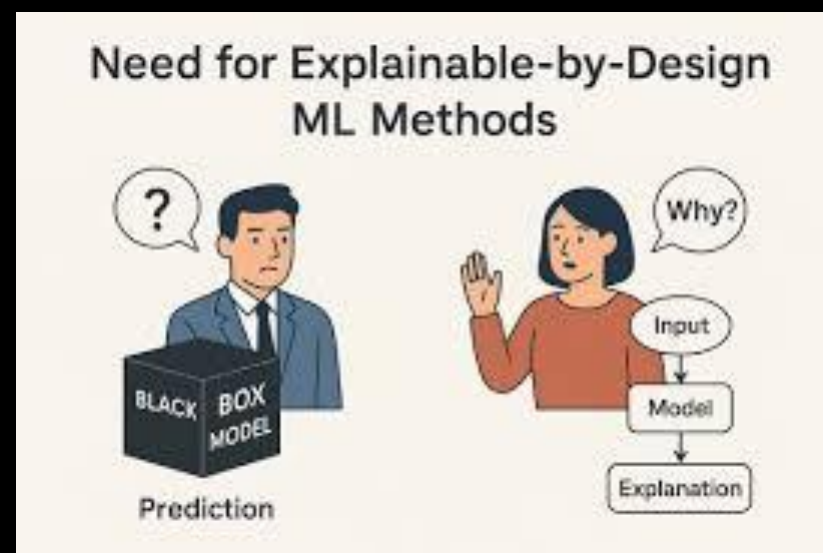
Used for:

- reduce complexity
- retaining most of the information in the dataset.

Explainability Matters

"Why did the model make this prediction?"

This is becoming very important because many machine learning models (Random Forest, XGBoost, Neural Networks) can achieve high accuracy but behave like a black box.



GLOBAL EXPLANATIONS (XAI)

How does the model behave overall?



A. FEATURE IMPORTANCE

Shows which features are most influential.

EXAMPLE

Predicting house prices

Feature	Importance
Size	45%
Location	30%
Bedrooms	15%
Age	10%

INTERPRETATION



Size contributes the most.



PROS

- Fast to compute
- Easy to understand



LIMITATION

- Can be biased toward:
 - high-cardinality features
 - correlated variables



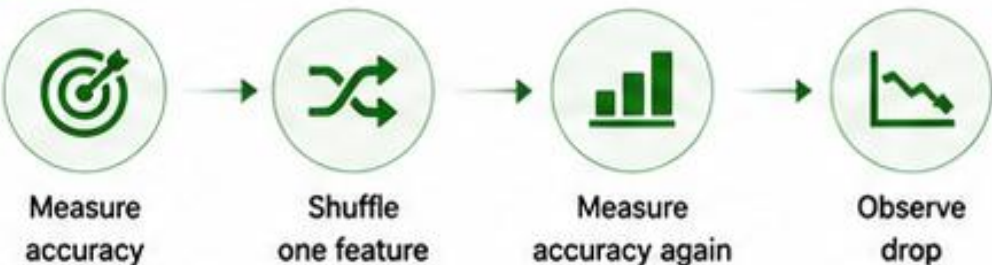
Better Alternative: Permutation Importance



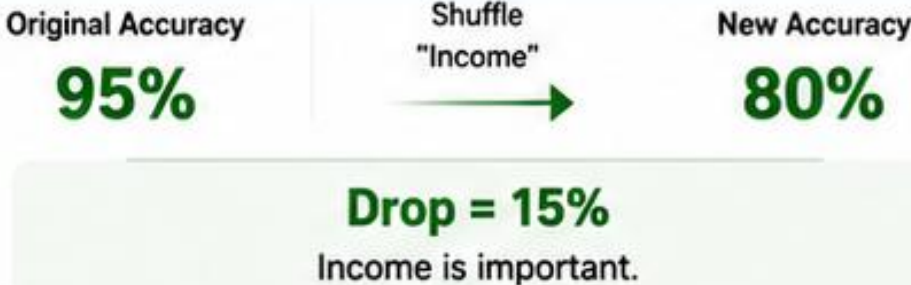
B. PERMUTATION IMPORTANCE

Measures the impact of a feature by observing the drop in model accuracy.

IDEA



EXAMPLE



PROS

- More reliable than basic feature importance
- Model agnostic



LIMITATION

- Computationally expensive



C. SHAP (GLOBAL VIEW)

SHAP (SHapley Additive exPlanations) based on game theory.

WHAT IT TELLS

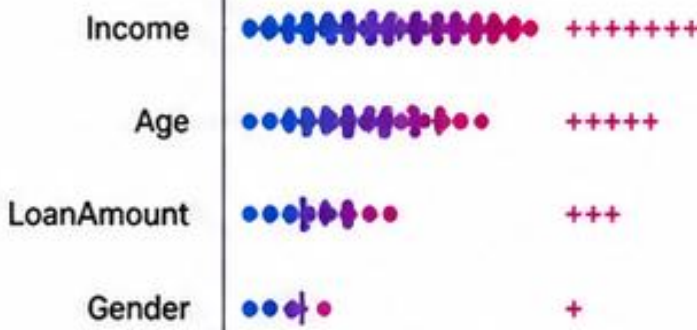


How much each feature contributes to predictions.

WHY SHAP?

- ✓ More accurate
- ✓ Consistent
- ✓ Works for most models

SHAP GLOBAL PLOT (EXAMPLE)



INTERPRETATION

Income is the strongest factor.



PROS

- Most accurate attribution
- Consistent & fair
- Model agnostic (TreeSHAP, KernelSHAP, etc.)



LIMITATION

- Slower to compute compared to other methods



AT A GLANCE

METHOD	BEST FOR	KEY TAKEAWAY
Feature Importance	Quick overview of important features	Fast and simple, but can be biased
Permutation Importance	Reliable ranking of feature impact	Measures real impact on model performance
SHAP (Global View)	Deep, accurate, and consistent explanations	Best for trustworthy and model-agnostic insights

LOCAL EXPLANATIONS (XAI)

Why did **THIS** specific prediction happen?



Local explanations focus on a **single prediction**, not the whole model.



A. LIME

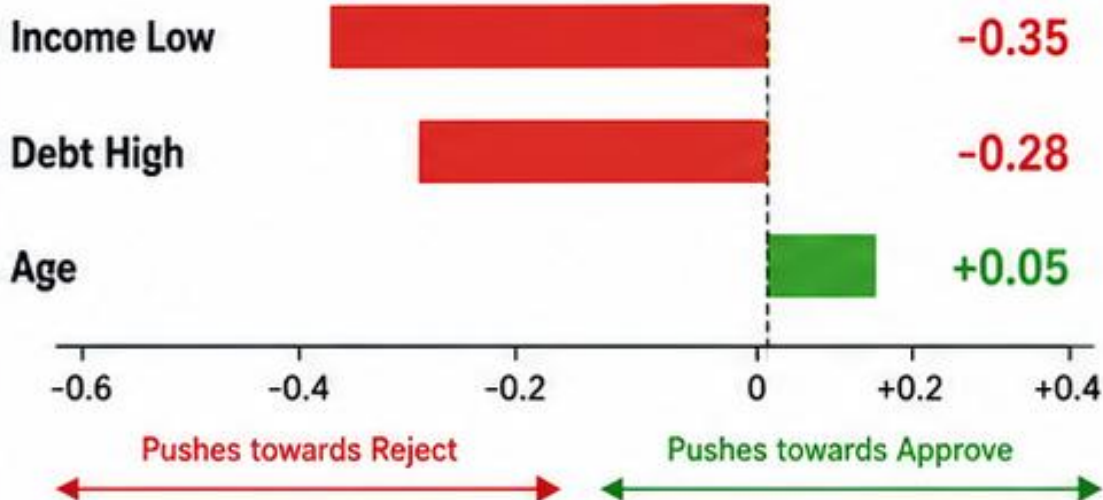
Local Interpretable Model-agnostic Explanations



Scenario: Loan Application
Customer A was denied a loan.

Prediction:
REJECTED

Feature Contributions (Local)



How LIME Works

1. Create nearby synthetic samples
2. Observe model predictions
3. Train a simple (local) model
4. Explain decision locally



PROS

- Fast and easy to understand
- Model agnostic (works with any model)



LIMITATIONS

- Can be unstable
- Depends on how samples are generated



B. SHAP (FOR INDIVIDUAL PREDICTIONS)

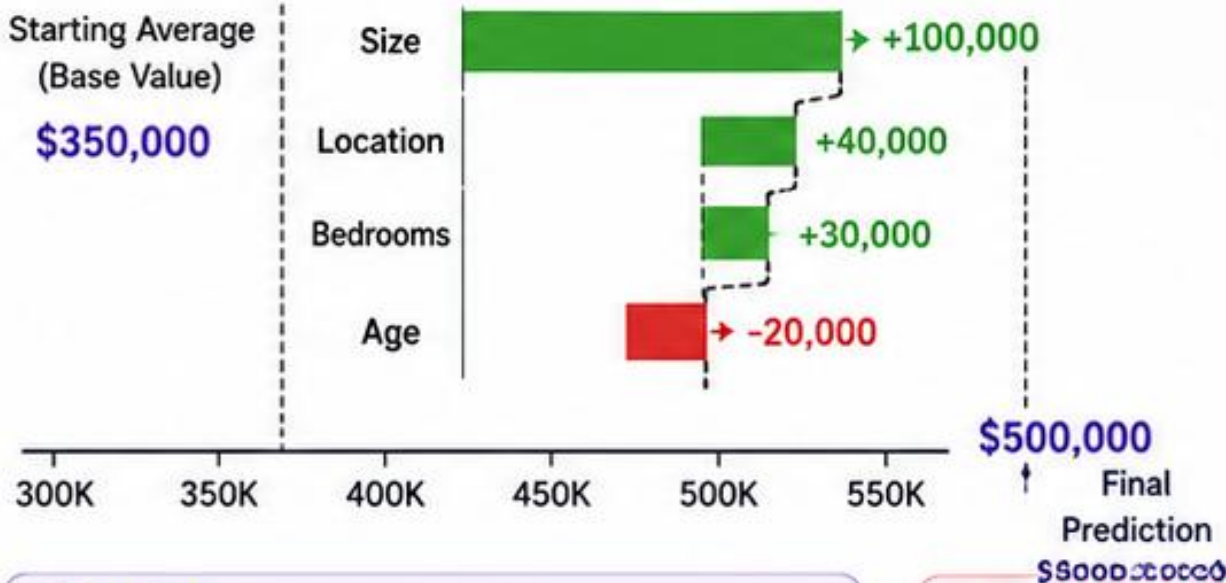
SHapley Additive exPlanations



Scenario: House Price Prediction
Explaining one specific prediction.

Predicted House Price:
\$500,000

SHAP Waterfall Explanation (Local)



PROS

- Consistent and theoretically sound
- Accurate feature attributions
- Works with most models



LIMITATIONS

- Computationally expensive (for large models/data)
- Requires more resources

How SHAP Works

1. Start from the average prediction
2. Add feature contributions
3. Sum up to the final prediction



KEY TAKEAWAY



LIME

Explains a single prediction using a simple local model around that prediction.



SHAP

Explains a single prediction using Shapley values, showing how each feature contributes.



Both methods help you understand **WHY** a specific prediction happened.

PDP vs ICE PLOTS

How does changing a feature affect predictions?

 PDP shows the average effect. ICE shows the effect for each individual.



1. PARTIAL DEPENDENCE PLOT (PDP)

Shows the average effect of a feature on the predicted outcome.



EXAMPLE

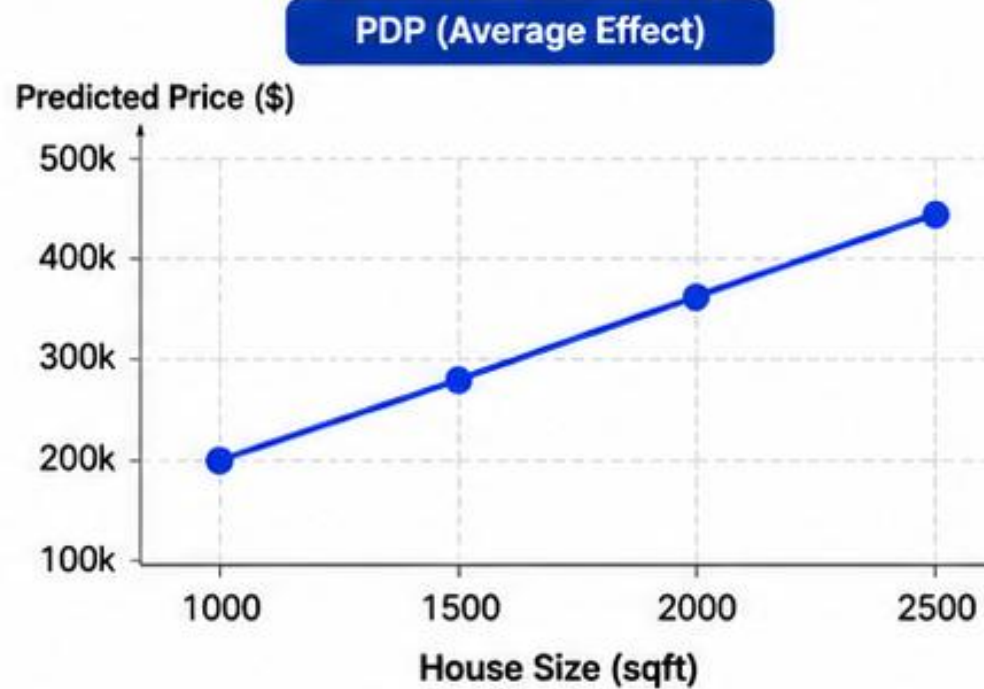
Feature: House Size (sqft)

House Size (sqft)	Predicted Price
1000	\$200k
1500	\$280k
2000	\$360k
2500	\$430k



INTERPRETATION

On average, as house size increases, the predicted price increases.



CODE EXAMPLE

```
from sklearn.inspection import PartialDependenceDisplay
PartialDependenceDisplay.from_estimator(
    model, X_test, ["Size"]
)
```



2. ICE PLOT (INDIVIDUAL CONDITIONAL EXPECTATION)

Shows how the prediction changes for each individual observation.



EXAMPLE

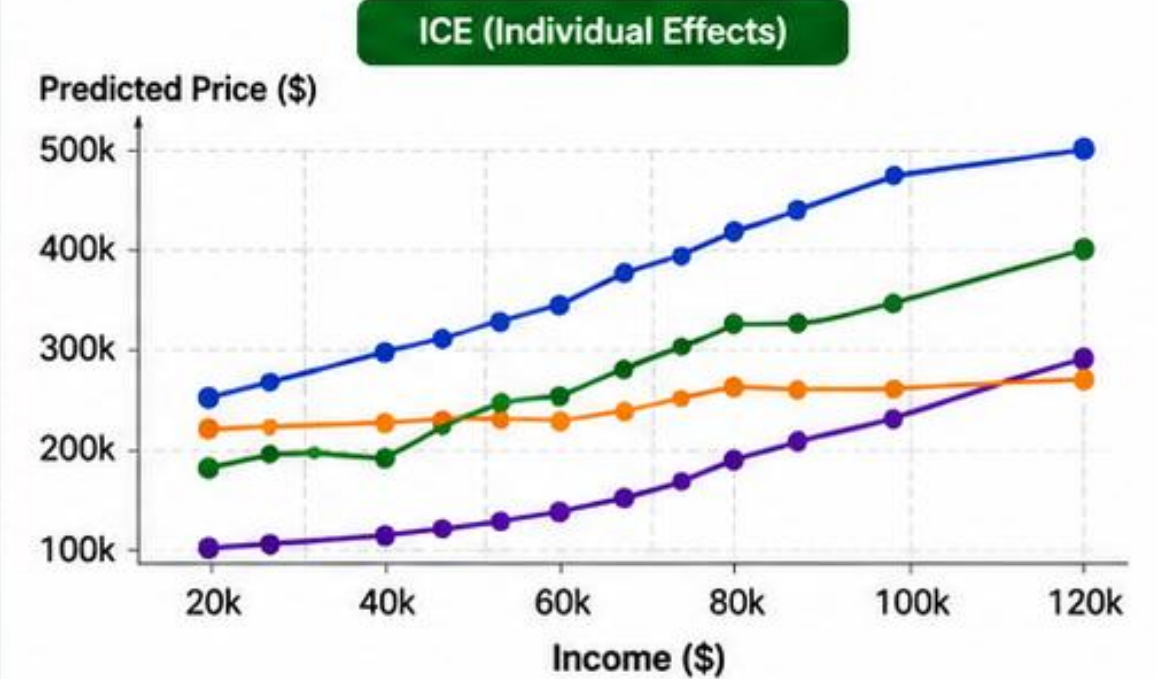
Feature: Income (\$)

- Customer 1
- Customer 2
- Customer 3
- Customer 4



INTERPRETATION

Different customers respond differently to changes in income. Some benefit more than others.



CODE EXAMPLE

```
from sklearn.inspection import PartialDependenceDisplay
PartialDependenceDisplay.from_estimator(
    model, X_test, ["Income"], kind="individual"
)
```



PDP vs ICE
AT A GLANCE

Aspect	PDP (Partial Dependence Plot)	ICE (Individual Conditional Expectation)
 What it shows	Average effect of a feature	Individual effect for each observation
 Visualization	One line	Many lines
 Ease of reading	Easier to read and interpret	More detailed, can be complex
 Type of explanation	Global explanation	Local-ish explanation (per observation)



KEY TAKEAWAY

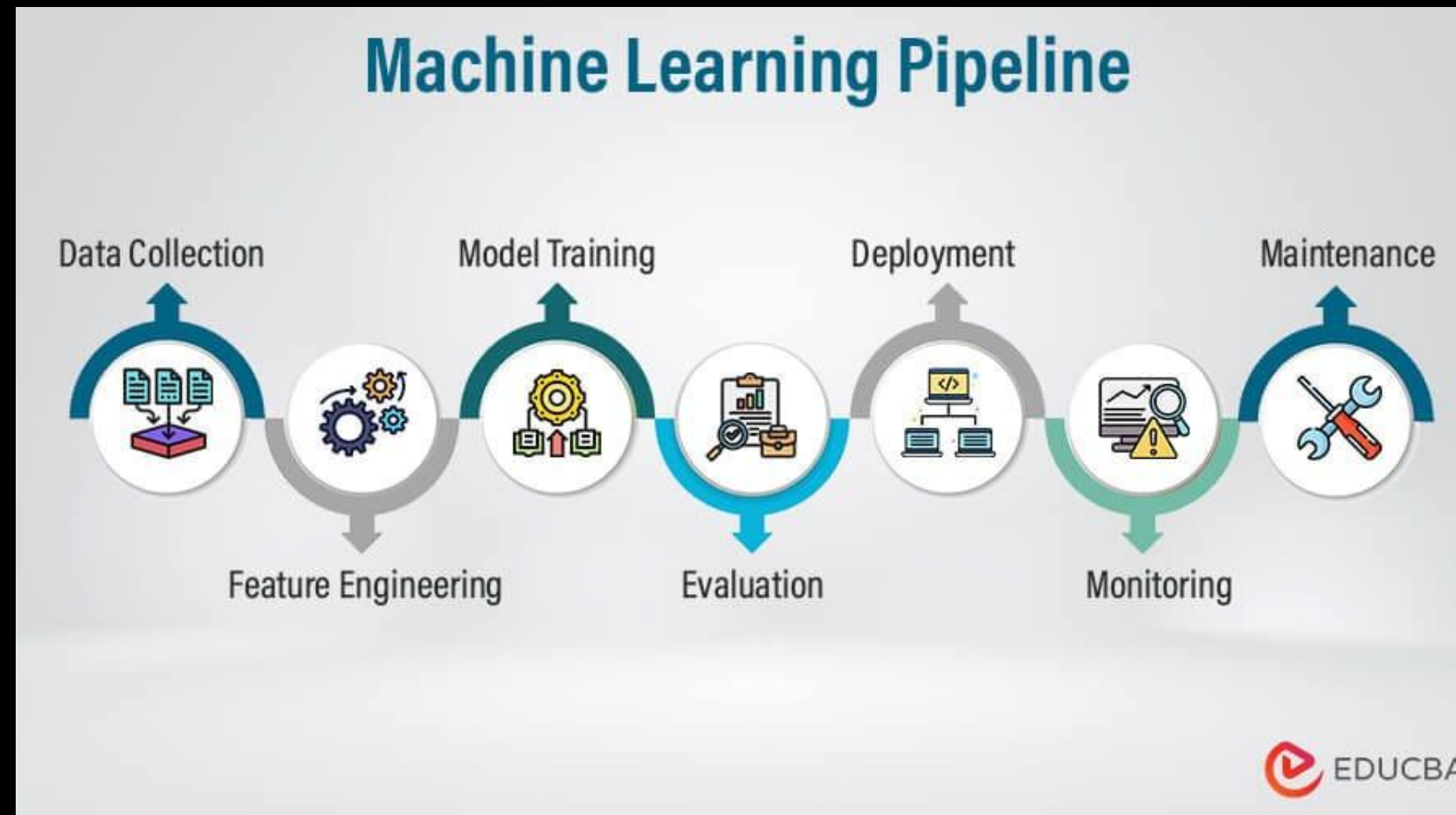
- ✓ Use PDP to understand the overall (average) trend.
- ✓ Use ICE to uncover differences among individuals.
- ✓ ICE can reveal heterogeneity that PDP hides.



Together, PDP and ICE give a complete picture!

Pipelines

A sequence of steps that automatically transforms data and trains a model in the correct order.





Pipelines

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)
```




Model Deployment

In practice, there are two common ways to deploy a machine learning model:

1. Using a model file
2. Using a model through an API

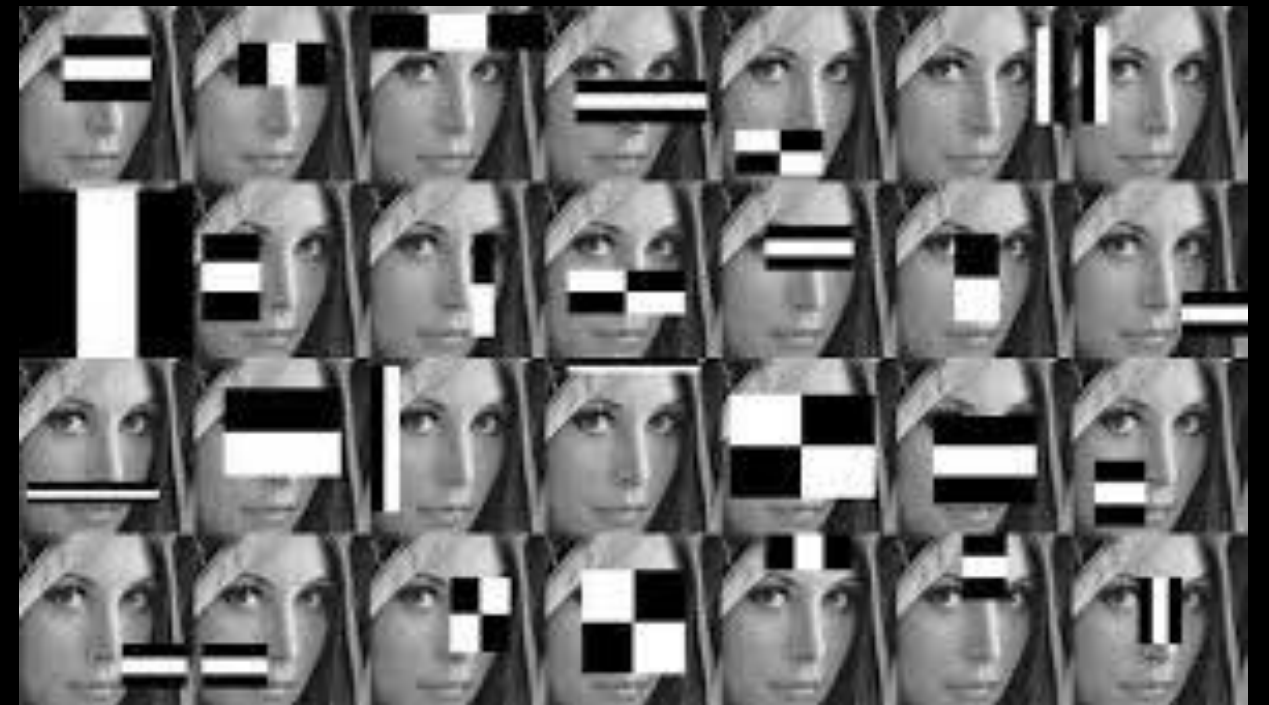
Both methods use the same idea. The model is already trained and We only send input then The model returns output

PRE-TRAINED MODEL

Model Deployment

Deployment Using a Model File (Haar Cascade Classifier)

- Pre-trained Supervised Learning model
- Used for object detection (e.g. face detection)
- Model is saved as an XML file



Model Deployment

Deployment Using a API (GPT Model)

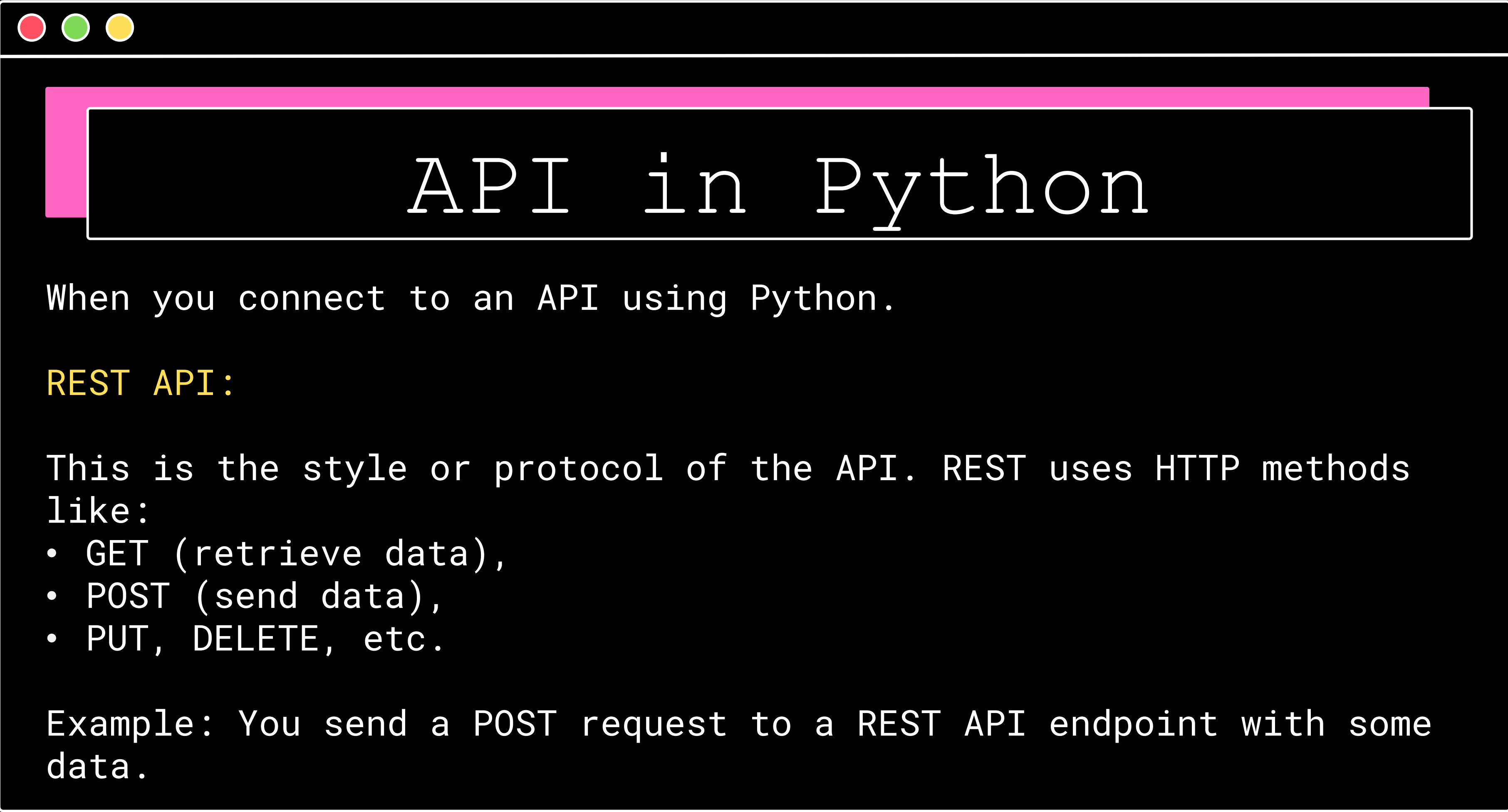
- Model is hosted in the cloud
- Model is accessed through an API
- No local model file required



Model Deployment

We can deploy our model using an API. A common approach for python is using FastAPI





API in Python

When you connect to an API using Python.

REST API:

This is the style or protocol of the API. REST uses HTTP methods like:

- GET (retrieve data),
- POST (send data),
- PUT, DELETE, etc.

Example: You send a POST request to a REST API endpoint with some data.



API in Python

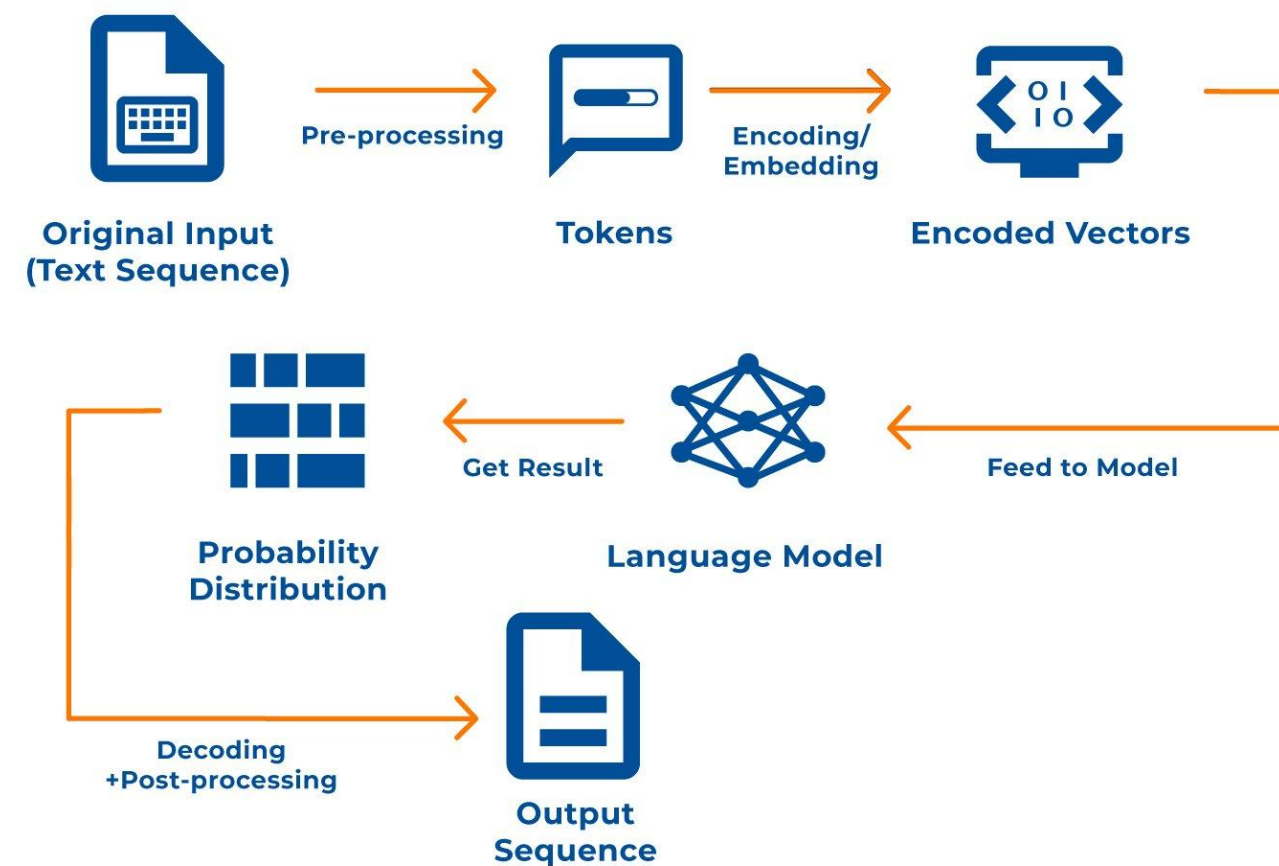
When you connect to an API using Python.

JSON (JavaScript Object Notation):

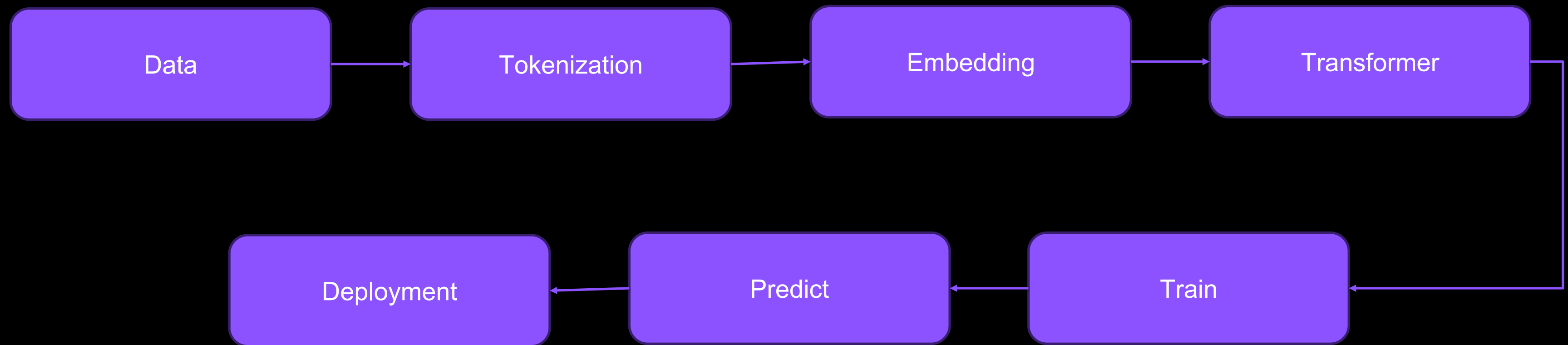
- This is the data format used to send or receive information. Python can handle JSON easily using the json module.
- You send a Python dictionary (converted to JSON) to the API, and the API replies in JSON too.

GPT Model

How do **GPT Models** work?



GPT Model



GPT Model

Tokenization

breaking text into smaller pieces that a machine can work with.

Text is split into tokens. Tokens can be:

Words

Sub-words

Characters

Tokens	Characters
11	43

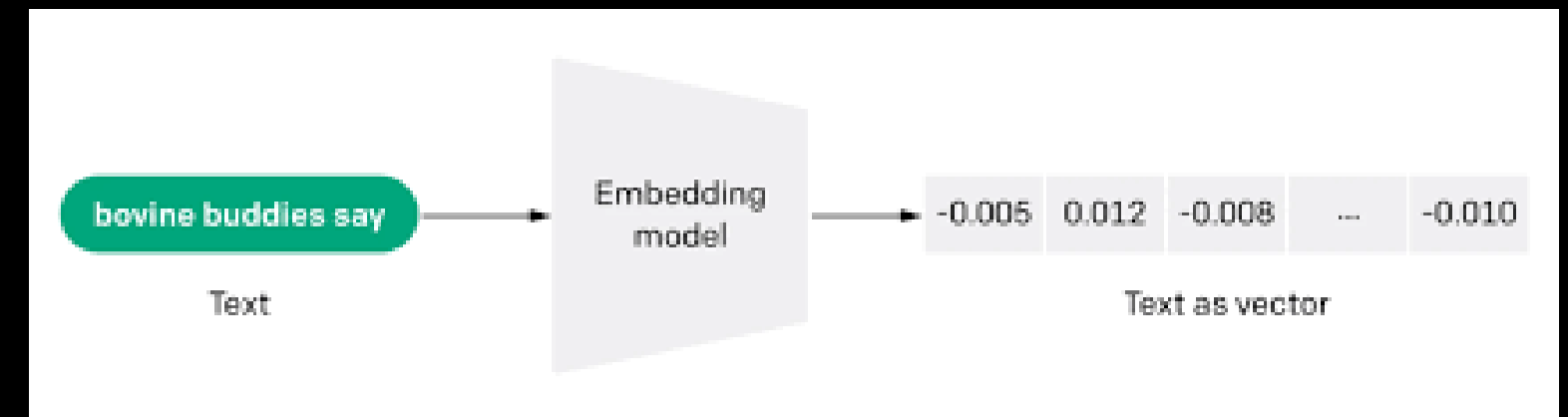
We need to stop anthropomorphizing ChatGPT.

GPT Model

Embedding

numbers that represent meaning. So we convert each token into a vector of numbers that captures what the token means and how it relates to other tokens.

"I" → [0.21, -0.34, 0.87, ...]
"learn" → [0.55, 0.12, -0.44, ...]
"ML" → [0.91, -0.08, 0.66, ...]





GPT Model

Transformer

GPT uses a Transformer neural network

Contains Millions to billions of parameters Parameters = learned weights

Designed specifically for language

Reads and understands context, not just single words

"The book on the table is blue"



GPT Model

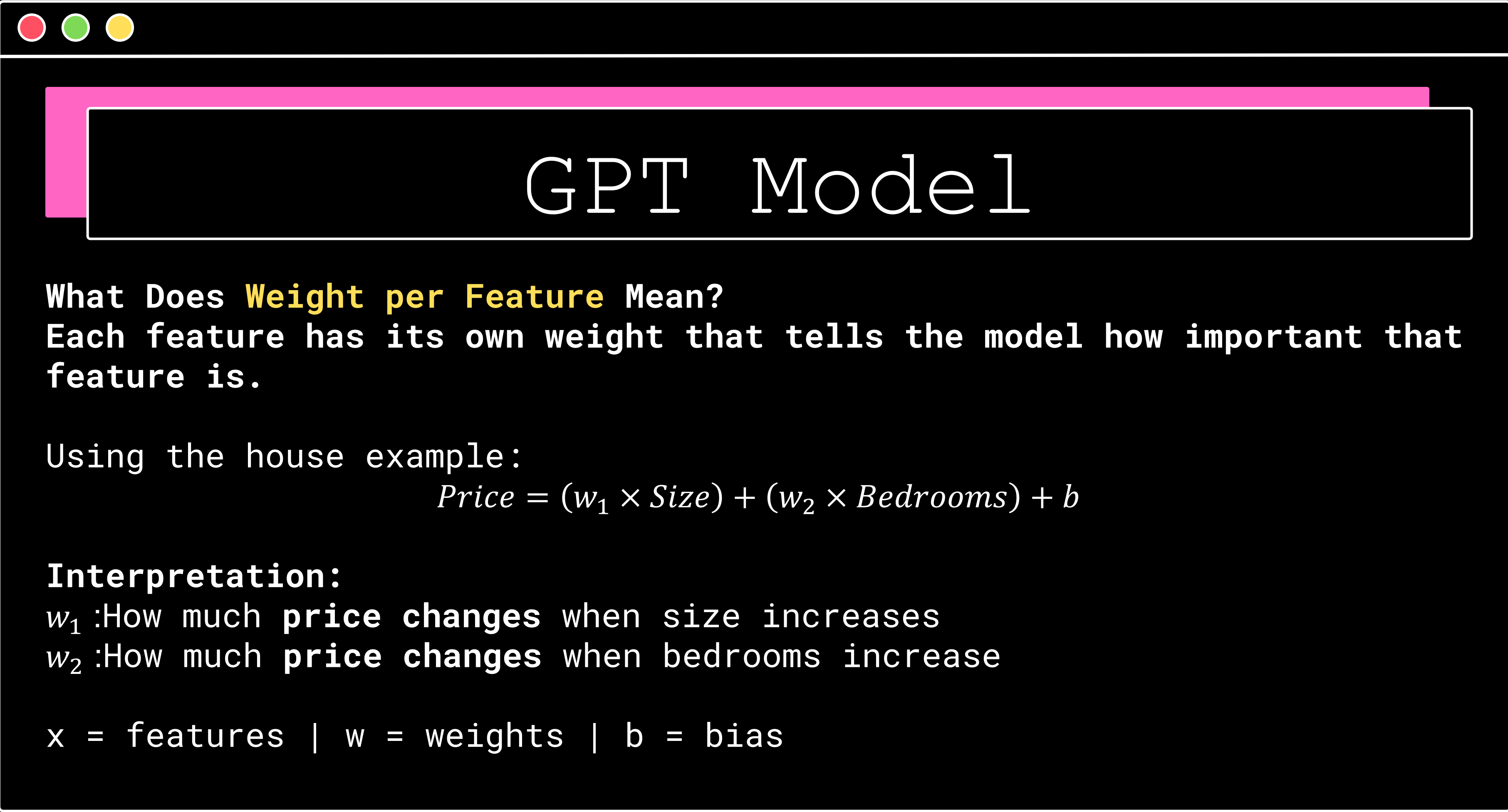
Parameters = numbers the model adjusts while learning. Each parameter is a weight

Weights control:

Importance of words

Relationships between tokens

Context influence



GPT Model

What Does **Weight per Feature** Mean?

Each feature has its own weight that tells the model how important that feature is.

Using the house example:

$$Price = (w_1 \times Size) + (w_2 \times Bedrooms) + b$$

Interpretation:

w_1 : How much **price changes** when size increases

w_2 : How much **price changes** when bedrooms increase

x = features | w = weights | b = bias



GPT Model

Training = adjusting model parameters so predictions become more accurate.

No matter how advanced the model is, training always follows this loop

Predict → Measure error → Adjust → Repeat

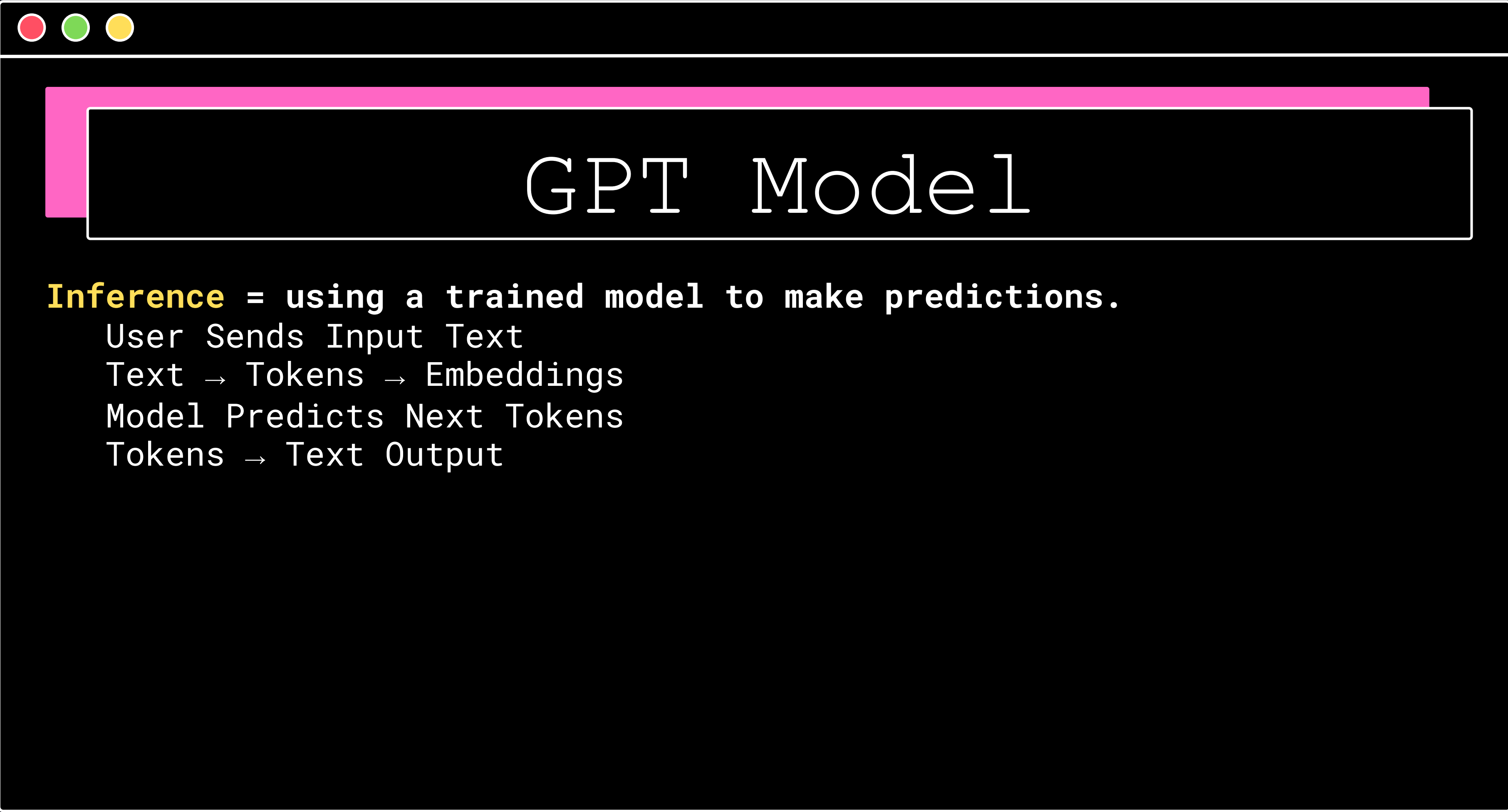
CORE TRAINING

- Predict the next token

- Loss Function (Error Measurement)

- Parameter Updates (Learning Happens Here)

- Iterations (Why Training Takes Long)



GPT Model

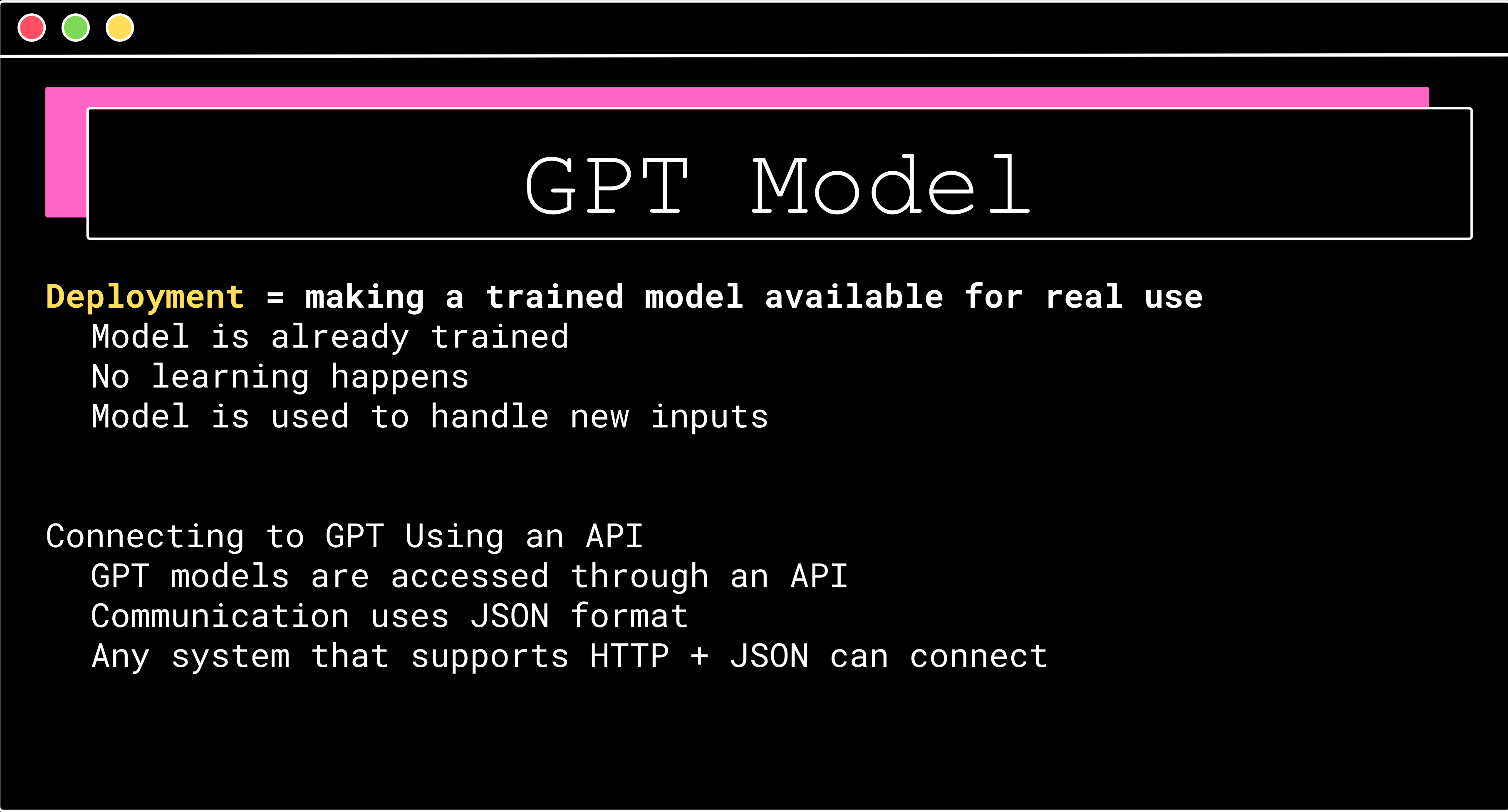
Inference = using a trained model to make predictions.

User Sends Input Text

Text → Tokens → Embeddings

Model Predicts Next Tokens

Tokens → Text Output



GPT Model

Deployment = making a trained model available for real use

- Model is already trained

- No learning happens

- Model is used to handle new inputs

Connecting to GPT Using an API

- GPT models are accessed through an API

- Communication uses JSON format

- Any system that supports HTTP + JSON can connect

NLP

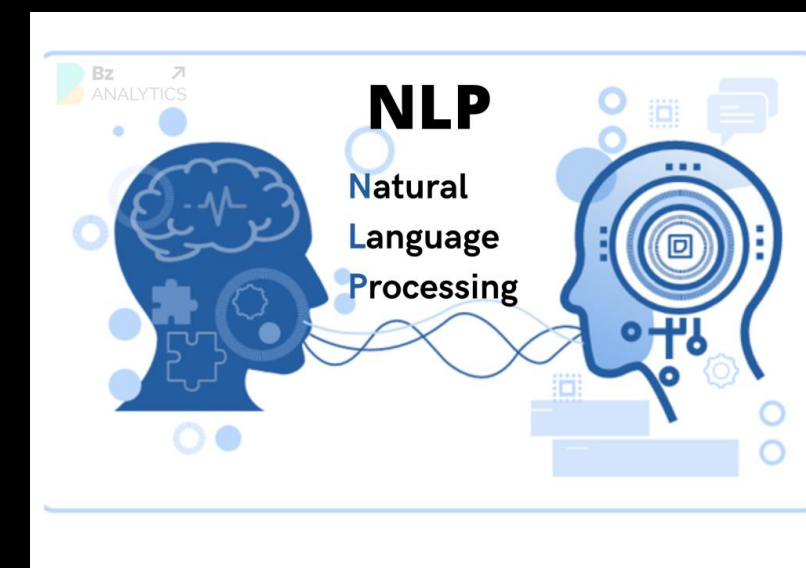
Artificial Intelligence

└─ Machine Learning

└─ Deep Learning

└─ Natural Language Processing (NLP)

└─ GPT



LAB 5

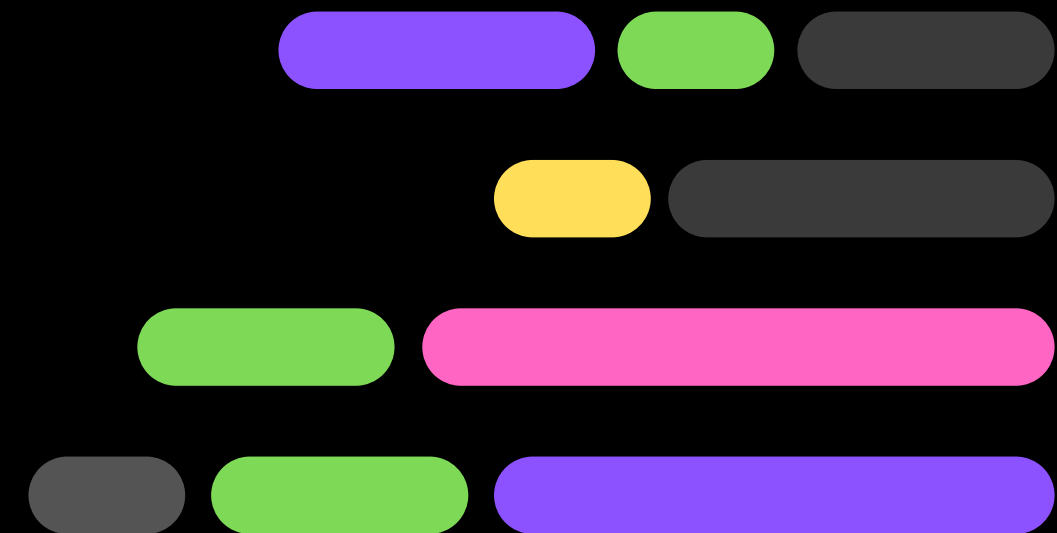
Building Chatbot

LAB 6

Chatbot Memory



CONCLUSION



FURTHER LEARNING

- Real World Application (WEB)



FURTHER LEARNING

- Real World Application (Data Science)



FURTHER LEARNING

- Real World Application (Machine Learning)



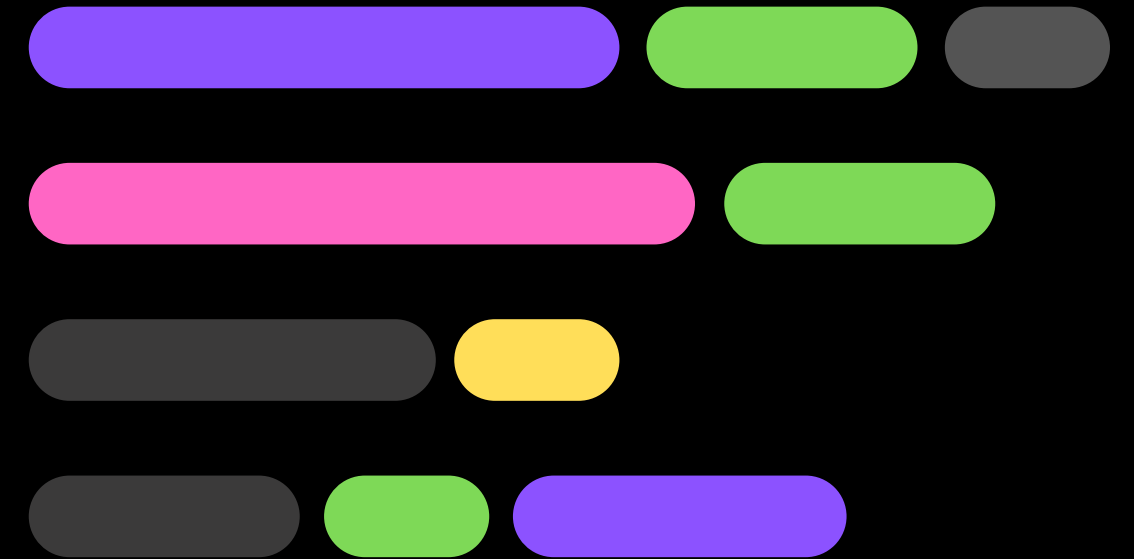
FURTHER LEARNING

- Real World Application (Automation)





THANK YOU!



SUFI FIRDAUS BIN FAKHRURRAZEY
sufi@iverson.com.my